
Overview & Revision

- ❑ Overview
- ❑ Revision of C/C++ Programming

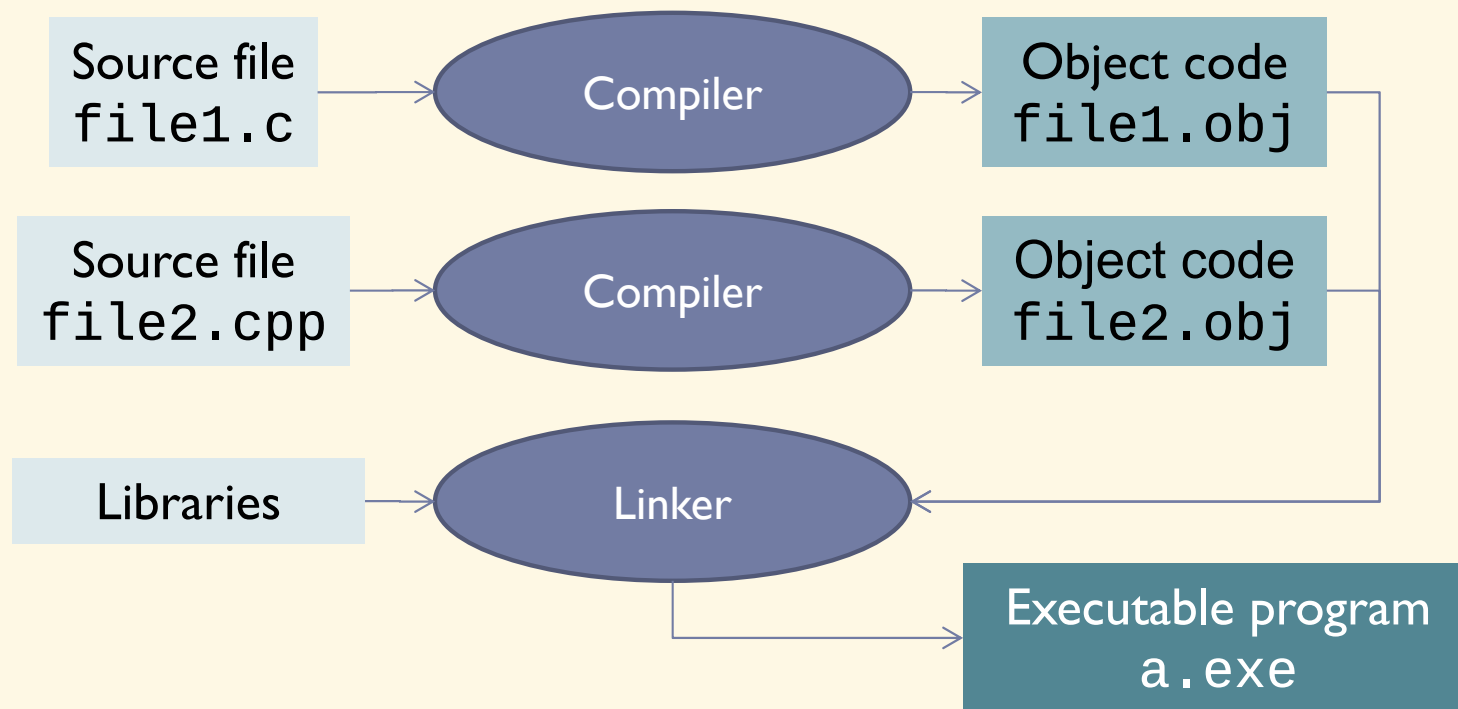
Overview

Introduction

- ▶ Data Structures & Algorithms (ET2100, ET2101E, ET2105E)
- ▶ Evaluation:
 - ▶ Midterm: In-class participation + Exercises + Homework
 - ▶ Final: Project
- ▶ Compilers: gcc/g++ or MS Visual C++
- ▶ Textbooks:
 - ▶ *Data Structures using C (2nd Ed.)*, R. Thareja, Oxford University Press, 2014
 - ▶ *Data Structures and Algorithm Analysis in C++ (4th Ed.)*, M. Weiss, Pearson, 2013
 - ▶ *C Programming – A Modern Approach (2nd Ed.)*, K.N. King, W.W. Norton & Company, 2008

Compiling a C/C++ program

- **Compilation:** is the process of converting a source code (written by human) into a program in the form of machine codes that can be executed



Compiling a C/C++ program (*cont.*)

- ▶ Benefits of compiling each source file individually:
 - ▶ Easy to divide and manage parts of the program
 - ▶ Only need to modify the involved files when something need to change
 - shorten maintenance, update time
 - ▶ Only need to recompile modified files instead of everything
 - shorten compilation time
- ▶ Modern compilers are able to optimize compiled codes (both data and instructions)
- ▶ Some popular compilers: MS Visual C++, gcc/g++, clang, Intel C++ Compiler, Watcom C/C++,...

Environment setup

- ▶ **MS Visual C++:**

- ▶ <https://visualstudio.microsoft.com/>

- ▶ **gcc under Linux (Ubuntu)**

- ▶ `sudo apt update`
`sudo apt install build-essential`

- ▶ **gcc under Windows**

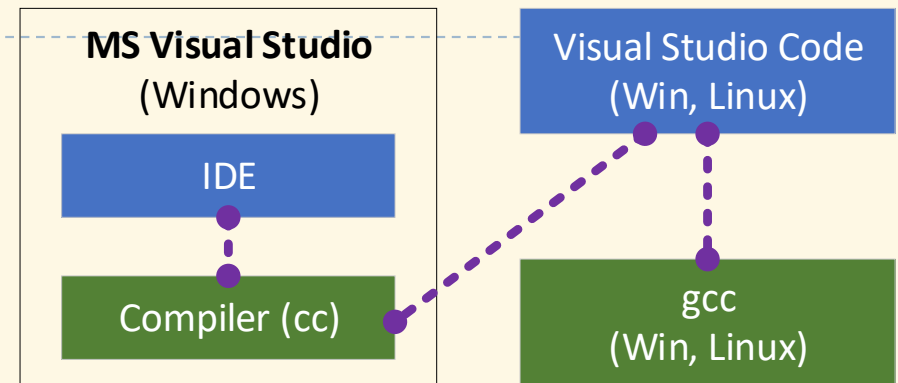
- ▶ Option 1: Windows Subsystem for Linux (WSL) → Ubuntu for WSL
 - ▶ Option 2: Use a MinGW-w64, POSIX, ucrt build:
 - ▶ <https://github.com/niXman/mingw-builds-binaries/releases>

- ▶ **Visual Studio Code:**

- ▶ <https://code.visualstudio.com/>
 - ▶ C/C++ Extension Pack

- ▶ **OnlineGDB:**

- ▶ <https://www.onlinegdb.com/>



Command line compilation

- ▶ gcc/g++

- ▶ Compile

- ```
gcc -c <filename.c [...]>
```

- ▶ Link

- ```
gcc -o <filename> <filename.o [...]>
```

- ▶ Visual C++

- ▶ Compile

- ```
cl /c <filename.c [...]>
```

- ▶ Link

- ```
link <filename.obj [...]> /OUT:<filename.exe>
```

Useful GCC flags

- Wall Turn on all warnings
- c <filename> Compile to object file
- o <filename> Link into output executable file
- g Include debug information
- I <folder> Use include folder
- l <filename> Use static library

► Example:

- gcc -Wall hello.c -o runhello
- ./runhello

Revision: C Programming

Display some text (export to screen)

► Syntax:

► `printf("Format string", <values>);`

► Typing symbols:

Symbol	Type	Symbol	Type
%f, %e, %g, %lf	double, float	%x	int (hex)
%d	int	%o	int (oct)
%c	char	%u	unsigned int
%s	string of characters	%p	pointer

► Formatting:

► `%[flags] [width] [.precision]type`

► Ex: `%+15.5f`

Read user input (typed from keyboard)

- ▶ Syntax:

- ▶ `scanf("Format string", <variable addresses>);`

- ▶ Examples:

- ▶ `int age;`
`scanf("%d", &age);`

- ▶ `float weight;`
`scanf("%f", &weight);`

- ▶ `char name[20];`
`scanf("%s", name);`

Variables and types

- ▶ Variables can hold values, which can be changed during runtime
- ▶ Variables need to be declared before using, with a type
- ▶ Global scope or limited within a function
- ▶ In standard C, internal variables need to be declared at the beginning of functions, before any instructions
- ▶ Variable declaration: `<type> <list of variables>;`
 - ▶ `int a, b, c;`
 - ▶ `unsigned char u;`
- ▶ Basic types:
 - ▶ `char, int, short, long`
 - ▶ `float, double`

Assignment

- ▶ To change the value of a variable with a new one
- ▶ Syntax:
 - ▶ `<variable> = <constant, variable> or <expression>;`
- ▶ Ex:
 - ▶ `count = 100;`
 - ▶ `value = cos(x);`
 - ▶ `i = i + 2;`
- ▶ Values of variables can be initialized at declaration (if not, the value is undefined):
 - ▶ `int count = 100;`
 - ▶ `char key = 'K';`

Constants

- ▶ Variables (yes, it's correct) whose values can not be changed at runtime
 - ▶ Declared by adding `const` keyword
 - ▶ Occupy memory (like any other variable)
- ▶ Ex:
 - ▶ `const double PI = 3.14159;`
 - ▶ `const char* name = "Nguyen Viet Tung";`
 - ▶ `PI = 3.14; /* error */`
- ▶ Another way to make a constant: using macro → not occupying memory (but un-typed)
 - ▶ `#define PI 3.14159`

Implementation dependency

- ▶ The size of basic types (`char`, `short`, `int`, `long`) are not exactly defined in the C standard, so they are implementation dependent
 - ▶ But the following points are always guaranteed:
 - ▶ $\text{size of char} \leq \text{size of short} \leq \text{size of int} \leq \text{size of long} \leq \text{size of long long}$
 - ▶ $\text{size of char} = \text{size of unsigned char}$, $\text{size of short} = \text{size of unsigned short}$, $\text{size of int} = \text{size of unsigned int}$,...
- ▶ If not specified, `short`, `int`, `long` and `long long` are signed by default, but `char` may not
 - ▶ `char` may be signed or unsigned depending on the compiler

Basic data (aka primitive) types

Type	Size (bytes)	Values
char	1	Character, integer
int	4	Integer
short	2	Integer
long	4	Integer
long long	8	Integer
float	4	Real (floating point)
double	8	Real (floating point)
void	0	Unspecified type

- ▶ Size of types may vary for machine and compiler, most commonly used is shown here
- ▶ Use sizeof operator to calculate the sizes of variables or data types in bytes:
 - ▶ `sizeof(x)`
 - ▶ `sizeof(int)`
 - ▶ `sizeof(15.6 * sin(y))`

Integer variable size and value range

► Signed and unsigned:

signed char	8 bits	$-128 \div 127$
signed short	16 bits	$-32768 \div 32767$
signed int	32 bits	$-2147483648 \div 2147483648$
signed long	32 bits	$-2147483648 \div 2147483648$
signed long long	64 bits	$-9.2 \times 10^{18} \div 9.2 \times 10^{18}$
unsigned char	8 bits	$0 \div 255$
unsigned short	16 bits	$0 \div 65535$
unsigned int	32 bits	$0 \div 4294967295$
unsigned long	32 bits	$0 \div 4294967295$
signed long long	64 bits	$0 \div 1.8 \times 10^{19}$

Numeric literals

► Integer literals

```
31          /* decimal int */
31U         /* unsigned int, or 31u */
037         /* octal */
0x1F        /* hexadecimal, or 0x1f */
31L         /* long */
31LU        /* unsigned long, or 31LU, 31ul, 31lu */
```

► Floating-point literals

```
123.4       /* double */
123.         /* double */
123.4F      /* float, or 123.4f */
123.F       /* float */
123.4L      /* long double */
1e-2        /* double */
123.4e-3    /* double */
.23e4       /* double */
.23e4F      /* float */
```

Character and string literals

▶ Character literals

```
'K'      /* normal character - 'K' */
'\113'   /* character given its octal code - 'K' */
'\x48'   /* character given its hexadecimal code - 'K' */
'\n'     /* newline character */
'\t'     /* tab character */
'\\'     /* backslash character */
'\\"'    /* double quote character */
'\0'     /* null character (marks end of string)*/
```

▶ Attn:

- ▶ In C, character literals always have type of int, so `sizeof('c')` is same as `sizeof(int)`
- ▶ In C++, character literals have type of char, so `sizeof('c')` is same as `sizeof(char)`

▶ String literals

```
"You have fifteen thousand new messages."
```

```
"I said, \"Crack, \" \"we're under attack!\"."
```

Type casting

- ▶ Conversion of value of an expression from one type to another
- ▶ Implicit casting:
 - ▶ `float a = 30;`
 - ▶ `int b = 'a';`
- ▶ Explicit casting:
 - ▶ `int a = (int)5.6; /* take integer part */`
 - ▶ `float f = (float)1/3;`
- ▶ Not any type can be casted into any other:
 - ▶ `char* s = 2.3; /* not compiled */`
 - ▶ `int x = "7"; /* compiled in C but incorrect */`
 - ▶ `int y = 12 + "34";`

Integer promotions

- ▶ Integer types smaller than `int` are promoted in binary and tertiary operations (except shift operations) to avoid data lost
 - ▶ If all values of the original type can be represented as an `int`, the value of the smaller type is converted to an `int`; otherwise, it is converted to an unsigned `int`
 - ▶ `signed char c1 = 100, c2 = 3, c3 = 4, r;`
`r = c1 * c2 / c3; /* 75 */`
- ▶ Integer conversion rank:
 - ▶ Ranks in increasing order: `signed char`, `short`, `int`, `long`, `long long`
 - ▶ Rank of unsigned type equals rank of corresponding signed type
- ▶ Usual arithmetic conversions
 - ▶ Integer promotions are performed if necessary
 - ▶ Operand with the type of lesser rank is converted to the type of the operand with greater rank

Enumeration (enum)

- ▶ Used to define the possible values of a data type
- ▶ Syntax:
 - ▶ `enum <type name> { <values> };`
- ▶ Ex:
 - ▶ `enum WildAnimal {Tiger, Leopard, Bear, Deer};`
 - ▶ `enum WeekDay { Mon = 2, Tue, Wed, Thu, Fri, Sat, Sun = 1 };`
- ▶ Usage:
 - ▶ `enum WildAnimal wa = Tiger;`
 - ▶ `wa = Leopard;`
 - ▶ `enum WeekDay n = Thu;`

Defining new types from old ones (typedef)

- ▶ Used define new names for old types that are shorter or with different significations
- ▶ Syntax:
 - ▶ `typedef <original type> <new name>;`
- ▶ Ex:
 - ▶ `typedef double Height;`
 - ▶ `typedef unsigned char byte;`
 - ▶ `typedef enum WildAnimal WA;`
- ▶ Usage
 - ▶ `Height d = 165.5;`
 - ▶ `byte b = 30;`
 - ▶ `WA wa = Tiger;`

Revision questions

1. Is a constant a variable?

2. What are the results?

a) `int a = 10/3;`

b) `double a = 10/3;`

c) `short a = -10;`

`unsigned char b = 10;`

`unsigned int c = 10;`

`printf("%d %d", a < b, a < c);`

d) `int a = -20;`

`unsigned int b = 10;`

`long long c = a + b;`

`printf("%d, %lld", a + b, c);`

e) `signed char a = -1;`

`unsigned char b = a;`

`printf("%d", b);`

f) `double x = 1.3, y = 1.6, z = 2.9;`

`printf("%d", z == (x + y));`

Google C++ Style Guide(*): Avoid using unsigned types unless you have a special good reason to do so.

* Full version at: <https://google.github.io/styleguide/cppguide.html>

Exercises

- ▶ Write programs to:
 1. Calculate the sum of the digits of a given integer.
 2. Determine whether a number is prime.
 3. Print all the first n prime numbers.
 4. Calculate π using Monte Carlo method.

Structures, Arrays, Pointers & Strings in C

Structures (struct)

- ▶ Used to define data types composed of sub-variables (members, fields)
- ▶ Syntax:
 - ▶ `struct <data type> { <member variables> };`
- ▶ Ex:
 - ▶

```
struct Student {  
    char name[20];  
    int birth_year;  
    int school_year;  
};
```
- ▶ Usage:
 - ▶ `struct Student st = {"Le Duc Tho", 1984, 56};`
 - ▶ `st.birth_year = 1985;`
 - ▶ `st.school_year = 54;`

Structure initialization

- ▶ `struct Student s1 = {"Le Duc Tho", 1984, 2002};`
- ▶ `struct Student s2 = {"Le Duc Tho", 1984};`
- ▶ Designated initializers:
`struct Student s3 = {
 .birth_year = 1984,
 .school_year = 2002,
 .name = "Le Duc Tho"
};`

Unnamed structures

- ▶ Sometimes, struct name may be omitted:

- ▶

```
struct {  
    float x, y;  
} p1, p2, pa[3];
```

- ▶

```
typedef struct {  
    // ...  
} Student;
```

```
Student s1, s2;
```

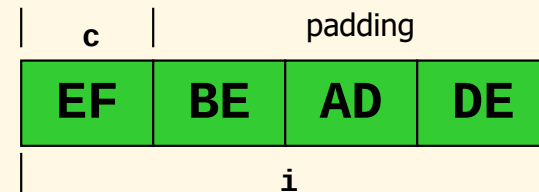
Unions

▶ Example:

```
▶ union AnElt {  
    int    i;  
    char   c;  
} elt1, elt2;
```

```
elt1.i = 4;  
elt2.c = 'a';  
elt2.i = 0xDEADBEEF;
```

- ▶ An element is an `int i` or a `char c`
- ▶ Size of union = size of the largest field



▶ Example:

```
▶ union color {  
    struct {unsigned char R,G,B,A;} s_color;  
    unsigned int i_color;  
};
```

Unions

- ▶ A union value itself doesn't "know" which case it contains

```
union AnElt {  
    int    i;  
    char   c;  
} elt1, elt2;  
  
elt1.i = 4;  
elt2.c = 'a';  
elt2.i = 0xDEADBEEF;  
  
if (elt1 currently has a char) ...
```



How should your program keep track whether elt1, elt2 hold an int or a char?



Basic answer: Another variable holds that info

Tagged unions

- ▶ Tag every value with its case, i.e., pair the type info together with the union

Enum must be external to struct,
so constants are globally visible

Struct field must be named

```
enum Union_Tag { IS_INT, IS_CHAR };

struct TaggedUnion {
    enum Union_Tag tag;
    union {
        int    i;
        char   c;
    } data;
};
```

Arrays

- ▶ Used to store multiple elements of a same type in memory at consecutive locations
- ▶ Are static pointers by nature
- ▶ Syntax:
 - ▶ `<type name> <variable name> [ <nº of elem.> ];`
- ▶ Ex:
 - ▶ `int age[6] = { 23, 50, 18, 40, 25, 33 };`

	23	50	18	40	25	33	
	0	1	2	3	4	5	

- ▶ Access elements: index starting from 0
 - ▶ `age[3] = 20;`
- ▶ Two-dimensional arrays (and multi-dimensional):
 - ▶ `float matrix[10][20];`
`matrix[5][15] = 1.23;`

Array initialization

- ▶ `int a1[10] = {1, 2, 3, 4, 5};`
- ▶ `int a2[] = {1, 2, 3, 4, 5};`
- ▶ Designated initializers:
`int a3[] = {1, 2, 3, [10] = 100, [14] = 200};`

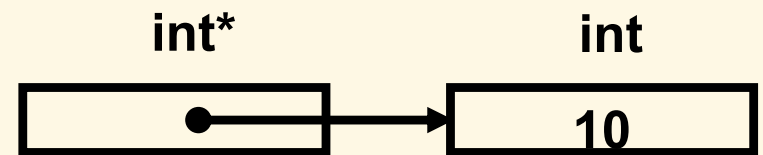
Compound types

- ▶ Used to combine multiple sub-variables of same or different types
 - ▶

```
typedef struct {  
    char name[20];  
    unsigned int age;  
    enum {Male, Female} sex;  
    struct {  
        char city[20];  
        char street[20];  
        int number;  
    } address;  
} Student;
```

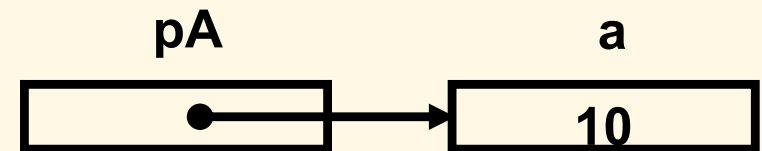
Pointers

- ▶ Pointer is variable whose value is address of some memory area of certain type
- ▶ Size of a pointer is same as that of `int`, but the size of memory area pointed by pointer is unknown (pointers do not have information about size)
- ▶ Declare pointers by `*` symbol:
 - ▶ `int *pInt;`
 - ▶ `char *pChar;`
 - ▶ `struct Student *pSt;`
- ▶ Access to value (indirection) also by `*`:
 - ▶ `int aInt = *pInt;` (`*pInt` is an `int` that `pInt` points to)
 - ▶ `*pChar = 'A';`
 - ▶ `printf("Value: %d", *pInt);`



Changing address of pointers

- ▶ As value of a pointer is address, when changing its value, the pointer will point to another memory area
- ▶ Assign new address to a pointer by “=” operator as normal
 - ▶ `int *pInt2;`
`pInt2 = pInt;`
- ▶ Address-of operator &: produces a pointer by taking address of a variable
 - ▶ `int a;`
`int* pA = &a; /* pA points to a */`
 - ▶ & is reversion of *: for any variable a, `&a` is equivalent to a; and for any pointer p, `&*p` is equivalent to p



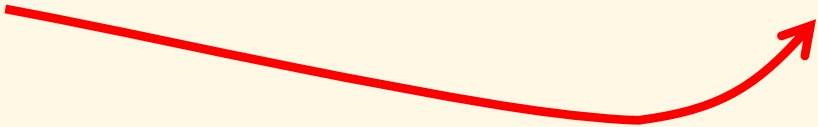
Illustration

```
▶ char c = 'A';  
  int *pInt;  
  short s = 50;  
  int a = 10;  
  
  pInt = &a;  
  *pInt = 100;
```

Addresses of variables in memory here is increasing only for illustration. In reality, variables are allocated in stack from higher memory to lower → first variable has highest address

Address	1500	1501	1502	1503	1504	1505	1506	1507	1508	1509	1510	1511
Variable	char c	int* pInt				short s		int a				...
Value	'A'	1507				50		100				...

```
pInt: 1507  
*pInt: 100  
&a: 1507  
a: 100
```



Void pointers (void*)

- ▶ Are pointers without type information
- ▶ Can be implicitly casted to any other pointer type, and vice versa (but not in C++)

```
▶ void* pVoid;    int *pInt;    char *pChar;
```

```
pInt = pVoid;    /* OK */
```

```
pChar = pVoid;   /* OK */
```

```
pVoid = pInt;    /* OK */
```

```
pVoid = pChar;   /* OK */
```

```
pChar = pInt;    /* error */
```

```
pChar = (char*)pInt; /* OK */
```

- ▶ Indirection cannot be applied to void* pointers
 - ▶ *pVoid /* error */
- ▶ void* pointers are used to access pure memory, or to manipulate variables of unknown type
 - ▶ memcpy(void* dest, const void* src, int size);

Null pointer (NULL)

- ▶ Is a pointer constant with address of 0, type of void*, which signifies a pointer that does not point to any memory
- ▶ Is macro declared as follows technically:
 - ▶ `#define NULL ((void*)0)`
- ▶ Impossible to assign values to null pointers
 - ▶ `int *pInt = NULL`
`*pInt = 100; /* error */`
- ▶ Distinguish null pointers with uninitialized ones (which point to random memory)
- ▶ Usually used to determine the validity of a pointer variable → to avoid errors, always assign NULL to pointer variables when unused
- ▶ As NULL is evaluated to 0, comparing a pointer to NULL can be ignored in logical expressions:
 - ▶ `if (p != NULL) ... → if (p) ...`

Pointer operations

- ▶ Increment/decrement: pointer points to the next/previous element (address increased/decreased by the size of the pointer type)

Address	1500	1501	1502	1503	1504	1505	1506	1507	1508	1509	1510	1511
			p - - (1502)		short *p (1504)		p++ (1506)					

- ▶ Addition/subtraction: similarly

Address	1500	1501	1502	1503	1504	1505	1506	1507	1508	1509	1510	1511
	p - 2 (1500)				short *p (1504)						p+3 (1510)	

- ▶ Comparison: 2 pointers of same type can be compared using address like comparing two integers (greater, smaller, equal,...)
- ▶ A pointer can be subtracted to another of the same type

Pointers and arrays

- ▶ Array is static pointer (whose address is fixed), which points to (stores the address of) its first element
 - ▶ It is possible to manipulate arrays like pointers, except changing their address
 - ▶

```
int arr[] = {1, 2, 3, 4, 5};  
int x;  
*arr = 10;           /* equivalent to: arr[0] = 10; */  
printf("%d", *(arr+2)); /* equivalent to: arr[2] */  
arr = &x;            /* error */
```
- ▶ A pointer can be used as array and be manipulated as array
 - ▶

```
int *p = arr;  
p[2] = 20;           /* same as: arr[2] = 20; */  
p = arr+2;           /* same as: p = &arr[2]; */  
p[0] = 30;           /* same as: arr[2] = 30; or: *p = 30; */
```
- ▶ Conclusion: pointers and arrays can be used interchangeably, depending on programmer's convenience

Pointers and arrays (cont.)

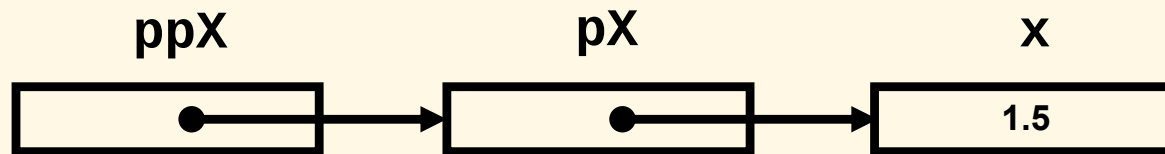
► Differences:

- Impossible to assign new address to an array variable
- Memory is allocated for array elements immediately at declaration (in stack)
- `sizeof( )` operator returns real size for arrays (total size of elements), while returns size of address for pointers
 - `float arr[5];` → `sizeof(arr)` returns 20 (5×4)
 - `float* p = arr;` → `sizeof(p)` returns 4 for 32-bit platforms
 - `sizeof(arr)/sizeof(arr[0])` → number of elements
- With pointer nature, it is possible to use negative index for arrays:
 - `int arr[] = {1, 2, 3, 4, 5};`
`int *p = arr + 2;`
`p[-1] = 10;` /* same as: `arr[1] = 10;` */

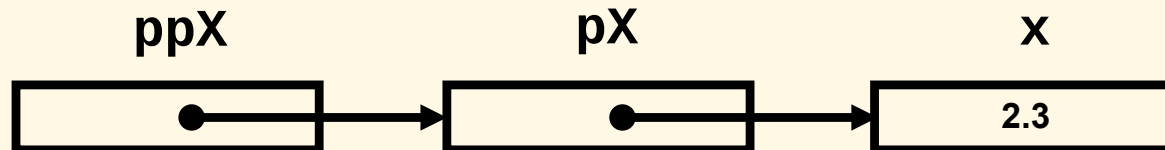
Pointers to pointers

- ▶ It is possible to declare pointers to pointers in C

```
float x = 1.5;  
float *pX = &x;  
float **ppX = &pX;  
printf("%f", **ppX);    /* output: 1.5 */
```



```
**ppX = 2.3;
```



- ▶ Can be considered as 2-dimensional arrays, arrays of arrays, pointers to arrays, or arrays of pointers

Pointers to struct, union

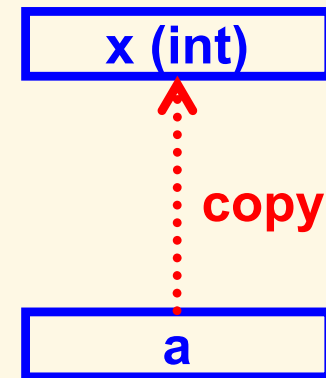
- ▶ For a pointer to struct or union, possible to use “->” operator to access member variables instead of “*” and “.”
 - ▶ `p->member` is equivalent to `(*p).member`
- ▶ Ex:
 - ▶

```
typedef struct {  
    int x, y;  
} Point;  
Point *pP = //...  
pP->x = 5;    /* same as: (*pP).x = 5; */  
(*pP).y = 7; /* same as: pP->y = 7; */
```

Passing parameters by value and by reference

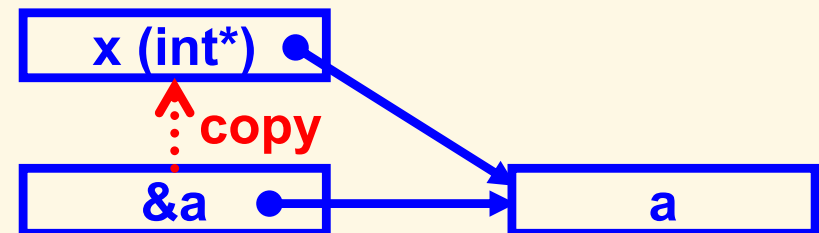
- ▶ Function parameters are temporary variables created on function calls and destroyed when functions end → assigning values to parameters does not have effect on original variables

```
▶ void assign10(int x) { x = 10; }  
  int main() {  
    int a = 20;  
    assign10(a);  
    printf("a = %d", a);  
    return 0;  
  }
```



- ▶ User pointers if desire to modify value of original variables

```
▶ void assign10(int *x)  
    { *x = 10; }  
  int a = 20;  
  assign10(&a);
```



- ▶ Pointer parameters are often used as an alternative way to return values to callers, as each function has only one real return value

Pointers returned from functions

- Issue of returning address of local variables:

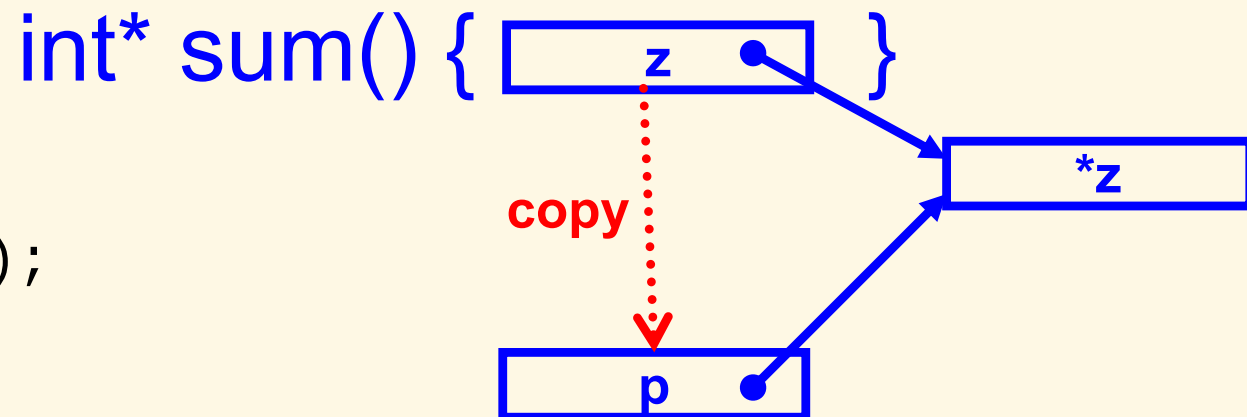
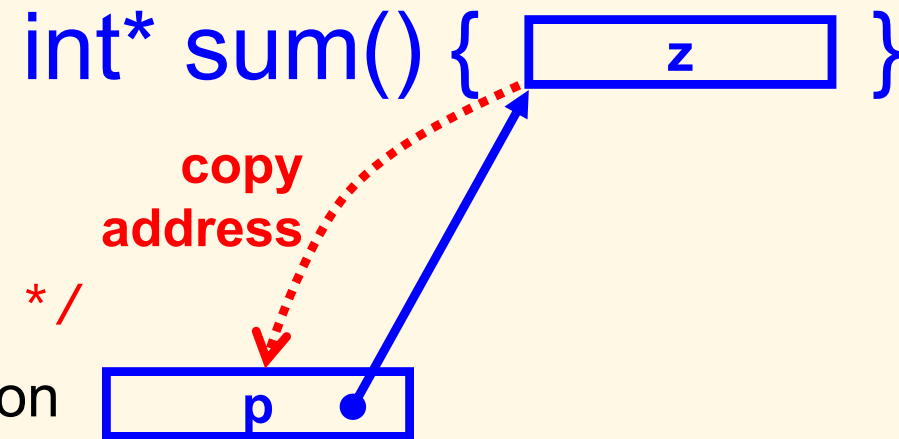
```
int* sum(int x, int y) {  
    int z = x+y;  
    return &z;  
}
```

```
int* p = sum(2, 3); /* error */
```

- Solution: allocate memory inside function

```
int* sum(int x, int y) {  
    int* z = (int*)malloc(sizeof(int));  
    *z = x+y;  
    return z;  
}
```

```
int* p = sum(2, 3);  
/* ... */  
free(p);
```



Function pointers

- ▶ Are pointers to functions → a data type in C, usually used to call functions that are unspecified at writing time
 - ▶ `double (*SomeOpt)(double, double);`
 - ▶ `typedef double (*OptFunc)(double, double);`
`OptFunc SomeOpt;`
- ▶ Ex:
 - ▶

```
double sum(double x, double y) { return x+y; }
double prod(double x, double y) { return x*y; }
int main() {
    double (*SomeOpt)(double, double) = &sum;
    SomeOpt(2., 5.);    /* same as: sum(2., 5.); */
    SomeOpt = prod;
    (*SomeOpt)(2., 5.); /* same as: prod(2., 5.); */
    return 0;
}
```
- ▶ Attn: using & in assignments and * in calls is optional

Dynamic memory allocation

- ▶ Memory is allocated for variables at declaration (in stack)
- ▶ When memory need to be allocated on demand (size unknown at writing time) → dynamic allocation (in heap)
- ▶ Allocate memory:
 - ▶ `#include <stdlib.h>`
 - ▶ `void* malloc(int size) /* size: number of bytes */`
 - ▶ `int *p = (int*)malloc(10*sizeof(int)); /*alloc 10 int*/`
 - ▶ `void* calloc(int num_elem, int elem_size)`
 - ▶ `void* realloc(void* ptr, int size)`
 - ▶ If allocation fails, returns NULL → need to check
- ▶ Release (deallocate) an allocated memory:
 - ▶ `void free(void* p);`
 - ▶ `free(p);`

Conclusion on pointers

- ▶ Pointer is a powerful notion of C compared to other languages, but is a double blade knife, as it is hard to debug pointer related errors → programmers must master and use pointers flexibly
- ▶ Pointers are usually used in following contexts:
 - ▶ Passing values from function to outside via parameters
 - ▶ Function pointers
 - ▶ Data structures: linked list, queue, dynamic array,...
- Will come back on related lessons

Strings of characters

- ▶ Are arrays of characters, terminated by '\0'
 - ▶ `char name[10] = "Tung";` (initialized by pointer)
 - ▶ `char name[10] = {'T', 'u', 'n', 'g', '\0'};` (by array)
 - ▶ `char *name = "Tung";` (pointer initialized by pointer)
 - ▶ `char *name = {'T', 'u', 'n', 'g', '\0'};` /* error */
- ▶ Ex: calculate length of string:
 - ▶ `for (n = 0; s[n]; n++) ;` /* more preferable */
 - ▶ `for (n = 0; *s; n++, s++) ;`
- ▶ Some popular string functions:
 - ▶ `#include <string.h>`
 - ▶ `int strlen(s)` → length of s
 - ▶ `char *strcpy(dst, src)` → copy string src to dst
 - ▶ `char *strcat(dst, src)` → concatenate src to the end of dst
 - ▶ `int strcmp(str1, str2)` → compare 2 strings, result: 1, 0, -1
 - ▶ `char *strstr(s1, s2)` → find position of s2 in s1

Exercise: String concatenation

- ▶ Implement a function `my_strcat(...)` to concatenate two strings given in parameter:
 - ▶ Input: two strings `s1` and `s2`
 - ▶ Output: a dynamically allocated string
- ▶ Example:
 - ▶ `my_strcat("hello_", "world!")`
 - ▶ Returns: `"hello_world!"`

Solution

```
char* my_strcat(const char* str1, const char* str2)
{
    int len1 = strlen(str1),
        len2 = strlen(str2);

    char* result = (char*)malloc((len1+len2+1) * sizeof(char));
    if (result == NULL) {
        printf("Memory allocation failure\n");
        return NULL;
    }

    strcpy(result, str1);
    strcpy(result + len1, str2);

    return result;
}
```

Question: Where comes free(...)?

Buffer overflow error

- ▶ Happens when program write more data than the size of buffer
 - ▶ Ex: copy a string of 10 characters into a buffer of 5 bytes

```
char s[5];  
strcpy(s, "0123456789"); /* error */
```
- ▶ This error is dangerous as it will cause unpredictable effects, especially may help user to take ownership of computer then do anything → need to control the length of input data to fit into memory allocated to variables
- ▶ Standard C functions do not check for buffer overflow error
→ use extended functions available since **C11**

String and memory processing functions

- ▶ `memcpy_s(void* dest, int size, const void* src, int count);`
- ▶ `memmove_s(void* dest, int size, const void* src, int count);`
- ▶ `strcpy_s(char* dest, int size, const char* src);`
- ▶ `strcat_s(char* dest, int size, const char* src);`
- ▶ `_strlwr_s(char* str, int size);`
- ▶ `_strupr_s(char* str, int size);`

Data reading functions

- ▶ `gets_s(char* str, int size);`
- ▶ `scanf_s(const char* format, ...);`
 - ▶ Added buffer-size checking parameters
 - ▶ `int i;`
`float f;`
`char c;`
`char s[10];`
`scanf_s("%d %f %c %s", &i, &f, &c, 1, s, 10);`
 - ▶ Similar to `fscanf_s()`, `sscanf_s()` functions

Revision questions

1. Are the following fragments working?

- a) `char name[100];
scanf("%s", &name);`
- b) `char name[100];
char* name2 = name;
scanf("%s", &name2);`
- c) `char* name;
scanf("%s", &name);`
- d) `int* sum(int a, int b) {
 int c = a + b;
 return &c;
}`

2. What are the results?

- a) `int a[] = {0, 1, 2, 3, 4, 5, 6, 7, 8};
int* b = &a[1];
printf("%d, %d", b - a, (int)b - (int)a);`

Exercises

- ▶ Write programs to:
 1. Convert a string into an integer number. Ex: "1234" → 1234.
 2. Convert an integer number into a string.
 3. Print a menu and ask the user to choose an option, then do a corresponding task. Use function pointers.

- ▶ What are the manners to return more than one values from a function?

Revision: C++ Programming

Overview

- ▶ Additional features compared to C:
 - ▶ Object-oriented programming (OOP)
 - ▶ Generic programming (template)
 - ▶ Many other small changes
- ▶ Small changes:
 - ▶ Conventional source files with .cpp extension
 - ▶ `main()` function can be declared with `void` return type:
 - ▶ `void main() { ... }`
 - ▶ Use `//` to comment until end of line
 - ▶ `area = PI * r * r;      // PI = 3.14`
 - ▶ Built-in `bool` type and `false`, `true` boolean literals:
 - ▶ `bool b1 = true, b2 = false;`
 - ▶ Variables and constants in C++ can be declared anywhere in functions (not limited to function beginning only), even in for loops
 - ▶ New function-like syntax for type casting: `int(5.32)`
 - ▶ `enum`, `struct`, `union` keywords become optional in variable declaration

Some more significant features...

- ▶ Reference types: are pointers by nature

- ▶ `int a = 5;`
`int& b = a;`
`b = 10;      // → a = 10`
 - ▶ `int& foo(int& x)`
`{ x = 2; return x; }`
`int y = 1;`
`foo(y);                      // → y = 2`
`foo(y) = 3;                  // → y = 3`

- ▶ Namespace

- ▶ `namespace ABC {`
`int x;`
`int setX(int y) { x = y; }`
`}`

```
ABC::setX(20);  
int z = ABC::x;
```

```
using namespace ABC;  
setX(40);
```

Some more significant features... (cont.)

- ▶ Dynamic memory allocation
 - ▶ Allocation: `new` and `new[]` operators
 - ▶ `int* a = new int;`
`float* b = new float(5.23f);`
`long* c = new long[5];`
 - ▶ Deallocation: `delete` and `delete[]` operators
 - ▶ `delete a;`
`delete[] c;`
 - ▶ Attn: do not mix `malloc()`/`free()` and `new/delete`:
 - ▶ Memory allocated by `malloc()` must be deallocated by `free()`
 - ▶ Memory allocated by `new` must be deallocated by `delete`
- ▶ Function overload (functions with same name but different parameters):
 - ▶ `int sum(int a, int b)` {...}
 - ▶ `int sum(int a, int b, int c)` {...}
 - ▶ `double sum(double a, double b)` {...}
 - ▶ `double sum(double a, double b, double c)` {...}
- ▶ Exception handling `try ... catch`: self-study

Input and output

- ▶ Example:

- ▶

```
#include <iostream>
using namespace std;
void main() {
    int n;
    cout << "Enter n: ";
    cin >> n;
    cout << "n = " << n << endl;
}
```

- ▶ Input/output with C++:

- ▶ Using iostream library
 - ▶ “cout <<” used for output to stdout (console by default)
 - ▶ “cin >>” used for input from stdin (keyboard by default)
 - ▶ Objects of C++ standard library defined in a namespace named std. If “using ...” is not used, one must use std::cout, std::cin, std::endl instead

Smart pointers (since C++11)

- ▶ Abstract type to simulate pointers, while providing additional features:
 - ▶ Memory management
 - ▶ Bound checking
- ▶ Example:

```
shared_ptr<int> p1(new int(10));  
shared_ptr<int> p2 = p1;  
*p2 = 20;
```

```
// memory is reclaimed when  
// the last reference is destroyed
```

More on C++ to revise

- ▶ Object and classes
 - ▶ Inheritance
 - ▶ Function & operator overload
- ▶ Generic programming with templates
- ▶ Modern C++
 - ▶ Move semantics
 - ▶ Lambda expressions
- ▶ Standard Template Library (STL)
- ▶ C++ Core Guidelines:
 - ▶ <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines>

Revision Questions

1. What is the equivalent mechanism for virtual methods in C?
2. Do the following fragments use a constructor or an assignment copy/casting operator?
 - a) `Cls o1;`
`Cls o2 = o1;`
 - b) `Cls o1, o2;`
`o2 = o1;`
3. Are the following fragments working correctly?
 - a) `Cls* p = (Cls*)malloc(3 * sizeof(Cls));`
 - b) `Cls* p = new Cls;`
`free(p);`

Exercises

1. May the following code work? If yes, what is the condition?

```
cls* x = nullptr;  
x->f();
```
2. Write a program to: print a menu and ask the user to choose an option, then do a corresponding task.
3. Implement the `shared_ptr<T>` class by yourself.

Homework

Write programs to:

1. Read an integer N , allocate memory for N integers and input data, then print the array in reverse order
2. Read an array of floating-point numbers without knowing nor asking for the number of elements a priori (extend the array on demand)
3. Read a string $s1$, then copy it to another string $s2$ (without using `strcpy` nor `memcpy`)
4. Read two strings $s1$ and $s2$, then compare to see if they are equal (without using `strcmp`)
5. Read a string, then split it into an array of words
6. Declare two arrays of increasing float values, then merge them into another array also in increasing order

Algorithm Flowchart & Pseudo-code

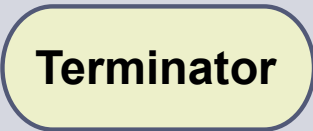
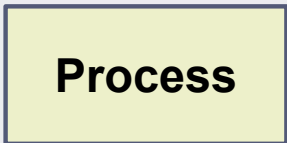
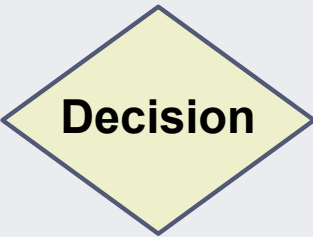
- ❑ Algorithm Flowchart
- ❑ Pseudo-code

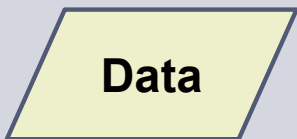
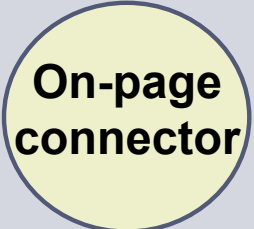
Algorithm Flowchart

Flowchart

- ▶ A graphical representation of the sequence of operations in an algorithm
- ▶ Shows the sequence of instructions in a program
- ▶ Helps visualize and understand how a program works

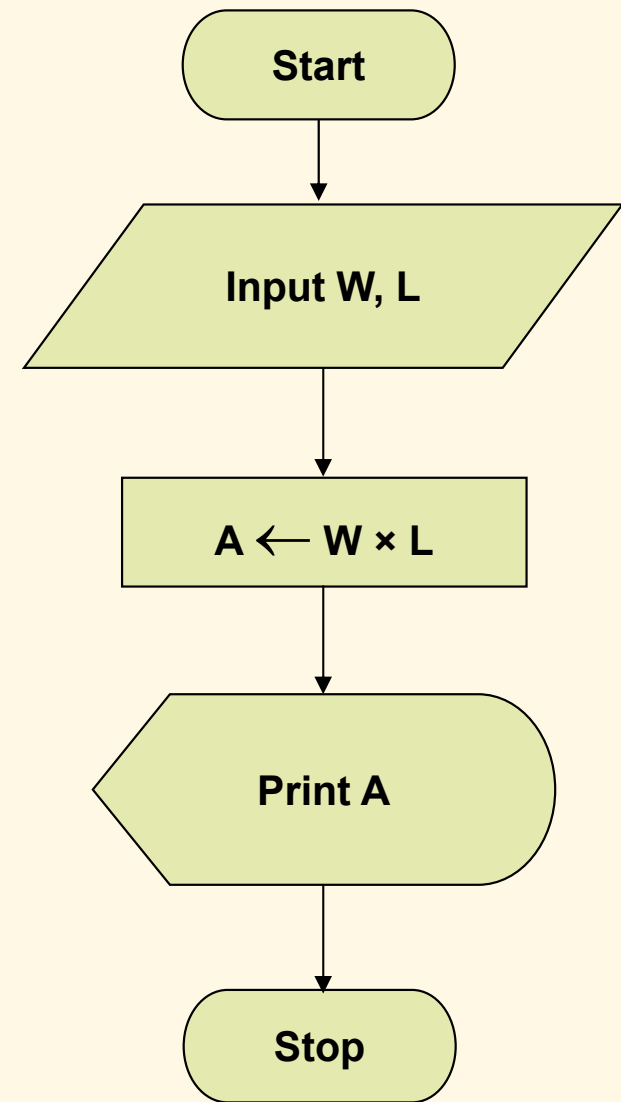
Basic Elements (*)

Element	Description
 Terminator	Beginning or end of a process
 Process	A step (task, action) in a process
 Arrow	Directional execution flow
 Decision	Conditional decision, this-or-that choice branch

Element	Description
 Data	Process input/output
 Display	Displayed output
 On-page connector	Flow continues at a target on the same chart (to avoid long arrows)
 Off-page connector	Flow continues at a target on another page

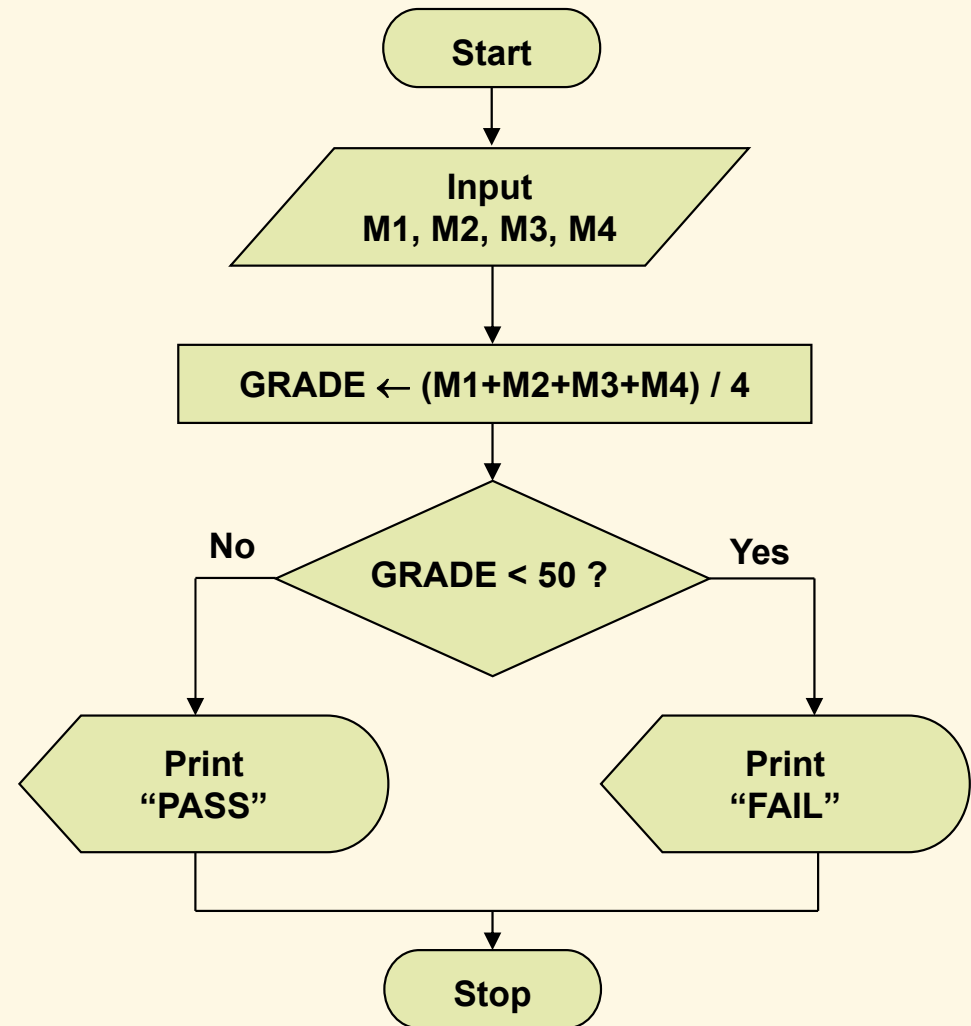
Example 1

- Write an algorithm and draw a flowchart that will read the two sides of a rectangle and calculate its area.



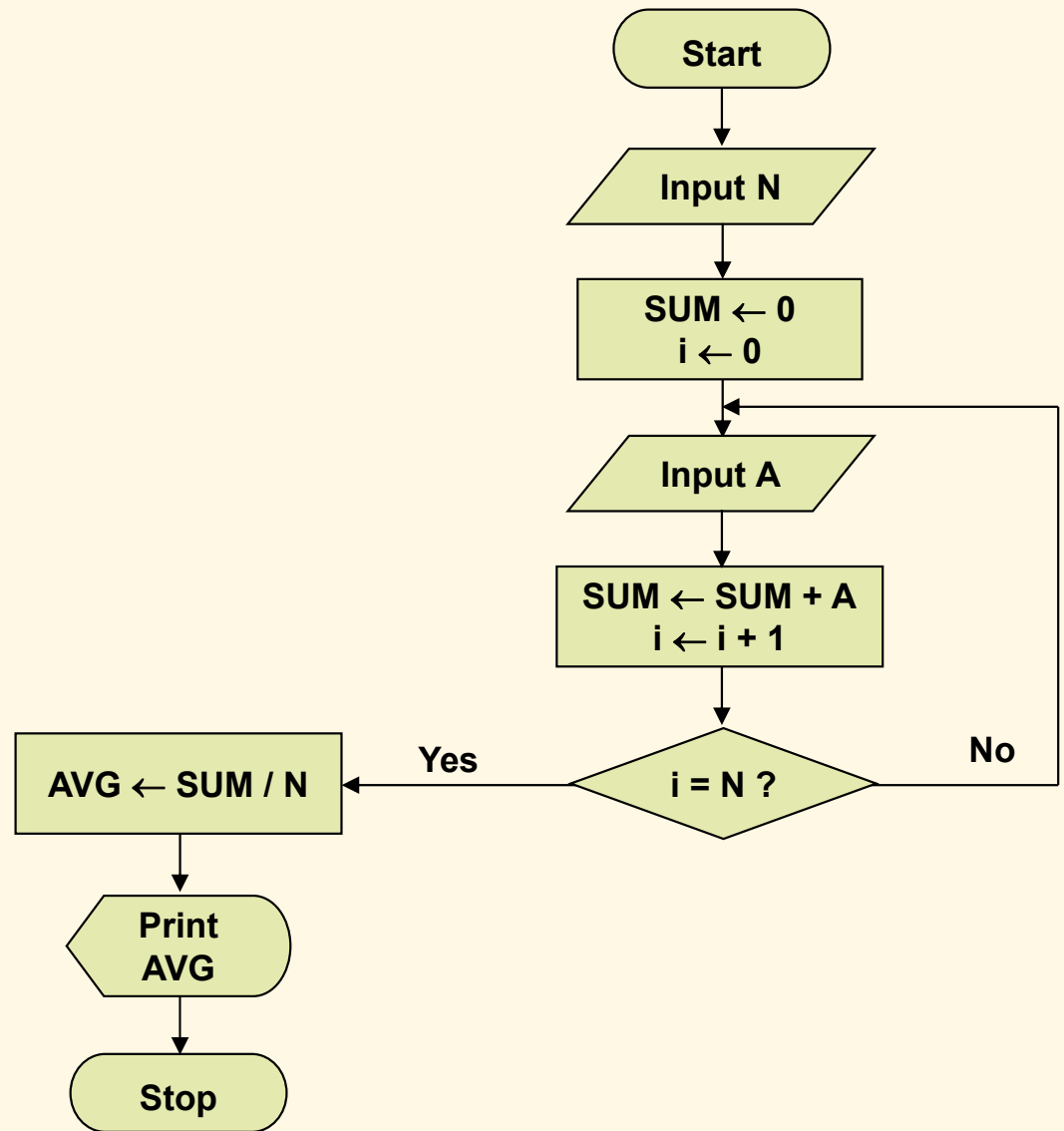
Example 2: Conditional Branch

- Write an algorithm to determine a student's final grade and indicate whether it is passing or failing. The final grade is calculated as the average of four marks.

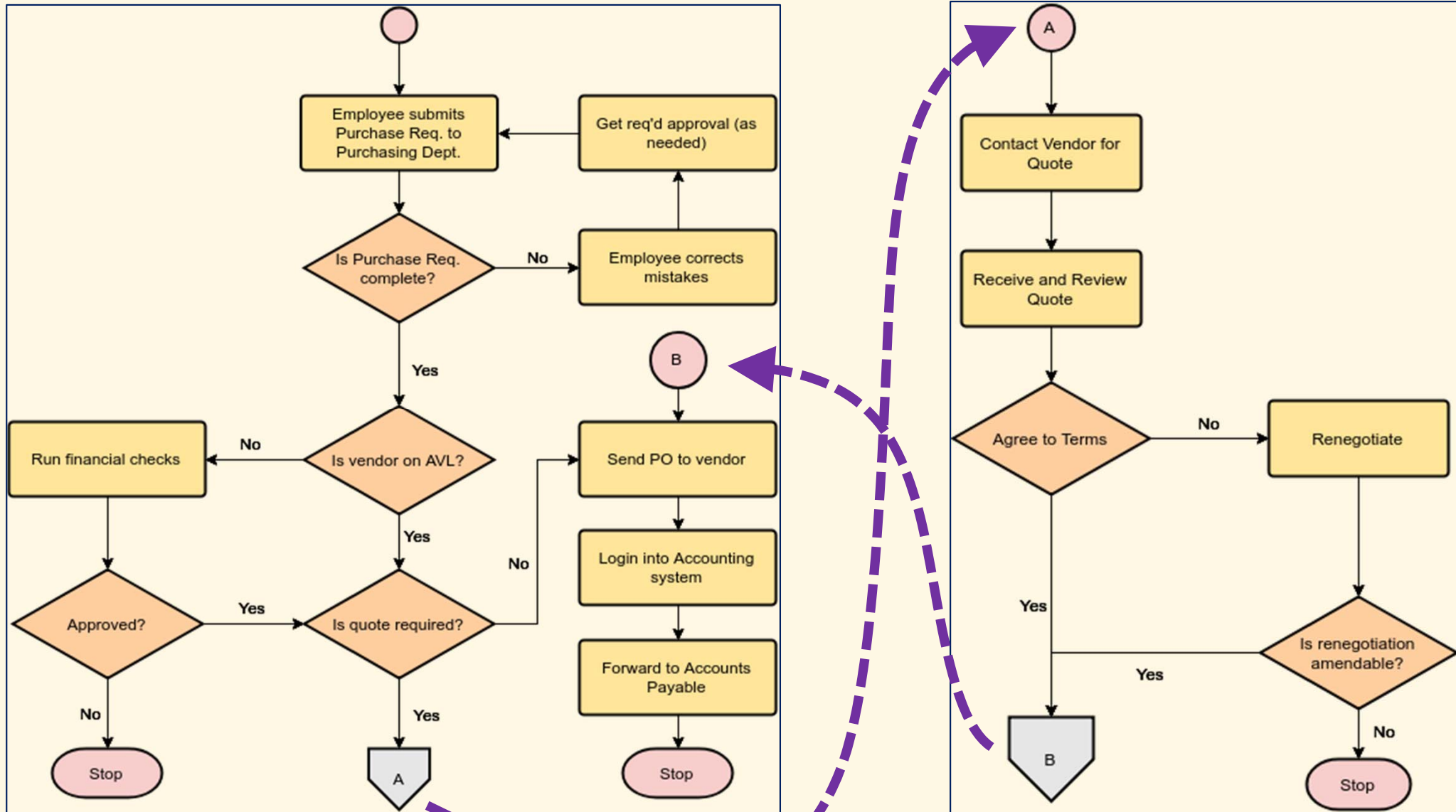


Example 3: Loop

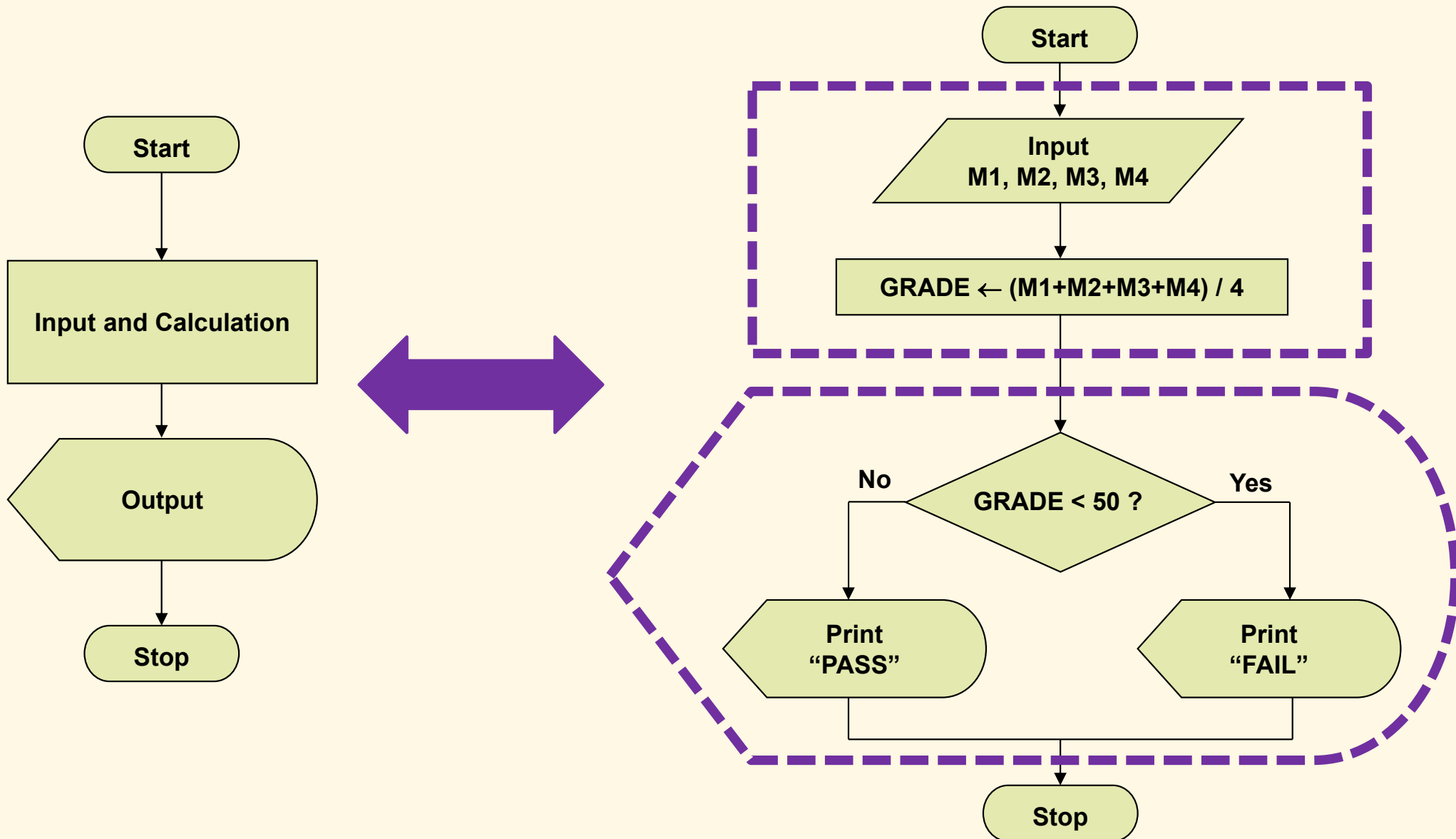
- Write an algorithm to determine the mean value of N numbers entered by user.



Linking Flowcharts



General and Detailed Flowcharts



Exercises

1. Make a flowchart to show how to solve quadratic equations: $ax^2 + bx + c = 0$.
2. Given an array of N integers, make a flowchart to print all the even numbers.
3. Make a flowchart of a ticket vending machine that accepts payment by cash and card.
4. Make a flowchart to calculate the taxi fare given the distance driven and time waited, with following rules:
 - ▶ Fare = Fare-by-distance + Fare-by-waiting-time
 - ▶ First 2km driven: \$5
 - ▶ Next every 0.1km driven: \$0.15
 - ▶ First 30min waited: \$3
 - ▶ Next every 10min waited: \$0.5

Pseudo-code



Pseudo-code

- ▶ Artificial and informal language that helps programmers develop algorithms
- ▶ May be combinations of computer language and spoken language

Common Statements

- ▶ Assignment
 - ▶ set *variable* to *value*
 - ▶ *variable* $\leftarrow$ *arithmetic-expression*
- ▶ If-then
 - ▶ if *condition* then
 statement
 [end/endif]
- ▶ If-then-else
 - ▶ if *condition* then
 statement-#1
else
 statement-#2
 [end/endif]

Common Statements (*cont'd*)

- ▶ Case-of

- ▶ case *expression* of

- condition-#1: statement-#1*

- condition-#2: statement-#2*

- ...

- condition-#n: statement-#n*

- others: default-statement*

- [end]

- ▶ Invoking procedures or functions

- ▶ call *proc*

- ▶ call *proc* with *parameters*

- ▶ call *func* with *parameters* returning *value*

Common Statements (*cont'd*)

▶ Loops

▶ repeat

statement

until *condition*

▶ do

statement

while *condition*

▶ while *condition* do

statement

▶ for each *element* in *list* do

statement

▶ for *variable* $\leftarrow$ *v1* [to|down to] *v2* do

statement

Procedures & Functions

- ▶ Example: Find the maximum element of an array.
- ▶ Pseudo-code

- ▶ Algorithm `ArrayMax(A, n)`:

Input: Array A storing n integers.

Output: The maximum element in A .

```
currentMax  $\leftarrow A[0]$   
for  $i \leftarrow 1$  to  $n-1$  do  
    if currentMax  $< A[i]$  then  
        currentMax  $\leftarrow A[i]$ 
```

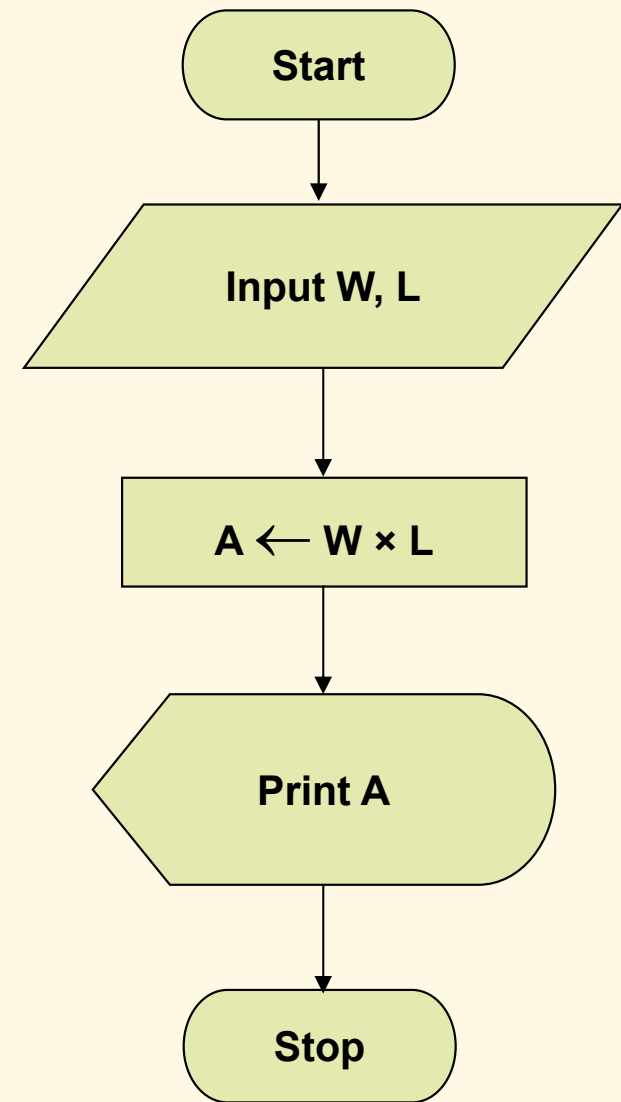
```
return currentMax
```


Example 1

Input W, L

$A \leftarrow W \times L$

Print A



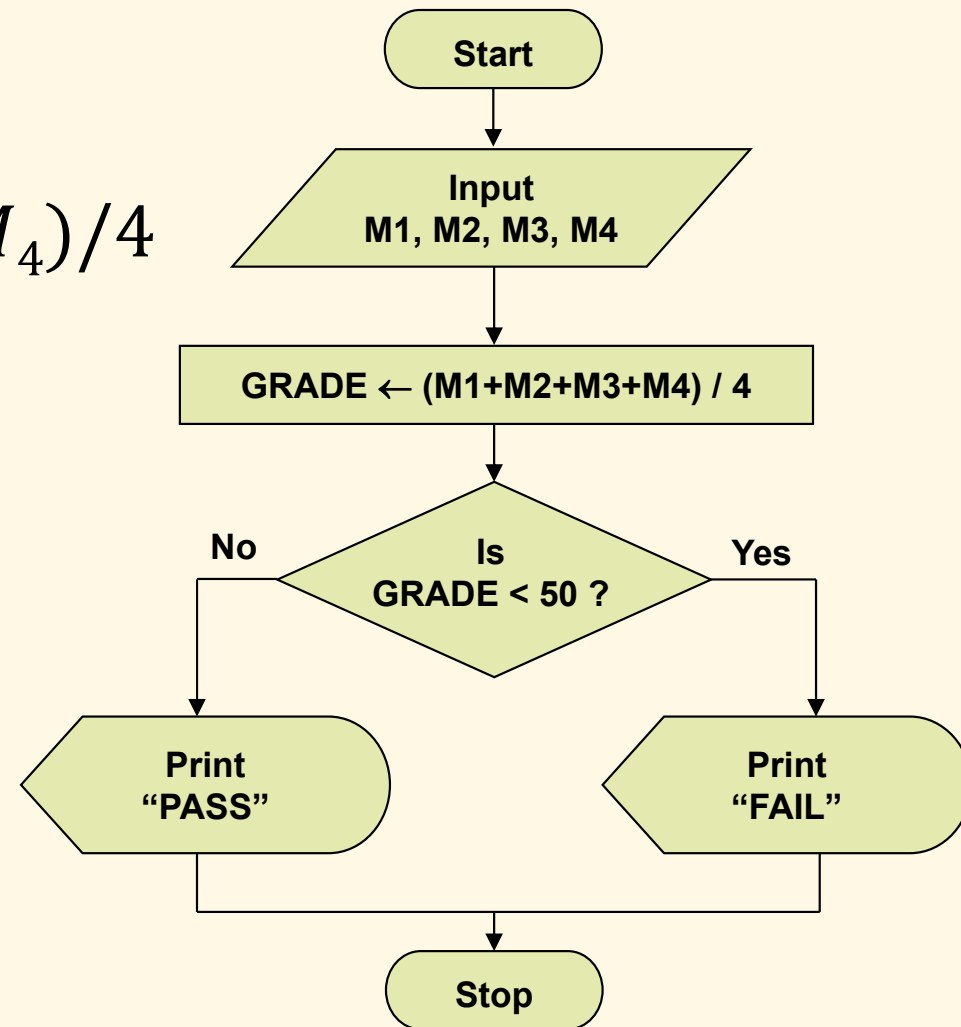
Example 2: Conditional Branch

Input M_1, M_2, M_3, M_4

$$GRADE \leftarrow (M_1 + M_2 + M_3 + M_4) / 4$$

if $GRADE < 50$ then
 Print “FAIL”

else
 Print “PASS”



Example 3: Loop

Input N

$SUM \leftarrow 0$

$i \leftarrow 0$

repeat

Input A

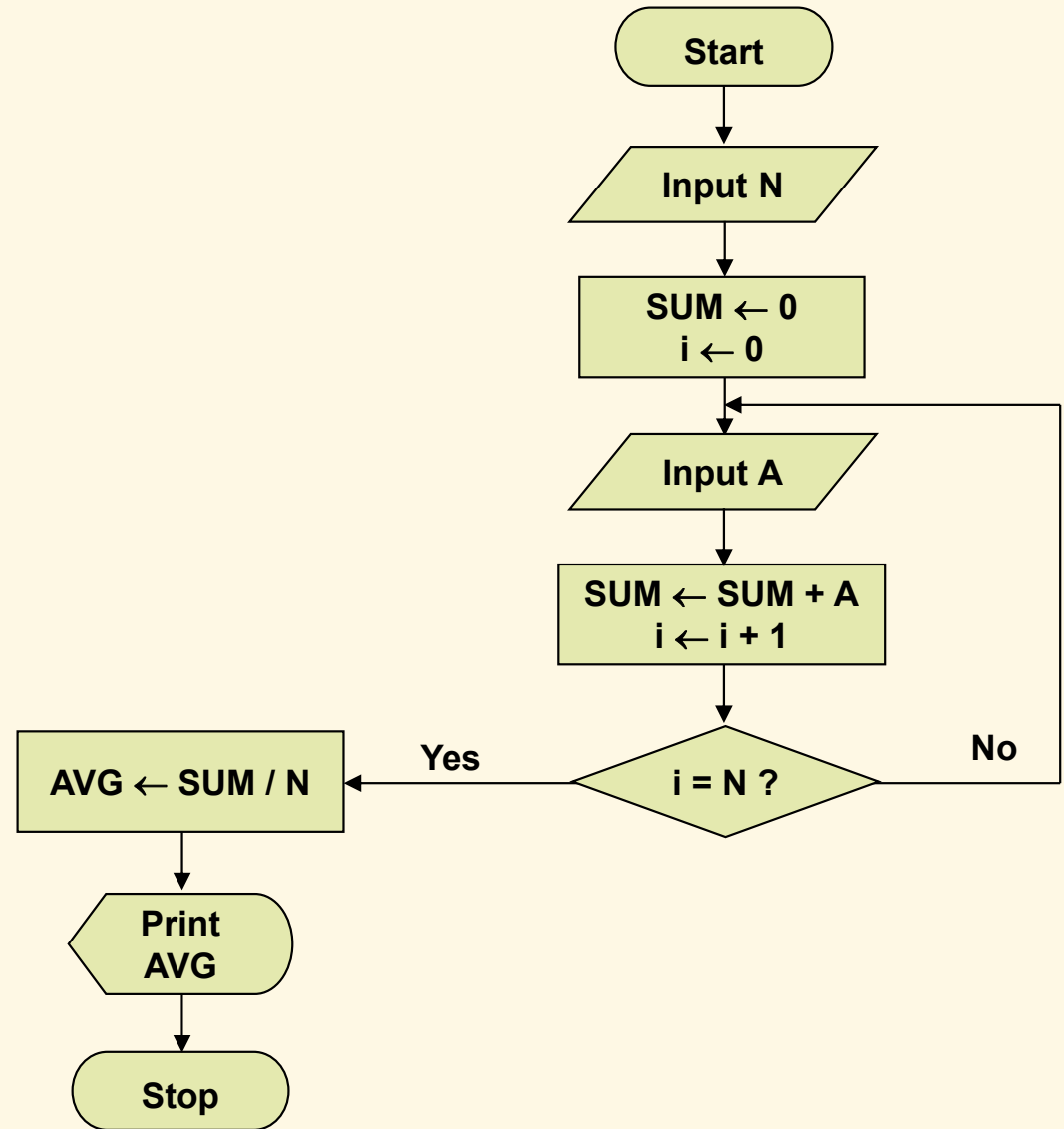
$SUM \leftarrow SUM + A$

$i \leftarrow i + 1$

until $i = N$

$AVG \leftarrow SUM / N$

Print AVG



Exercises

- ▶ Revisit the flowchart exercises and rewrite the algorithms using pseudo-code.

Algorithm Complexity, Recursion

- ❑ Algorithm Complexity
- ❑ Recursion

Algorithm Complexity

Introduction

- ▶ Algorithms need to be analyzed for their efficiency, in terms of
 - ▶ Running time → computational complexity
 - ▶ Size of used memory → memory complexity
- ▶ An algorithm may run faster/slower and use more/less on certain data sets than on others
 - many indicators for assessing the efficiency:
 - ▶ Average case
 - ▶ Best case (lower bound)
 - ▶ Worst case (upper bound)
 - ▶ Most common case
- ▶ How to measure complexity?
 - ▶ Experimental studies
 - ▶ Theoretical analysis with pseudo-code, flowcharts

Big-O notation

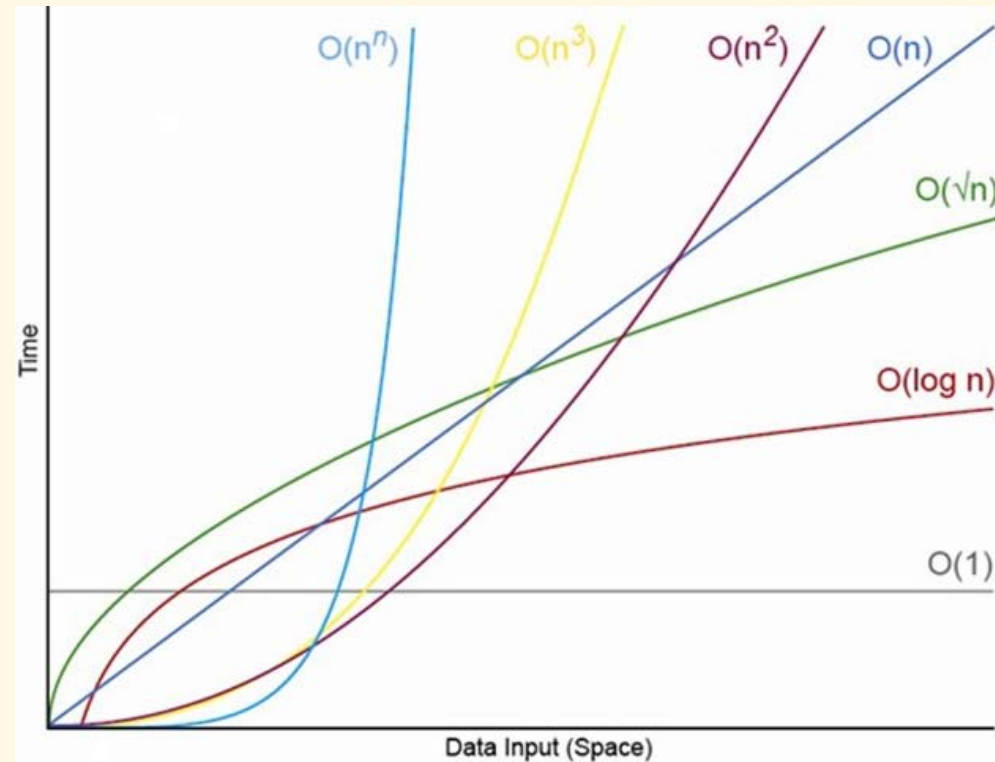
- ▶ Definition: Suppose $f(n)$ and $g(n)$ are non-negative functions of n . Then we say that $f(n)$ is $O(g(n))$ if there exists constants $C > 0$ and $N > 0$ such that for all $n > N$, $f(n) \leq Cg(n)$.
- ▶ This says that function $f(n)$ grows at a rate no faster than $g(n)$, thus $g(n)$ is an upper bound on $f(n)$
- ▶ Big-O expresses an upper bound on the growth rate of a function, for sufficiently large values of n
 - ▶ It represents the computational/memory complexity of algorithms

Examples

- ▶ For $f(n) = 3n + 5$ and $g(n) = n$ there are positive constants C and N such that $f(n) \leq Cg(n)$ for $n > N$
→ $3n + 5$ is $O(n)$
- ▶ $3n^2 + 5n + 4$ is $O(n^2)$
- ▶ $n + \sqrt{n}$ is $O(n)$
- ▶ $2^n + n^2$ is $O(2^n)$

Common Classes

- ▶ Constant: $O(1)$
- ▶ Linear: $O(n)$
- ▶ Quadratic: $O(n^2)$
- ▶ Polynomial: $O(n^k)$, $k \geq 1$
- ▶ Exponential: $O(a^n)$, $n > 1$
- ▶ Logarithmic: $O(\log n)$
- ▶ Factorial: $O(n!)$



- ▶ Efficiency comparison of classes
 - ▶ $O(1) < O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(3^n) < O(n!) < O(n^n)$

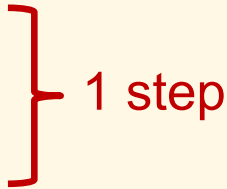
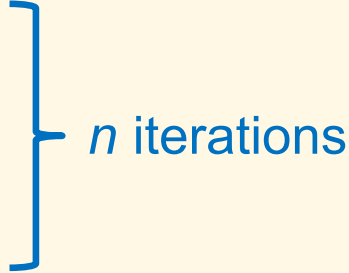
Asymptotic Notation

- ▶ It is correct to say $3n + 2$ is $O(n^3)$, $O(2^n)$
 - ➔ One should make approximation as tight as possible
 - ➔ Asymptotic notation
- ▶ Simple rule: Drop lower order terms and constant factors
 - ▶ $3n + 2$ is $O(n)$
 - ▶ $8n^2 \log n + 5n^2 + n$ is $O(n^2 \log n)$

Computational Complexity Analysis

- ▶ Break down into number of primitive operations
- ▶ Example 1: Find the maximum element of an array.
 - ▶ Algorithm `ArrayMax(A, n)`:
Input: Array A storing n integers.
Output: The maximum element in A .

```
currentMax ← A[0]
for i ← 1 to n-1 do
    if currentMax < A[i] then
        currentMax ← A[i]
```

 1 step  n iterations

```
return currentMax
```

Computational Complexity Analysis (*cont'd*)

- ▶ Example 2: Compute prefix averages.
 - ▶ Algorithm PrefixAverages(X, n):
Input: Array X of n numbers.
Output: Array A of n numbers such that $A[i]$ is the average of $X[0..i]$.

Initialize A as array of n numbers.

for $i \leftarrow 0$ to $n-1$ do

$a \leftarrow 0$

 for $j \leftarrow 0$ to i do

$a \leftarrow a + X[j]$

$A[i] \leftarrow a/(i+1)$

} 1 step } i iterations
($i = 0..n-1$)

} n iterations

return A

Computational Complexity Analysis (*cont'd*)

- ▶ Example 3: Compute prefix averages.
 - ▶ Algorithm BetterPrefixAverages(X, n):
Input: Array X of n numbers.
Output: Array A of n numbers such that $A[i]$ is the average of $X[0..i]$.

Initialize A as array of n numbers.

$a \leftarrow 0$

for $i \leftarrow 0$ to $n-1$ do

$a \leftarrow a + X[i]$

$A[i] \leftarrow a/(i+1)$

} 1 step } n iterations

return A

Memory Complexity Analysis

- ▶ Revisit above examples and determine the memory complexity

Recursion

Problems Defined Recursively

- ▶ There are many problems whose solution can be defined recursively
- ▶ Example 1: factorial function

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ 1 \times 2 \times 3 \times \dots \times (n-1) \times n & \text{if } n > 0 \end{cases} \quad (\text{closed-form solution})$$

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ (n-1)! \times n & \text{if } n > 0 \end{cases} \quad (\text{recursive solution})$$

Recursive Function

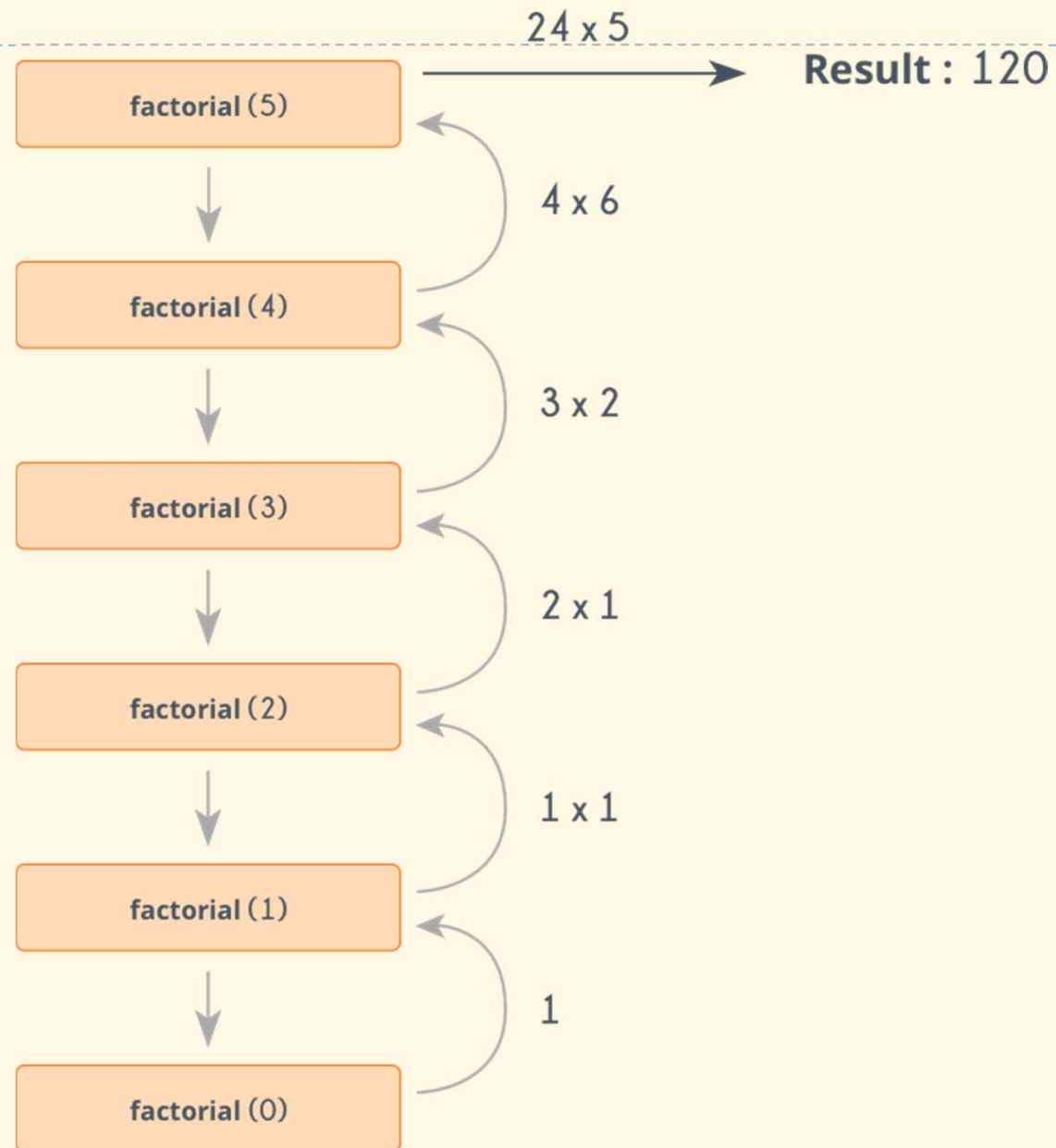
- ▶ A function that calls itself, either in direct or indirect manner
- ▶ Implementation

```
double factorialIterative(int n) {  
    double f = 1;  
    for (int i = 1; i <= n; i++)  
        f *= i;  
    return f;  
}
```

```
double factorialRecursive(int n) {  
    return n == 0 ? 1. : n * factorialRecursive(n-1);  
}
```

```
printf("5! = %lf\n", factorialIterative(5));  
printf("5! = %lf\n", factorialRecursive(5));
```

Explanation



Example 2

- ▶ Calculate x^n , $n \in \mathbb{N}$

- ▶ Implementation

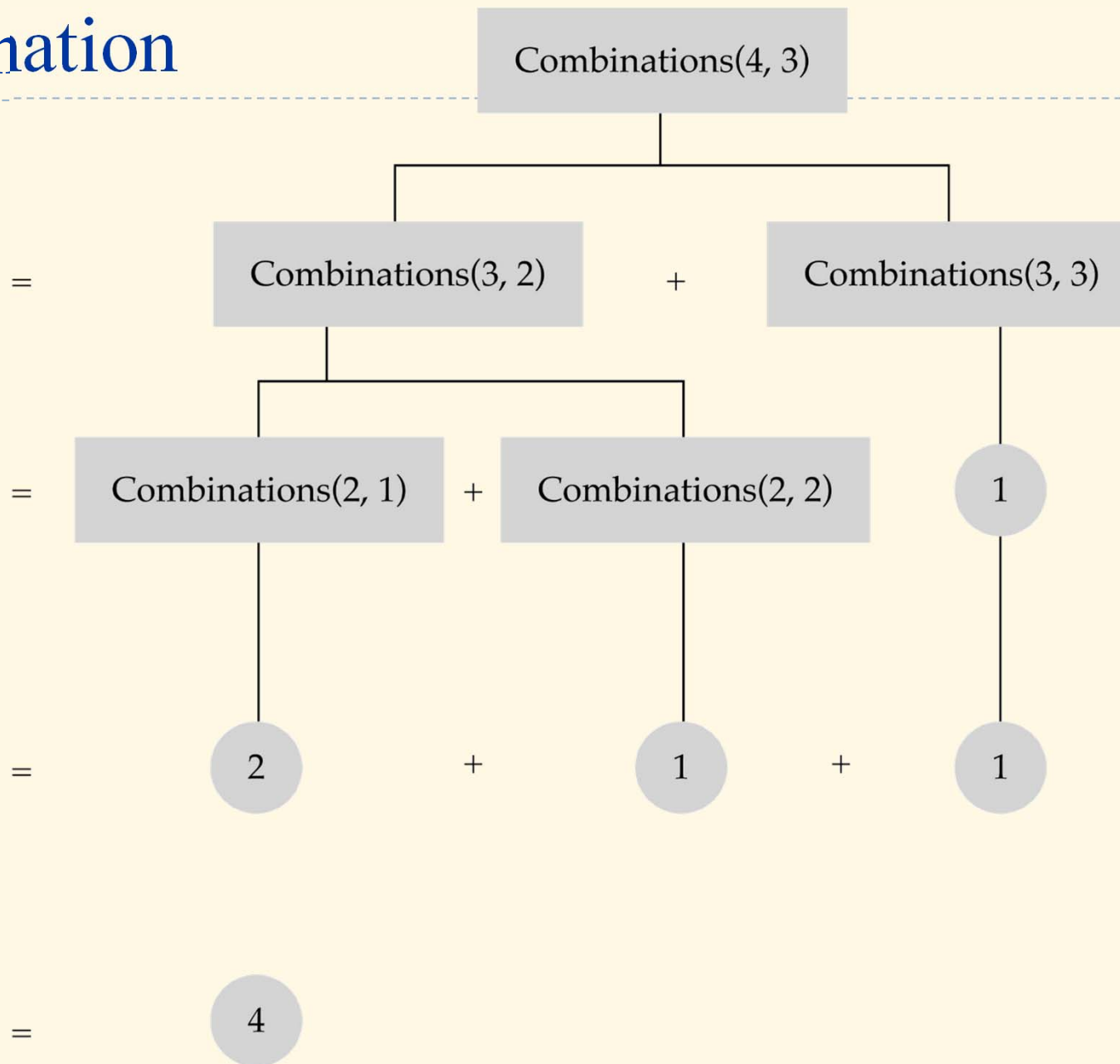
```
double powerIterative(double x, int n) {  
    double y = 1;  
    for (int i = 1; i <= n; i++)  
        y *= x;  
    return y;  
}
```

```
double powerRecursive(double x, int n) {  
    if (n == 0) return 1.;  
    double y = powerRecursive(x, n/2);  
    return (n % 2 == 0) ? y*y : y*y*x;  
}
```

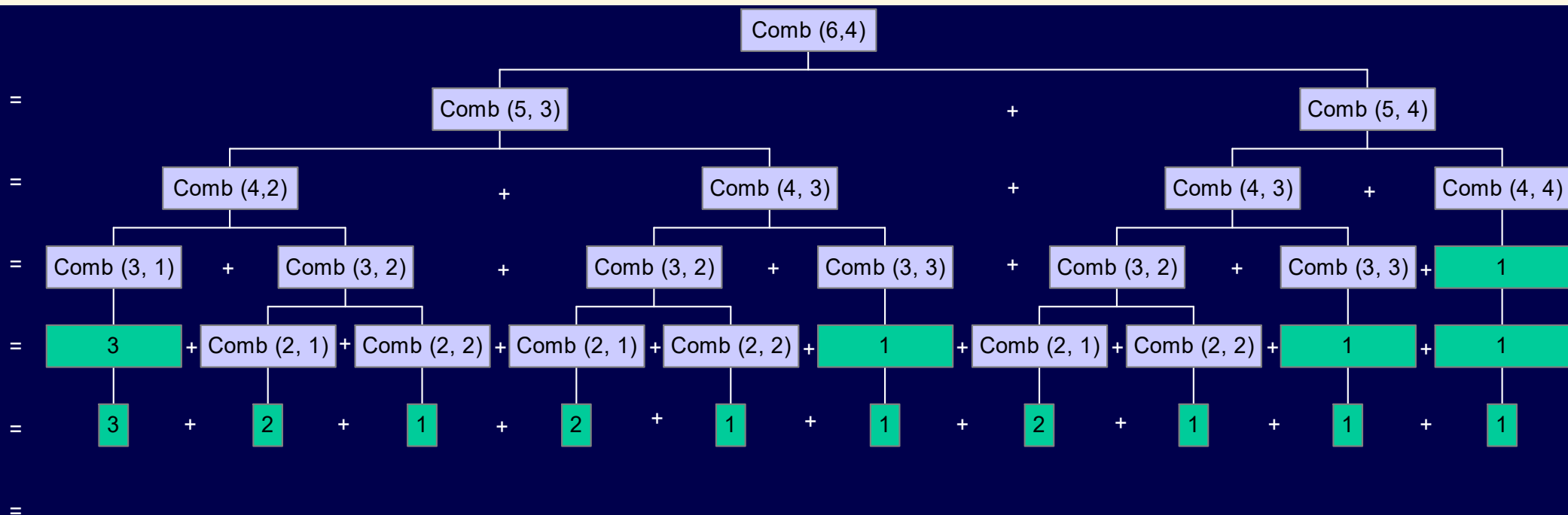
Example 3

- ▶ Calculate number of combinations of k elements from n .
- ▶ Closed-form solution:
 - ▶ $C_n^k = \frac{n!}{k!(n-k)!}$
- ▶ Recursive solution using Pascal's formula:
 - ▶ $C_n^k = \frac{n-k+1}{k} C_n^{k-1}$ with base case $C_n^1 = n$
- ▶ Binary recursive solution:
 - ▶ $C_n^k = C_{n-1}^{k-1} + C_{n-1}^k$ with base cases $C_n^1 = n$ and $C_n^n = 1$

Explanation



Recursion Can be Very Inefficient in Some Cases



Recursion vs. Iteration

- ▶ Iteration can be used in place of recursion
 - ▶ An iterative algorithm uses a looping construct
 - ▶ A recursive algorithm uses a branching structure
- ▶ Recursive solutions are often less efficient, in terms of both time and space, than iterative solutions
- ▶ Recursion can simplify the solution of a problem, often resulting in shorter, more easily understood source code

How to Write a Recursive Function?

- ▶ Determine the size factor
- ▶ Determine the base case(s) (the one for which you know the answer)
- ▶ Determine the general case(s) (the one where the problem is expressed as a smaller version of itself)
- ▶ Verify the algorithm

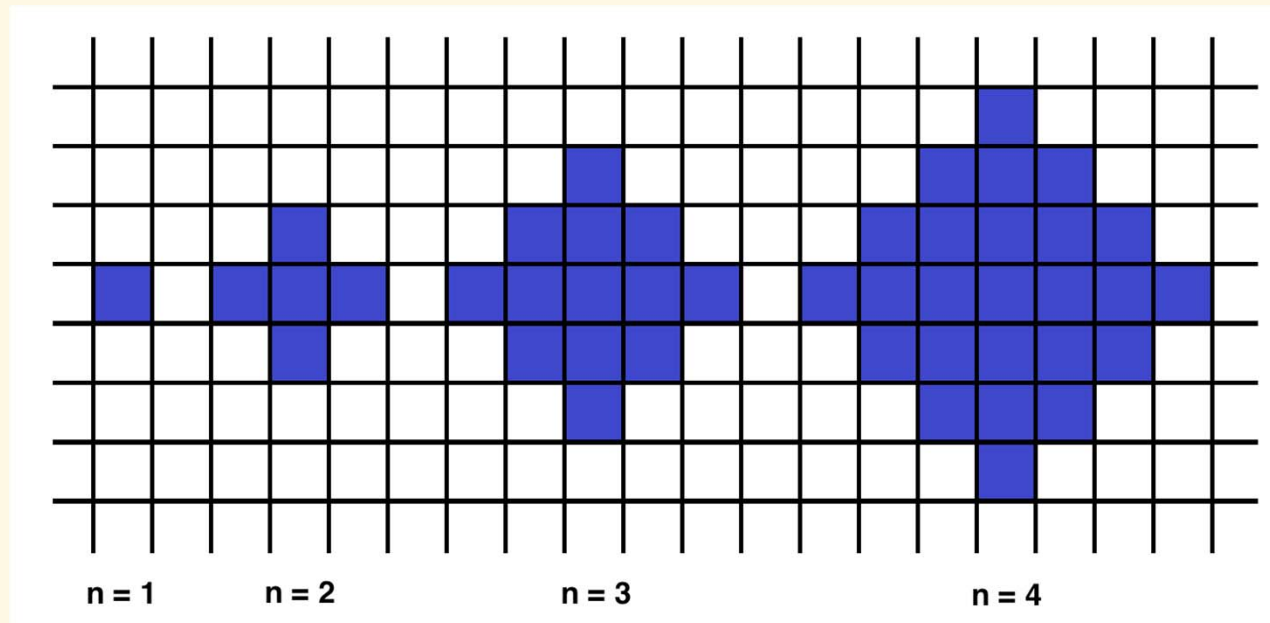
Exercises

1. Write the pseudo-code to express the algorithm to look-up a word from a dictionary.
 - ▶ Function `WordLookup( $D$ ,  $W$ )`
Input: Dictionary D as an array of N words
already sorted in alphabetical order.
Word W to be looked-up.
Output: Index of W found in D ,
or -1 if not found.

... (to do)
2. Analyze the complexity of above examples.

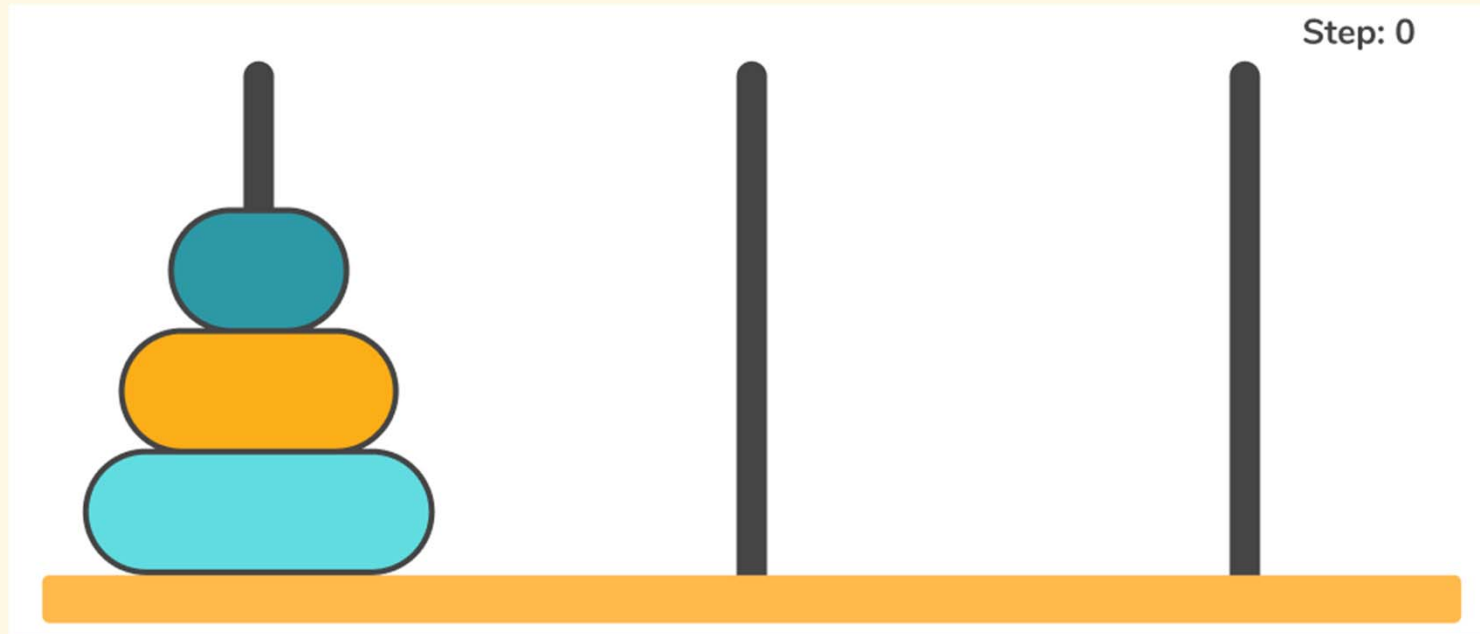
Exercises

3. The so-called n -interesting polygon is defined as in the picture. Write a **recursive function** to find the area of a polygon for a given n .



Exercises

4. The Tower of Hanoi is a puzzle consisting of 3 rods and n disks. The task is to move all disks to another rod, but: only the uppermost disk can be moved, and larger disks cannot be placed on top of smaller disks. Write a recursive function to solve the problem given n .



Dynamic Arrays & Linked Lists

- ❑ Dynamic Arrays
- ❑ Singly Linked Lists
- ❑ Doubly Linked Lists

Introduction: dynamic array

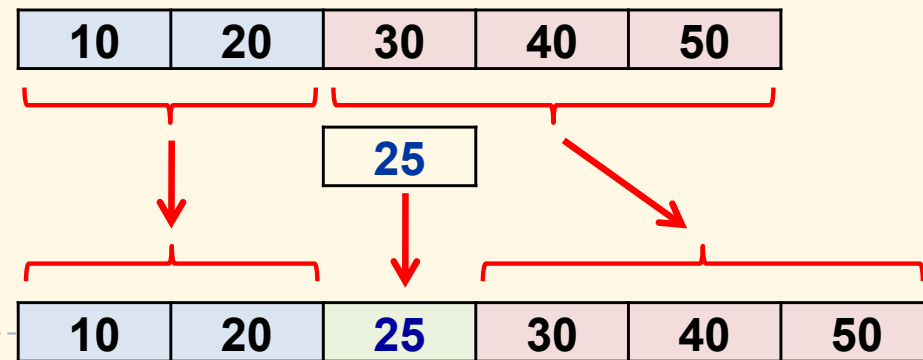
- ▶ Conventional array in C has fix number of elements
- ▶ Dynamic array is array with variable number of elements: actually a pointer and a variable indicating n° of elements

```
▶ int n = 5;  
   int* arr = (int*)malloc(n*sizeof(int));
```

- ▶ Insert an element into dynamic array:

```
▶ pos = 2; val = 25;  
   arr1 = (int*)malloc((n+1)*sizeof(int));  
   memcpy(arr1, arr, pos*sizeof(int));  
   memcpy(arr1+pos+1, arr+pos, (n-pos)*sizeof(int));  
   arr1[pos] = val;  
   free(arr);  
   arr = arr1; n++;
```

- ▶ Similarly for element removal

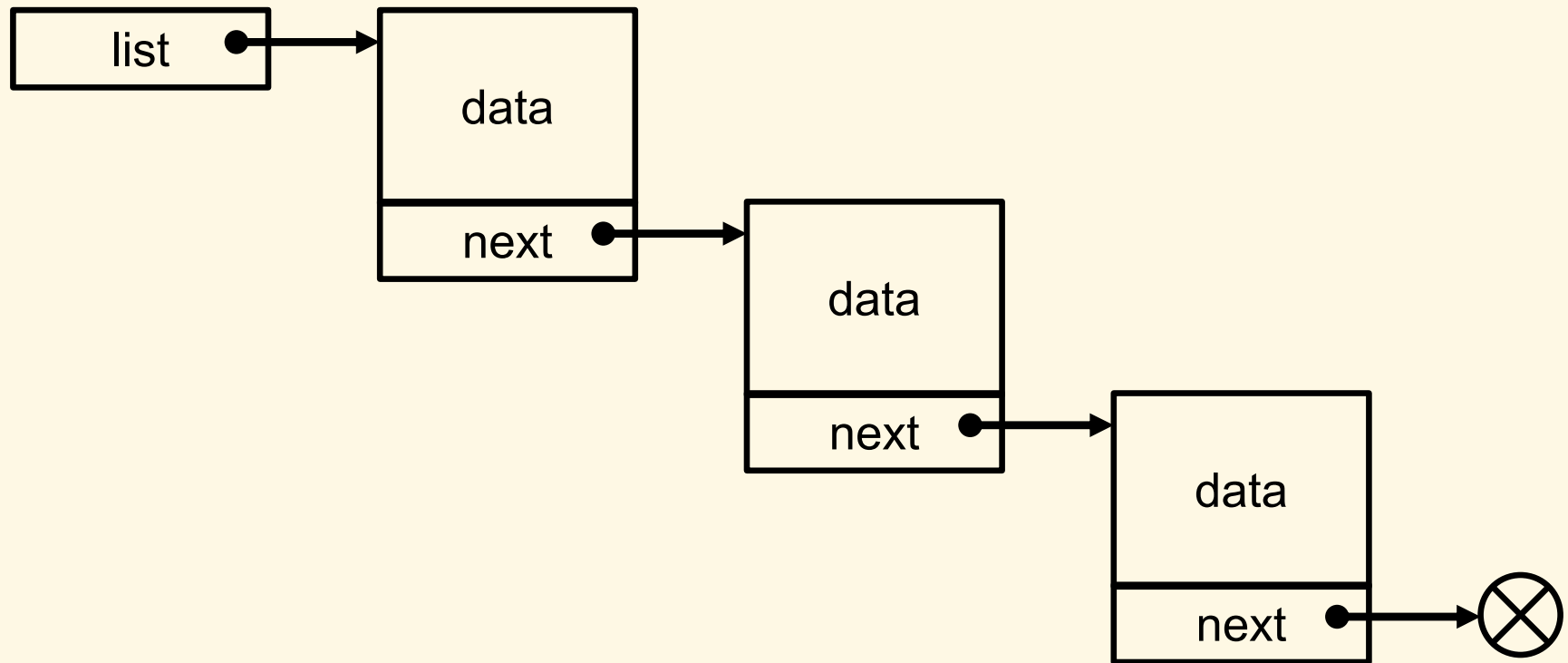


Overview

- ▶ Above example shows that dynamic arrays are slow in inserting/removing elements because of moving large memory areas, especially when there are many elements
→ necessity of a more flexible data structures
- ▶ Frequently used data structures:
 - ▶ Stack
 - ▶ Queue
 - ▶ Linked list
 - ▶ Dynamic array (vector)
 - ▶ Map, dictionary, hash table
 - ▶ Set
 - ▶ Tree
 - ▶ Graph

Linked list

- ▶ Group of nodes linked together by pointers and represent a sequence of elements. For presentation:
 - ▶ A pointer to the first node (or NULL for empty list)
 - ▶ Each node includes 2 members: data, and a pointer to the next node
 - ▶ The next pointer of last element is NULL



Declaration of linked list

- ▶ Example for int elements:

```
typedef int LLIST_DATA;
```

```
struct _LIST_ELEM;
```

```
typedef struct _LIST_ELEM LIST_ELEM, *LLIST;
```

```
struct _LIST_ELEM {  
    LLIST_DATA data;  
    LIST_ELEM* next;  
};
```

Operations of linked list

- ▶ Necessary functions:

- ▶ `void llInit(LLIST* l);`
- ▶ `void llInsertHead(LLIST* l, LLIST_DATA data);`
- ▶ `void llInsertTail(LLIST* l, LLIST_DATA data);`
- ▶ `void llInsertAfter(LLIST* l, LIST_ELEM* a, LLIST_DATA data);`
- ▶ `void llDeleteHead(LLIST* l);`
- ▶ `void llDeleteTail(LLIST* l);`
- ▶ `void llDeleteAfter(LLIST* l, LIST_ELEM* a);`
- ▶ `void llDeleteAll(LLIST* l);`
- ▶ `LIST_ELEM* llSeek(LLIST l, int i);`
- ▶ `void llForEach(LLIST l, LLCALLBACK func, void* user);`
- ▶ `int llLength(LLIST l);`

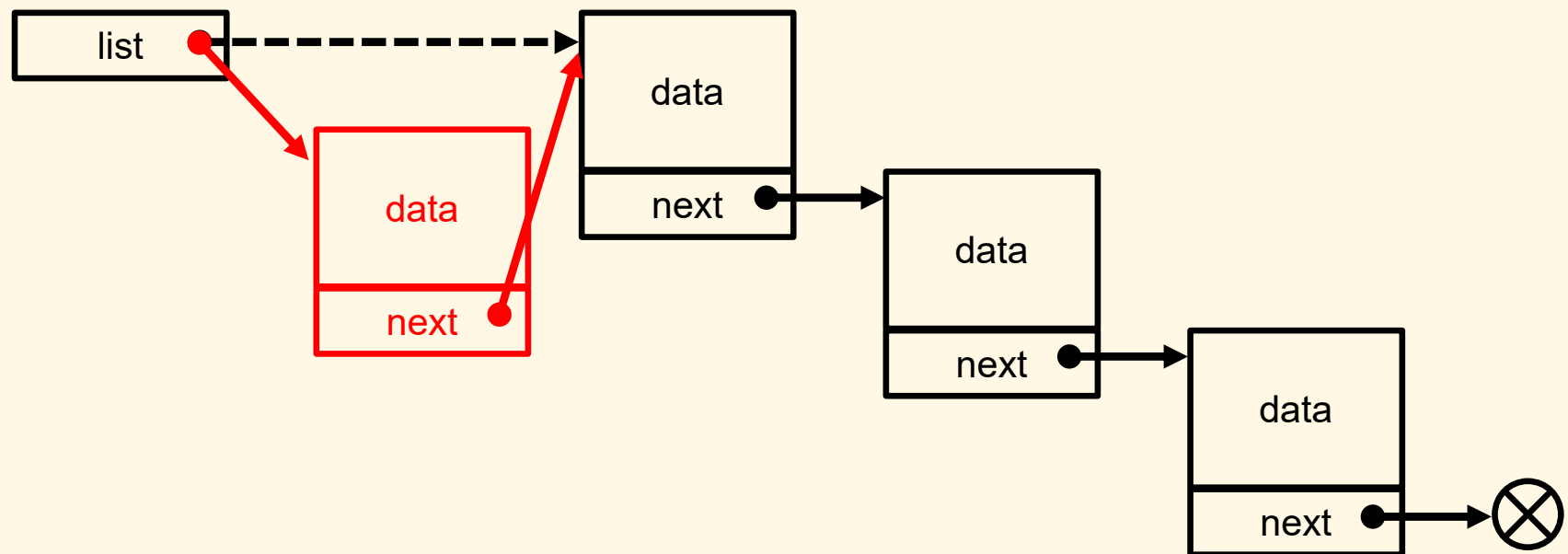
Initialization

```
void llInit(LLIST* l) {  
    *l = NULL;  
}
```

- ▶ Initialized to NULL → empty list

Insert element to head of a list

```
void llInsertHead(LLIST* l, LLIST_DATA data) {  
    LIST_ELEM* e = (LIST_ELEM*)malloc(sizeof(LIST_ELEM));  
    e->data = data;  
    e->next = *l;  
    *l = (LLIST)e;  
}
```

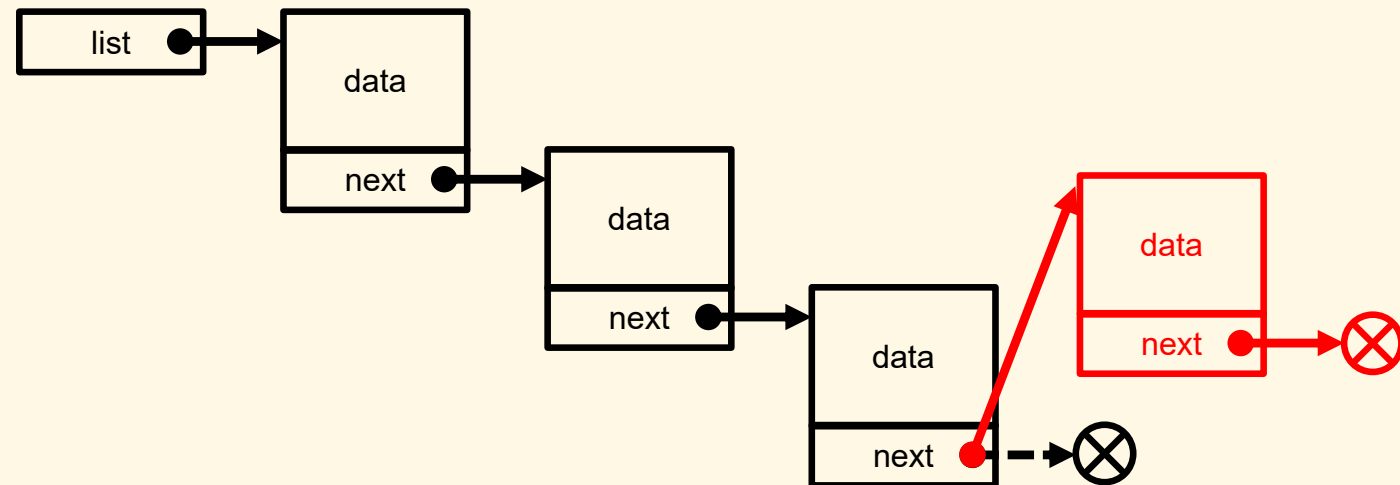


Insert element to tail of a list

```
void llInsertTail(LLIST* l, LLIST_DATA data) {  
    LIST_ELEM* e = (LIST_ELEM*)malloc(sizeof(LIST_ELEM));  
    e->data = data;  
    e->next = NULL;
```

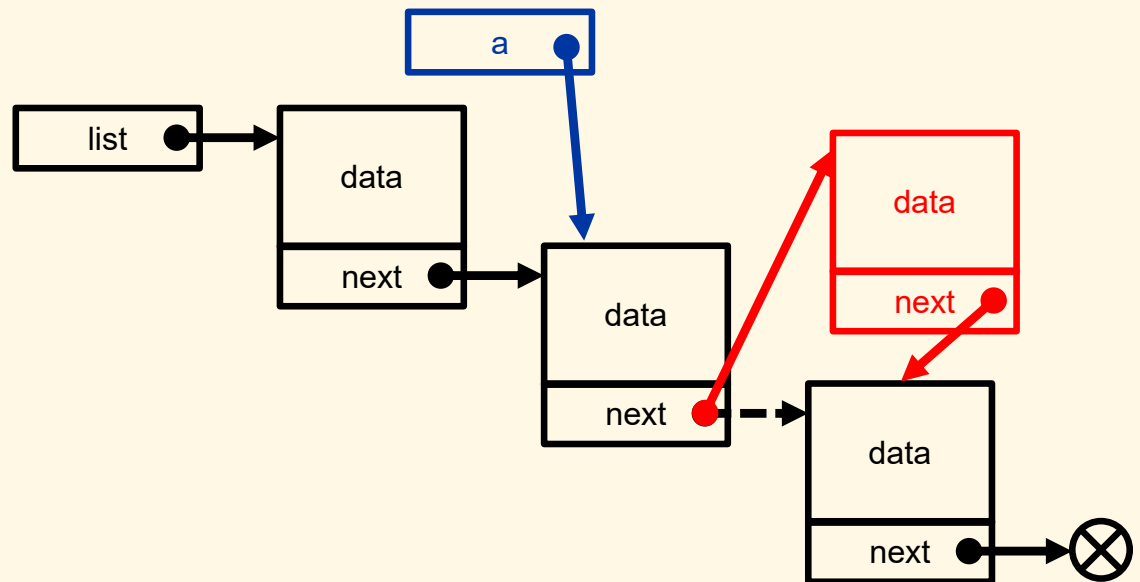
```
    if (!*l) {  
        *l = e;  
        return;  
    }
```

```
    LIST_ELEM* p;  
    for (p = *l; p->next; p = p->next) ;  
    p->next = e;  
}
```



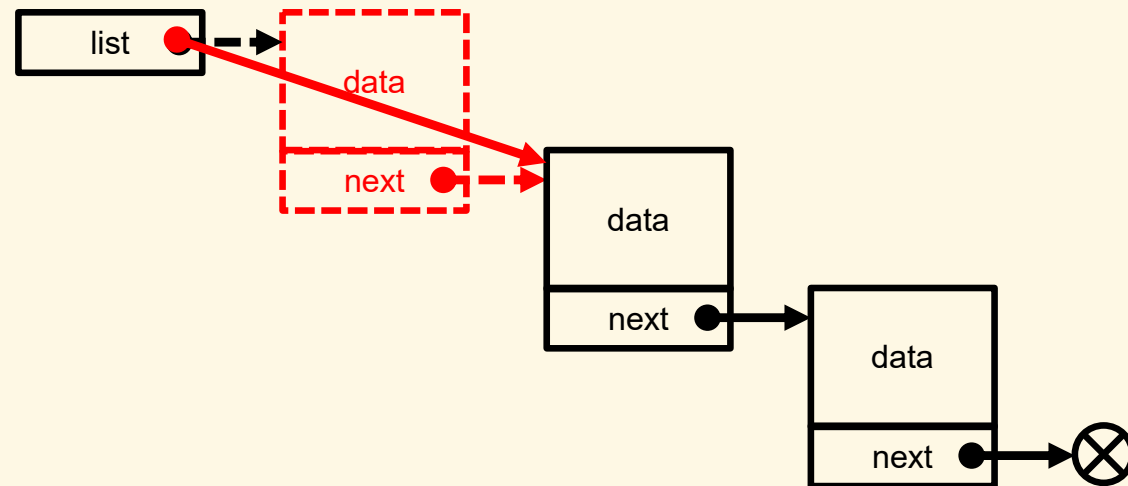
Insert elements in the middle

```
void llInsertAfter(LLIST* l, LIST_ELEM* a, LLIST_DATA data) {  
    if (!a) return;  
  
    LIST_ELEM* e = (LIST_ELEM*)malloc(sizeof(LIST_ELEM));  
    e->data = data;  
    e->next = a->next;  
    a->next = (LLIST)e;  
}
```



Delete first element

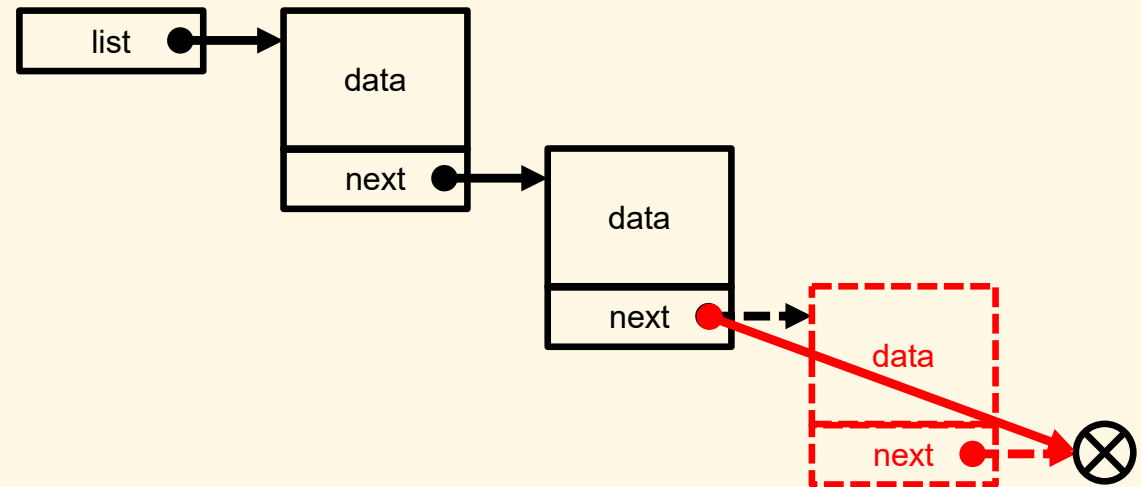
```
void llDeleteHead(LLIST* l) {  
    if (!*l) return;  
  
    LIST_ELEM* p = (*l)->next;  
    free(*l);  
    *l = (LLIST)p;  
}
```



Delete last element

```
void llDeleteTail(LLIST* l) {  
    if (!*l) return;
```

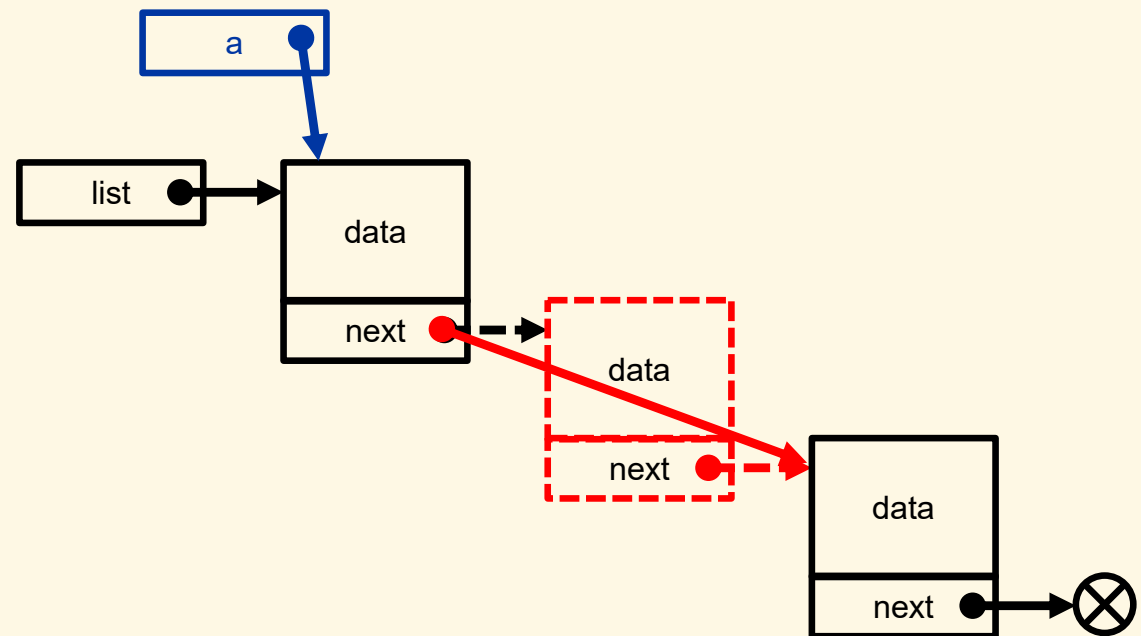
```
    if (!(*l)->next) {  
        free(*l);  
        *l = NULL;  
        return;  
    }
```



```
    LIST_ELEM* p;  
    for (p = *l; p->next->next; p = p->next) ;  
    free(p->next);  
    p->next = NULL;  
}
```


Delete elements in the middle

```
void llDeleteAfter(LLIST* l, LIST_ELEM* a) {  
    if (!a || !a->next) return;  
  
    LIST_ELEM* p = a->next;  
    a->next = p->next;  
  
    free(p);  
}
```



Delete all elements

```
void llDeleteAll(LLIST* l) {  
    LIST_ELEM *p, *i;  
    for (i = *l; i; i = p) {  
        p = i->next;  
        free(i);  
    }  
  
    *l = NULL;  
}
```

Count the number of elements

```
int llLength(LLIST l) {  
    int c;  
    for (c = 0; l; c++)  
        l = l->next;  
  
    return c;  
}
```

Iterate a list

- ▶ Find the i^{th} element of a list

```
LIST_ELEM* llSeek(LLIST l, int i) {  
    for (; i>0 && l; i--)  
        l = l->next;  
    return (LIST_ELEM*)l;  
}
```

- ▶ Do some operation to all elements of a list

```
typedef void (*LLCALLBACK)(LLIST_DATA, void*);  
  
void llForEach(LLIST l, LLCALLBACK func, void* user) {  
    for (; l; l = l->next)  
        func(l->data, user);  
}
```

Usage example

```
#include <stdio.h>
#include "llist.h"

void printList(LLIST l) {
    printf("%d elements: ( ",
           llLength(l));
    for (; l; l = l->next)
        printf("%d ", l->data);
    printf(") \n");
}

void listSum(int data, void* user) {
    int* pS = (int*)user;
    *pS += data;
}

int main() {
    LLIST l;
    LIST_ELEM* p;
    int i, s;

    llInit(&l);

    for (i = 0; i < 5; i++) {
        llInsertTail(&l, i);
        llInsertHead(&l, -i);
    }
    printList(l);

    p = llSeek(l, 1);
    llDeleteAfter(&l, p);
    llDeleteHead(&l);
    llDeleteTail(&l);
    printList(l);

    s = 0;
    llForEach(l, listSum, (void*)&s);
    printf("Sum of values: %d\n", s);

    llDeleteAll(&l);
    printList(l);

    return 0;
}
```

Comparison of Computational Complexity

Operation	Array	Linked List
Insert to head	$O(n)$	$O(1)$
Insert to tail	$O(n)$	$O(n)$
Insert in the middle	$O(n)$	$O(1)$
Delete at head	$O(n)$	$O(1)$
Delete at tail	$O(n)$	$O(n)$
Delete in the middle	$O(n)$	$O(1)$
Delete all	$O(1)$	$O(1)$
Get element by index	$O(1)$	$O(n)$
Count elements	$O(1)$	$O(n)$
Iterate over elements	$O(n)$	$O(n)$

Properties of linked lists

- ▶ Advantages/drawbacks of linked lists compared to dynamic arrays:
 - + Flexible: easy to insert/remove elements, sort or change the order of elements without moving them in memory
 - + No need to allocate a large consecutive memory area to store all elements
 - Need additional memory to store next pointers
 - Sequential access: impossible to directly access arbitrary elements (or random access), but need to iterate from the beginning
- ▶ Beside basic form (singly linked list), there are other forms: doubly linked list, ordered linked list, stack, queue, cycle linked list,... and can be implemented in many different ways depending on specific problems

Exercises

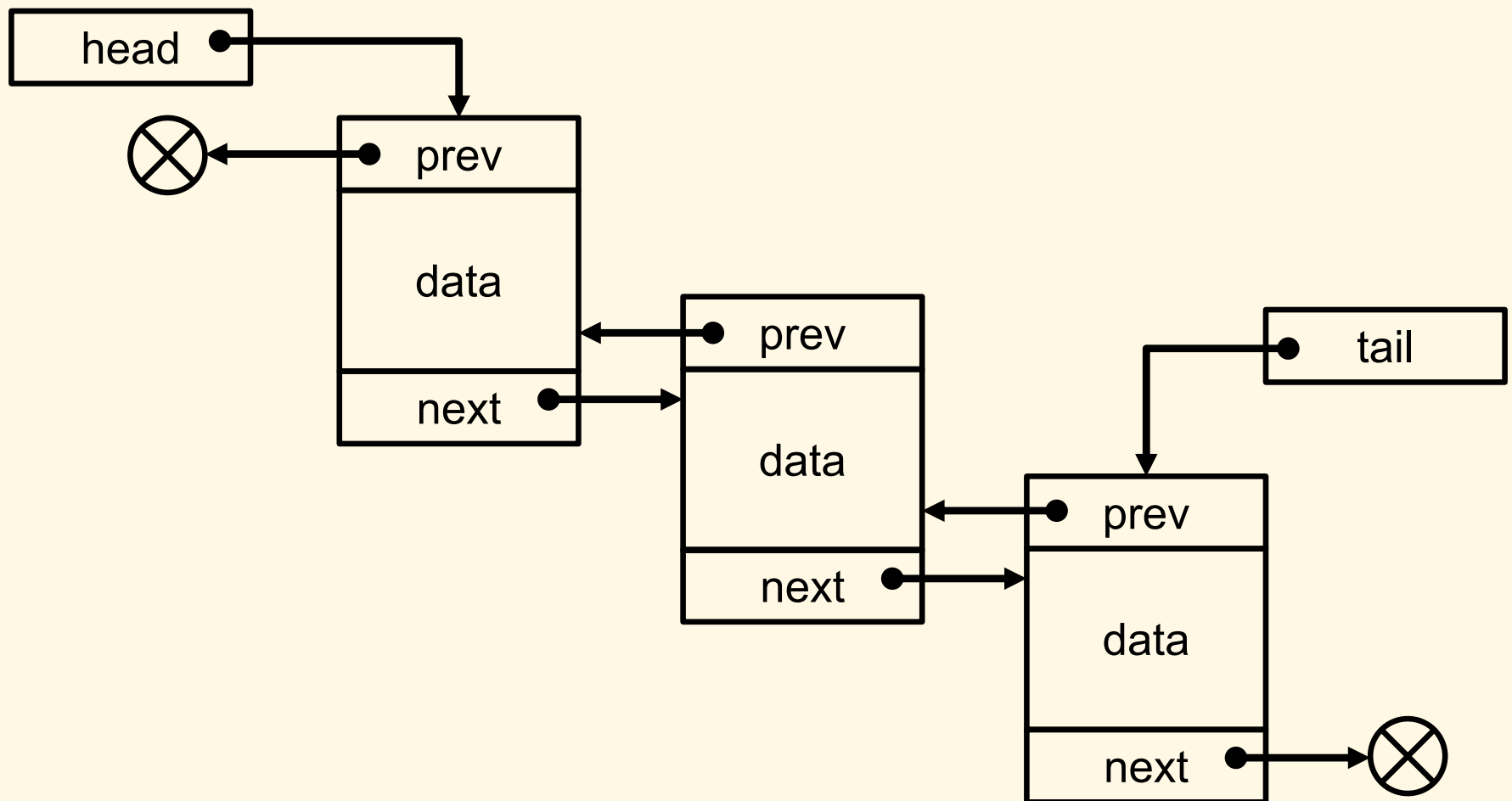
1. Calculate the average value of a linked list by two ways: iteration, using `llForEach()` function.
2. Write `llInsertBefore(l, a, v)` function to insert a new element with value `v` before element `a` in the list `l`
3. Write `llDeleteAt(l, a)` function to delete element `a` in the list `l`
4. Write `llConvert(int* arr, int count)` function to convert an array into a linked list without using functions from the library.
5. Given two linked lists `l1` and `l2`, write `llInsertListAfter(&l1, l2, p)` function to insert `l2` into `l1` after element `p`.
6. Implement a linked list that could be used for different types (Hint: make data field dynamic).

Doubly Linked Lists

Overview

- ▶ Singly linked list (SLL) only allows to iterate in forward direction
 - ▶ Each element has only a pointer to the next one
 - ▶ Impossible to find an element given the following one
- ▶ Doubly linked list (DLL):
 - ▶ Possible to move forward and backward at any time
 - ▶ Each element has a 2nd pointer to the previous one

How DLLs work



Exercise

1. Implement a doubly linked list.

Homework

1. Write `llReverse(LLIST* l)` function to reverse the order of a linked list without creating new nodes.
2. Same as Prob. 1, but do recursively.

Stacks & Queues

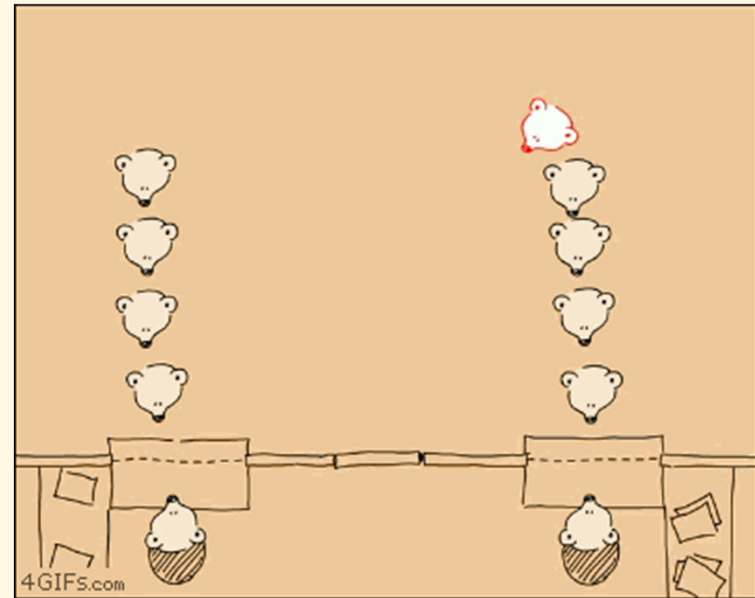
- ❑ Stacks
- ❑ Queues
- ❑ Separation of interface and implementation

Overview

- ▶ Access restriction
 - ▶ Arrays: random
 - ▶ Linked lists: sequential
 - ▶ Stacks and Queues: limited



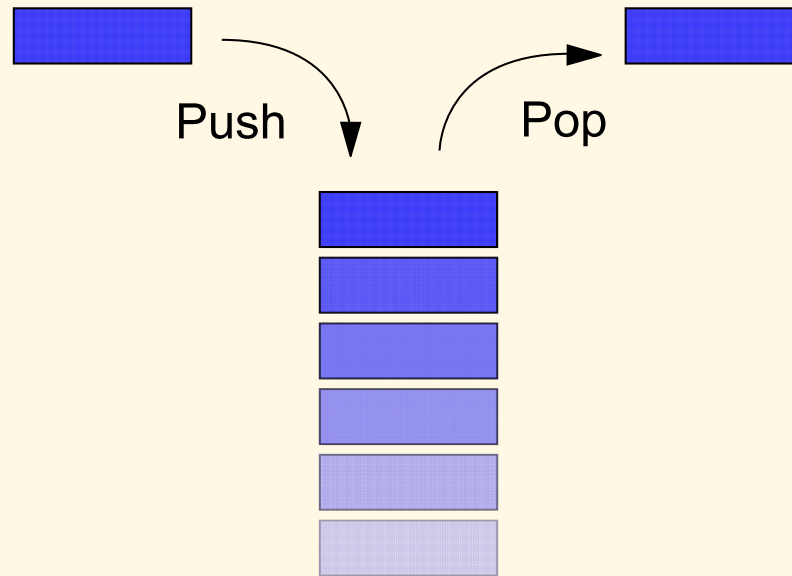
Stack



Queue

Stacks

What's a stack?



- ▶ Linear data structure which can only be accessed at one of its ends
- ▶ LIFO (last in, first out) basis
- ▶ Two ways of implementation:
 - ▶ Using array (static or dynamic)
 - ▶ Using linked list (dynamic)

Operations

- ▶ `create()`: creates a stack
- ▶ `destroy()`: destroys a stack
- ▶ `push()`: inserts an element
- ▶ `pop()`: removes last inserted element (optionally returns the element)
- ▶ `top()`: returns the last inserted element without removing it
- ▶ `isEmpty()`: checks if the stack is empty
- ▶ `isFull()`: checks if the stack is full

Static implementation

► Definitions

```
typedef int STACK_DATA;  
#define MAX_ELEMENTS 20
```

```
typedef struct {  
    STACK_DATA elements[MAX_ELEMENTS];  
    int top;  
} STACK;
```

► Initialization

```
void stInit(STACK* s) {  
    s->top = -1;  
}
```

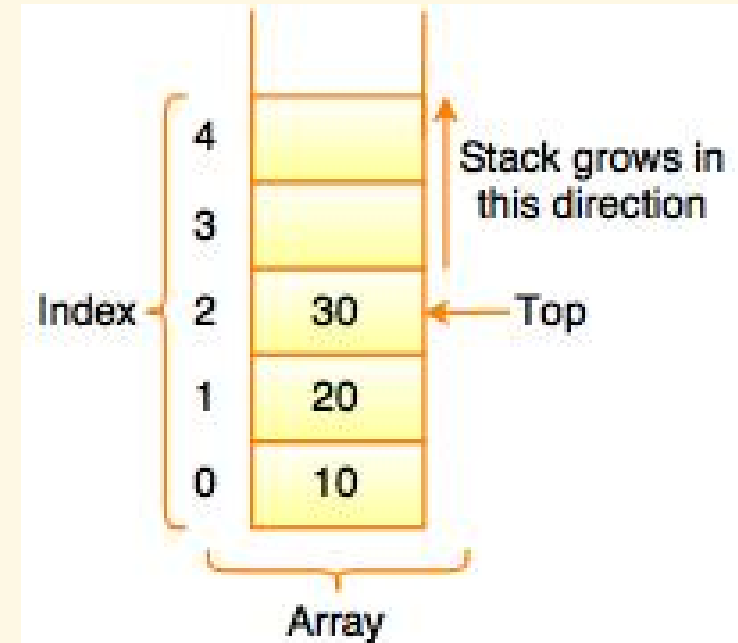
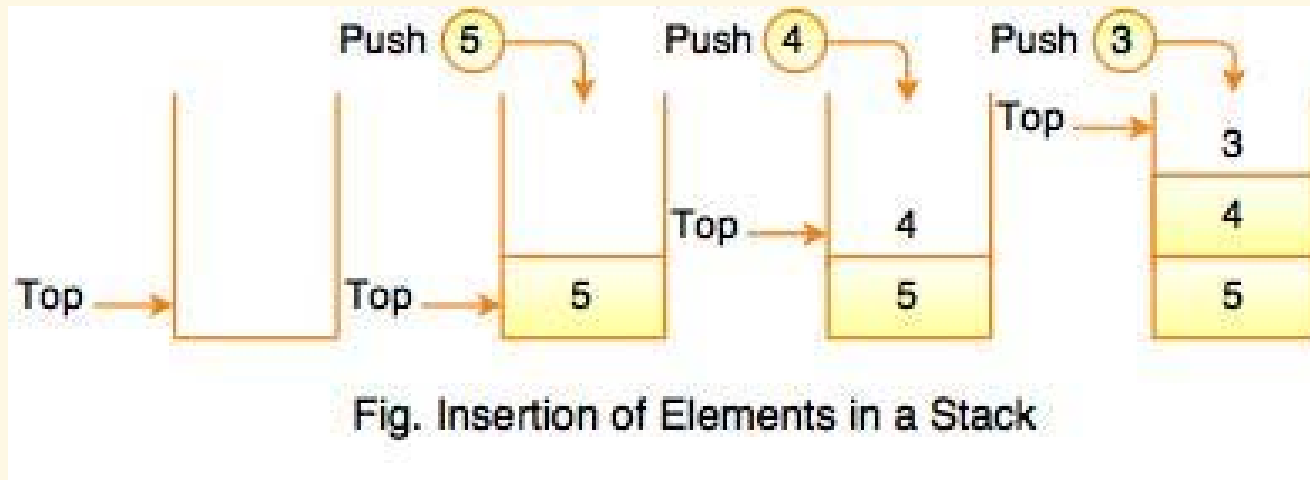


Fig. Implementation Stack using Array

Push



```
void stPush(STACK* s, STACK_DATA data) {  
    if (stIsFull(s)) return;  
  
    s->top++;  
    s->elements[s->top] = data;  
}
```

Pop

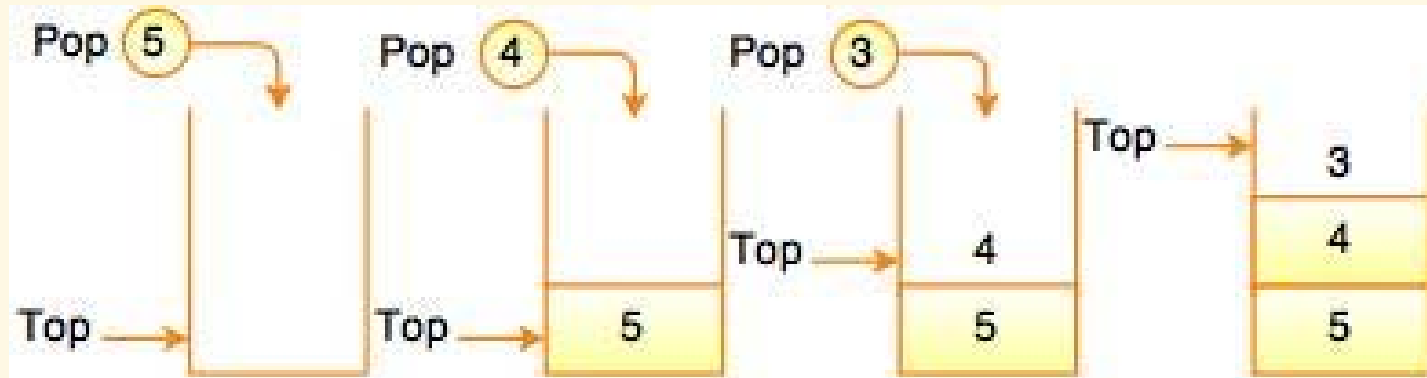


Fig. Deletion of Elements in a Stack

```
void stPop(STACK* s) {  
    if (stIsEmpty(s)) return;  
  
    s->top--;  
}
```

Get top value

```
STACK_DATA stTop(STACK* s) {  
    return s->elements[s->top];  
}
```

Is empty? Is full?

```
int stIsEmpty(STACK* s) {  
    return s->top == -1;  
}
```

```
int stIsFull(STACK* s) {  
    return s->top == MAX_ELEMENTS - 1;  
}
```

Clear all

```
void stDestroy(STACK* s) {  
    s->top = -1;  
}
```


Dynamic implementation

► Definitions

```
typedef int STACK_DATA;
```

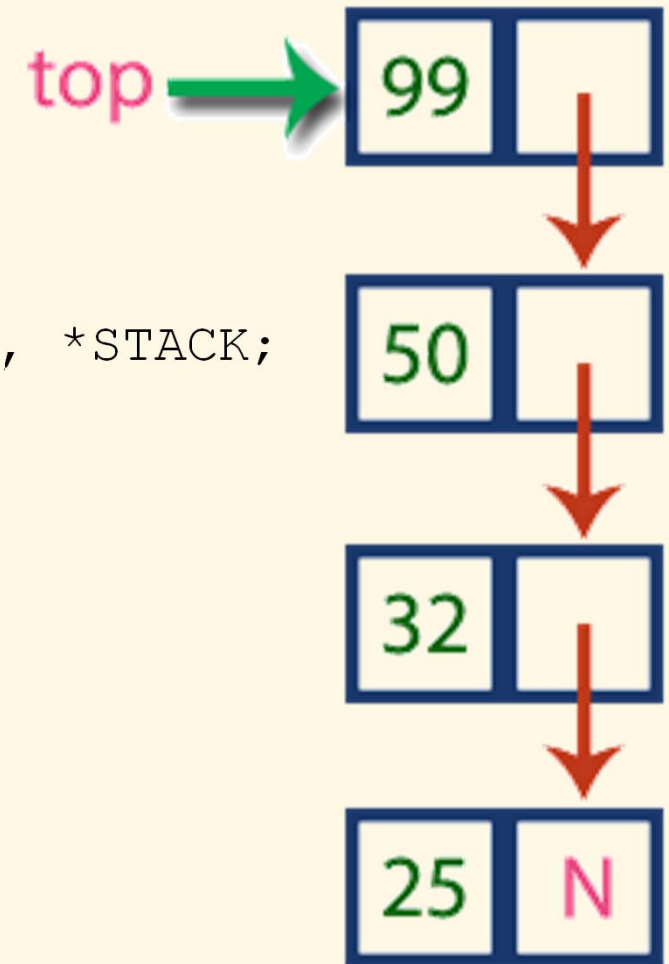
```
struct _STACK_ELEM;
```

```
typedef struct _STACK_ELEM STACK_ELEM, *STACK;
```

```
struct _STACK_ELEM {  
    STACK_DATA data;  
    STACK_ELEM* next;  
};
```

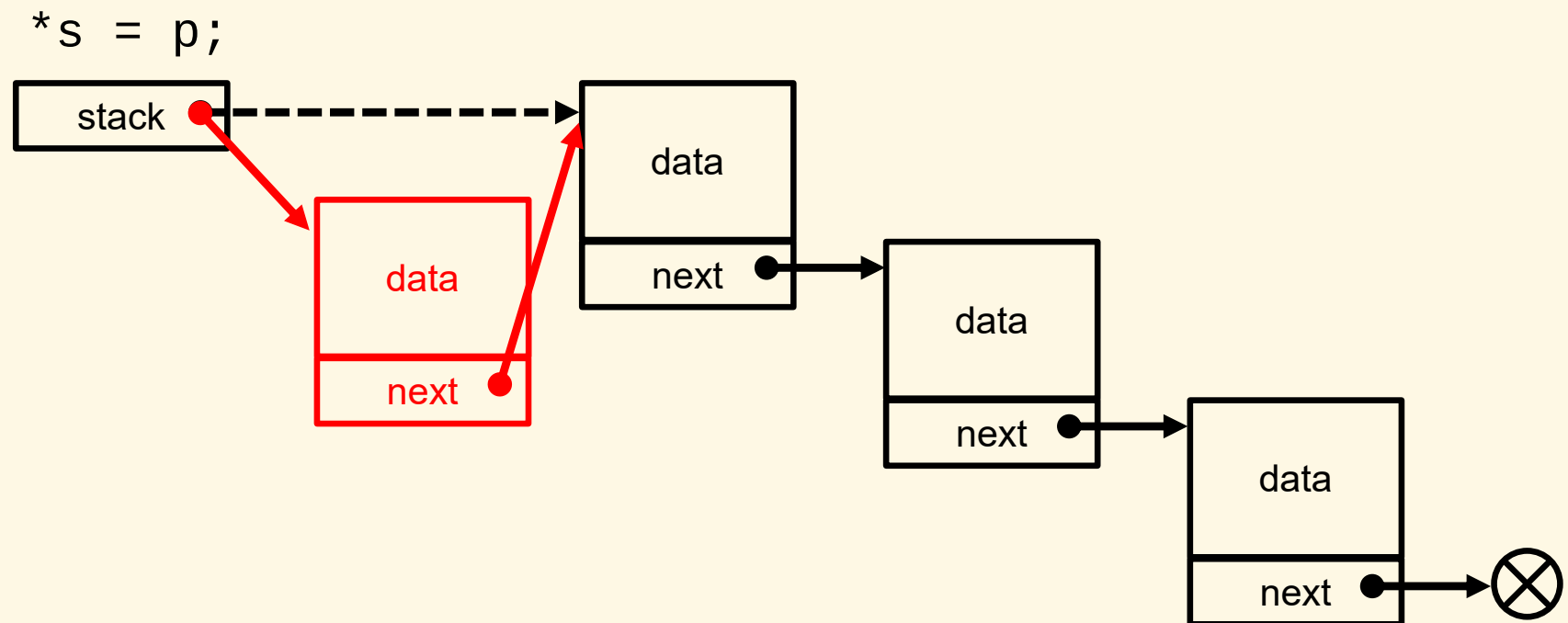
► Initialization

```
void stInit(STACK* s) {  
    *s = NULL;  
}
```

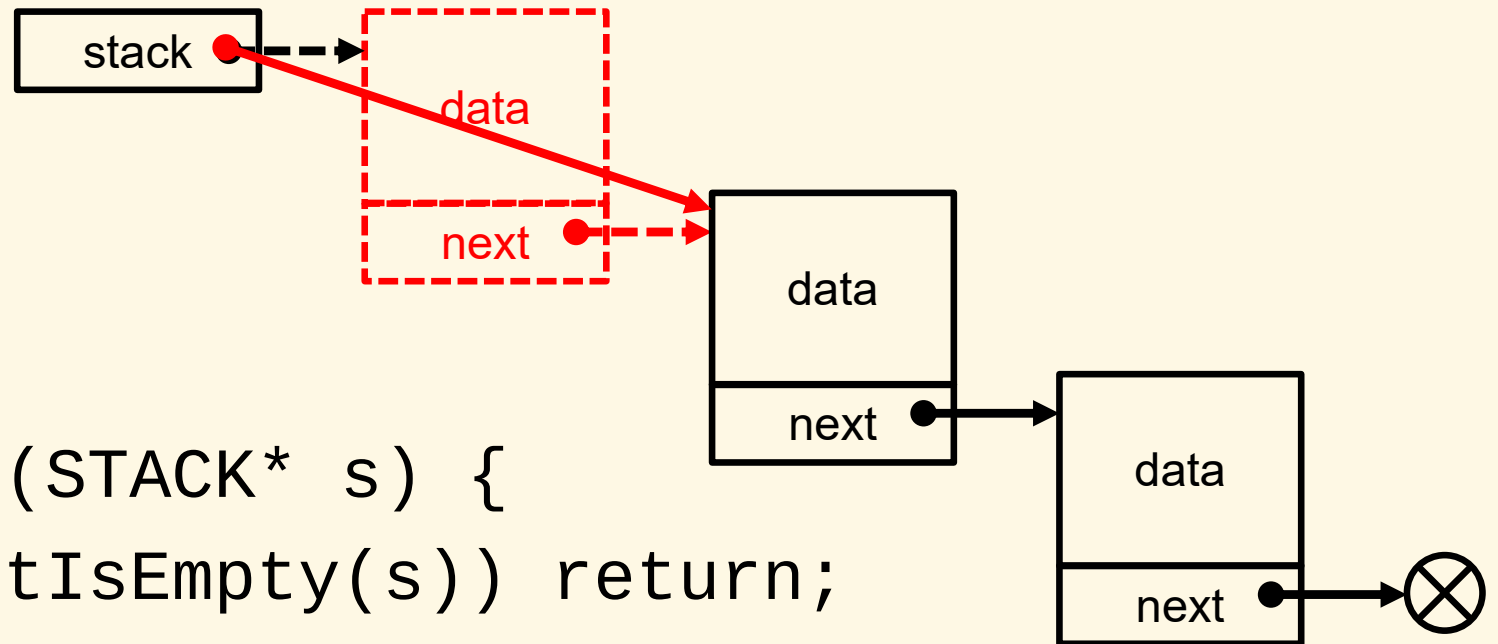


Push

```
void stPush(STACK* s, STACK_DATA data) {  
    STACK_ELEM* p = (STACK_ELEM*)malloc(  
        sizeof(STACK_ELEM));  
  
    p->data = data;  
    p->next = *s;  
  
    *s = p;  
}
```



Pop



```
void stPop(STACK* s) {  
    if (stIsEmpty(s)) return;
```

```
    STACK_ELEM* p = *s;  
    *s = (*s)->next;  
    free(p);
```

```
}
```

Get top value

```
STACK_DATA stTop(STACK* s) {  
    return (*s)->data;  
}
```

Is empty? Is full?

```
int stIsEmpty(STACK* s) {  
    return *s == NULL;  
}
```

```
int stIsFull(STACK* s) {  
    return 0;  
}
```

Clear all

```
void stDestroy(STACK* s) {  
    while (*s) {  
        STACK_ELEM* p = *s;  
        *s = (*s)->next;  
        free(p);  
    }  
}
```

Applications of stacks

- ▶ Memory representation for function calls in computer program execution
- ▶ Undo operation in editors
- ▶ History in web browsers
- ▶ Alternative implementation of recursive functions
- ▶ Math expression evaluation (reverse Polish notation)

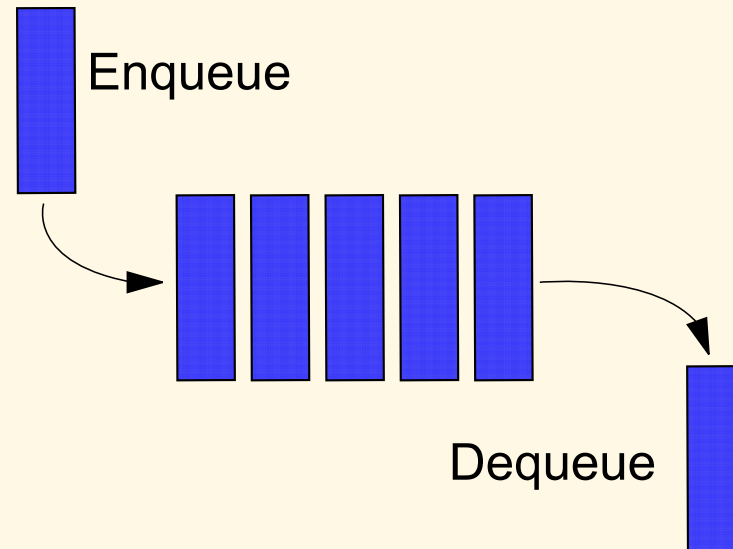
Separation of interface and implementation

Overview

- ▶ Motivation:
 - ▶ A problem can have different methods/mechanisms in implementation, but they share a same or similar usage
 - ▶ Reducing implementation detail exposed to the user
 - ▶ Reducing physical coupling
- ▶ C & C++ languages allow to separate the interface (.h files) from its implementation (.c/.cpp files)

Queues

What's a queue?



- ▶ Linear data structure which can only be accessed at its two ends
- ▶ FIFO (first in, first out) basis
- ▶ Two ways of implementation:
 - ▶ Using array (static or dynamic)
 - ▶ Using linked list (dynamic)

Operations

- ▶ `create()`: creates a queue
- ▶ `destroy()`: destroys a queue
- ▶ `enqueue()`: inserts an element
- ▶ `dequeue()`: removes first inserted element (optionally returns the element)
- ▶ `top()`: returns the first inserted element without removing it
- ▶ `isEmpty()`: checks if the queue is empty
- ▶ `isFull()`: checks if the queue is full

Static implementation

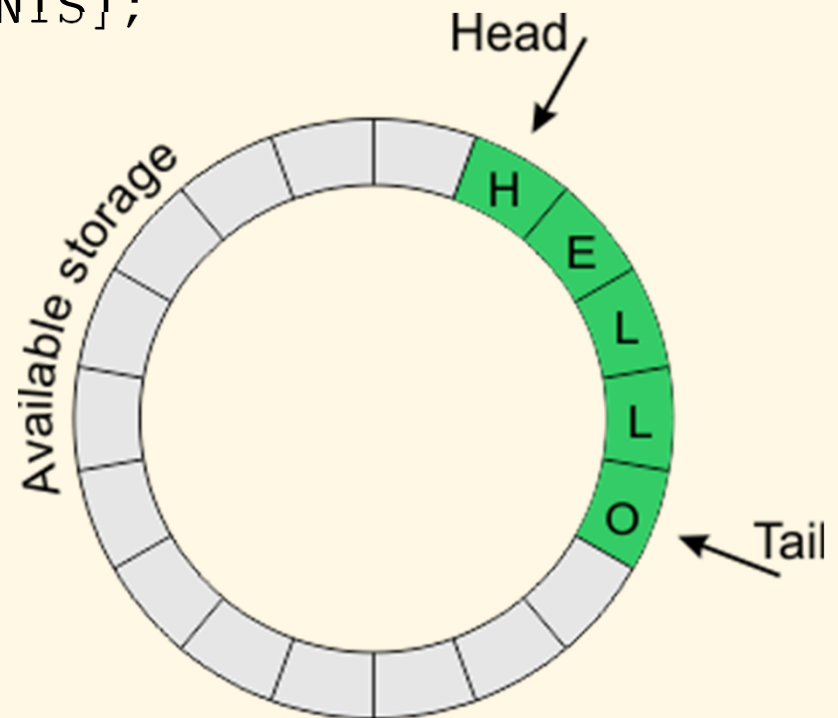
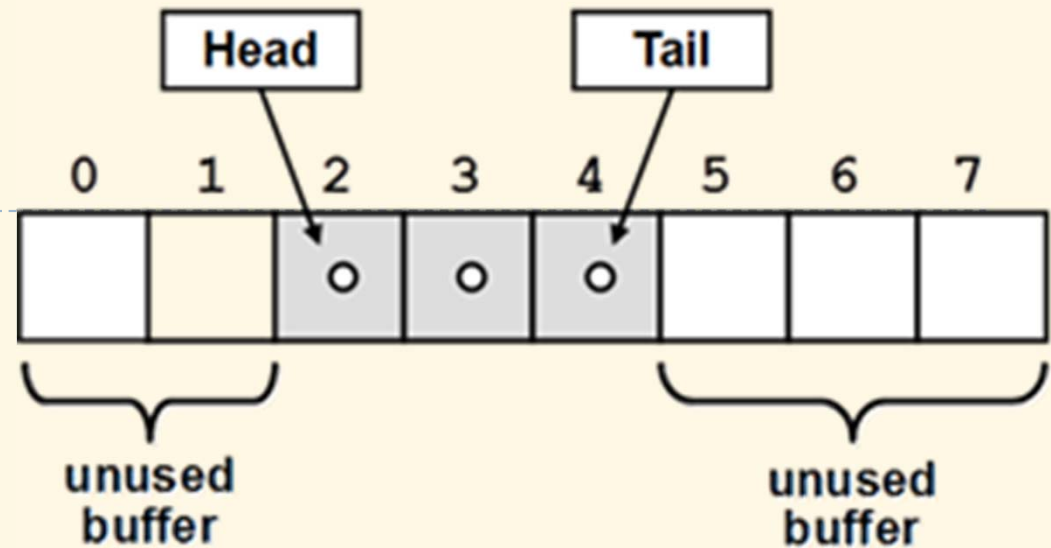
► Definitions

```
typedef int QUEUE_DATA;  
#define MAX_ELEMENTS 20
```

```
typedef struct {  
    QUEUE_DATA elements[MAX_ELEMENTS];  
    int head;  
    int count;  
} QUEUE;
```

► Initialization

```
void qInit(QUEUE* s) {  
    s->head = 0;  
    s->count = 0;  
}
```



Enqueue

enqueue 7
enqueue 42
enqueue 18
enqueue 99

7	42	18	99				
---	----	----	----	--	--	--	--

Start = 0

End = 3

```
void qEnqueue(Queue* s, Queue_Data data) {  
    if (qIsFull(s)) return;  
  
    int i = (s->head + s->count) % MAX_ELEMENTS;  
    s->elements[i] = data;  
  
    s->count++;  
}
```

Dequeue

dequeue
dequeue

7	42	18	99				
---	----	----	----	--	--	--	--

Start = 2

End = 3

```
void qDequeue(QUEUE* s) {  
    if (qIsEmpty(s)) return;  
  
    s->count--;  
  
    if (++ s->head == MAX_ELEMENTS)  
        s->head = 0;  
}
```

Get first value

```
QUEUE_DATA qHead(QUEUE* s) {  
    return s->elements[s->head];  
}
```


Is empty? Is full?

```
int qIsEmpty(QUEUE* s) {  
    return s->count == 0;  
}
```

```
int qIsFull(QUEUE* s) {  
    return s->count == MAX_ELEMENTS;  
}
```

Clear all

```
void qDestroy(QUEUE* s) {  
    s->head = 0;  
    s->count = 0;  
}
```

Dynamic implementation

► Definitions

```
typedef int QUEUE_DATA;
```

```
struct _QUEUE_ELEM;
```

```
typedef struct _QUEUE_ELEM QUEUE_ELEM;
```

```
struct _QUEUE_ELEM {  
    QUEUE_DATA data;  
    QUEUE_ELEM* next;  
};
```

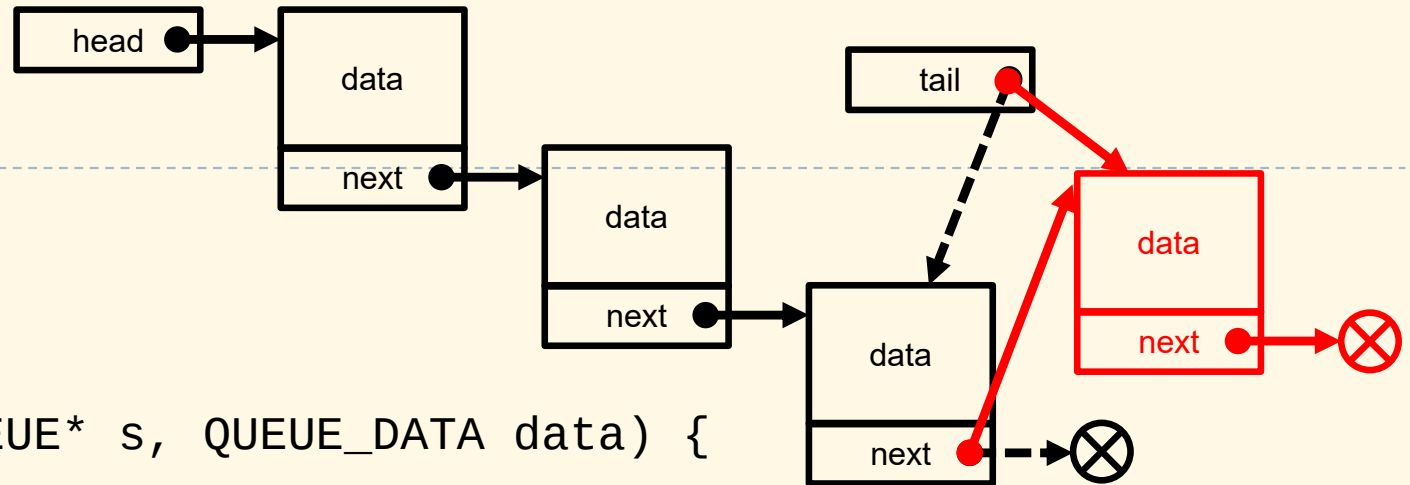
```
typedef struct {  
    QUEUE_ELEM* head;  
    QUEUE_ELEM* tail;  
} QUEUE;
```



Initialization

```
void stInit(QUEUE* s) {  
    s->head = s->tail = NULL;  
}
```

Enqueue



```
void qEnqueue(Queue* s, Queue_Data data) {  
    if (qIsFull(s)) return;
```

```
    Queue_Elem* p = (Queue_Elem*)malloc(  
        sizeof(Queue_Elem));
```

```
    p->data = data;
```

```
    p->next = NULL;
```

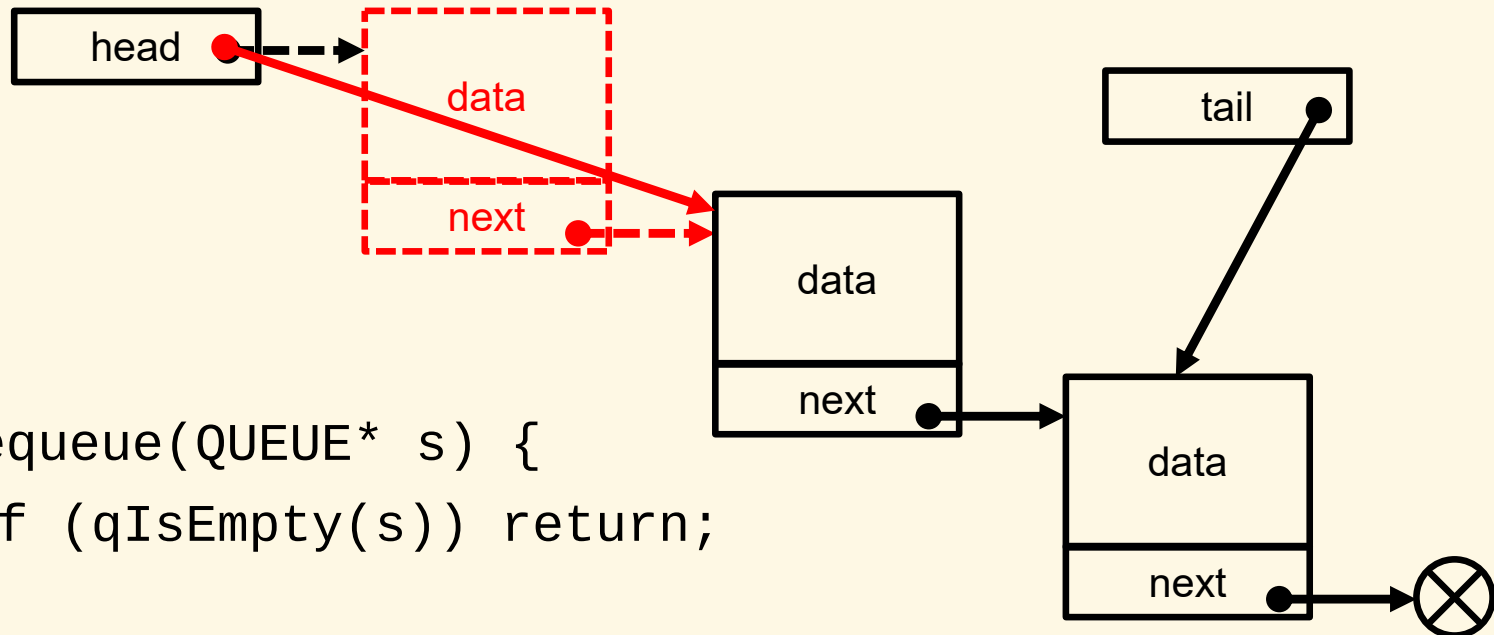
```
    if (s->tail) s->tail->next = p;
```

```
    s->tail = p;
```

```
    if (!s->head) s->head = p;
```

```
}
```

Deque



```
void qDequeue(Queue* s) {  
    if (qIsEmpty(s)) return;  
  
    QueueElem* p = s->head;  
    s->head = p->next;  
    free(p);  
  
    if (!s->head) s->tail = NULL;  
}
```

Get first value

```
QUEUE_DATA qHead(QUEUE* s) {  
    return s->head->data;  
}
```

Is empty? Is full?

```
int qIsEmpty(QUEUE* s) {  
    return s->head == NULL;  
}
```

```
int qIsFull(QUEUE* s) {  
    return 0;  
}
```


Clear all

```
void qDestroy(QUEUE* s) {  
    while (s->head) {  
        QUEUE_ELEM* p = s->head;  
        s->head = p->next;  
        free(p);  
    }  
  
    s->head = NULL;  
    s->tail = NULL;  
}
```

Applications of queues

- ▶ Data transfer and communication buffer
- ▶ Reading/writing buffer
- ▶ Implementation of client/server services

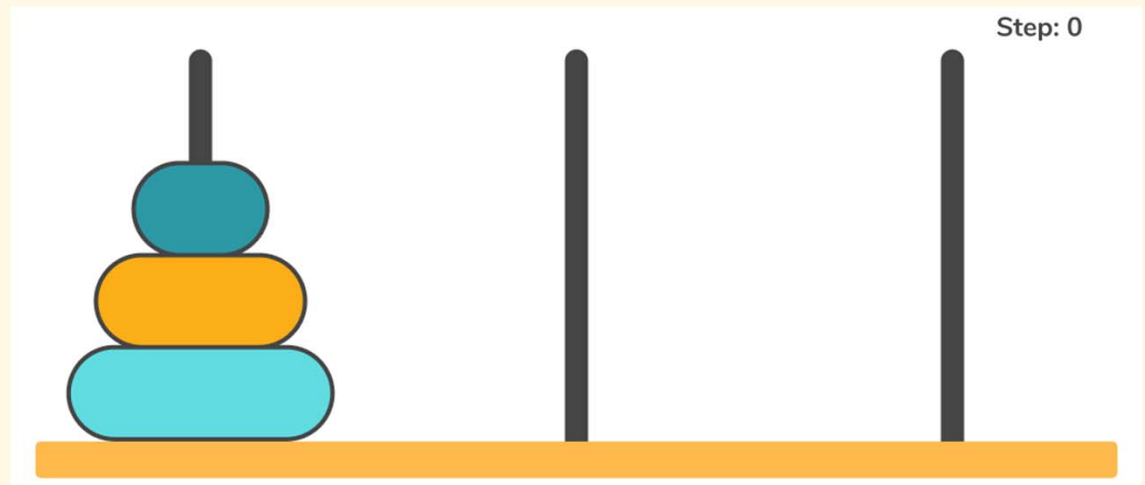
Priority Queues

Priority queue

- ▶ A type of queue that arranges elements based on their priority values
 - ▶ Elements with higher priority values are typically retrieved before elements with lower priority values
 - ▶ The priority value is usually stored in a member variable, but can also be a function of the element data
- ▶ Several implementation methods:
 - ▶ Using an unsorted array or linked list
 - ▶ Using a sorted array or linked list
 - ▶ Using a heap
 - ▶ Using a binary search tree

Exercises: Stack

1. Write a program which reads integers and prints them in the normal and reversed orders.
2. Implement the reverse Polish (postfix) notation for simple expression evaluation.
3. The Tower of Hanoi is a puzzle consisting of 3 rods and n disks. The task is to move all disks to another rod, but:
 - ▶ Only the uppermost disk can be moved
 - ▶ Larger disks cannot be placed on top of smaller disks



Write a function to solve the problem with help of a stack, given the number of disks n .

Exercises: Queue

1. Write a program to simulate a task producer-consumer work queue.
2. Implement a priority queue with help of a static array.
3. Implement a priority queue with help of a sorted linked list.

Sorting

- ☐ Bubble sort
- ☐ Insertion sort
- ☐ Selection sort
- ☐ Heap sort
- ☐ Merge sort
- ☐ Quicksort

Overview

- ▶ Arranging data in a certain order
- ▶ Purposes:
 - ▶ Visualization
 - ▶ Searching
- ▶ Strategies to sort data
 - ▶ Let's imagine how we arrange cards or documents in order
- ▶ How to judge the efficiency of a sorting algorithm?
 - ▶ Time taken
 - ▶ Memory used
 - ▶ Data stored in an array or a linked list

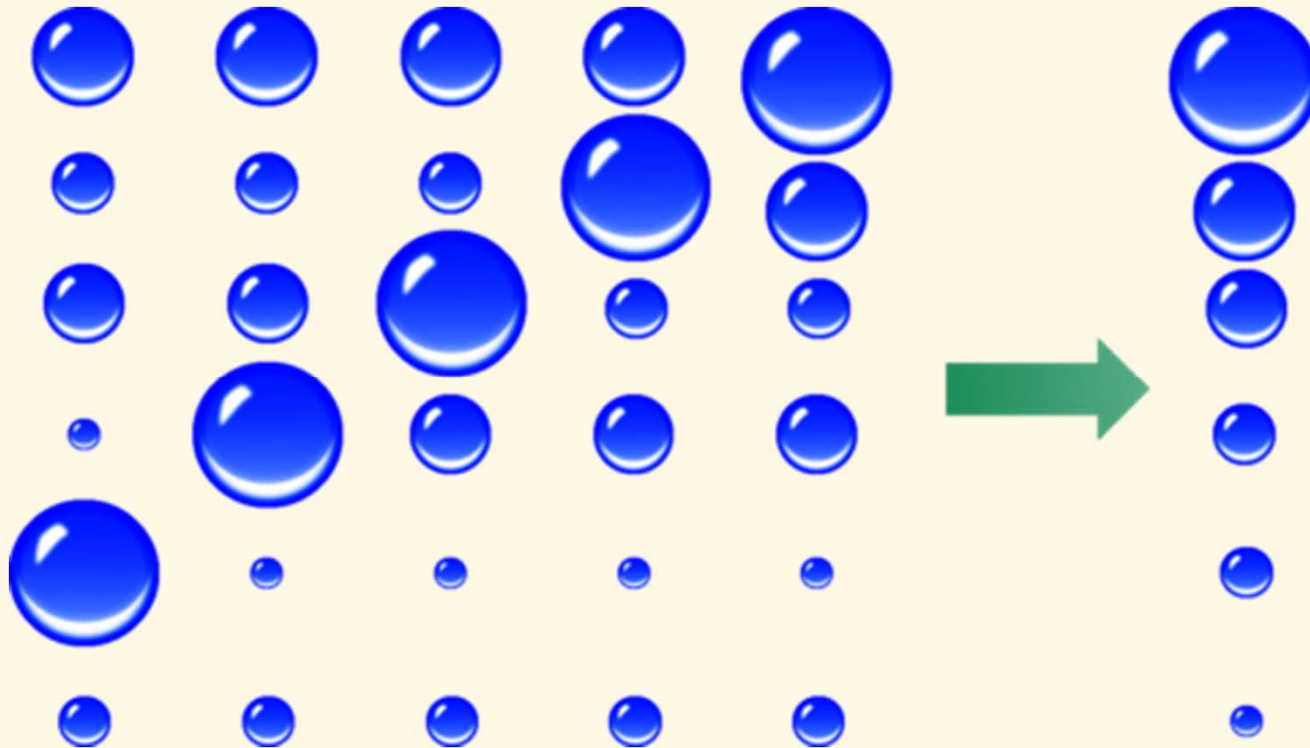


Bubble sort

Overview

- ▶ Principle

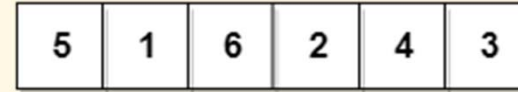
- ▶ Bigger elements bubble toward the end of the list



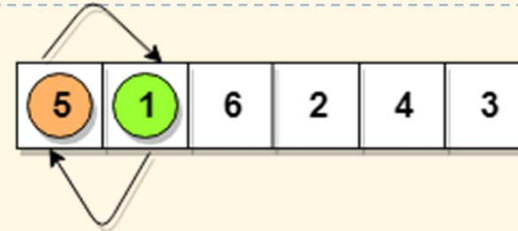
Algorithm

- ▶ Repeat until the array is sorted:
 - ▶ Step through the array
 - ▶ Compare each pair of **adjacent items**
 - ▶ Swap them if they are in the wrong order

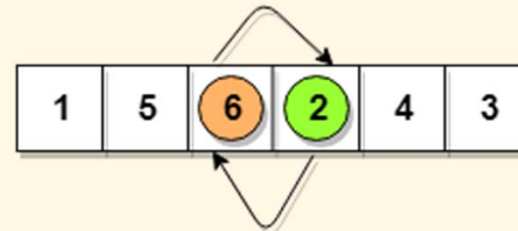
$5 > 1$
so interchange



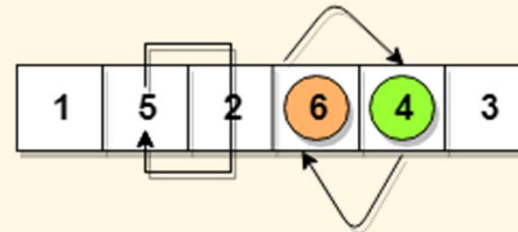
$5 < 6$
No swapping



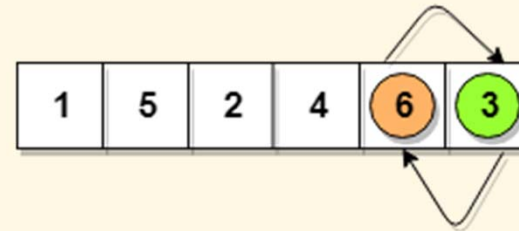
$6 > 2$
so interchange



$6 > 4$
so interchange



$6 > 3$
so interchange



This is
first
iteration

Do
similarly
for next
iterations
until the
array is
sorted

Naïve version (without termination checking)

```
void sortBubbleSimple(int* a, int n) {  
    int i, j;  
    for (i = n; i > 0; i--) {  
        for (j = 0; j < i - 1; j++) {  
            if (a[j] > a[j + 1])  
                swap(a, j, j + 1);  
        }  
    }  
}
```

With termination checking

```
void sortBubbleImproved(int* a, int n) {  
    int i, j, sorted = 0;  
    for (i = n; !sorted && i > 0; i--) {  
        sorted = 1;  
        for (j = 0; j < i - 1; j++) {  
            if (a[j] > a[j + 1]) {  
                swap(a, j, j + 1);  
                sorted = 0;  
            }  
        }  
    }  
}
```

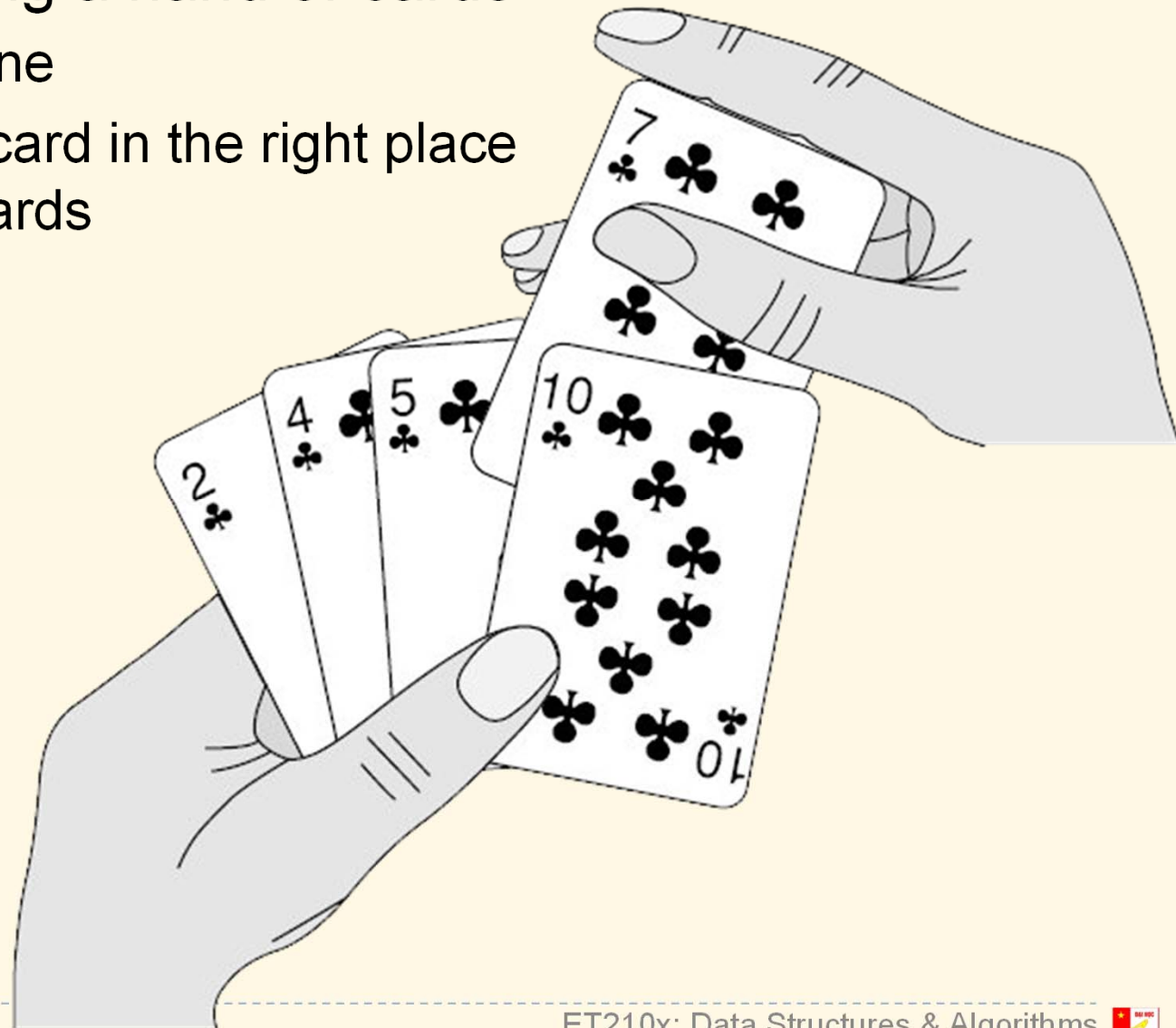
Complexity

- ▶ Computation:
 - ▶ Worst case: when array is in reversed order
 - ▶ Number of comparisons = no. of swaps = $n(n - 1)/2$
 - ▶ Complexity: $O(n^2)$
 - ▶ Average case: $O(n^2)$
 - ▶ Best case: when the array is already sorted
 - ▶ $O(n^2)$ for simple algorithm
 - ▶ $O(n)$ for improved algorithm
- ▶ Memory consumption: $O(1)$
- ▶ Generally not suitable in real applications
 - ▶ Good use cases
 - ▶ Data is almost sorted
 - ▶ Number of elements is small
- ▶ Can be implemented recursively

Insertion sort

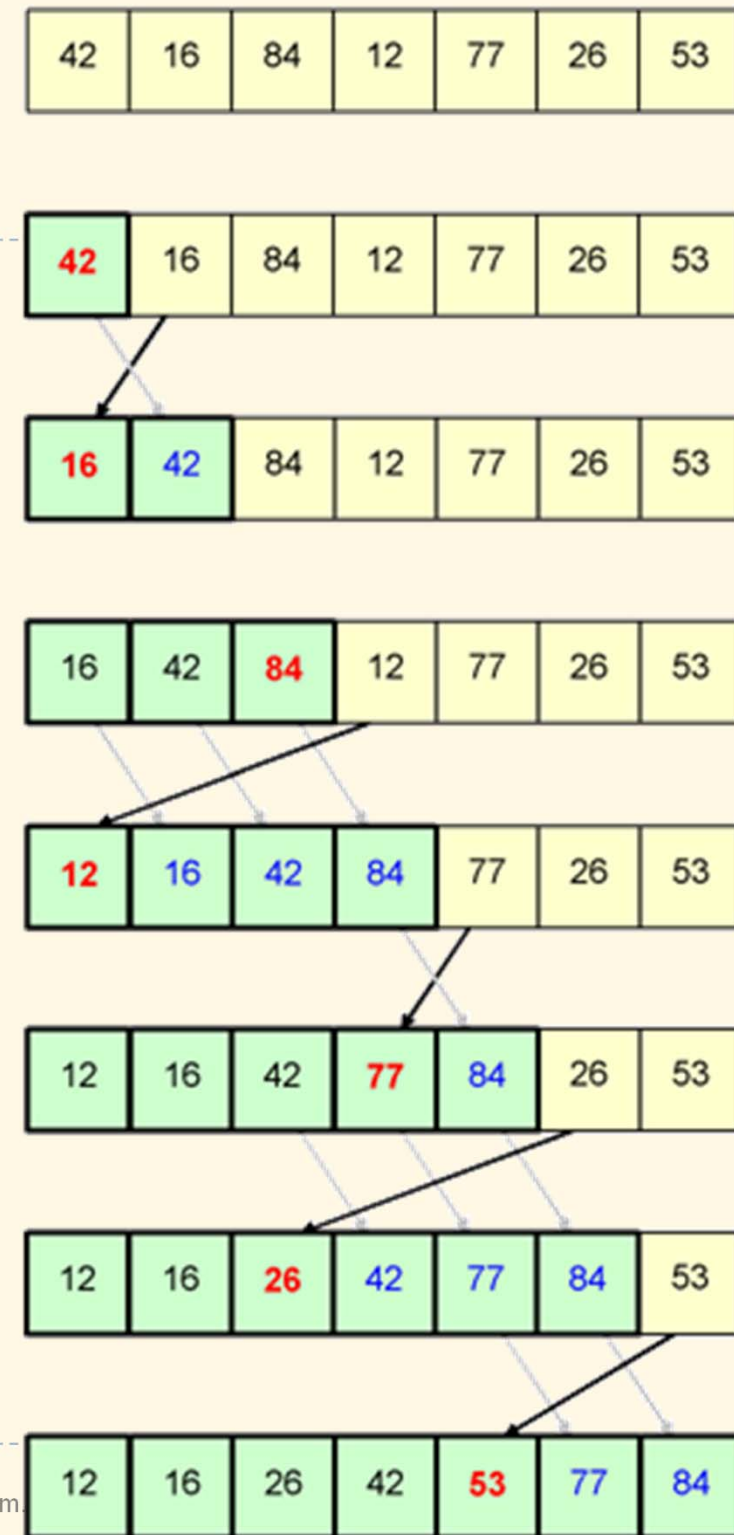
Overview

- ▶ Principle: Like sorting a hand of cards
 - ▶ Pick cards one by one
 - ▶ Each time, insert a card in the right place among the sorted cards



Algorithm

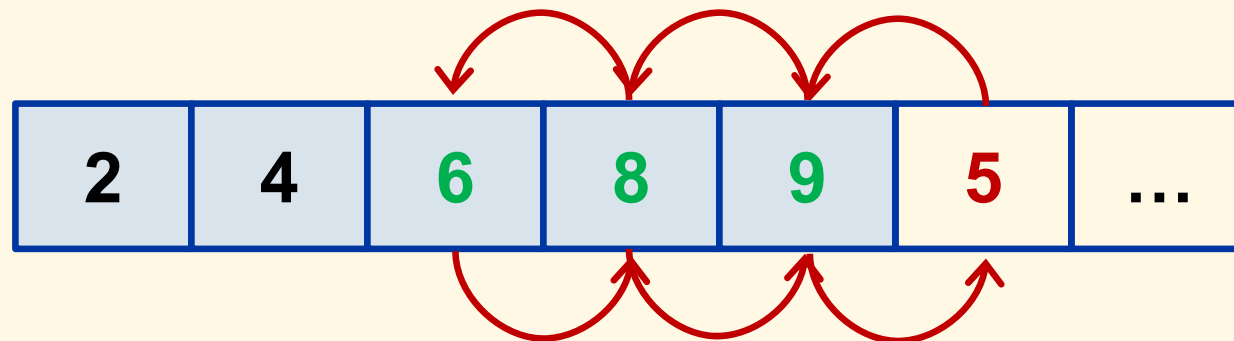
- ▶ Go through the array from left to right:
 - ▶ Pick the first element in the **unsorted part** (right part)
 - ▶ Find the right place for that element in the **sorted part** (left part)
 - ▶ Move (insert) the element to the found place
 - ▶ Expand the sorted part by 1 element, and shrink the unsorted part
- ▶ Attn: After the i^{th} iteration, the i left most elements are sorted part



Simple version

```
void sortInsertionSimple(int* a, int n) {  
    int i, j;  
    for (i = 1; i < n; i++) {  
        for (j = i; j > 0 && a[j] < a[j-1]; j--)  
            swap(a, j, j-1);  
    }  
}
```

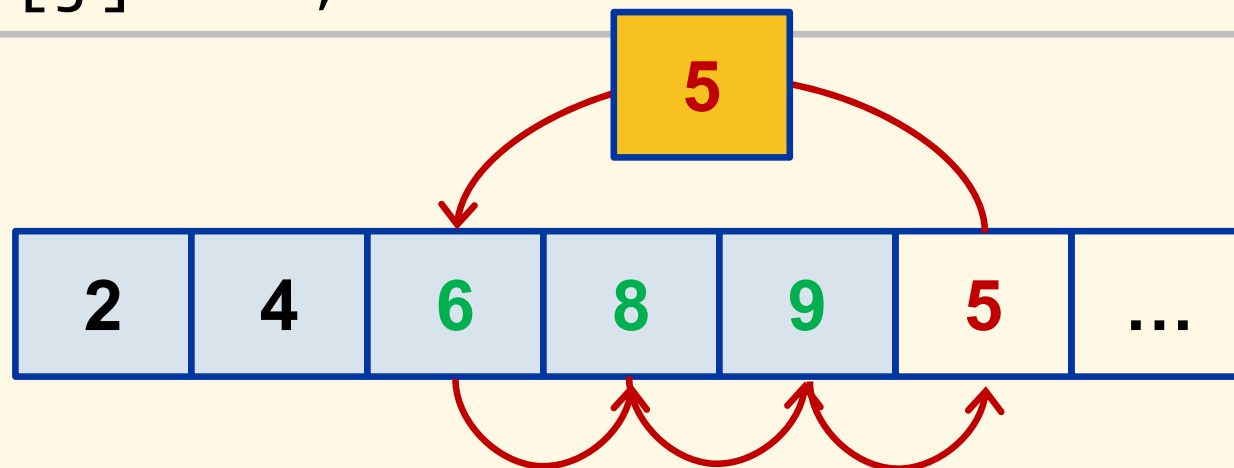
insertion



Better version

```
void sortInsertionImproved(int* a, int n) {  
    int i, j, t;  
    for (i = 1; i < n; i++) {  
        t = a[i];  
        for (j = i; j > 0 && t < a[j-1]; j--)  
            a[j] = a[j-1];  
        a[j] = t;  
    }  
}
```

insertion



Complexity

- ▶ Computation:
 - ▶ Worst case: when array is in reversed order
 - ▶ Number of comparisons = no. of moves = $n(n - 1)/2$
 - ▶ Complexity: $O(n^2)$
 - ▶ Average case: $O(n^2)$
 - ▶ Best case: when the array is already sorted
 - ▶ $O(n)$ for simple algorithm
 - ▶ $O(n)$ for improved algorithm
- ▶ Memory consumption: $O(1)$
- ▶ Can be implemented recursively

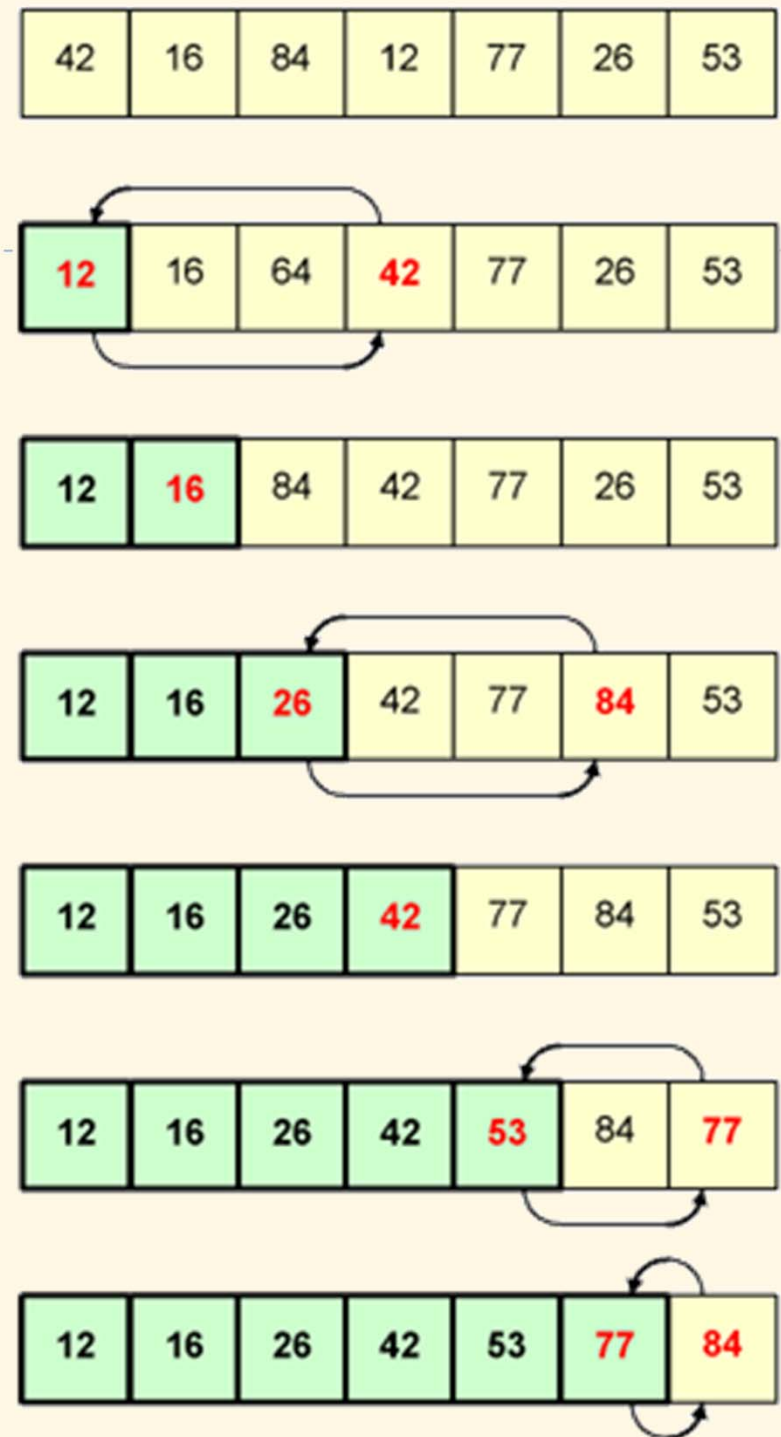
Selection sort

Overview

- ▶ For insertion sort, we pick the element first, then find the place to put it
- ▶ For selection sort, we do in the reverse way: find the place first, then find the element to put there

Algorithm

- ▶ Go from left to right of the array
 - ▶ Pick the **smallest element** from the **unsorted part**
 - ▶ Swap it with the first element in the unsorted part
 - ▶ Expand the **sorted part** by one
- ▶ Attn: After the i^{th} iteration, the i left most elements are sorted part



Implementation

```
void sortSelectionSimple(int* a, int n) {  
    int i, j;  
    for (i = 0; i < n-1; i++) {  
        for (j = i+1; j < n; j++) {  
            if (a[i] > a[j])  
                swap(a, i, j);  
        }  
    }  
}
```


Implementation

```
void sortSelectionImproved(int* a, int n) {
    int i, j, mi;
    for (i = 0; i < n-1; i++) {
        for (mi = i, j = i+1; j < n; j++) {
            if (a[j] < a[mi]) mi = j;
        }

        if (mi != i) swap(a, i, mi);
    }
}
```

Complexity

- ▶ Computation:
 - ▶ Worst case: when array is in reversed order
 - ▶ Number of comparisons: $n(n - 1)/2$
 - ▶ Number of swaps: n
 - ▶ Complexity: $O(n^2)$
 - ▶ Average case: $O(n^2)$
 - ▶ Best case: $O(n^2)$ when the array is already sorted
- ▶ Memory consumption: $O(1)$
- ▶ Can be implemented recursively

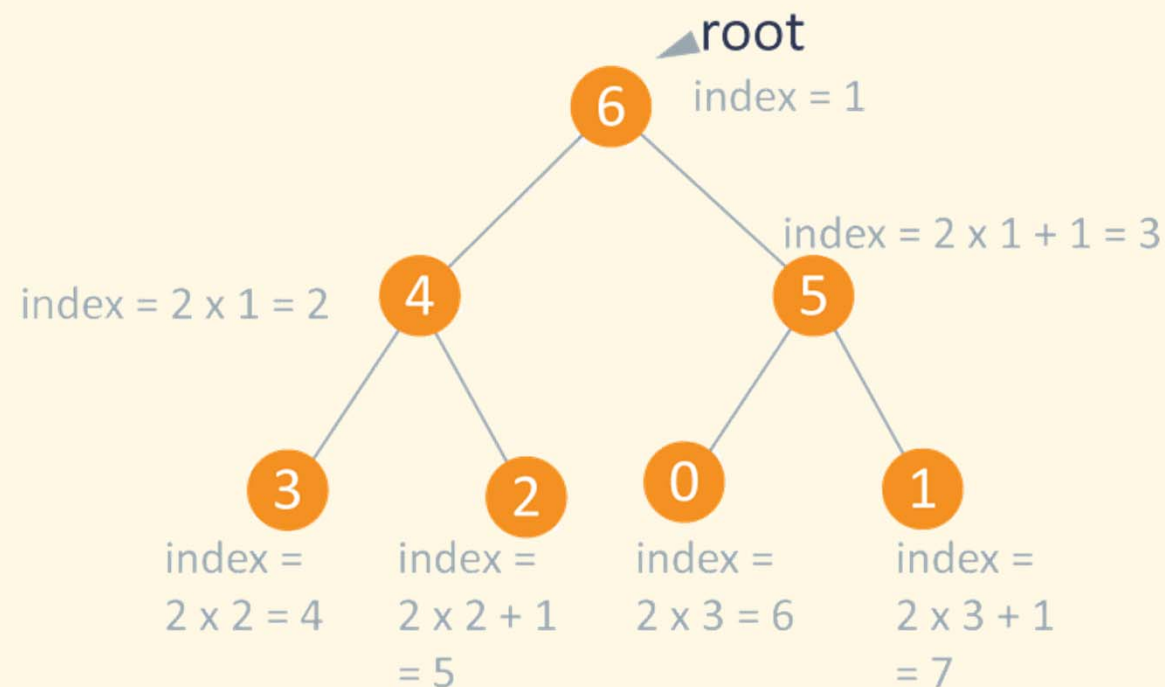
Exercises

- ▶ Rewrite the bubble, insertion, selection sort algorithms as recursive functions

Heap sort

Overview

- ▶ Based on a binary heap that helps to easily find the max (or min) elements
- ▶ Similar principle to Selection Sort, but the way to find the max elements is different
- ▶ Binary heap:
 - ▶ A complete binary tree
 - ▶ Two types: min heap, max heap
 - ▶ Value of each node in a max heap is greater than its children's
- ▶ No additional tree is needed since the tree is maintained by the array itself



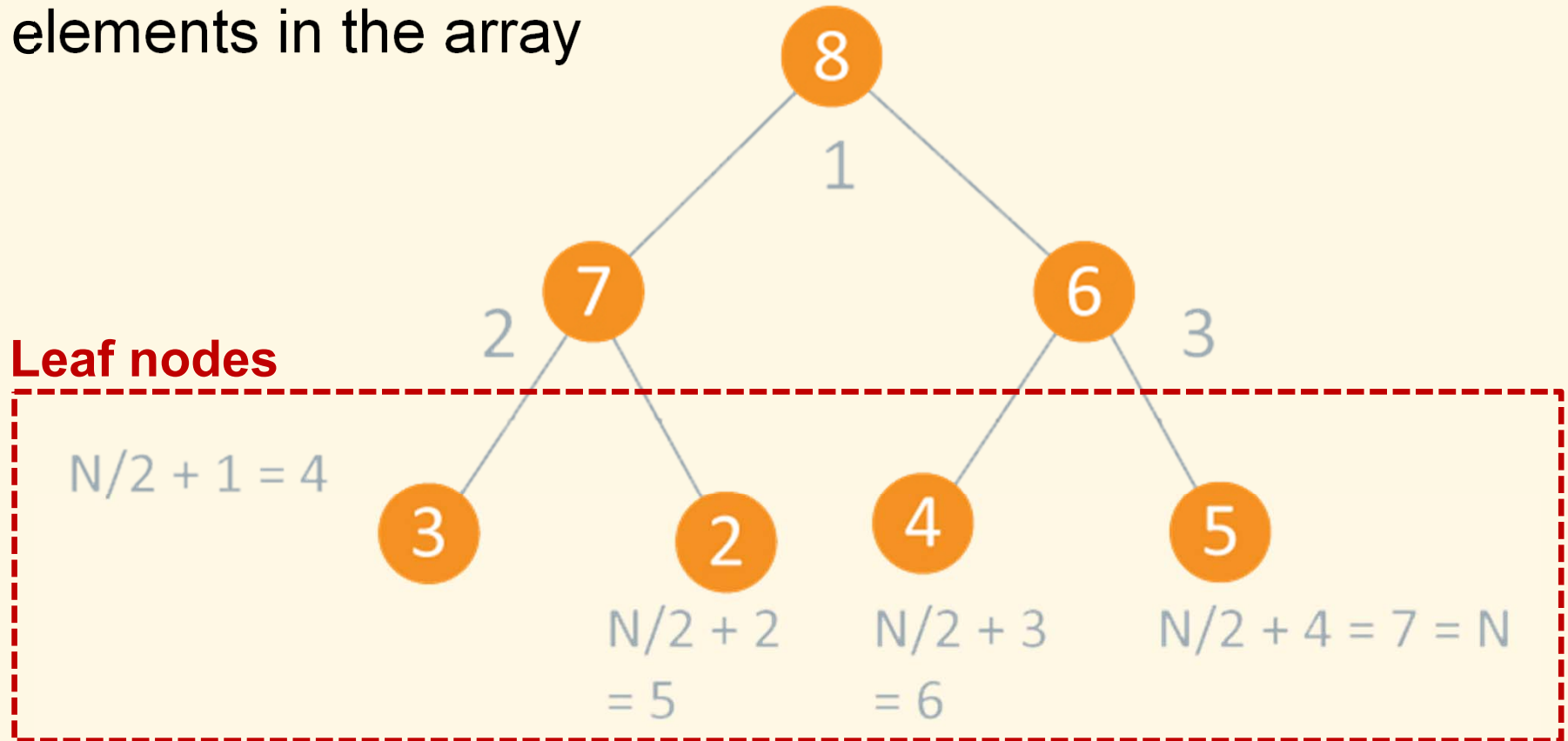
Arr	6	4	5	3	2	0	1
	1	2	3	4	5	6	7

Algorithm

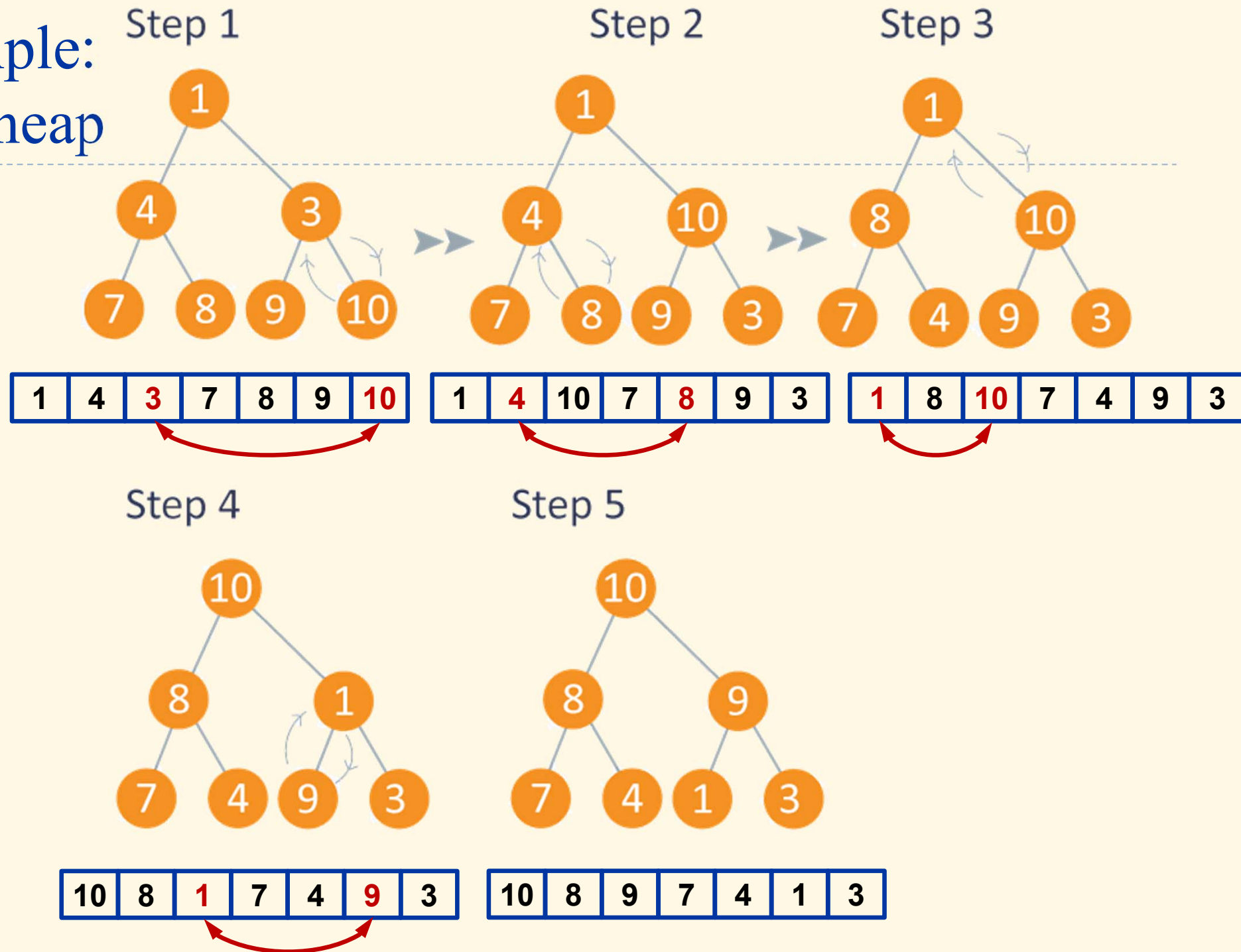
- ▶ Build a binary max heap
- ▶ Repeat until the array is sorted:
 - ▶ Swap the max (first) and the last elements in the array
 - ▶ Reduce the data size by 1
 - ▶ Correct the heap because of the root node's new value
- ▶ Attn:
 - ▶ In a max heap, the root element has the max value
 - ▶ The first element in the array corresponds to the root in the tree

Building max heap

- ▶ For each node, make sure that its children are smaller
- ▶ Check from lower nodes up to the root
- ▶ No need to check leaf nodes → Only check the first $n/2$ elements in the array

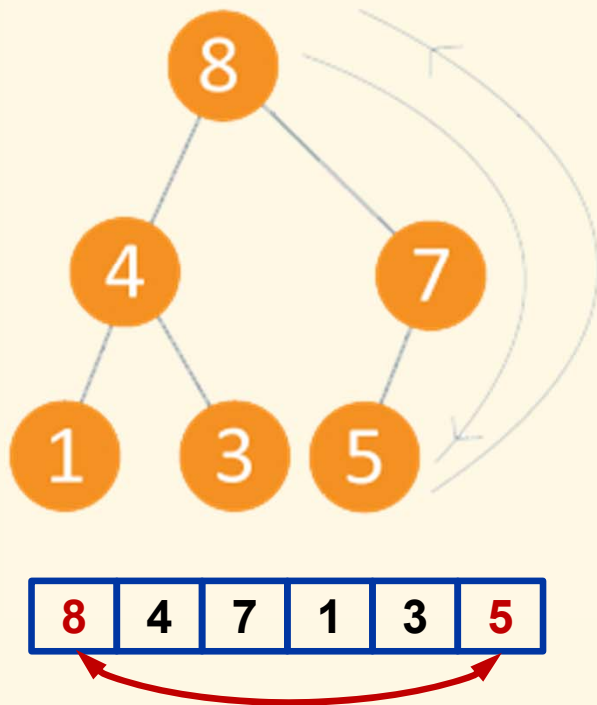


Example: Max heap

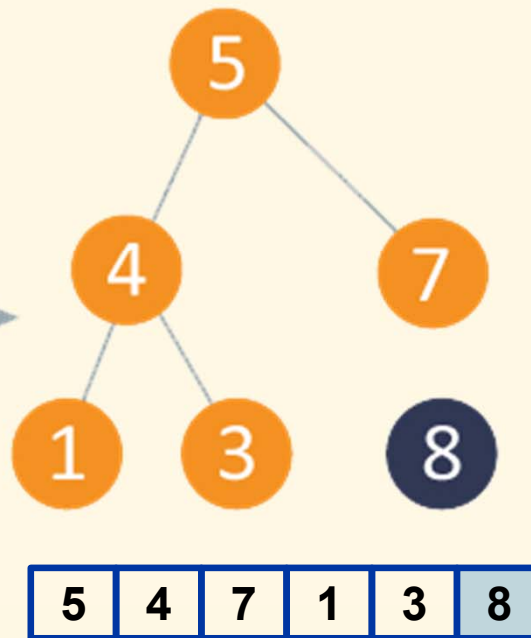


Example: Heap sort

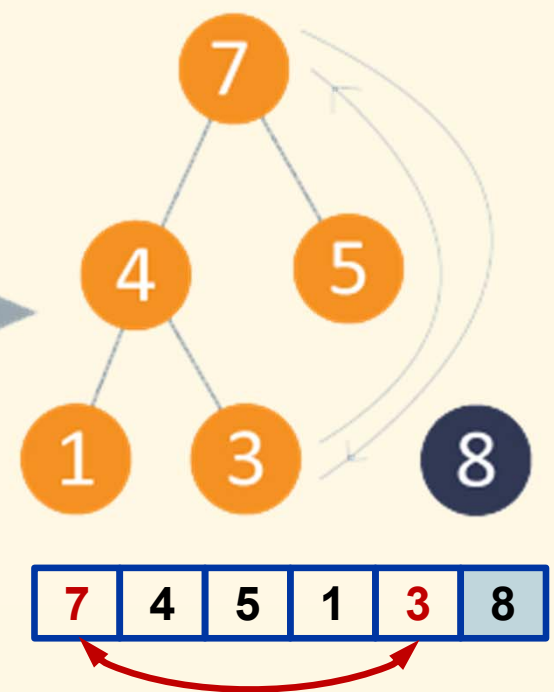
Step 1: Max heap



Step 2: Arrange

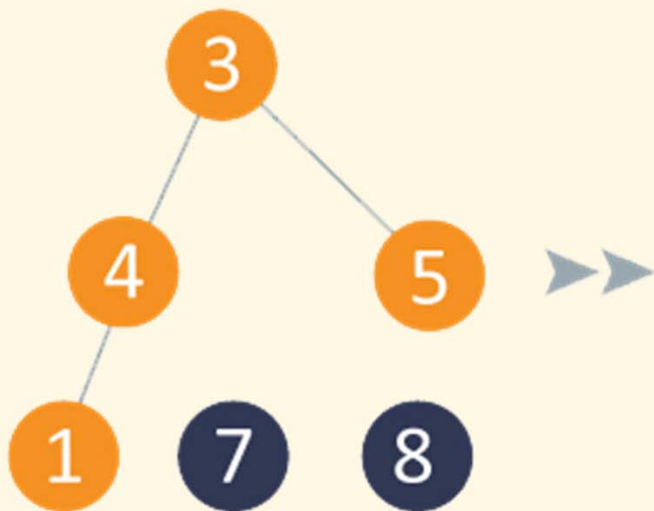


Step 3: Max heap

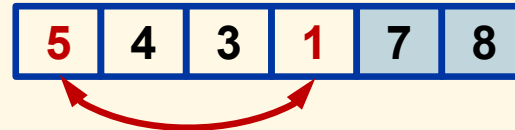
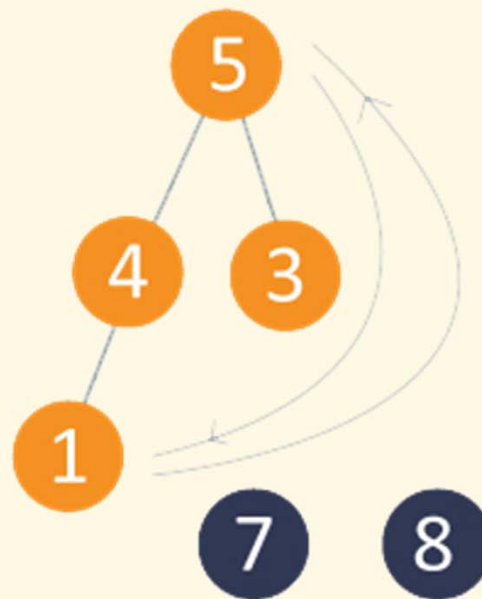


Example: Heap sort (*cont'd*)

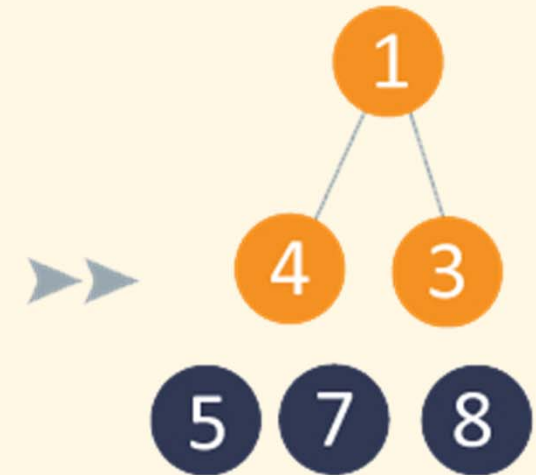
Step 4: Arrange



Step 5: Max heap

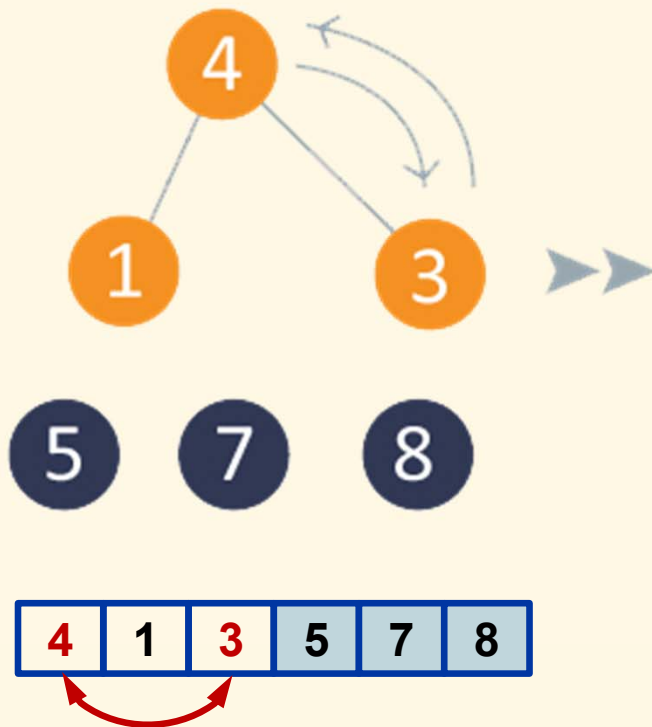


Step 6: Arrange

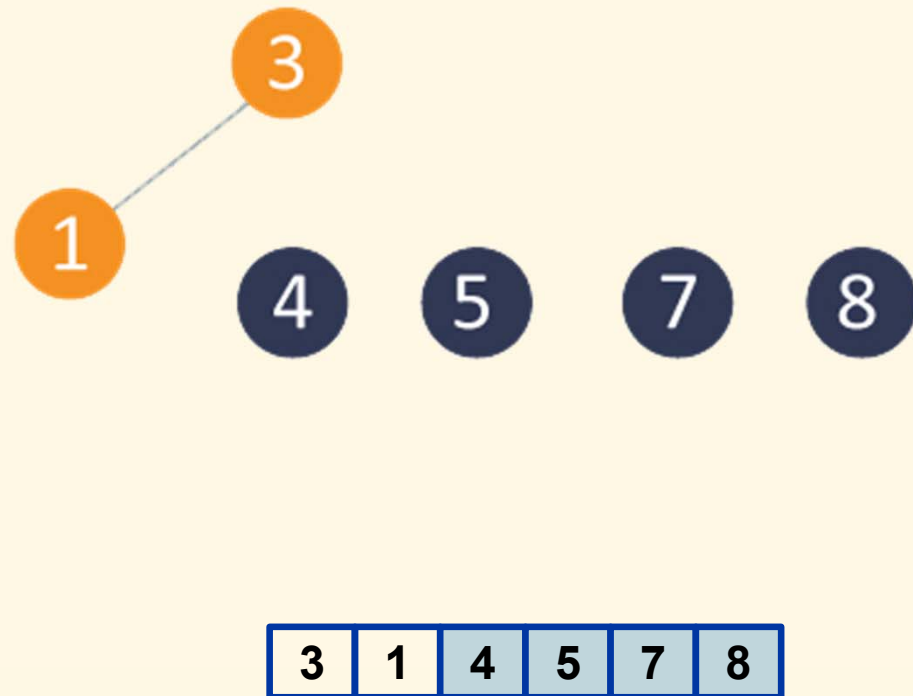


Example: Heap sort (*cont'd*)

Step 7: Max heap

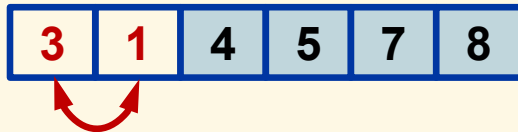


Step 8: Arrange



Example: Heap sort (*cont'd*)

Step 9: Max heap



Step 10: Arrange



Implementation: Correct heap node

```
void correctHeapNode(int* a, int n, int i) {
    int l = i*2 + 1,
        r = l + 1,
        m = i;
    if (l < n && a[l] > a[m]) m = l;
    if (r < n && a[r] > a[m]) m = r;

    if (m != i) {
        swap(a, i, m);
        correctHeapNode(a, n, m);
    }
}
```

Implementation: Max heap

```
void buildHeap(int* a, int n) {  
    for (int i = n/2; i >= 0; i--)  
        correctHeapNode(a, n, i);  
}
```

Implementation: Heap sort

```
void sortHeap(int* a, int n) {  
    buildHeap(a, n);  
  
    for (int i = n; i > 1; i--) {  
        swap(a, 0, i-1);  
        correctHeapNode(a, i-1, 0);  
    }  
}
```

Complexity

▶ Computation

- ▶ Correct node: $O(\log n)$
- ▶ Build heap: $O(n \log n)$
- ▶ Main loop: $O(n \log n)$
- ▶ Overall: $O(n \log n)$

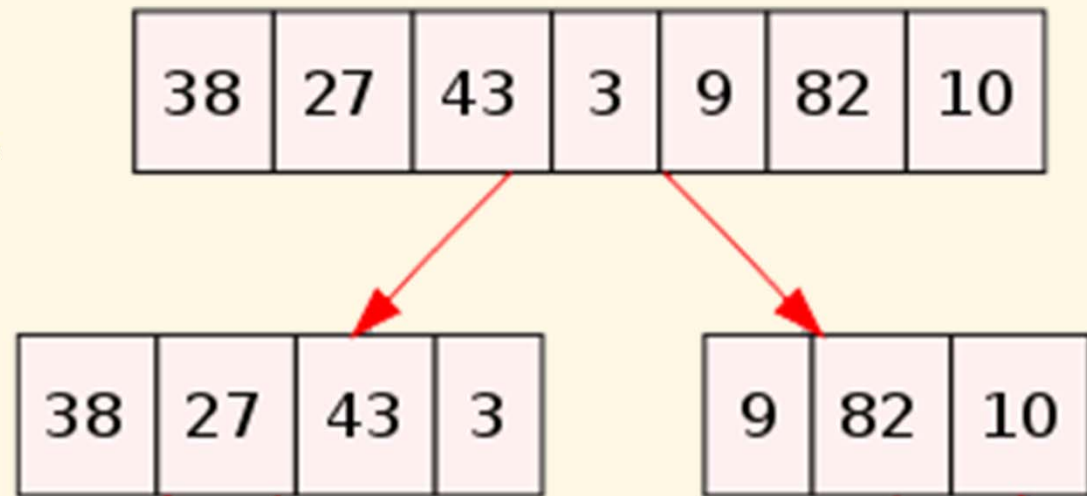
▶ Memory (stack)

- ▶ Correct node: $O(\log n)$
- ▶ Build heap: $O(\log n)$
- ▶ Main loop: $O(\log n)$
- ▶ Overall: $O(\log n)$

Merge sort

Overview

- ▶ Initial idea:
 - ▶ Bubble, insertion, selection sorts all have complexity of $O(n^2)$
 - ➔ The computational time grows quickly when n is large
 - ➔ Subdivide the problem into smaller tasks
- ▶ Divide and conquer approach (recursive implementation)
 - ▶ Divide into two parts
 - ▶ Sort each parts separately
 - ▶ Merge the two sorted parts



Algorithm

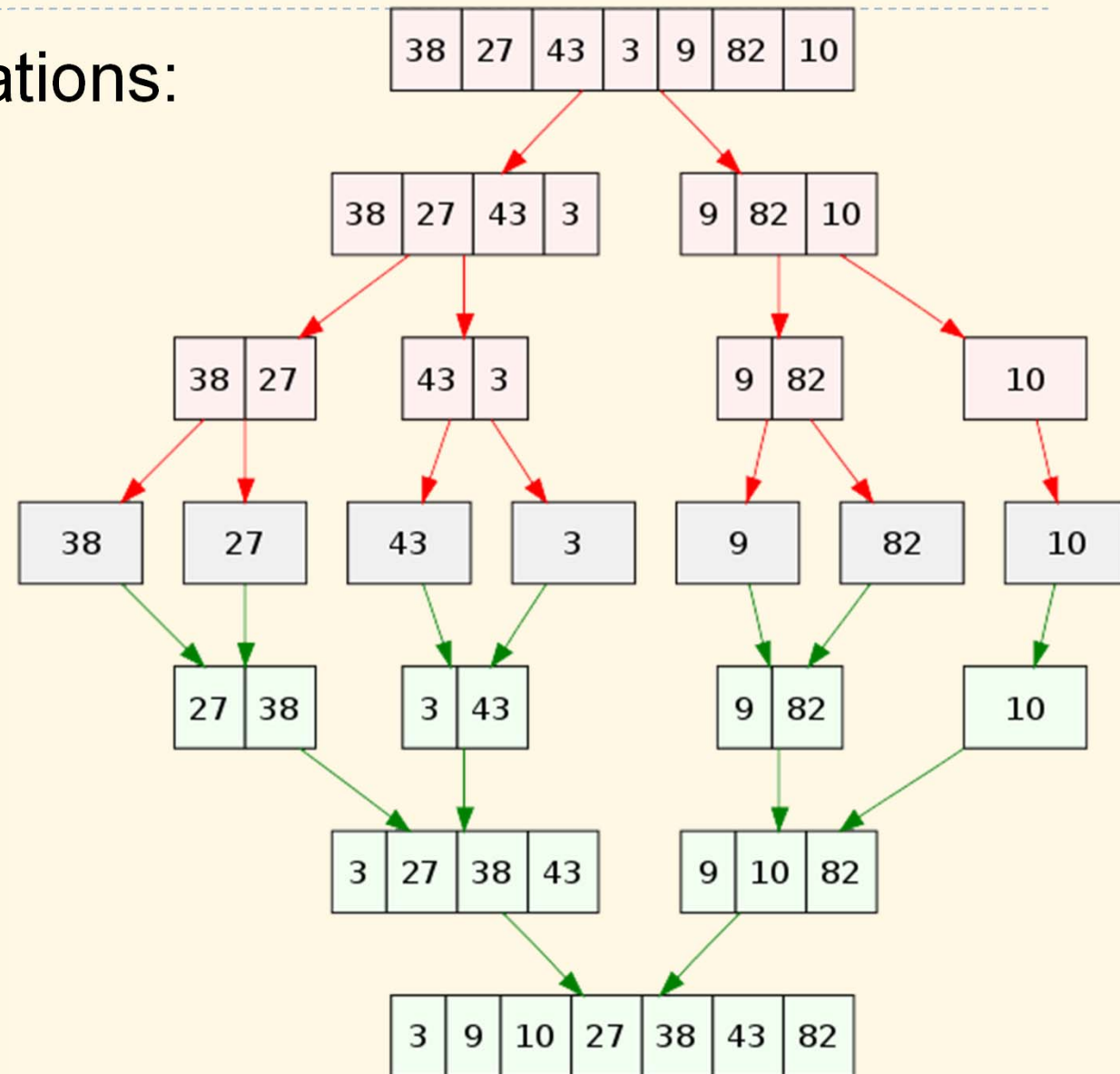
- ▶ Based on two operations:

- ▶ Division

- ▶ Divide continuously until the arrays become elementary (with 1 item)

- ▶ Merging

- ▶ Combine two sorted arrays



Implementation

```
void sortMergePart(int* a, int l, int r) {  
    int m = (l + r)/2;  
    if (m > l) sortMergePart(a, l, m);  
    if (m + 1 < r) sortMergePart(a, m + 1, r);  
    mergePartsXXX(a, l, m, r);  
}
```

```
void sortMerge(int* a, int n) {  
    sortMergePart(a, 0, n-1);  
}
```

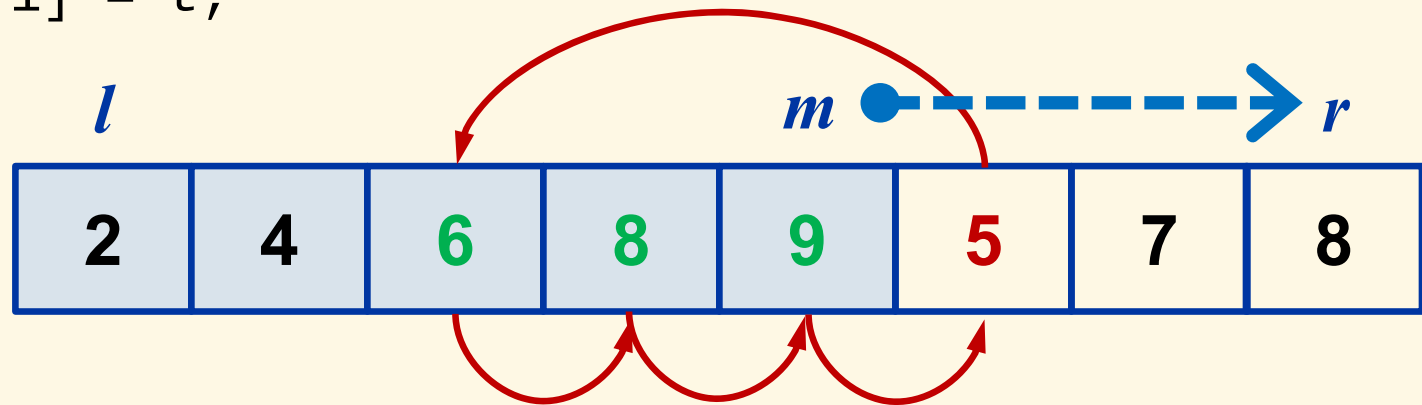
Merge arrays using auxiliary space (standard)

```
void mergeParts_aux(int* a, int l, int m, int r) {
    int pn = m - l + 1, qn = r - m;
    int *p = (int*)malloc(pn * sizeof(int)),
        *q = (int*)malloc(qn * sizeof(int));
    memcpy(p, a + l, pn * sizeof(int));
    memcpy(q, a + m + 1, qn * sizeof(int));
    for (int i = 0, j = 0; i < pn || j < qn; ) {
        if ((i == pn) || (j < qn && p[i] > q[j])) {
            a[l + i + j] = q[j];
            j++;
        } else {
            a[l + i + j] = p[i];
            i++;
        }
    }
    free(p);
    free(q);
}
```

Merge arrays without auxiliary space (in place)

- ▶ Since the two arrays are consecutive

```
void mergeParts_inPlace(int* a, int l, int m, int r) {  
    int i, t;  
    for (; m < r; m++) {  
        t = a[m+1];  
        for (i = m; i >= l && a[i] > t; i--)  
            a[i+1] = a[i];  
        a[i+1] = t;  
    }  
}
```



Complexity

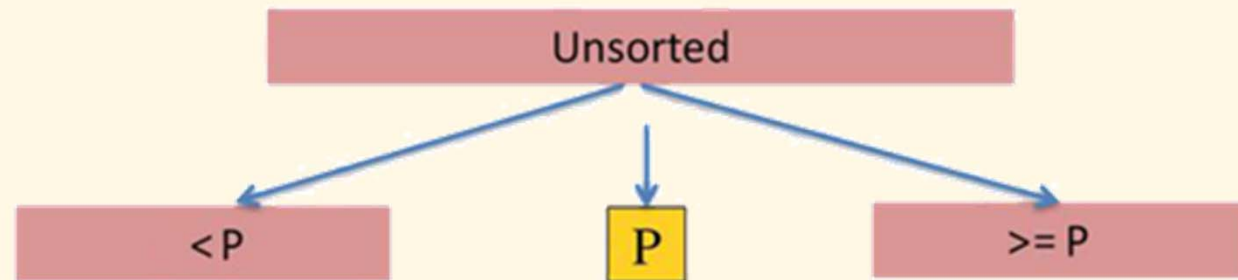
Complexity	Standard method	In-place method
Computation	$O(n \log n)$	$O(n^2 \log n)$
- Division	$O(\log n)$	$O(\log n)$
- Merging	$O(n)$	$O(n^2)$
Memory	$O(n)$	$O(\log n)$
- Division (stack)	$O(\log n)$	$O(\log n)$
- Merging (heap)	$O(n)$	$O(1)$

Quicksort

Overview

- ▶ Divide and conquer approach
- ▶ Principle:
 - ▶ Partition the data into two smaller sets using a threshold value (called pivot)
 - ▶ Do the same with the two subsets

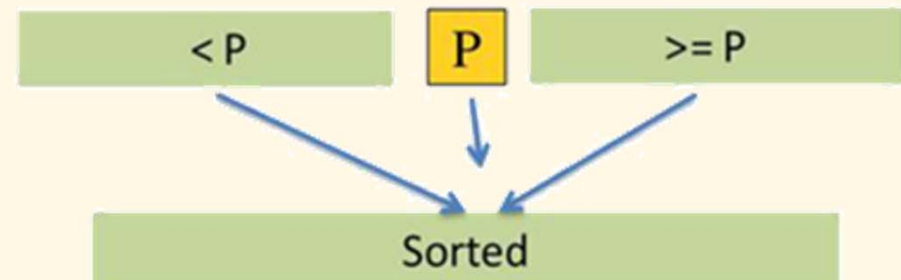
Divide: Pick a pivot, partition into groups



Conquer: Return array when length ≤ 1



Combine: Combine sorted partitions and pivot

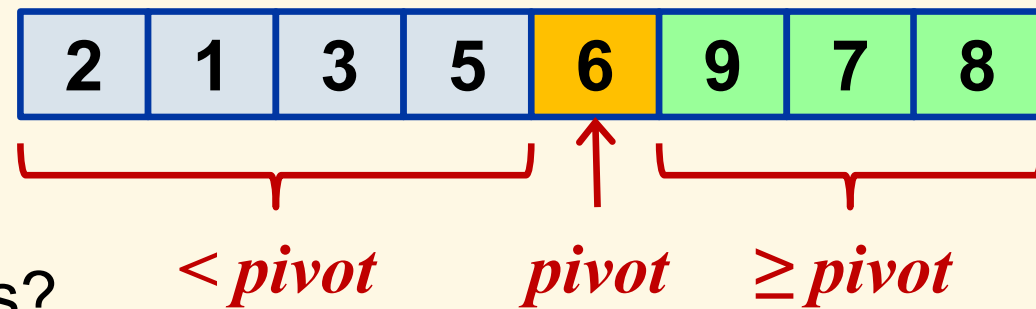


Algorithm

- ▶ If the array size is greater than 1
 - ▶ Pick a pivot element
 - ▶ Partition the array so that
 - ▶ Elements $<$ the pivot go to the left
 - ▶ Elements $\geq$ the pivot go to the right
 - ▶ Pivot is in between
 - ▶ Do the same with the left part
 - ▶ Do the same with the right part

- ▶ The problems:

- ▶ How to partition?
 - ▶ How to choose good pivots?



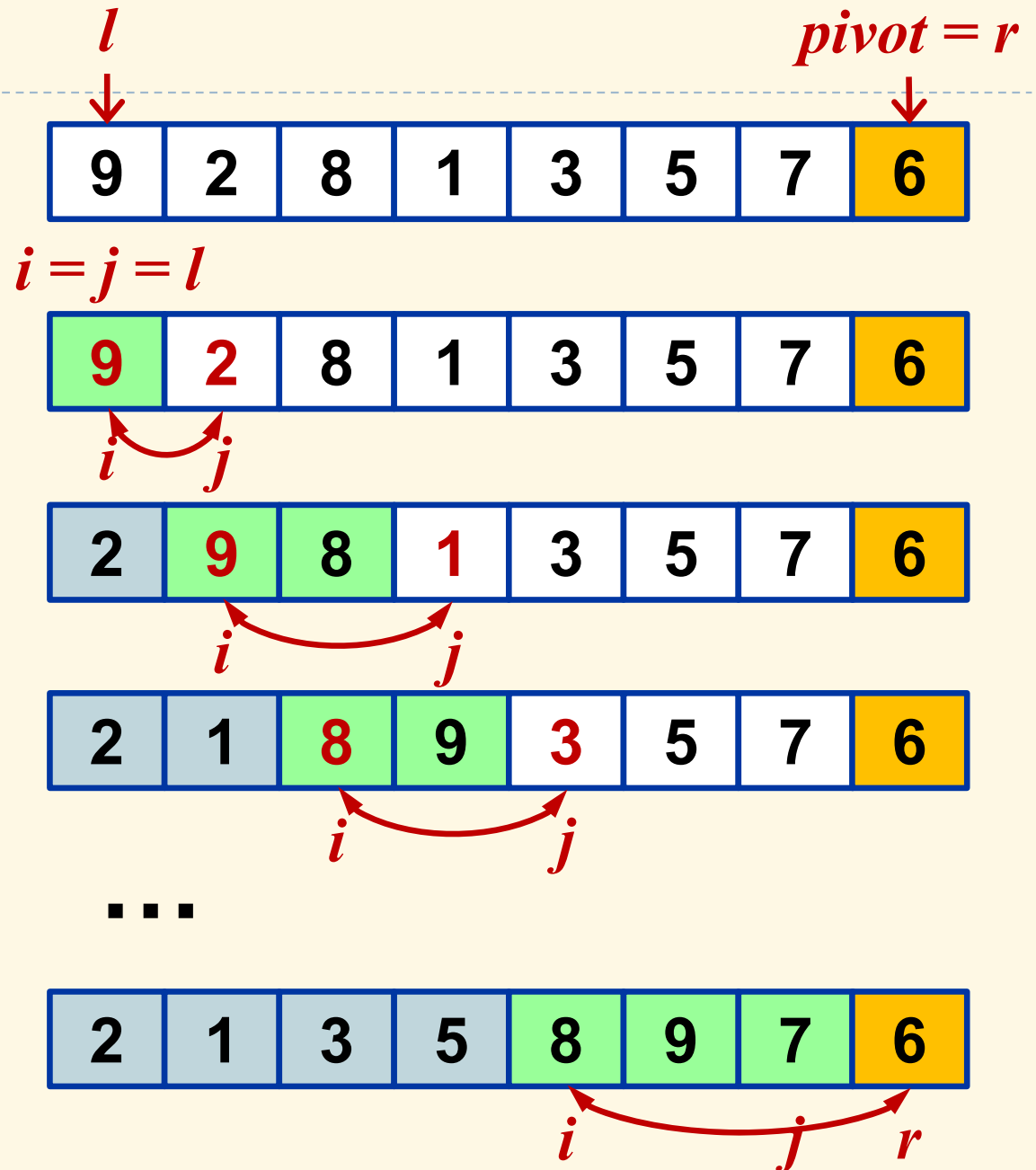
Implementation: Quicksort

```
void quicksortRange(int* a, int l, int r) {  
    if (l < r) {  
        int pi = qsPartitionXXX(a, l, r);  
        quicksortRange(a, l, pi - 1);  
        quicksortRange(a, pi + 1, r);  
    }  
}
```

```
void quicksort(int* a, int n) {  
    quicksortRange(a, 0, n - 1);  
}
```

Lomuto partition

- ▶ Pick last element as pivot
- ▶ i : index of the first element $\geq$ pivot; starts with l
- ▶ Repeat until $j = r$
 - ▶ j : find element $<$ pivot
 - ▶ Swap elements at i and j
 - ▶ Increase i
- ▶ Swap elements at i and r

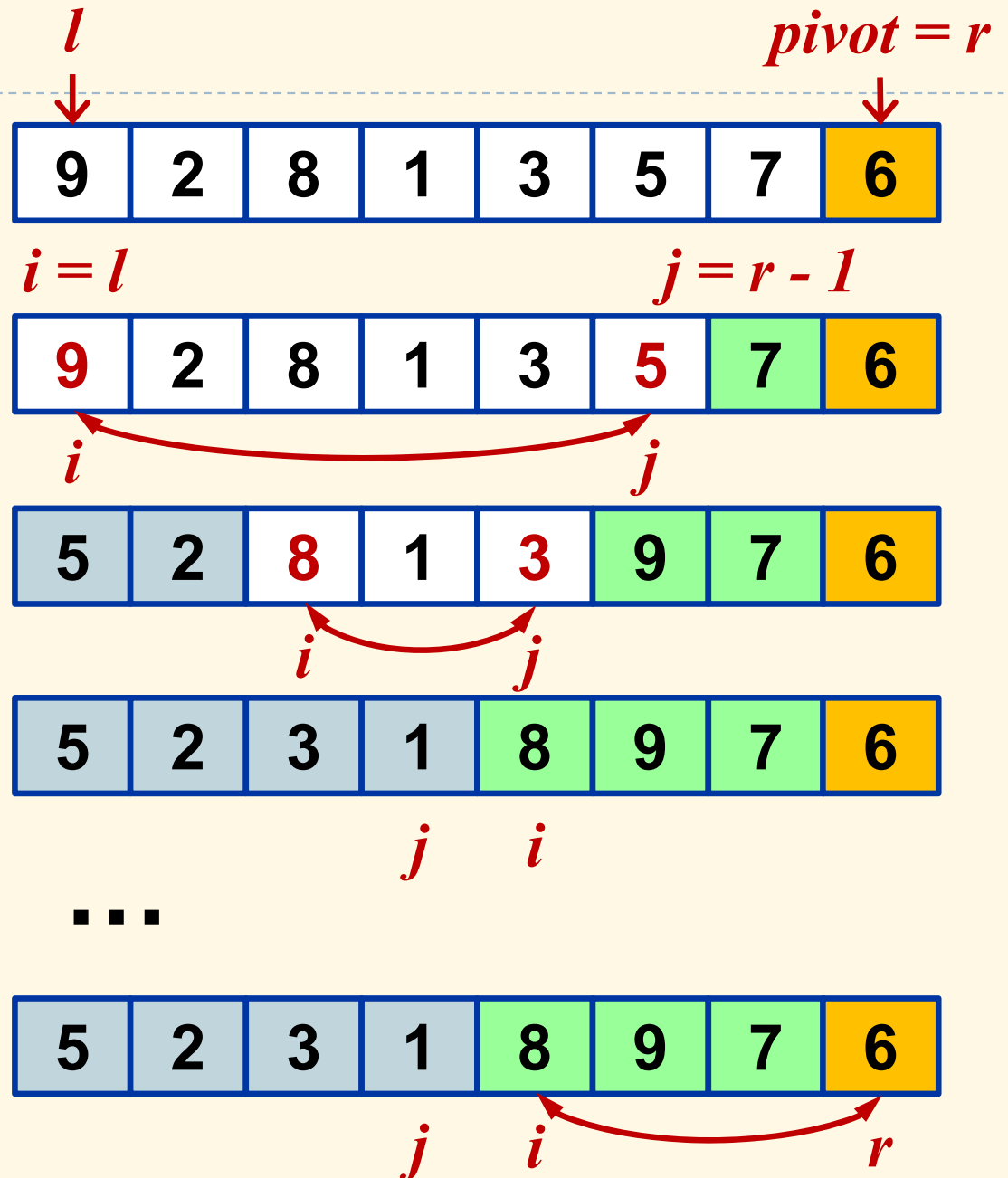


Implementation: Lomuto partition

```
int qsPartitionLomuto(int* a, int l, int r) {  
    int pivot = a[r];  
    int i, j;  
  
    for (i = l, j = l; j < r; j++) {  
        if (a[j] < pivot)  
            swap(a, i++, j);  
    }  
  
    swap(a, i, r);  
    return i;  
}
```

Hoare partition

- ▶ Pick last element as pivot
- ▶ i and j run from two ends
- ▶ Repeat until $i \geq j$
 - ▶ i : find element $\geq$ pivot forward
 - ▶ j : find element $<$ pivot backward
 - ▶ Swap elements at i and j
- ▶ Swap elements at i and r



Implementation: Hoare partition

```
int qsPartitionHoare(int* a, int l, int r) {  
    int pivot = a[r];  
    int i, j;  
  
    for (i = l, j = r - 1; i < j;) {  
        for (; a[i] < pivot && i < j; i++);  
        for (; a[j] >= pivot && i < j; j--);  
        swap(a, i, j);  
    }  
  
    if (a[r] > a[i]) return r;  
  
    swap(a, i, r);  
    return i;  
}
```

Edge case: When the chosen pivot has the greatest value, keep it as the last element.

Complexity

Complexity	Worst case	Best case	Average case
Computation	$O(n^2)$	$O(n \log n)$	$O(n \log n)$
- Partition	$O(n)$	$O(n)$	$O(n)$
- Recursion	$O(n)$	$O(\log n)$	$O(\log n)$
Memory (stack)	$O(n)$	$O(\log n)$	$O(\log n)$
- Partition	$O(1)$	$O(1)$	$O(1)$
- Recursion	$O(n)$	$O(\log n)$	$O(\log n)$

- ▶ Worst case: when the algorithm always pick the smallest or largest element as pivot

Good strategy to choose pivots?

- ▶ Any element can be chosen as pivot (no need to be last/first one)
- ▶ Pivot being first or last element leads to worst case when the array is already sorted
- ▶ Other strategies:
 - ▶ Middle element
 - ▶ Random element
 - ▶ Median of {first, last, middle}

Summary

Complexity

Algorithm	Worst case	Best case	Average case	Memory
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(\log n)$
Merge (std)	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Merge (in place)	$O(n^2 \log n)$	$O(n^2 \log n)$	$O(n^2 \log n)$	$O(\log n)$
Quicksort	$O(n^2)$	$O(n \log n)$	$O(n \log n)$	$O(\log n)$

- ▶ Quicksort is considered as best choice for most of cases in practice

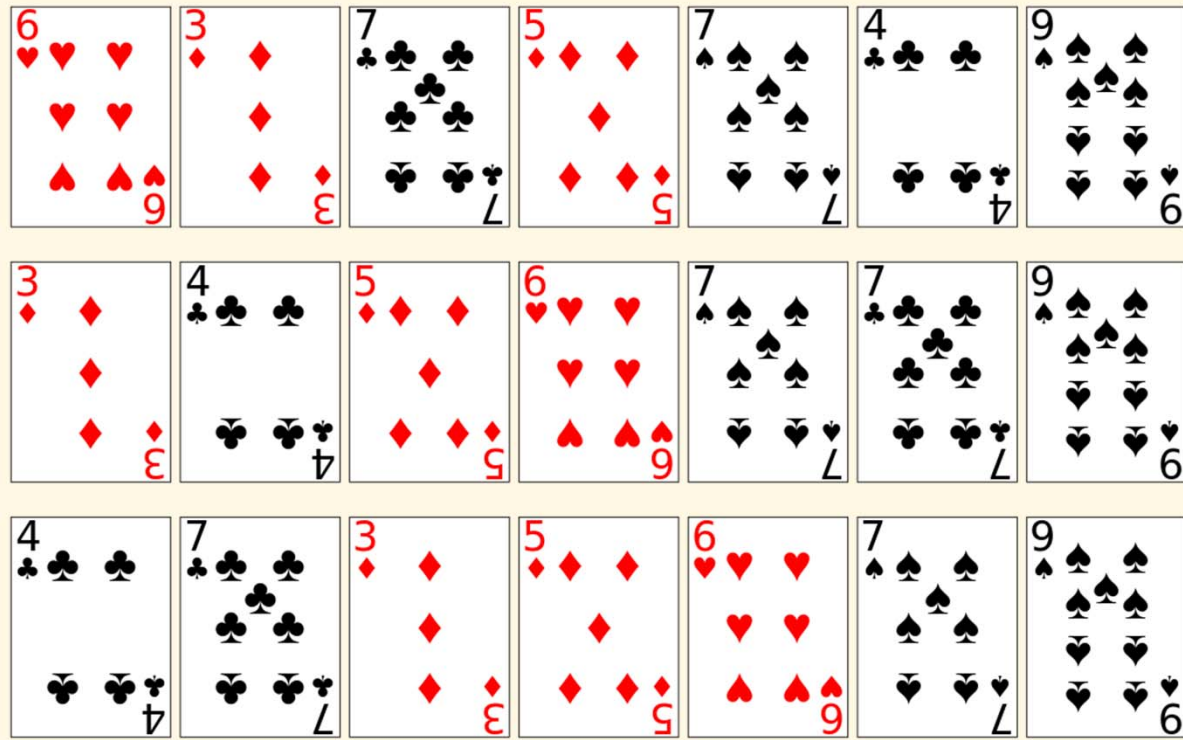
Other considerations

Algorithm	Recursive?	Parallel processors?	Parallel disks?	Cache efficient?
Bubble	—	—	—	—
Insertion	—	—	—	—
Selection	—	—	—	—
Heap	—	—	—	—
Merge (std)	+	+	+	+
Merge (in place)	+	+	+	+
Quicksort	+	+	—	+

+ good — bad

Sort stability

- ▶ A sorting algorithm is stable if items with same key preserve their order in the output as they were in the input
- ▶ Stable algorithms by nature: bubble, insertion, merge
- ▶ Unstable algorithms: selection, heap, quicksort
- ▶ Unstable algorithms can be made stable by
 - ▶ Adding original order into account of the comparison



Application example of sorting a list using primary and secondary keys: first sorting the cards by rank (using any sort), then doing a stable sort by suit

Further technical discussion

- ▶ How to avoid swapping large array elements?
- ▶ How to generalize data type and sort order?
- ▶ Which algorithms can be implemented with linked lists?
- ▶ Why don't we partition into more parts for divide-and-conquer algorithms?
- ▶ Other sorting algorithms
 - ▶ Bucket sort, shuffle sort, comb sort, Shell sort,...
 - ▶ Hybrid sorts
- ▶ Other kinds of sorting:
 - ▶ Sorting into groups (e.g., Dutch national flag problem)

ANSI C `qsort()` function

► Definition

```
void qsort(void* base, size_t num, size_t size,  
           int (*compare)(const void*, const void*));
```

► Example

```
int strorder(const void* a, const void* b) {  
    const char* sa = *(const char**)a,  
               * sb = *(const char**)b;  
    return strcmp(sa, sb);  
}
```

```
const char* s[] = {"abc", "xyz", ...};  
const int n = sizeof(s) / sizeof(s[0]);  
qsort(s, n, sizeof(s[0]), strorder);
```

Searching

- ❑ Linear search
- ❑ Self-organized list
- ❑ Binary search
- ❑ List verification

Overview

- ▶ Look for information of interest
- ▶ Information can be in lots of forms
 - ▶ Text
 - ▶ Number
 - ▶ Signals
 - ▶ Image, video
- ▶ Strategies and techniques:
 - ▶ Linear
 - ▶ Sentinel
 - ▶ Binary
 - ▶ Hierarchical
 - ▶ Pattern matching
 - ▶ ...



Linear search

Linear (sequential) search

- ▶ Sequentially visit all the elements in a list or array
- ▶ Compare the given key with each element
 - ▶ If the element is found, return its index or pointer
 - ▶ Return -1 or NULL otherwise
- ▶ The list or array is unorganized (elements in any order)

Example

```
int search(char* str, int n, char v) {  
    int t;  
    for (t = 0; t < n && v != str[t]; t++);  
  
    return t < n ? t : -1;  
}
```

```
char* str = "asdf";  
int index = search(str, 4, 's');  
printf("%d", index);
```

Sentinel

- ▶ Note that each iteration requires two conditions to be checked and one statement to be executed
- ▶ We can avoid checking for the end of the array on every iteration by inserting the target as an extra “sentinel” element at the end of the array
 - ➔ No more need to check for end-of-array condition

Example

```
int search(char* str, int n, char v) {  
    str[n] = v;  
  
    int t;  
    for (t = 0; v != str[t]; t++);  
  
    str[n] = 0;  
  
    return t < n ? t : -1;  
}
```

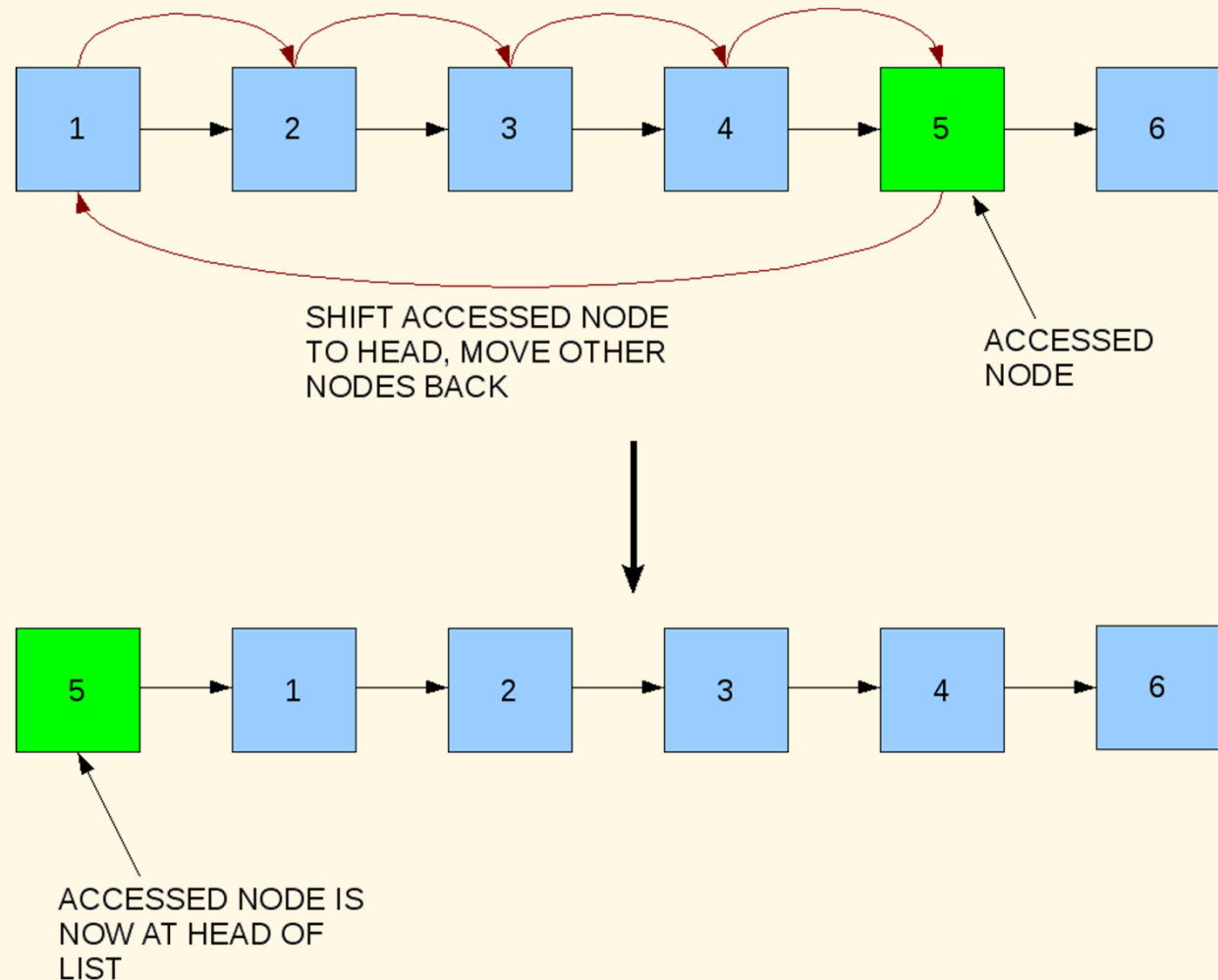
Self organizing list

Idea

- ▶ List that reorders its elements based on some heuristic
 - ▶ Improve overall average access time
 - ▶ Frequently accessed elements are closed to the head
- ▶ Approaches
 - ▶ Move to front method
 - ▶ Transpose method
 - ▶ Count method
- ▶ Applications
 - ▶ Language processing
 - ▶ Caching algorithms

Move-to-front method

- ▶ Any element searched/accessed is moved to the front
- ▶ Pros
 - ▶ No more memory needed
 - ▶ Simple
- ▶ Cons
 - ▶ May over-reward



Transpose method

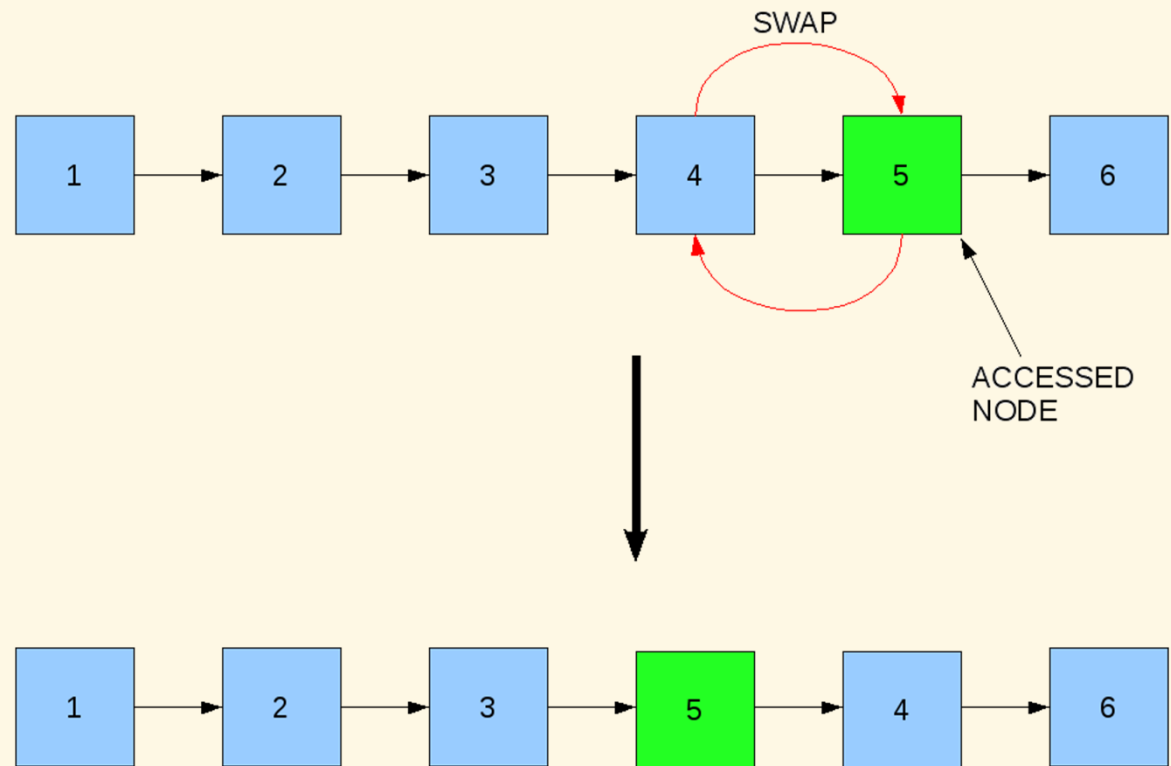
- ▶ Searched/accessed element is swapped with the next one

- ▶ Pros

- ▶ No more memory needed
- ▶ Simple

- ▶ Cons

- ▶ Slow convergence



Count method

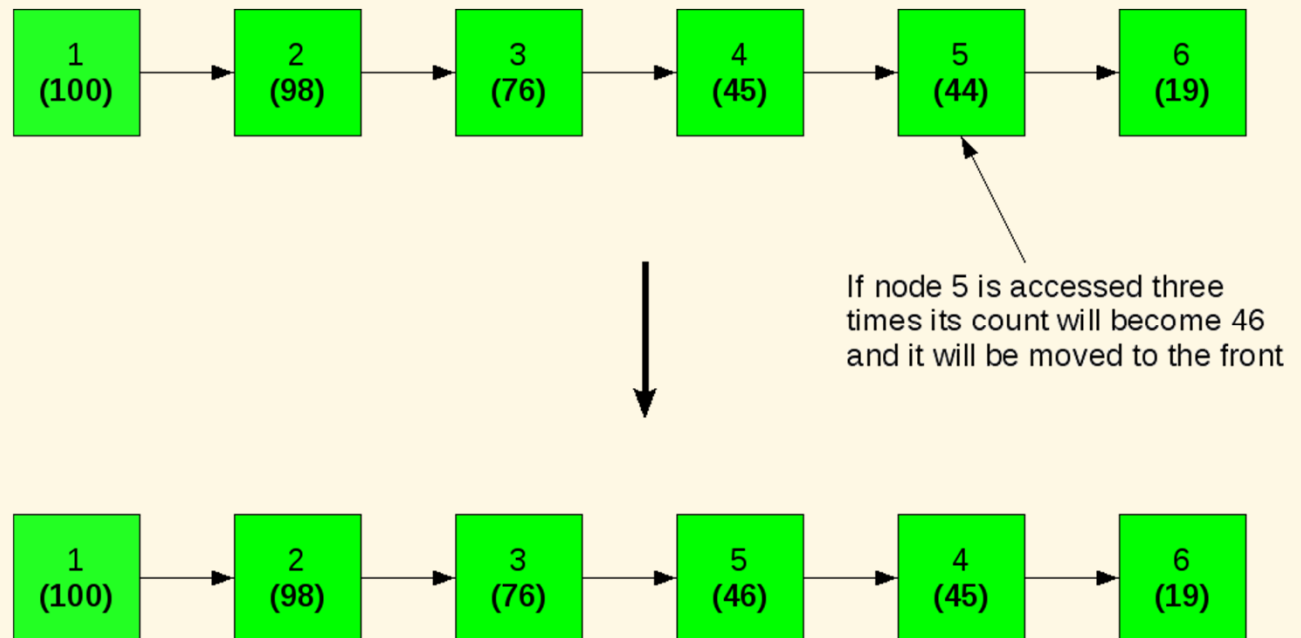
- ▶ The nodes are arranged based on the number of times they are searched for

- ▶ Pros

- ▶ More exact

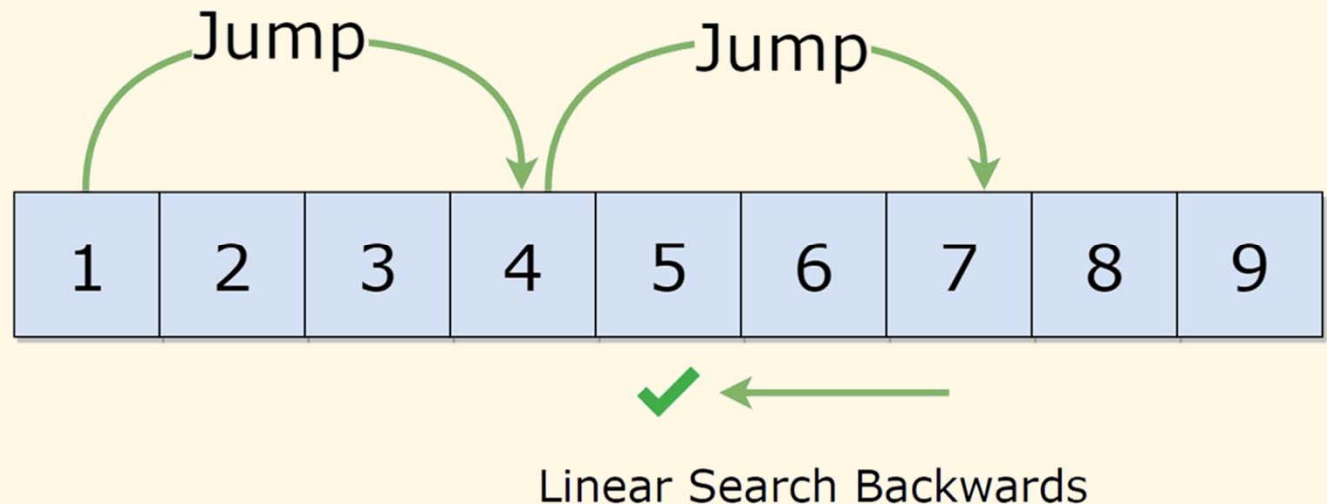
- ▶ Cons

- ▶ Memory for counters
 - ▶ Not adaptive when access pattern changes



Jump search

Principle



- ▶ When data is sorted
 - ▶ Jump multiple steps (m) ahead each time to quickly narrow down the range of search
 - ▶ Linear search once the range is found
- ▶ Worst number of iterations: $n/m + m$
 - ▶ Minimal when $m = \sqrt{n}$

Implementation

```
int jump_search(int a[], int n, int v) {
    int i, j = 0, m = (int)sqrt(n);

    for (i = 0; i < n && a[i] <= v; j = i, i += m)
        if (a[i] == v) return i;

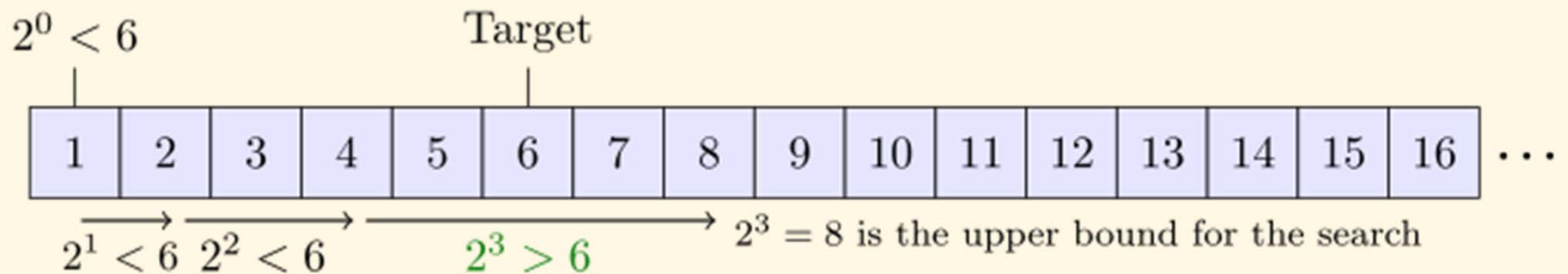
    if (i >= n) {
        i = n - 1;
        if (a[i] < v) return -1;
    } else i--;

    for (; i >= j; i--)
        if (a[i] == v) return i;

    return -1;
}
```

Further notes

- ▶ Recursively apply jump search for found ranges
- ▶ Jump search with exponentially increasing size
→ Exponential search



- ▶ Better estimation for jump targets
→ Interpolation search:

$$p = i_{start} + \frac{v - a_{start}}{a_{end} - a_{start}} (i_{end} - i_{start})$$

(not applicable for non-scalar data types)

Exercise

- ▶ Implement a jump search such that when going backward, jump search is also used
 - ▶ Note that, this will make the search recursive, and it may go forward then backward multiple times
- ▶ Implement a jump search with exponentially increasing size for double-valued arrays.

Binary search

Principle

1	3	5	6	10	11	14	25	26	40	41	78
---	---	---	---	----	----	----	----	----	----	----	----

- ▶ Similar to the way we look up words in a dictionary
 - ▶ Require the array/listed to be sorted
- ▶ Divide-and-conquer technique
 - ▶ The key is compared with the element in the middle of the list
 - ▶ If the key is smaller, restrict the search to the first half of the list
 - ▶ Otherwise, search the second half of the list
- ▶ Extremely good in reducing the number of condition checking times

Illustration

- Search key = 78

1	3	5	6	10	11	14	25	26	40	41	78
---	---	---	---	----	----	----	----	----	----	----	----

1	3	5	6	10	11	14	25	26	40	41	78
---	---	---	---	----	----	----	----	----	----	----	----

$11 < 78$

14	25	26	40	41	78
----	----	----	----	----	----

$26 < 78$

40	41	78
----	----	----

$41 < 78$

78

$78 = 78$

- Only 4 iterations needed to find

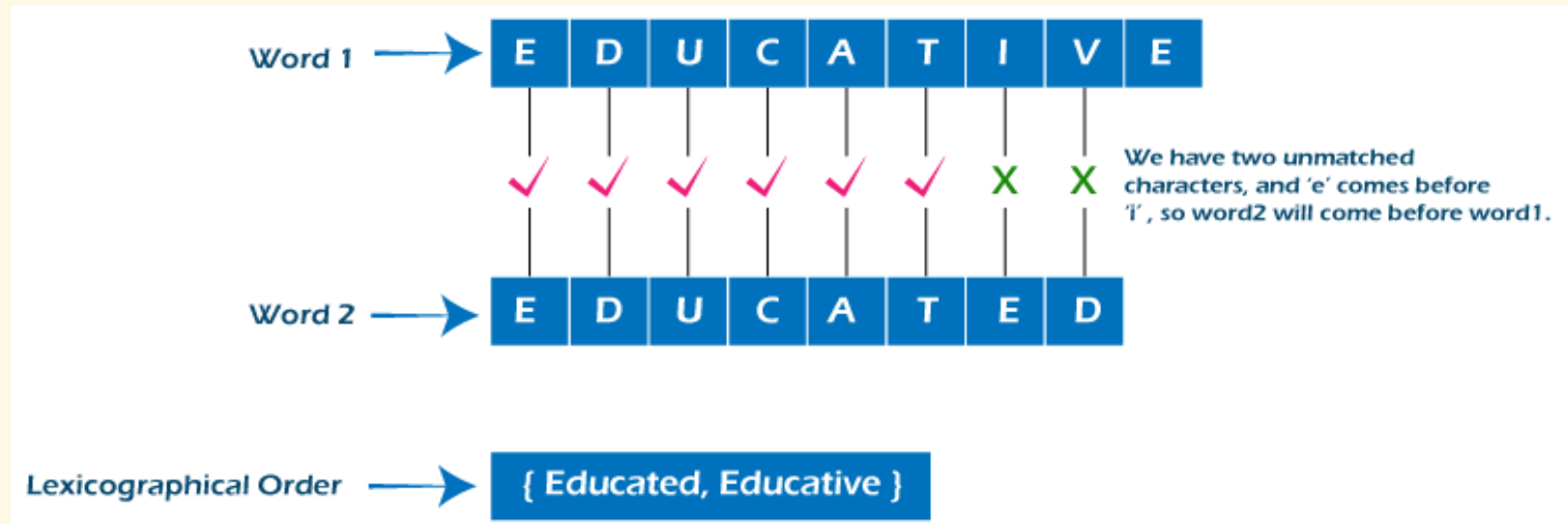
Implementation (iterative)

```
int search(int list[], int key, int lo, int hi) {  
    int mid;  
  
    while (lo <= hi) {  
        mid = (lo + hi) / 2;  
  
        if (list[mid] < key)  
            lo = mid + 1;  
        else if (list[mid] > key)  
            hi = mid - 1;  
        else return mid;  
    }  
  
    return -1;  
}
```

Implementation (recursive)

```
int search(int list[], int key, int lo, int hi) {  
    if (lo > hi) return -1;  
  
    int mid = (lo + hi) / 2;  
  
    if (list[mid] < key)  
        return search(list, key, mid + 1, hi);  
  
    if (list[mid] > key)  
        return search(list, key, lo, mid - 1);  
  
    return mid;  
}
```

String comparison with lexicographic order



- ▶ `strcmp(s1, s2)` function, returns
 - ▶ 1 if s1 is “greater” than s2
 - ▶ 0 if s1 is identical to s2
 - ▶ -1 if s1 is “lower” than s2
- ▶ Attention
 - ▶ Character comparison is based on ASCII code
 - ▶ 'a' < 'd', 'B' < 'M'
 - ▶ "adding" < "addition"

Big-O of search algorithms

- ▶ Use n as number of elements in the array/list
- ▶ Estimate the number of iterations needed to find the key
 - ▶ Average value (most important)
 - ▶ Worst case
 - ▶ Best case
- ▶ Sequential search: $O(n)$
- ▶ Jump search: $O(\sqrt{n})$
- ▶ Binary search: $O(\log n)$

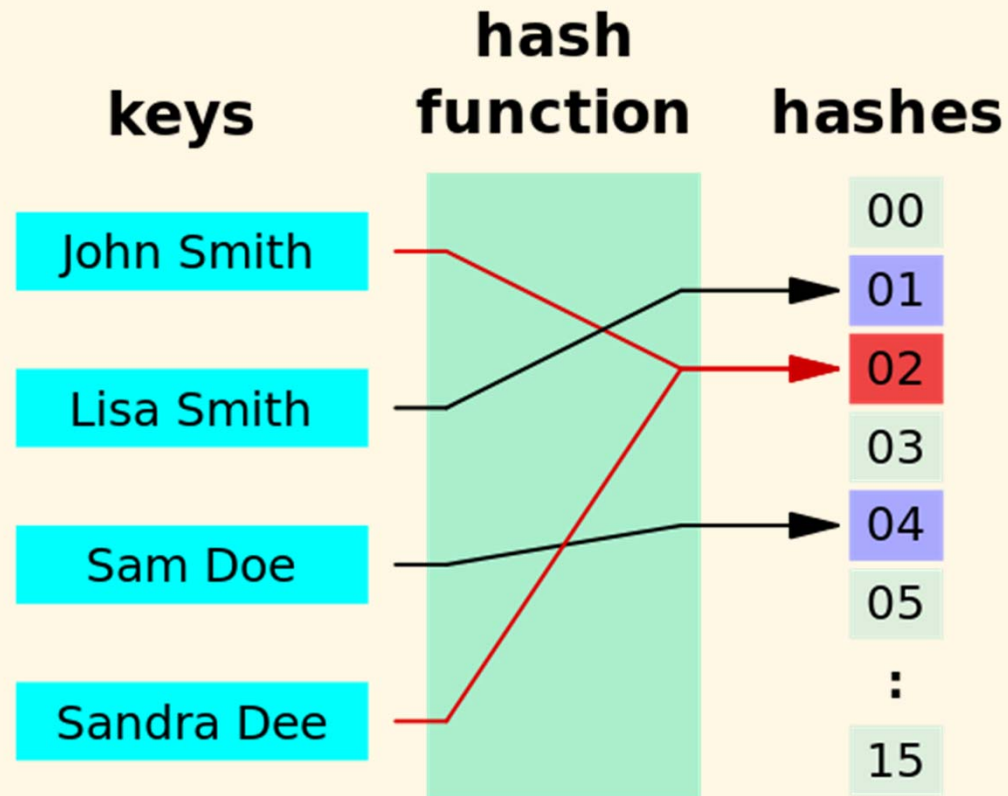
Exercises

- ▶ Implement a binary search for string-valued arrays.
- ▶ Do a real test to compare sequential, jump, binary search algorithms.

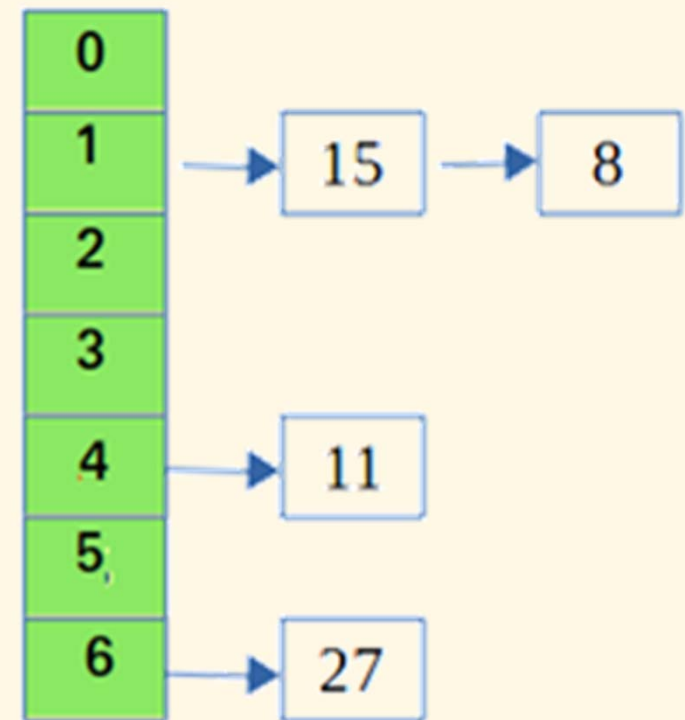
Hash function

Overview

- ▶ Arrange the data into groups with deterministic keys



Chaining



- ▶ To be revisited later!

List verification

Overview

- ▶ Compare two lists to verify that they are identical or identify the discrepancies
- ▶ Example: international revenue service (e.g., employee vs. employer)
- ▶ Complexity
 - ▶ Unordered lists: $O(mn)$
 - ▶ Ordered lists: $O(m + n)$

Homework

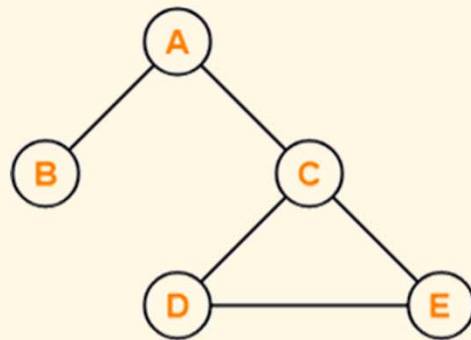
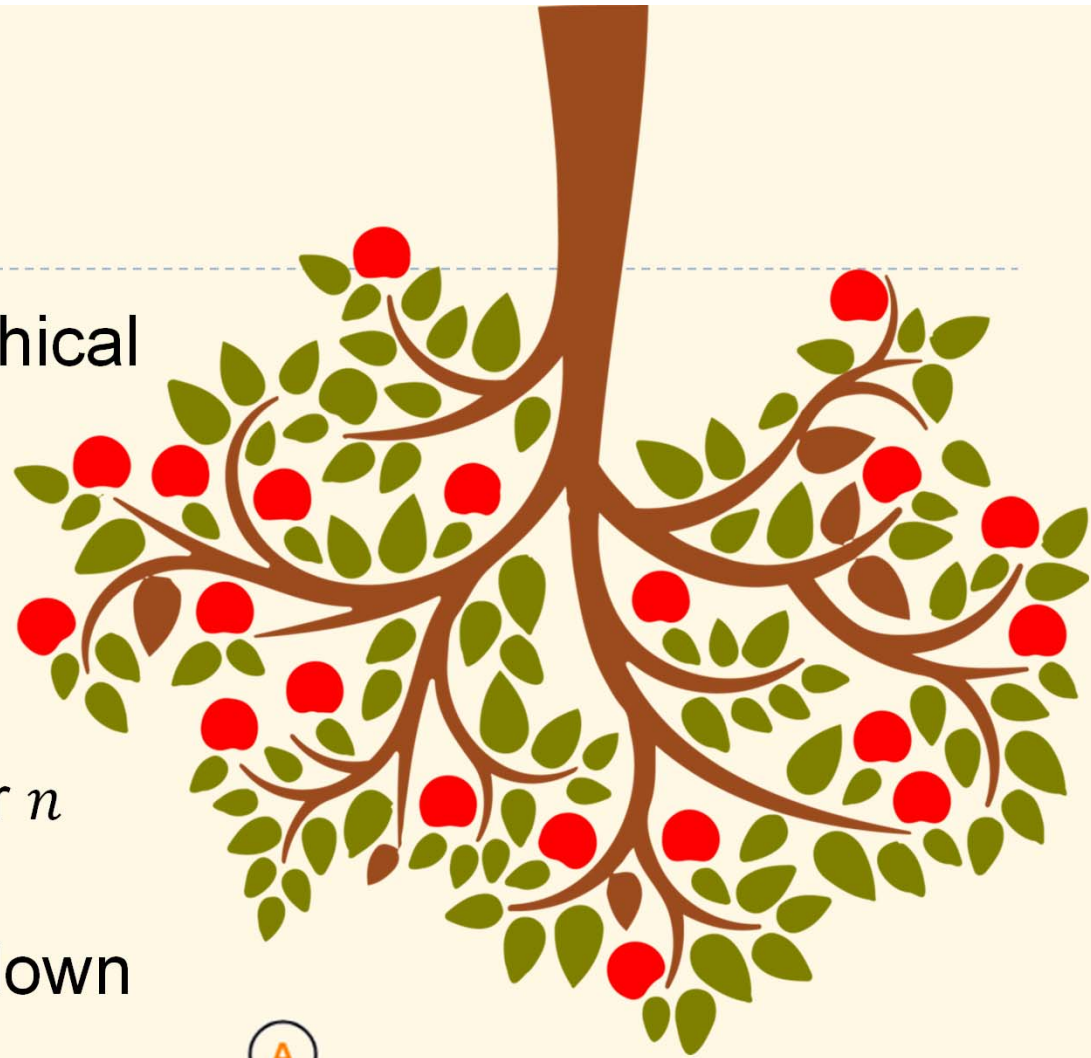
1. Implement a self organizing list using linked list.
2. Implement a jump search with position estimation by interpolation for double-valued arrays.

Trees

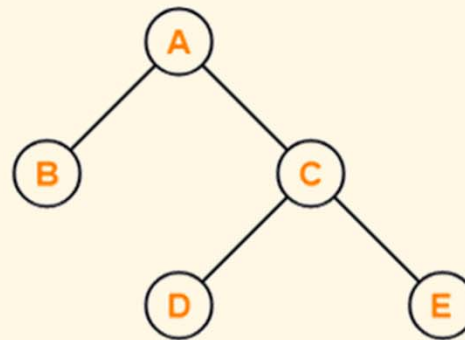
- ❑ General trees
- ❑ Binary trees
- ❑ Threaded binary trees
- ❑ Binary search trees
- ❑ Depth first & breadth first search

Trees

- ▶ Used to represent hierarchical data
- ▶ A special form of graph
 - ▶ Minimally connected
 - ▶ No loops, no circuits
 - ▶ Always has $n - 1$ edges for n vertices
- ▶ Usually depicted upside down



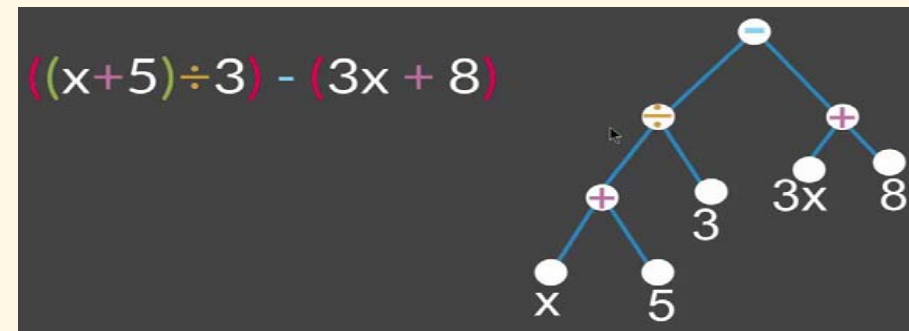
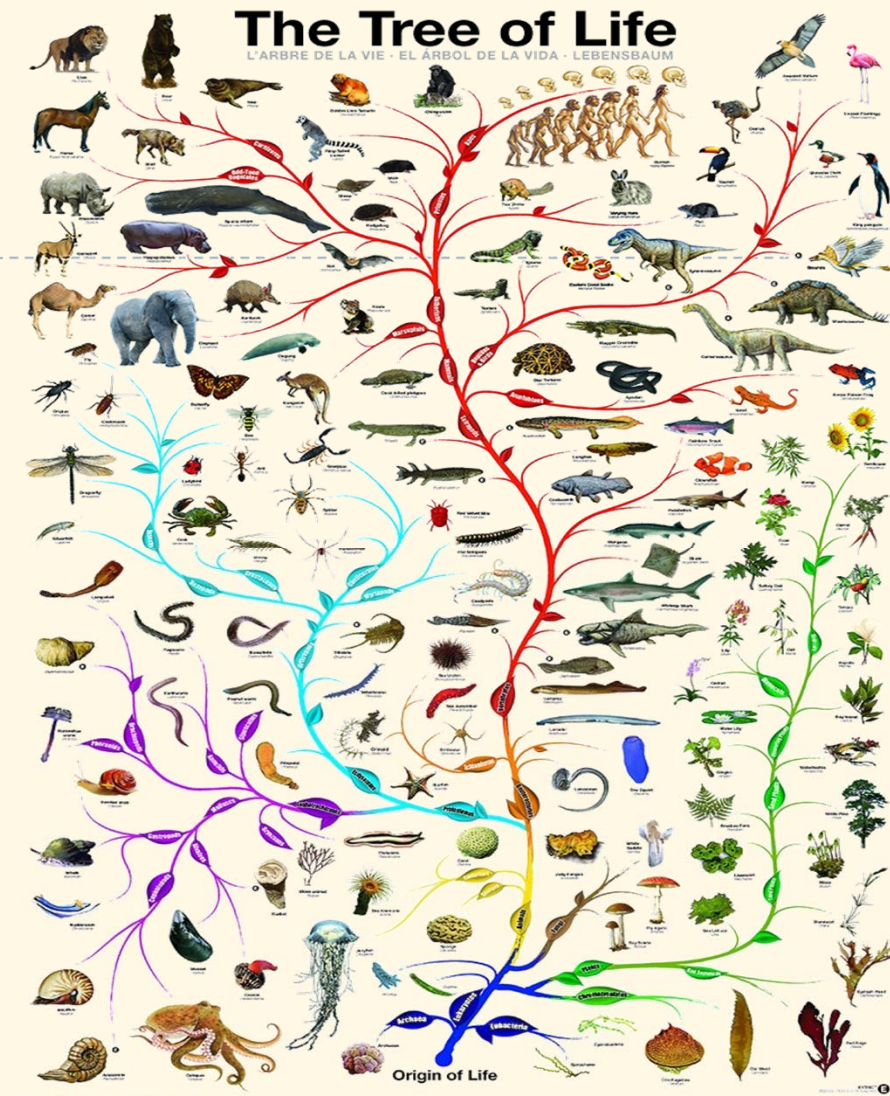
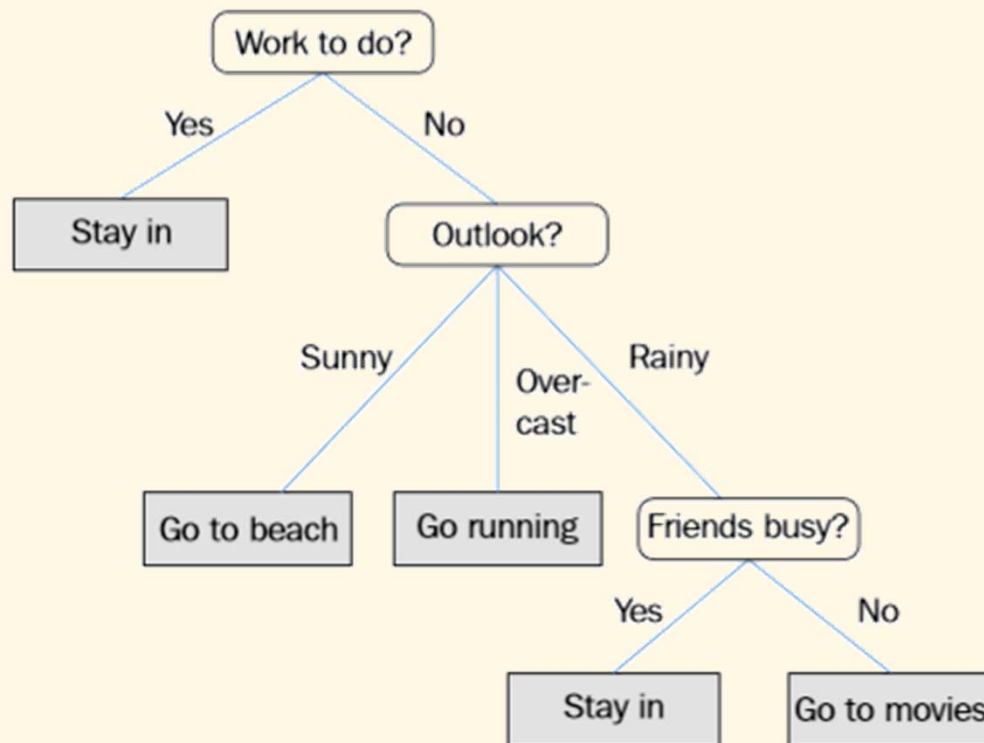
X



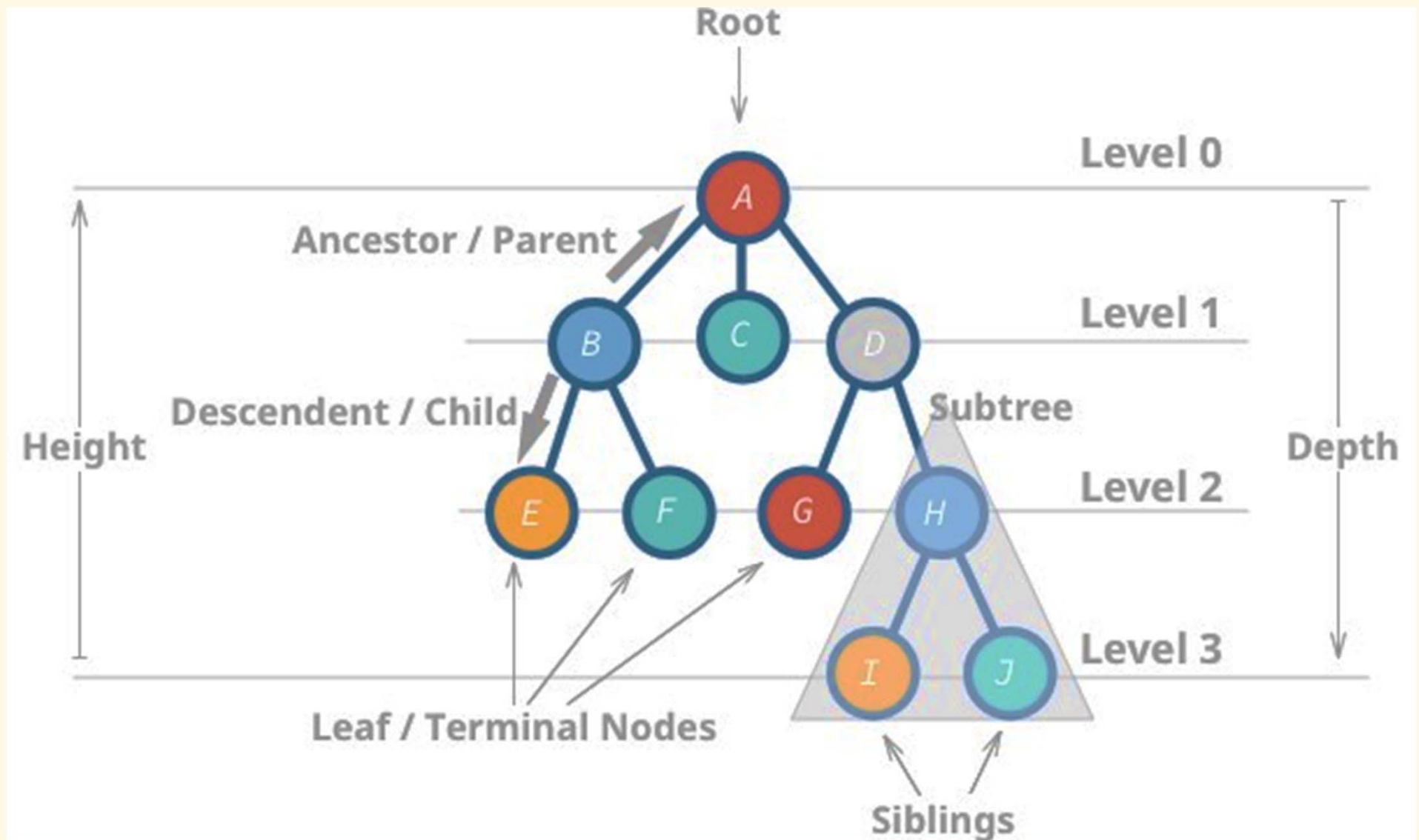
✓

Application examples

- ▶ Classification
- ▶ Decision making
- ▶ Data storage, sorting, searching
- ▶ Computational methods



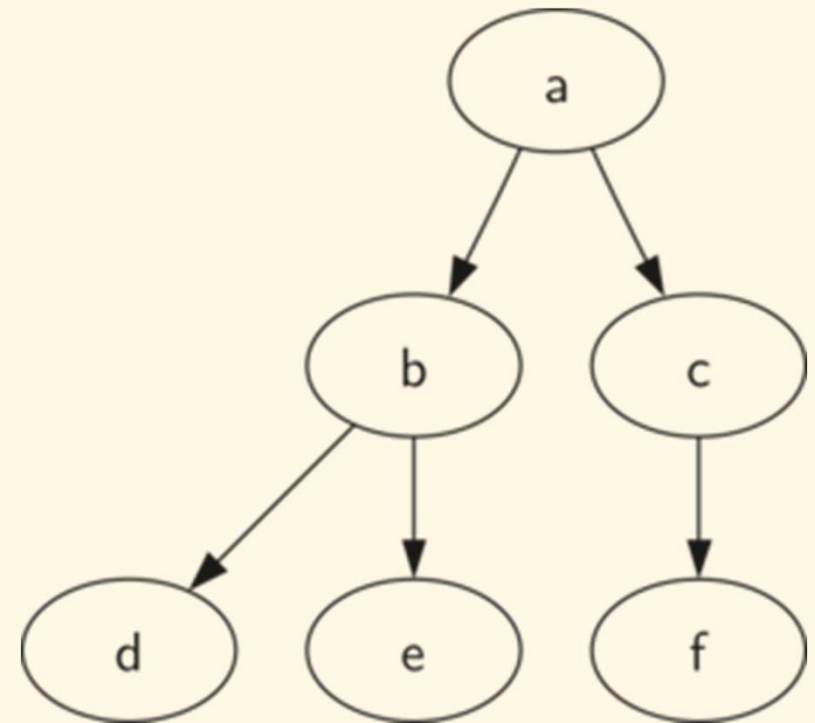
Terminology



List representation

- ▶ Each node has a list of zero or more child nodes

```
(a, [  
  (b, [  
    (d, []),  
    (e, [])  
  ]),  
  (c, [  
    (f, [])  
  ])  
)
```



→ (a, [(b, [(d, ~~[]~~), (e, ~~[]~~)]), (c, [(f, ~~[]~~)])])

Implementation

- ▶ Two approaches
 - ▶ Using an **array** to represent the children of each node
 - ▶ Using a **linked list** to represent the children of each node
- ▶ Possible to design in possessive or non-possessive manner
 - ▶ **Possessive**: a parent node takes over its child nodes, and is responsible to destroy them on its destruction
 - ▶ **Non-possessive**: a parent does not take over its child nodes, and leave them to destroy themselves

Exercises

1. Implement a simple general tree using an array (with possessive and non-possessive methods).
2. Implement a simple general tree using a linked list.
3. Write a function to save a tree to a string, and another to reconstruct the original tree from a string.

Binary tree

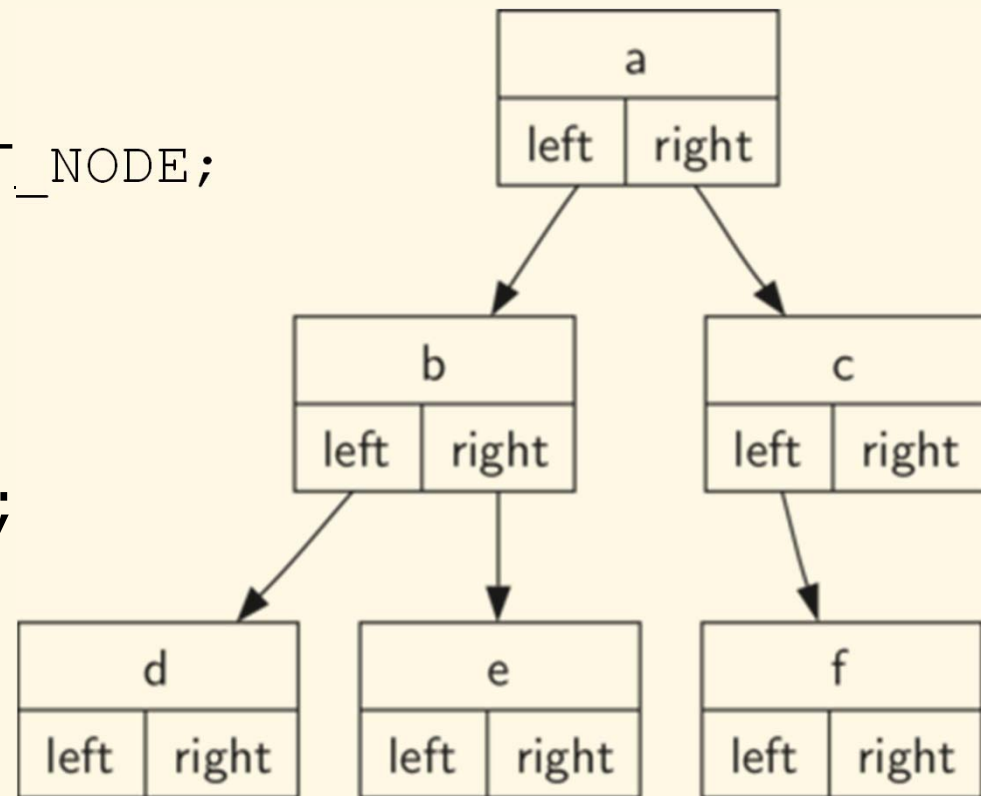
Definitions (with `int` data type)

- ▶ A tree in which each node can have at most 2 children: *left subtree* and *right subtree*

```
struct _BT_NODE;
```

```
typedef struct _BT_NODE BT_NODE;
```

```
struct _BT_NODE {  
    int data;  
    BT_NODE *left, *right;  
};
```



Basic operations

- ▶ `BT_NODE* btCreateNode(int v);`
- ▶ `BT_NODE* btCreateNodeLR(int v,
BT_NODE* l, BT_NODE* r);`
- ▶ `void btAddToLeft(BT_NODE* p, BT_NODE* c);`
- ▶ `void btAddToRight(BT_NODE* p, BT_NODE* c);`
- ▶ `void btDeleteLeft(BT_NODE* p);`
- ▶ `void btDeleteRight(BT_NODE* p);`
- ▶ `void btDeleteNode(BT_NODE** n);`

- ▶ `int btIsLeaf(BT_NODE* n);`

- ▶ `BT_NODE* btLeftMost(BT_NODE* n);`
- ▶ `BT_NODE* btRightMost(BT_NODE* n);`

Ways to construct a tree (1)

```
BT_NODE* a = btCreateNode(1);
```

```
BT_NODE* b = btCreateNode(2);
```

```
BT_NODE* c = btCreateNode(3);
```

```
BT_NODE* d = btCreateNode(4);
```

```
btAddToLeft(a, b);
```

```
btAddToRight(a, c);
```

```
btAddToRight(b, d);
```


Ways to construct a tree (2)

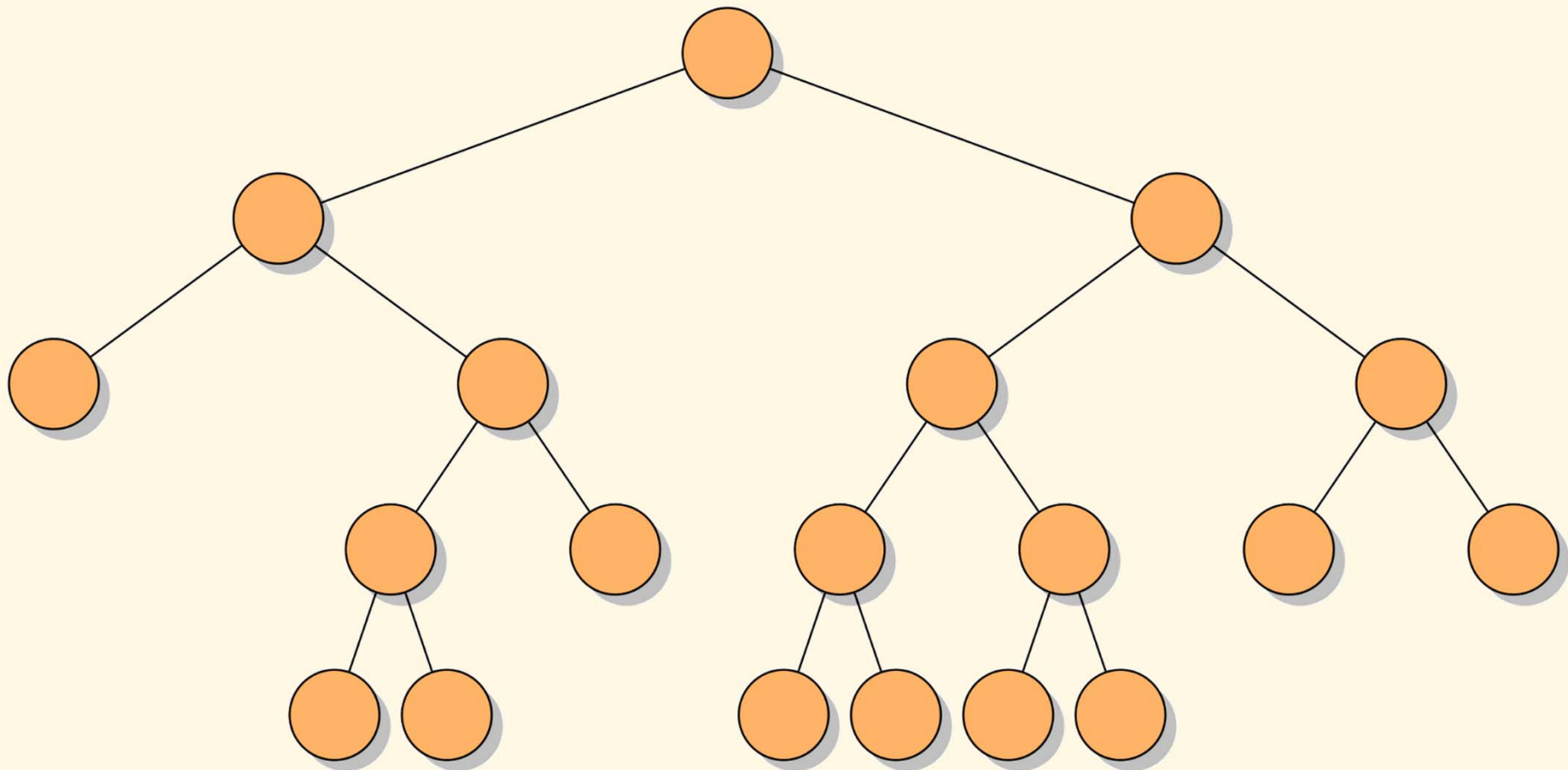
```
BT_NODE* a = btCreateNode(1);  
a->left = btCreateNode(2);  
a->right = btCreateNode(3);  
a->left->right = btCreateNode(4);
```

Ways to construct a tree (3)

```
BT_NODE* a =  
    btCreateNodeLR(1,  
        btCreateNodeLR(2,  
            NULL,  
            btCreateNodeLR(4, NULL, NULL)),  
        btCreateNodeLR(3, NULL, NULL));
```

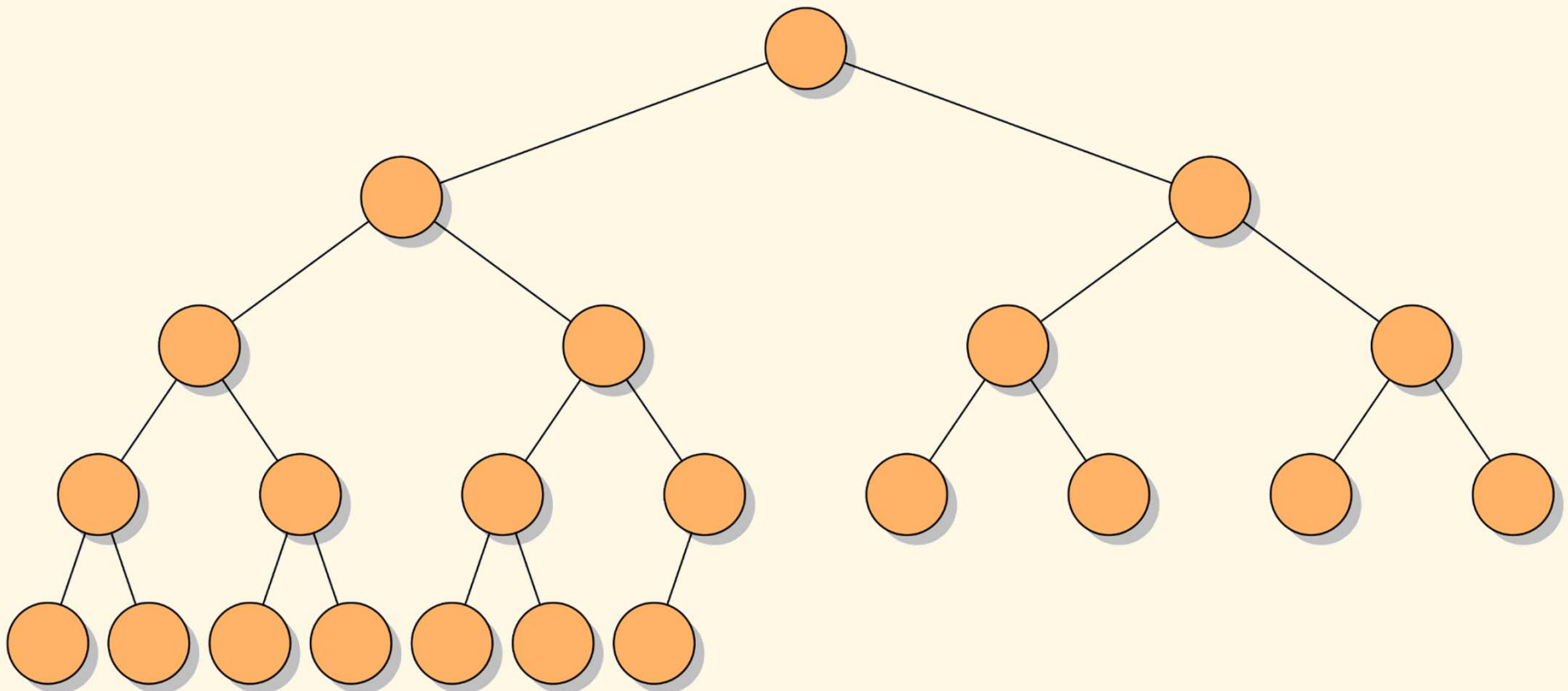
Full binary tree

- ▶ In a full binary tree, all nodes have either 0 or 2 children



Complete binary tree

- ▶ In a complete binary tree, every level, except possibly the last, is completely filled, and all nodes are as far left as possible



More operations

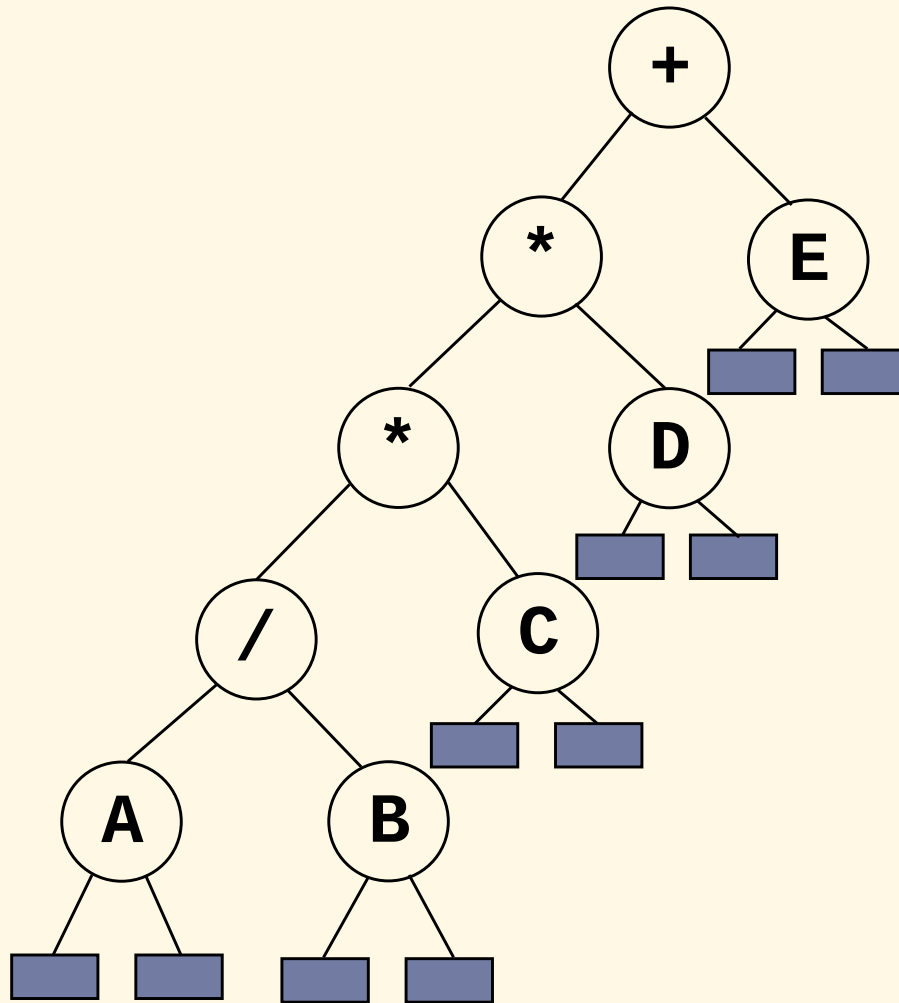
- ▶ `int btDepth(BT_NODE* n);`
- ▶ `int btCount(BT_NODE* n);`
- ▶ `int btIsFull(BT_NODE* n);`
- ▶ `int btIsComplete(BT_NODE* n);`

- ▶ `BT_NODE* btClone(BT_NODE* n);`
- ▶ `void btForEach(BT_NODE* n,
void (*callback)(int data, void* user),
void* user);`

- ▶ `void btPrintTree(BT_NODE* n);`
- ▶ `void btPrintList(BT_NODE* n);`

Binary tree traversals

► Arithmetic expression using binary tree



inorder traversal (LVR)

A / B * C * D + E

(infix expression)

preorder traversal (VLR)

+ * * / A B C D E

(prefix expression)

postorder traversal (LRV)

A B / C * D * E +

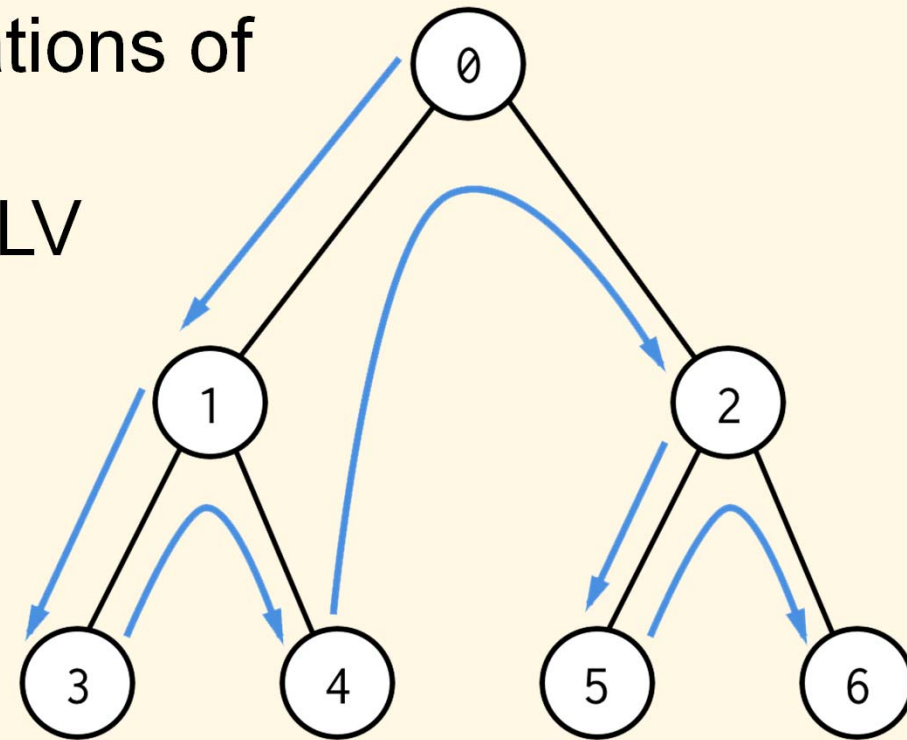
(postfix expression)

level order traversal

+ * E * D / C A B

Binary tree traversals: Depth first search (DFS)

- ▶ Let:
 - ▶ L: moving left
 - ▶ R: moving right
 - ▶ V: visiting node
- ▶ There are 6 possible combinations of traversal:
LVR, LRV, VLR, VRL, RVL, RLV
- ▶ Two ways of implementation
 - ▶ Recursive
 - ▶ Iterative (using a stack)



DFS implementation using recursion (VLR)

```
void btDepthFirstSearch(BT_NODE* n) {  
    if (n == NULL) return;  
  
    // do something with n->data...  
  
    if (n->left != NULL)  
        btTraverseDFSRecursive(n->left);  
  
    if (n->right != NULL)  
        btTraverseDFSRecursive(n->right);  
}
```


DFS implementation using recursion (LVR)

```
void btDepthFirstSearch(BT_NODE* n) {  
    if (n == NULL) return;  
  
    if (n->left != NULL)  
        btTraverseDFSRecursive(n->left);  
  
    // do something with n->data...  
  
    if (n->right != NULL)  
        btTraverseDFSRecursive(n->right);  
}
```

DFS implementation using recursion (LRV)

```
void btDepthFirstSearch(BT_NODE* n) {  
    if (n == NULL) return;  
  
    if (n->left != NULL)  
        btTraverseDFSRecursive(n->left);  
  
    if (n->right != NULL)  
        btTraverseDFSRecursive(n->right);  
  
    // do something with n->data...  
}
```

DFS implementation using stack (VLR)

```
void btDepthFirstSearch(BT_NODE* n) {  
    STACK stack;  
    init(&stack);  
    push(&stack, n);  
  
    while (!empty(stack)) {  
        n = top(stack);  
        pop(&stack);  
  
        printf("%d ", n->data);  
  
        if (n->right != NULL) push(&stack, n->right);  
        if (n->left != NULL) push(&stack, n->left);  
    }  
}
```

DFS implementation using stack (LVR)

```
void btDepthFirstSearch(BT_NODE* n) {  
    STACK stack;  
    init(&stack);  
  
    for (;;) {  
        for (; n != NULL; n = n->left) push(&stack, n);  
        if (empty(stack)) break;  
  
        n = top(stack);  
        pop(&stack);  
        printf("%d ", n->data);  
  
        n = n->right;  
    }  
}
```

DFS implementation using stack (LRV)

```
void btDepthFirstSearch(BT_NODE* n) {
    STACK stack;
    init(&stack);

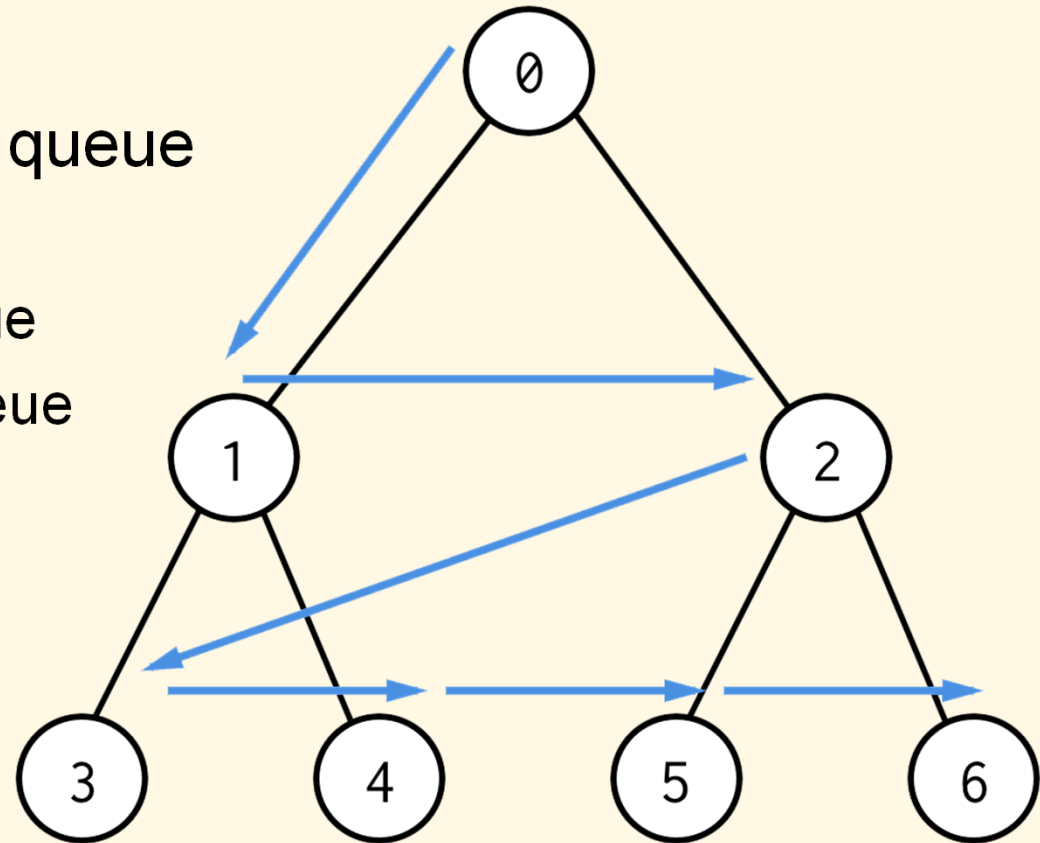
    for (;;) {
        for (; n != NULL; n = n->left) {
            push(&stack, n);
            push(&stack, n);
        }
        if (empty(stack)) break;

        n = top(stack);
        pop(&stack);

        if (!empty(stack) && top(stack) == n) n = n->right;
        else {
            printf("%d ", n->data);
            n = NULL;
        }
    }
}
```

Binary tree traversals: Breadth first search (BFS)

- ▶ Go across to siblings before visiting all nodes on a given level in left-to-right order
- ▶ Implementation (using a queue):
 - ▶ Add root node to queue
 - ▶ For a given node from the queue
 - ▶ Visit node
 - ▶ Add nodes left child to queue
 - ▶ Add nodes right child to queue



BFS implementation using queue

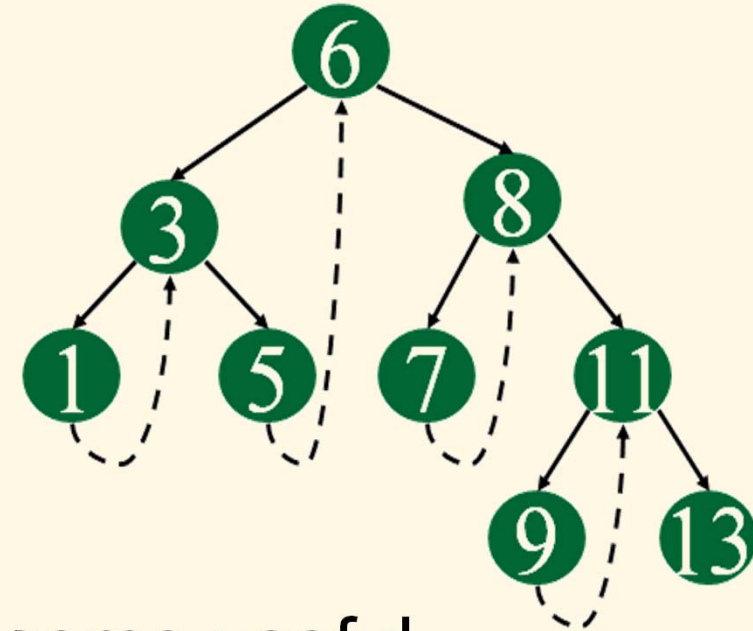
```
void btBreadthFirstSearch(BT_NODE* n) {  
    QUEUE queue;  
    init(&queue);  
    enqueue(&queue, n);  
  
    while (!empty(queue)) {  
        n = top(queue);  
        dequeue(&queue);  
  
        printf("%d ", n->data);  
  
        if (n->left != NULL) enqueue(&queue, n->left);  
        if (n->right != NULL) enqueue(&queue, n->right);  
    }  
}
```

Threaded binary tree (TBT)

Overview

- ▶ Two many null pointers in current representation of binary trees (waste of memory)

- ▶ n : number of nodes
- ▶ number of non-null links: $n - 1$
- ▶ total links: $2n$
- ▶ null links: $2n - (n - 1) = n + 1$



➔ Replace these null pointers with some useful “threads”

- ▶ NULL right pointers is made to point to the in-order successor (*single right threaded*)

Representation

```
struct _TBT_NODE {  
    int data;  
    TBT_NODE *left, *right;  
    int isRightThread;  
}
```

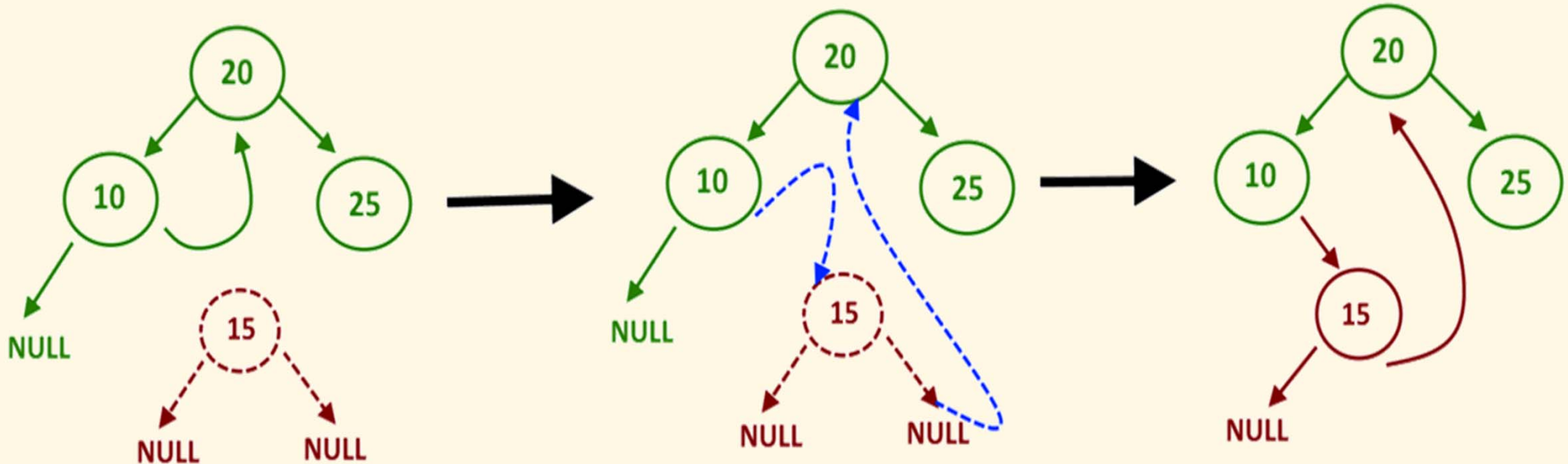
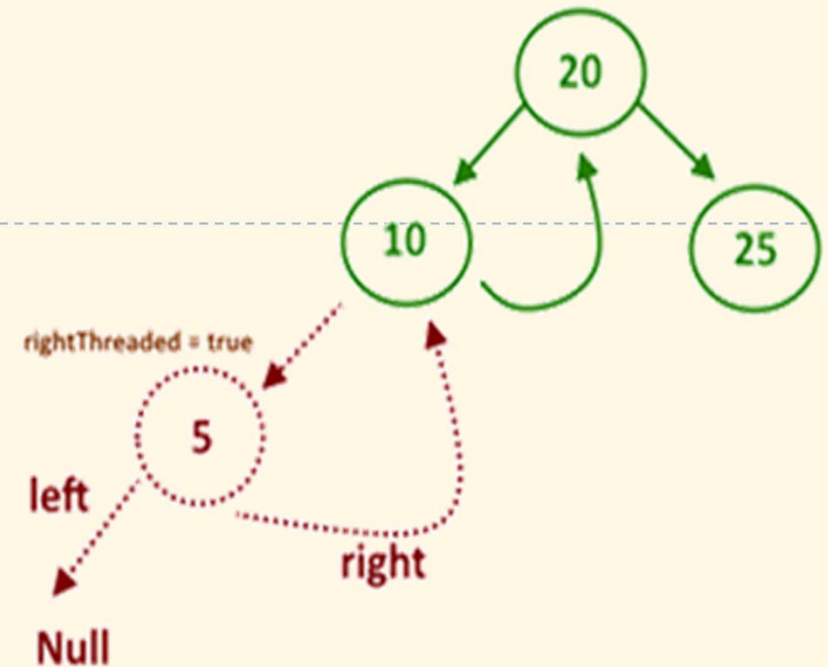
DFS traversal

- ▶ No more stack neither recursion

```
void tbtTraverse(BT_NODE* n) {  
    BT_NODE* cur = btLeftMost(n);  
  
    while (cur != NULL) {  
        printf("%d ", cur->data);  
  
        if (cur->isRightThread)  
            cur = cur->right;  
        else  
            cur = btLeftMost(cur->right);  
    }  
}
```

Insertion

- ▶ 2 cases
 - ▶ To the left of parent node
 - ▶ To the right of parent node

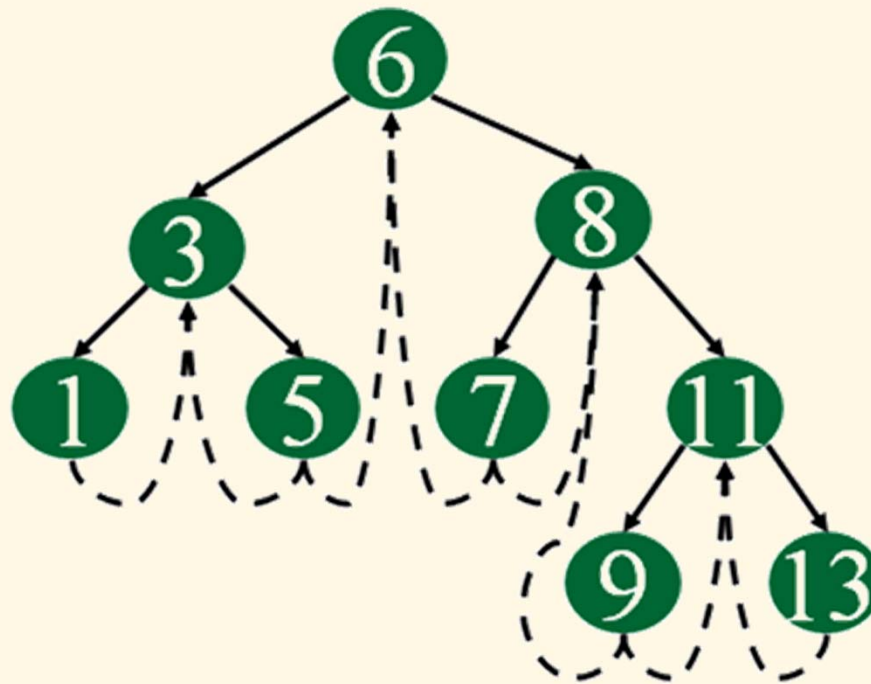


Deletion

- ▶ Also 2 cases
 - ▶ Delete left subtree
 - ▶ Delete right subtree

Double threaded binary tree

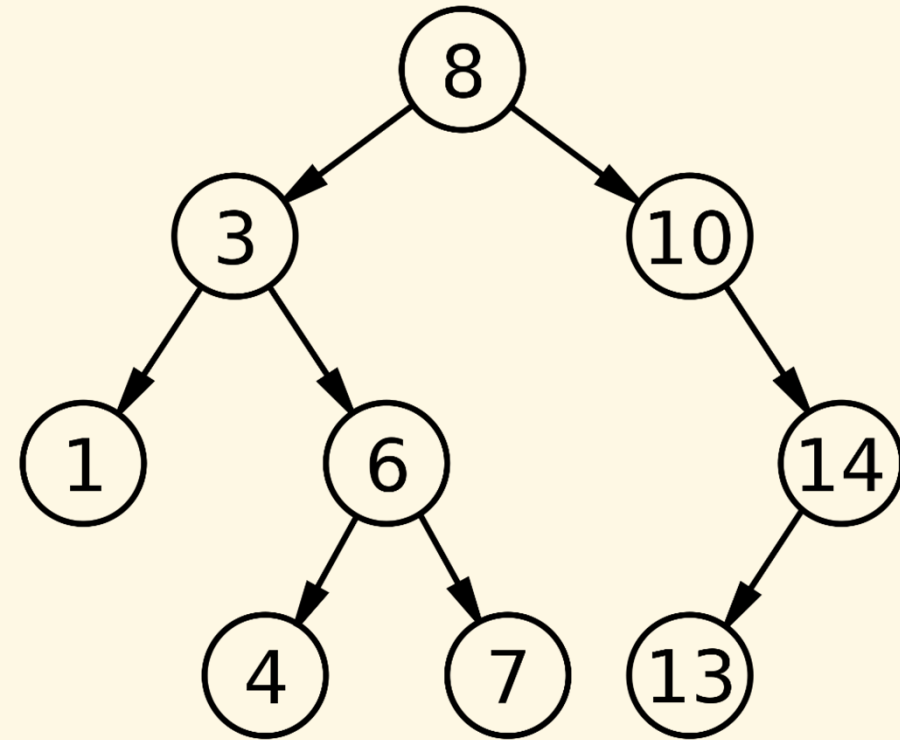
- ▶ Both left and right NULL pointers are made to point to in-order predecessor and in-order successor, respectively
- ▶ The predecessor threads are useful for reverse in-order traversal and post-order traversal



Binary search tree (BST)

Overview

- ▶ Binary tree can be used in binary search
- ▶ However, the nodes need to be organized in a specific order: for every node,
 - ▶ the values on left branch must be smaller than its one
 - ▶ the values on the right branch must be greater its one
 - ▶ there are not two nodes with the same key
- ▶ Note that subtrees are also BST



Operations (with `int` data type)

- ▶ `void bstInsertNode(BT_NODE** n, int x);`
- ▶ `void bstDeleteNode(BT_NODE** n, int x);`
- ▶ `BT_NODE* bstSearch(BT_NODE* n, int x);`

Insert node in BST

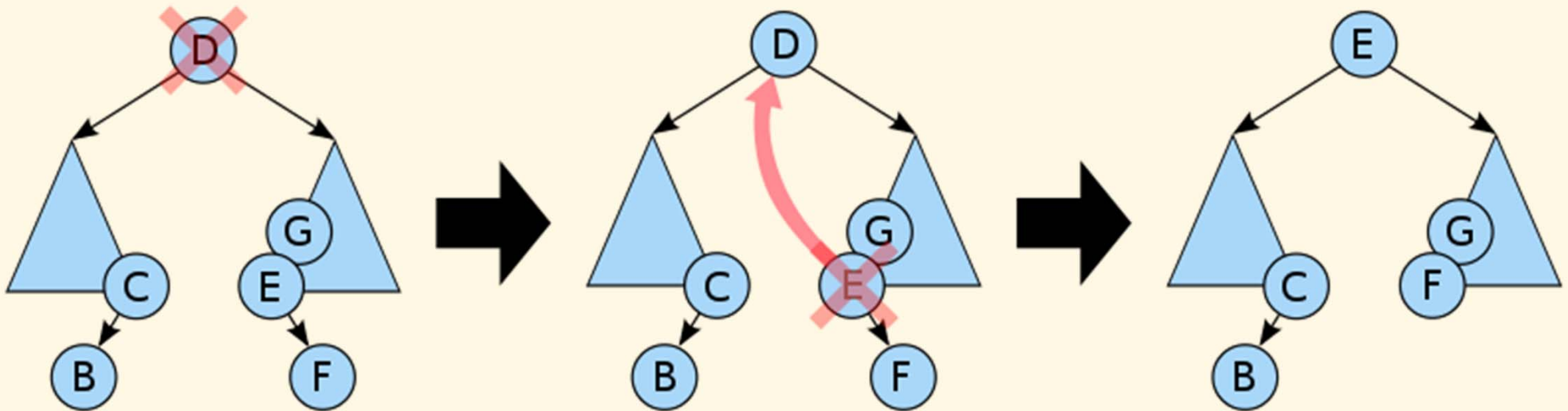
```
void bstInsertNode(BT_NODE** n, int x) {
    if (*n == NULL) {
        *n = (BT_NODE*)malloc(sizeof(BT_NODE));
        (*n)->data = x;
        (*n)->left = NULL;
        (*n)->right = NULL;
    } else if (x < (*n)->data)
        bstInsertNode(&(*n)->left, x);
    else if (x > (*n)->data)
        bstInsertNode(&(*n)->right, x);
}
```

Search node in BST

```
BT_NODE* bstSearch(BT_NODE* n, int x) {  
    if (n == NULL) return NULL; // not found  
    if (n->data == x) return n; // found  
  
    if (n->data > x) return bstSearch(n->left, x);  
    return bstSearch(n->right, x);  
}
```

Delete node in BST

- ▶ 3 cases
 - ▶ Delete a node with no children
 - ▶ Delete a node with one child
 - ▶ Delete a node with two children (*see picture*)



BST shapes

- ▶ The order of supplying the data determines where it is placed in the BST, which determines the shape of the BST
- ▶ Create BSTs from the same set of data presented each time in a different order:
 - ▶ 17 4 14 19 15 7 9 3 16 10
 - ▶ 9 10 17 4 3 7 14 16 15 19
 - ▶ 19 17 16 15 14 10 9 7 4 3
- ➔ Balanced search trees to avoid tree degeneration
 - ▶ AVL trees
 - ▶ 2-3-4 trees
 - ▶ Red-black trees

AVL tree *(for reference)*

Overview

- ▶ Operations on binary search tree are $O(h)$ - where h is tree height
 - ▶ Tree must be balanced so that $h \approx \log n$
- ▶ Add and remove operations do not necessarily maintain that balance
- ▶ We seek ways to maintain balance and thus maintain efficiency

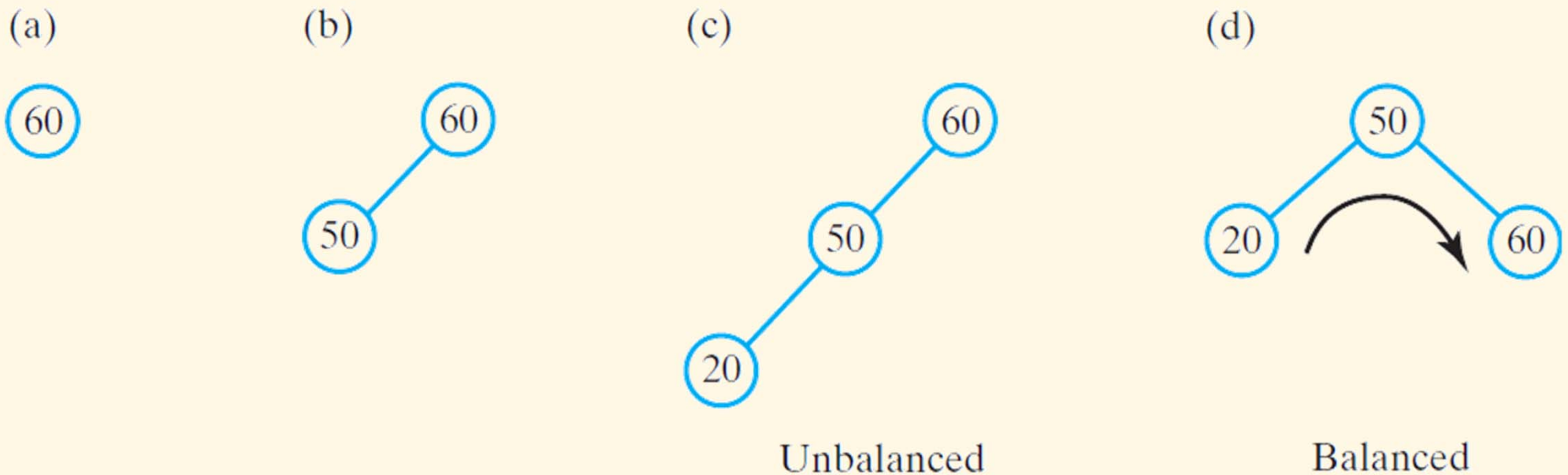
AVL tree

- ▶ Binary search tree that rearranges nodes whenever it becomes unbalanced
 - ▶ Happens during addition or removal of nodes
- ▶ Uses
 - ▶ Left rotation
 - ▶ Right rotation
 - ▶ Balanced node – the root of a balanced tree



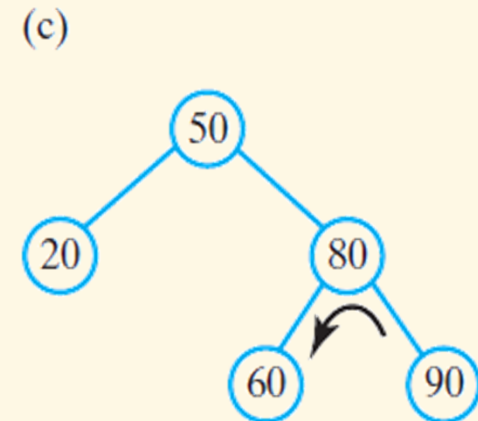
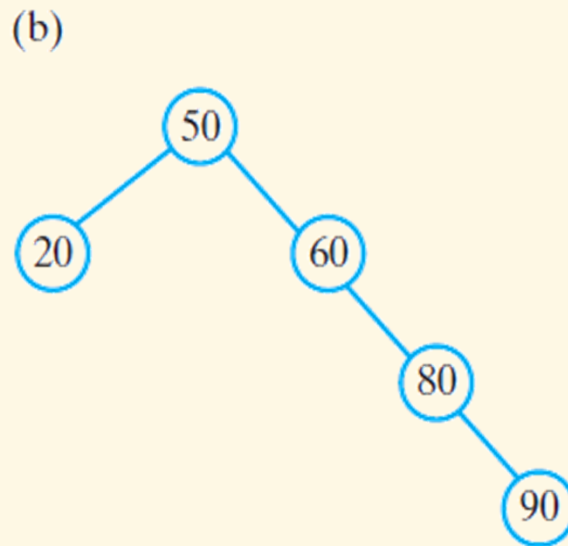
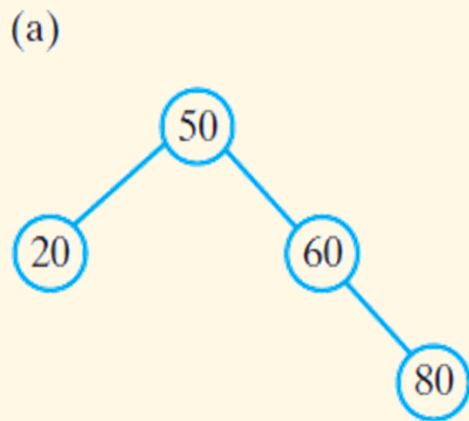
AVL tree

- ▶ (a, b, c) After inserting 60, 50 and 20 into an initially empty binary search tree, the tree is not balanced
- ▶ (d) A corresponding AVL tree rotates its nodes to restore balance



AVL tree

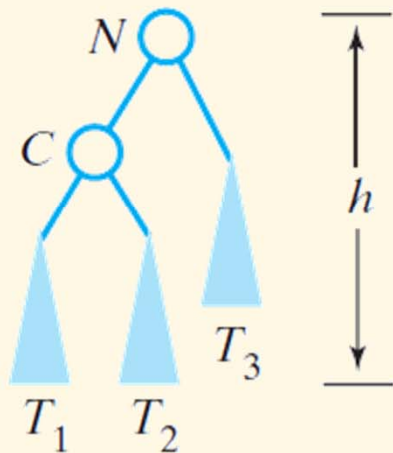
- ▶ (a) Adding 80 to the tree in Figure 27-1d does not change the balance of the tree
- ▶ (b) A subsequent addition of 90 makes the tree unbalanced
- ▶ (c) A left rotation restores its balance



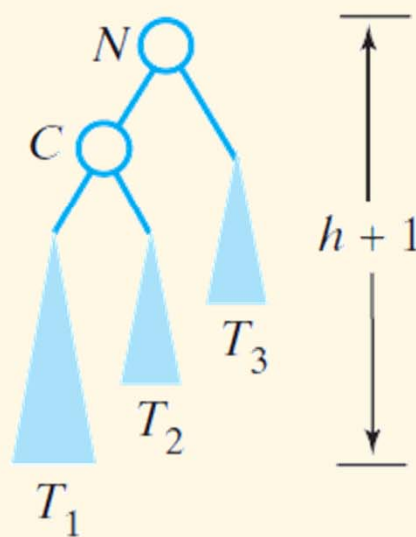
AVL tree

- Before and after an addition to an AVL subtree that requires a right rotation to maintain its balance

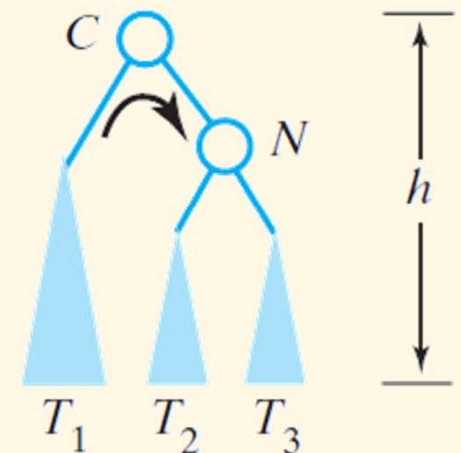
(a) Before addition



(b) After addition

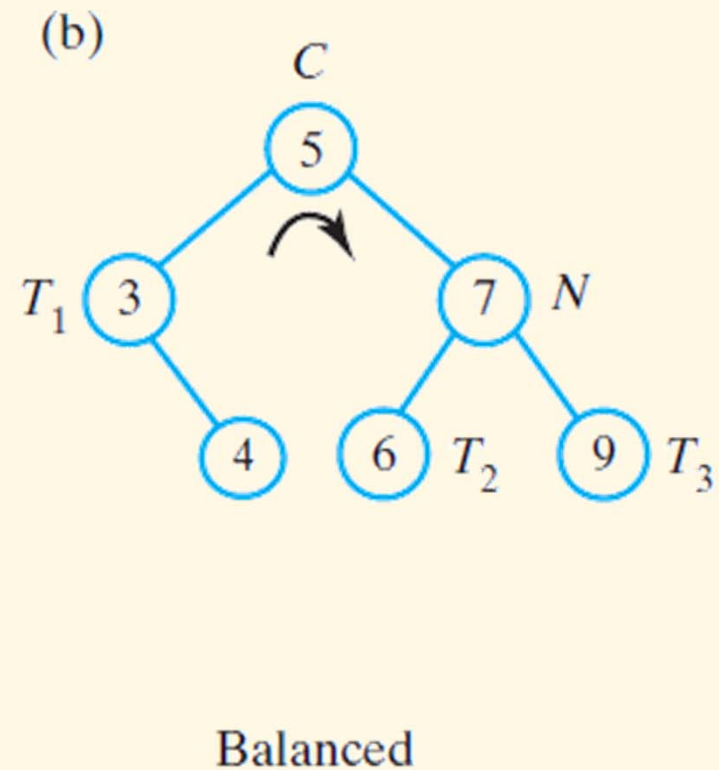
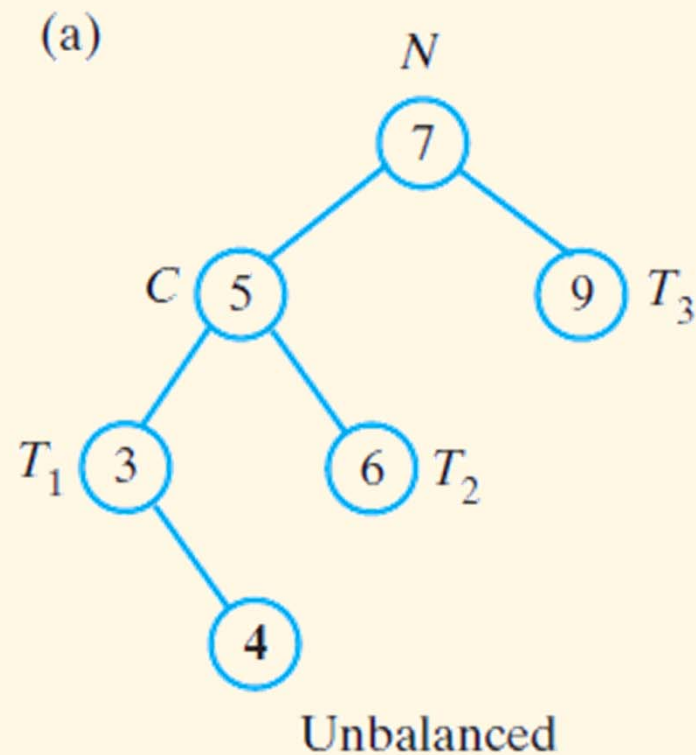


(c) After right rotation



AVL tree

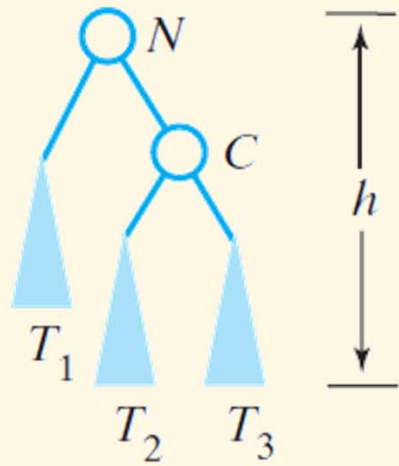
- Before and after a right rotation restores balance to an AVL tree



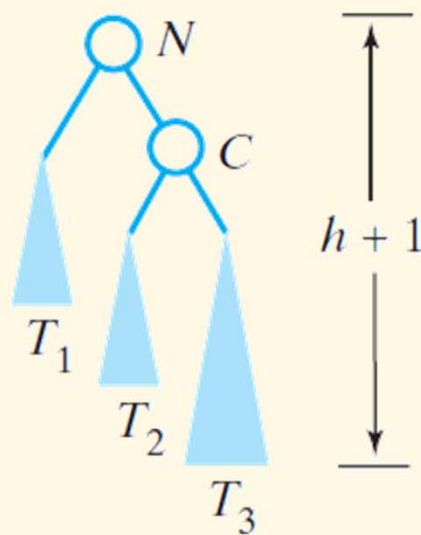
AVL tree

- Before and after an addition to an AVL subtree that requires a left rotation to maintain its balance

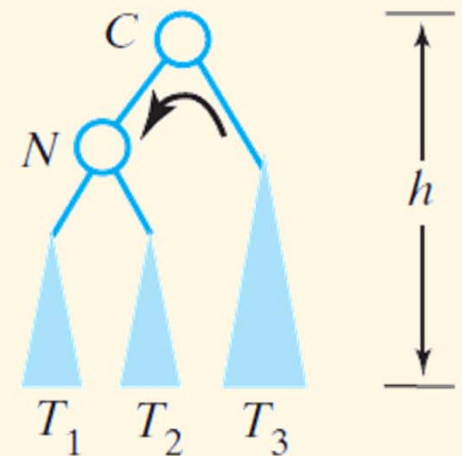
(a) Before addition



(b) After addition



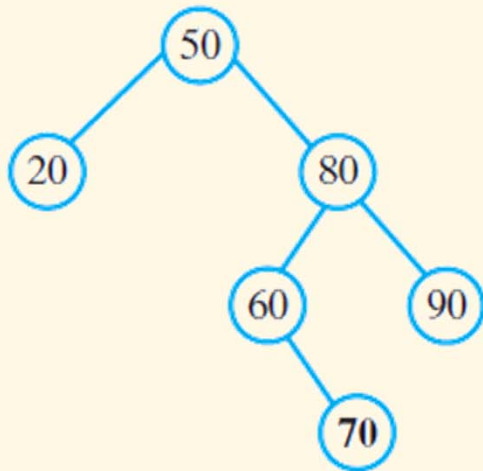
(c) After left rotation



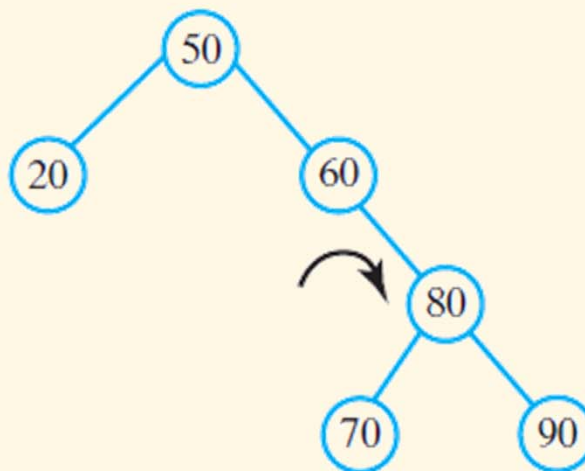
AVL tree

- ▶ (a) Adding 70 to the tree destroys its balance
- ▶ (b, c) To restore the balance, perform both a right rotation and a left rotation

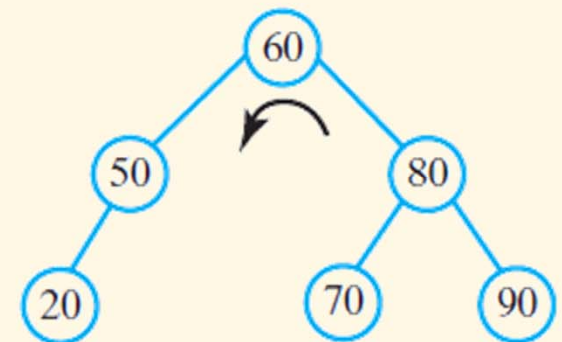
(a) After adding 70



(b) After right rotation



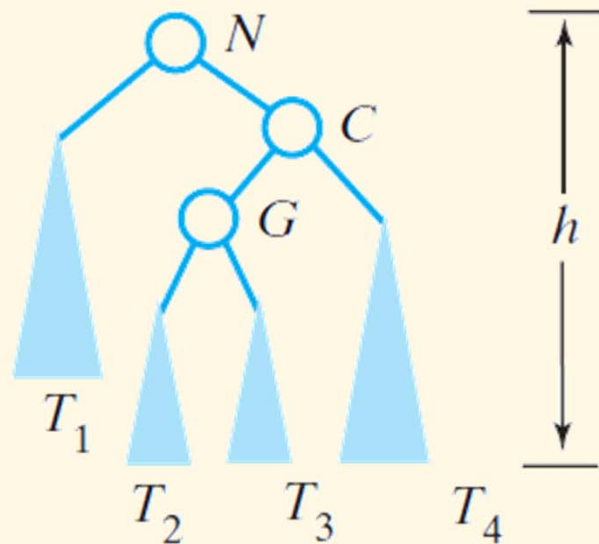
(c) After left rotation



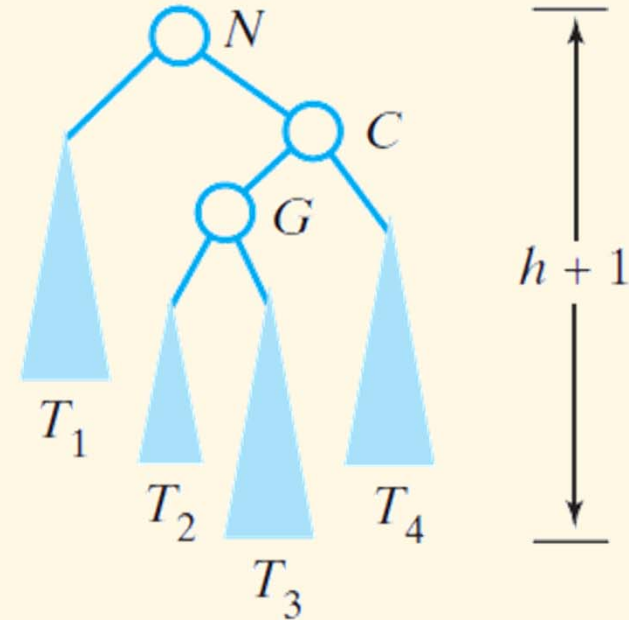
AVL tree

- Before and after an addition to an AVL subtree that requires both a right rotation and a left rotation to maintain its balance

(a) Before addition



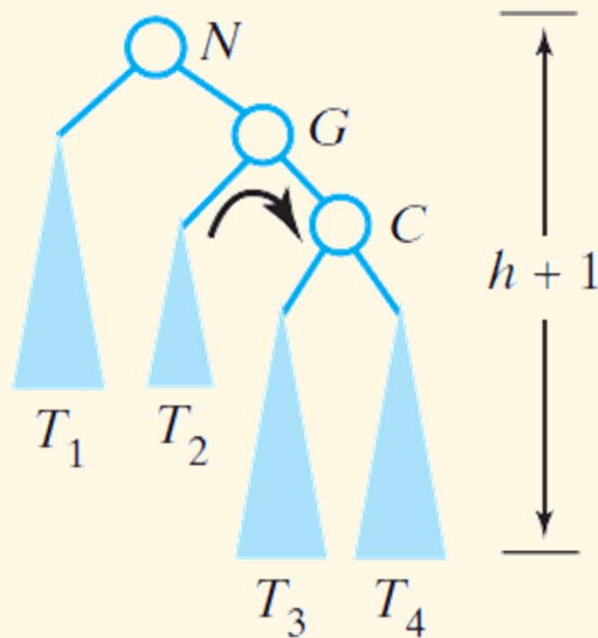
(b) After addition



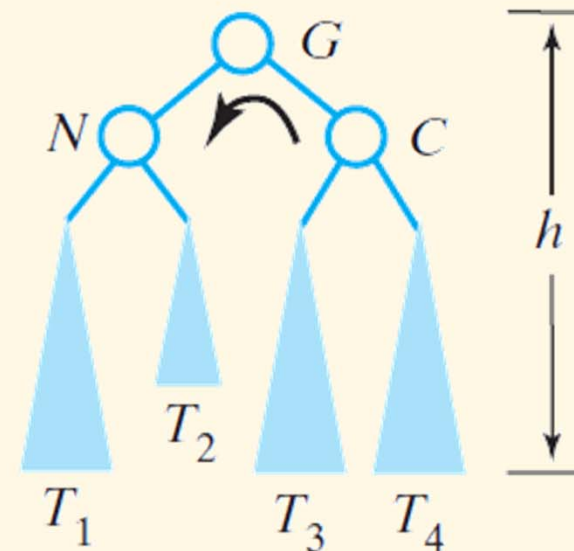
AVL tree

- Before and after an addition to an AVL subtree that requires both a right rotation and a left rotation to maintain its balance

(c) After right rotation



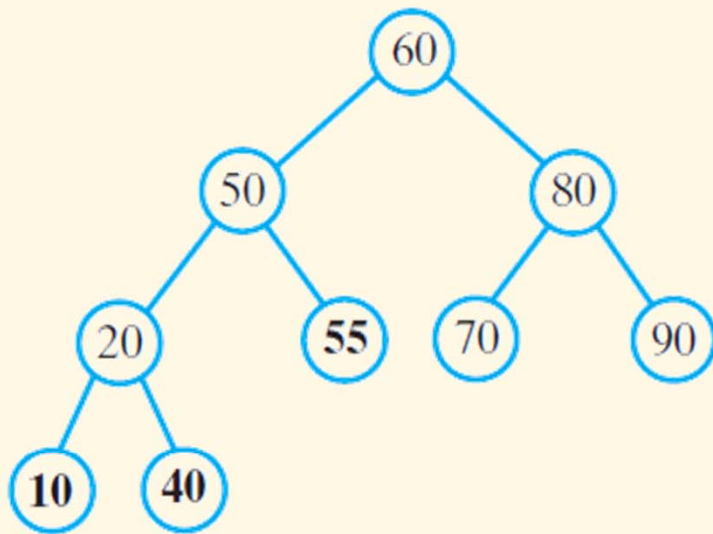
(d) After left rotation



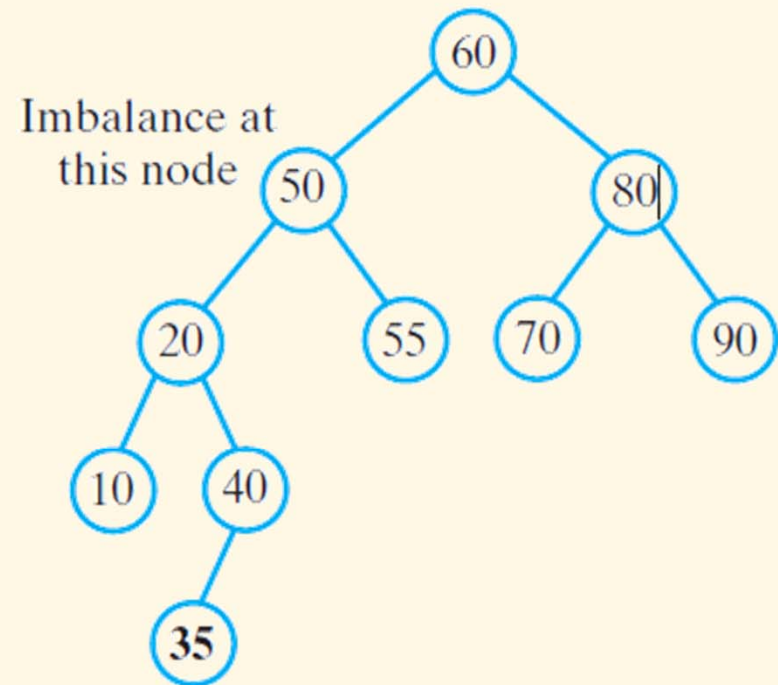
AVL tree

- ▶ (a) After additions that maintain its balance
- ▶ (b) After an addition that destroys the balance

(a) After adding 55, 10, and 40



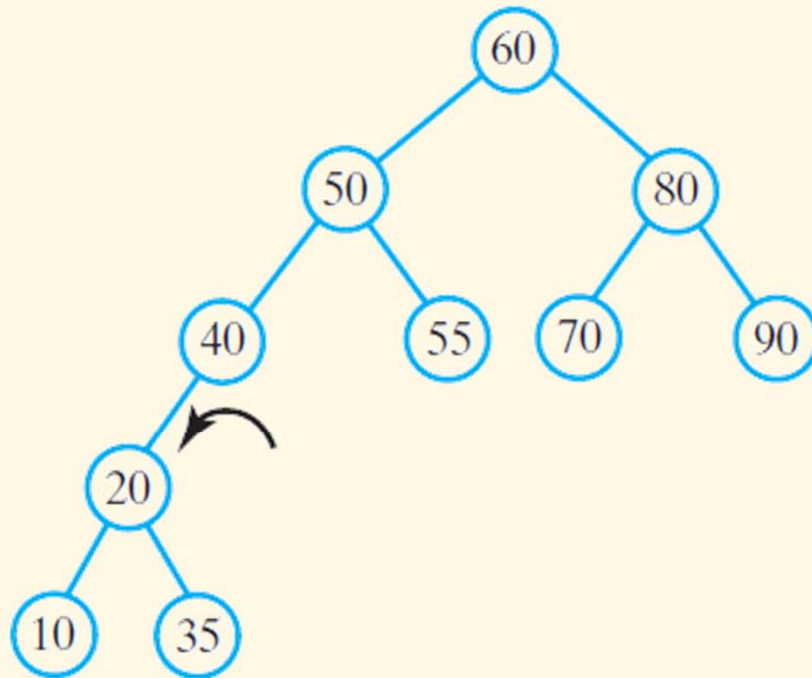
(b) After adding 35



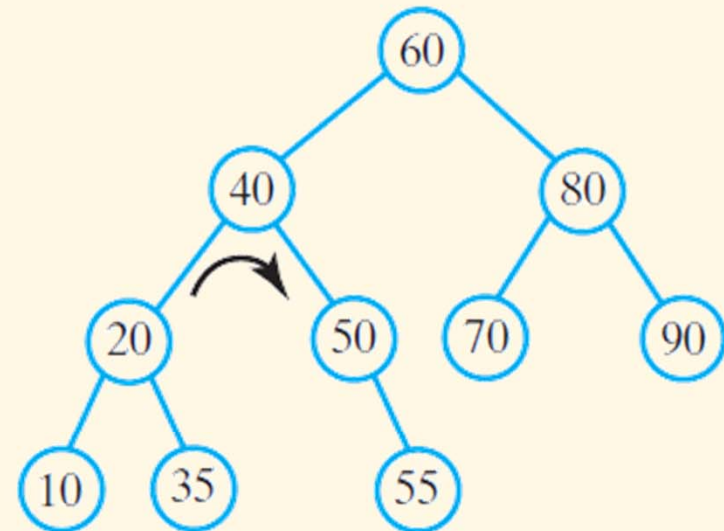
AVL tree

- ▶ (c) after a left rotation
- ▶ (d) after a right rotation

(c) After left rotation about 40



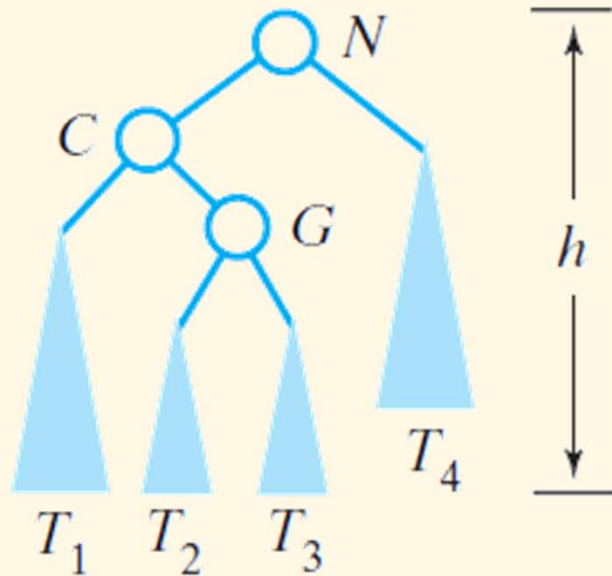
(d) After right rotation about 40



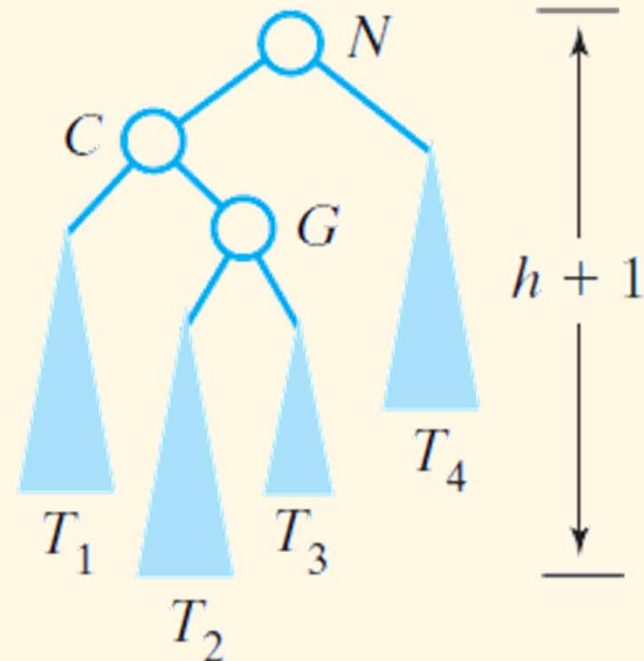
AVL tree

- Before and after an addition to an AVL subtree that requires both a left rotation and a right rotation to maintain its balance

(a) Before addition



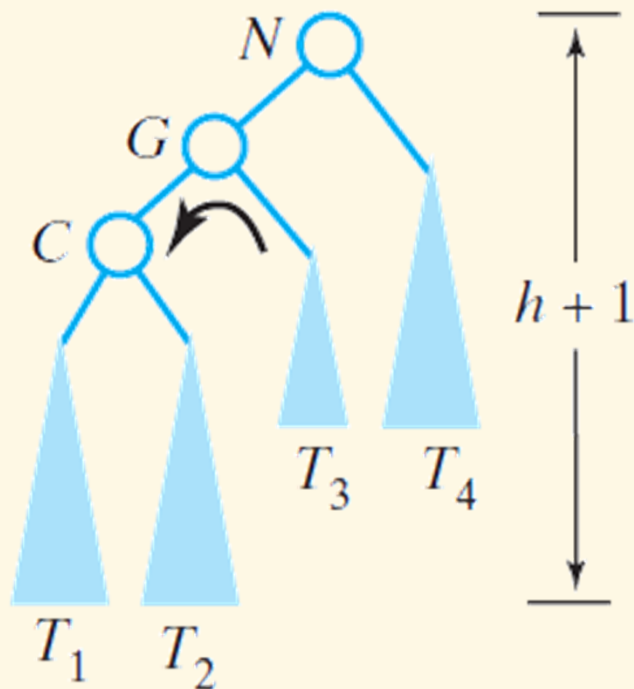
(b) After addition



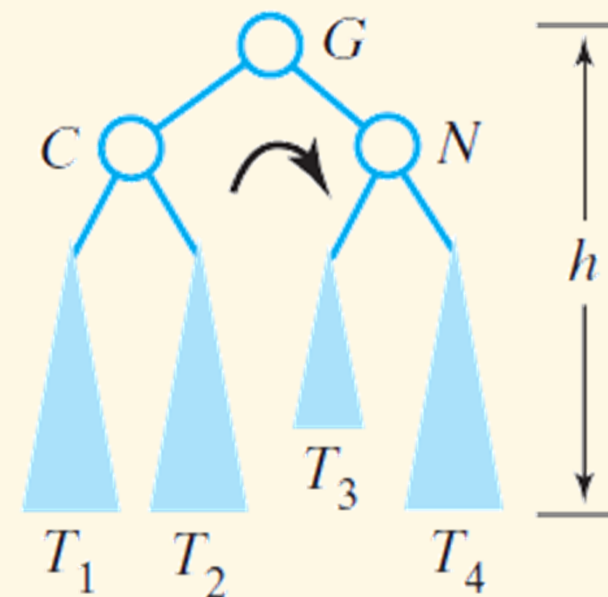
AVL tree

- Before and after an addition to an AVL subtree that requires both a left rotation and a right rotation to maintain its balance

(c) After left rotation

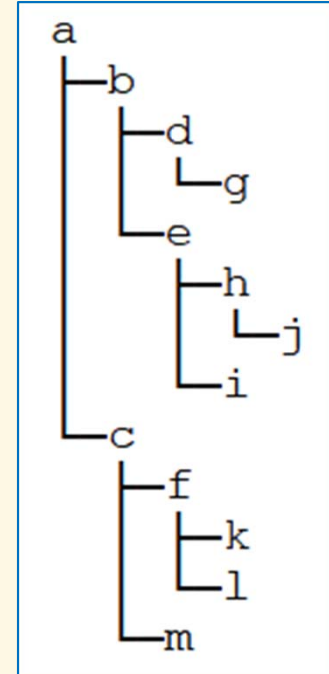


(d) After right rotation



Homework

1. Write a function to print a binary tree in a pretty way (see picture)
2. Write a function to calculate number of leaf nodes in a tree
3. Write a function to check if a binary tree is complete
4. Write a function to check if a binary tree is BST
5. Implement the deletion of nodes in BST
6. Write a program to create a binary tree representing the following expression, then evaluate the value using the tree



$$25^2/5 - (6 + 3)^2 \times (4 + 2)$$

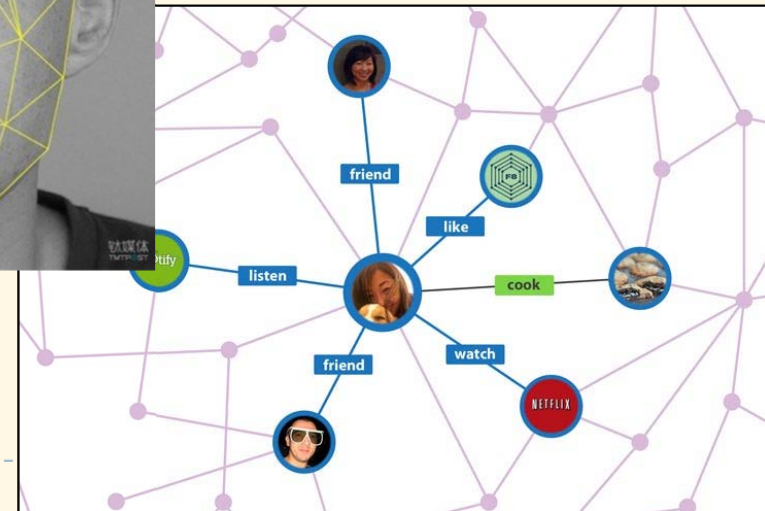
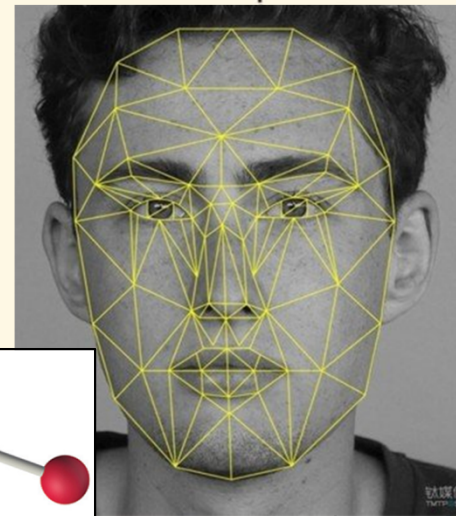
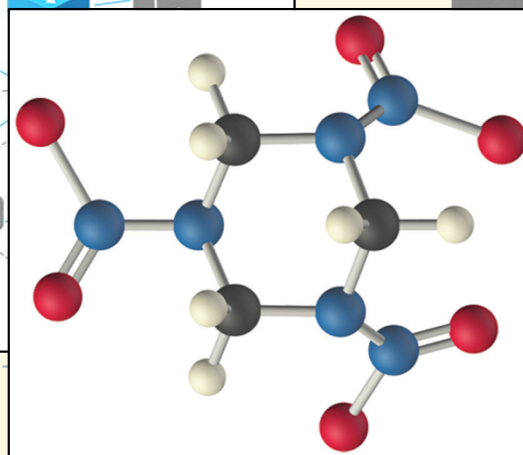
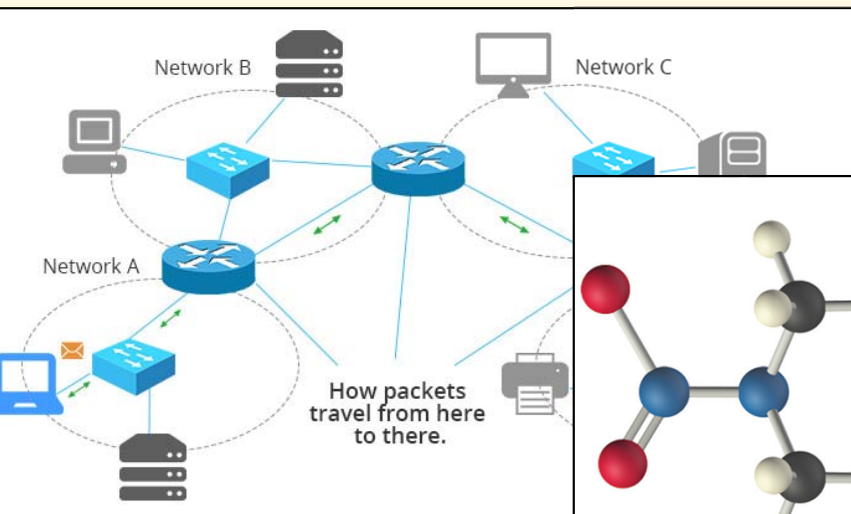
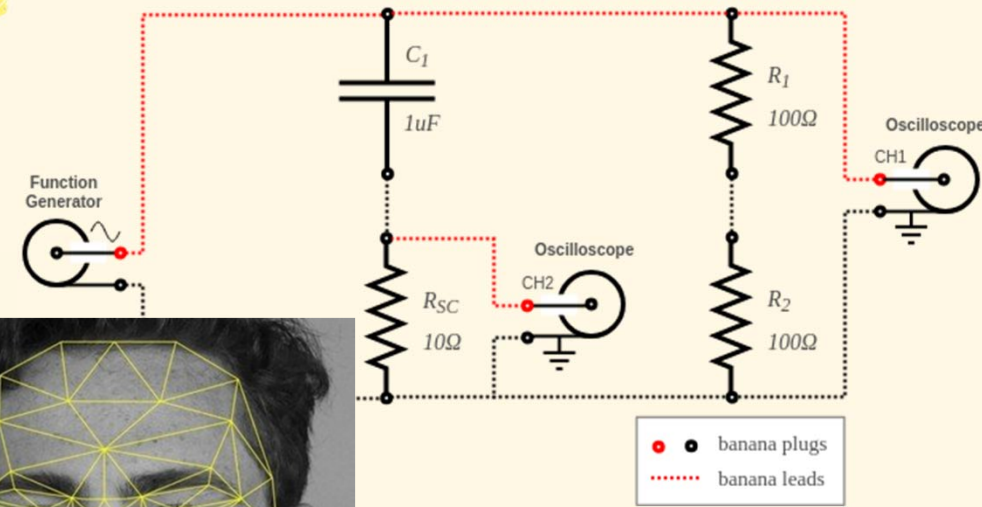
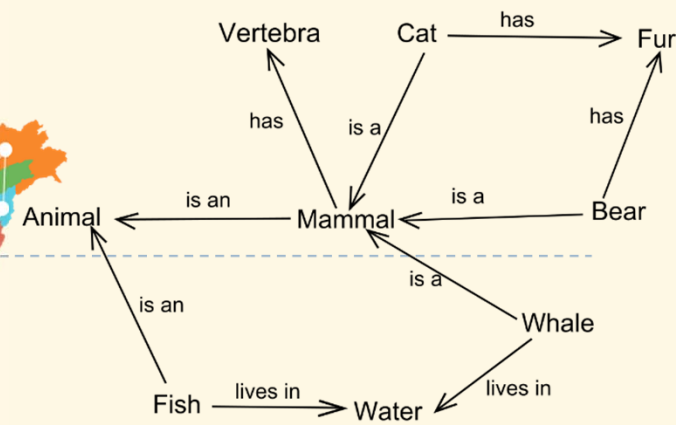
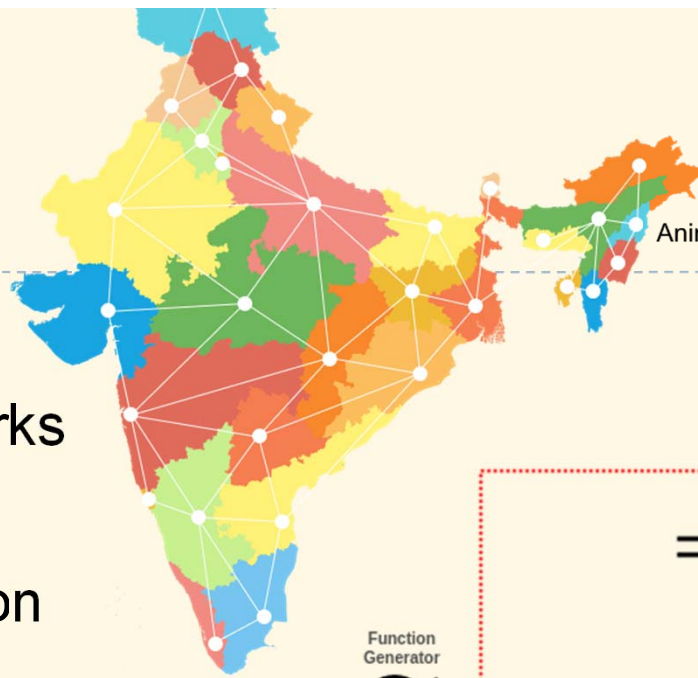
Graphs

- ❑ Definitions
- ❑ Implementation
- ❑ Traversal, search, shortest path

Overview of Graphs

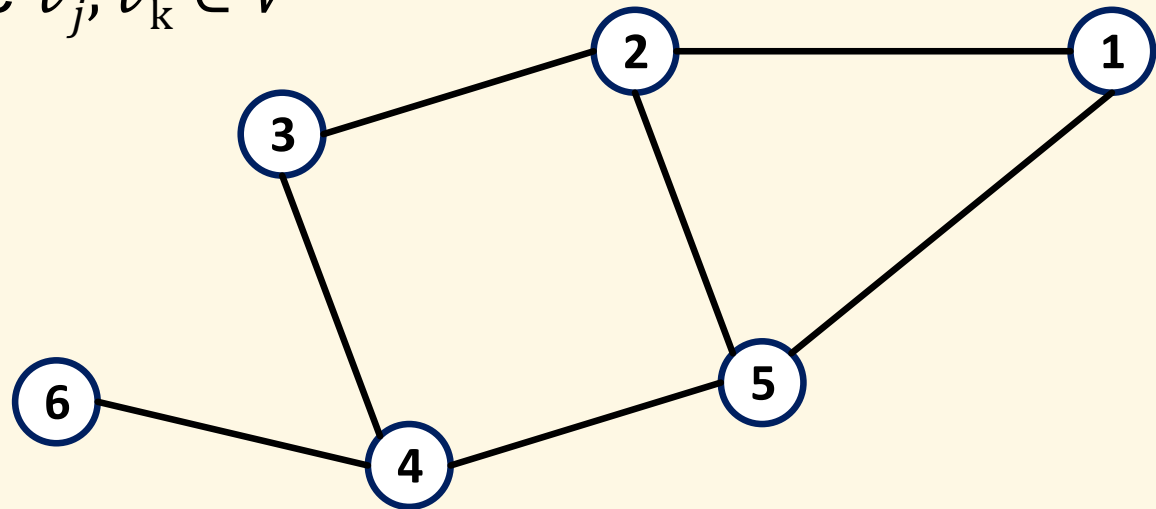
Introduction

- ▶ Traffic networks
- ▶ Communication networks
- ▶ Social networks
- ▶ Semantic representation
- ▶ Electrical circuits
- ▶ Chemical molecular structures
- ▶ ...



Definitions

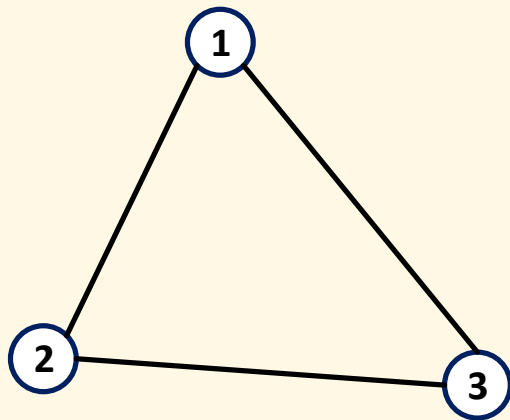
- ▶ Graph is a pair: $G = \langle V, E \rangle$, where:
 - ▶ $V = \{v_1, v_2, \dots, v_n\}$ are *vertices*, or nodes
 - ▶ $E = \{e_1, e_2, \dots, e_m\}$ are *edges*, or connections
 - ▶ $e_i = \langle v_j, v_k \rangle$ where $v_j, v_k \in V$



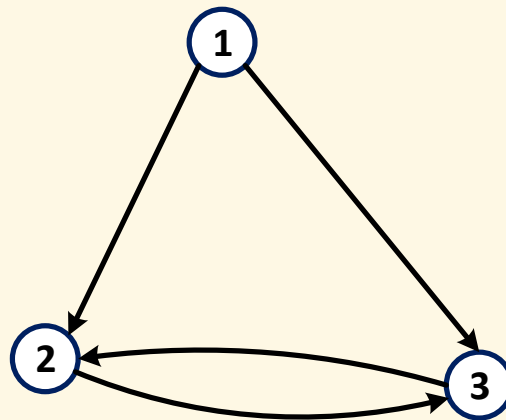
- ▶ Example
 - ▶ $V = \{1,2,3,4,5,6\}$
 - ▶ $E = \{\langle 1,2 \rangle, \langle 2,3 \rangle, \langle 2,5 \rangle, \langle 3,4 \rangle, \langle 4,5 \rangle, \langle 4,6 \rangle\}$
- ▶ Trees are graphs

Directivity

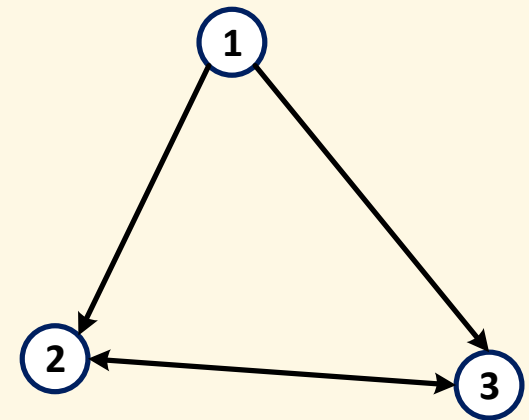
- ▶ **Undirected graph**: edges are unordered
 - ▶ $E_1 = \{\langle 1,2 \rangle, \langle 1,3 \rangle, \langle 2,3 \rangle\}$
 - ▶ $\langle u, v \rangle \in E$ implies that $\langle v, u \rangle \in E$
- ▶ **Directed graph**: edges are ordered
 - ▶ $E_2 = \{\langle 1,2 \rangle, \langle 1,3 \rangle, \langle 2,3 \rangle, \langle 3,2 \rangle\}$
- ▶ Trees can be either directed or undirected



G_1 : undirected



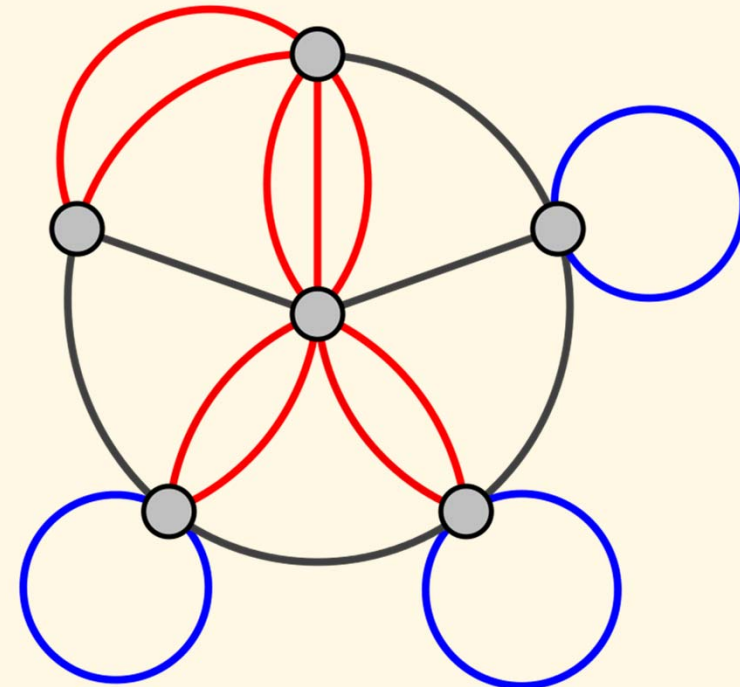
G_2 : directed



G_2 : directed with simplified drawing

Self-edges, multi-edges

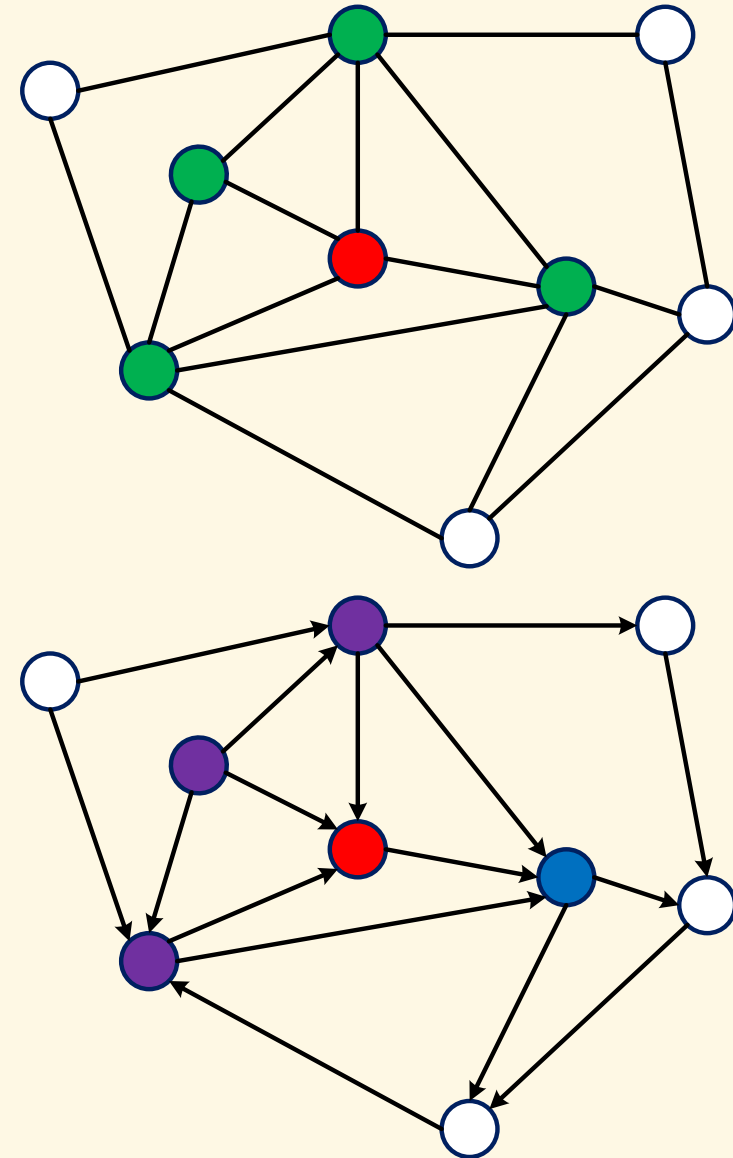
- ▶ A *self-edge* (aka loop edge) is an edge where its two vertices are identical: $\langle u, u \rangle$
- ▶ A *multi-edge* (aka parallel edge) is an edge that appears multiple times
- ▶ A graph is called *simple* iff it has no self-edge neither multi-edge
 - ▶ Otherwise, the graph is called *multi-graph*
- ▶ In normal convention, if no further note given, only simple graphs are considered



A multi-graph with **multi-edges** and **self-edges**

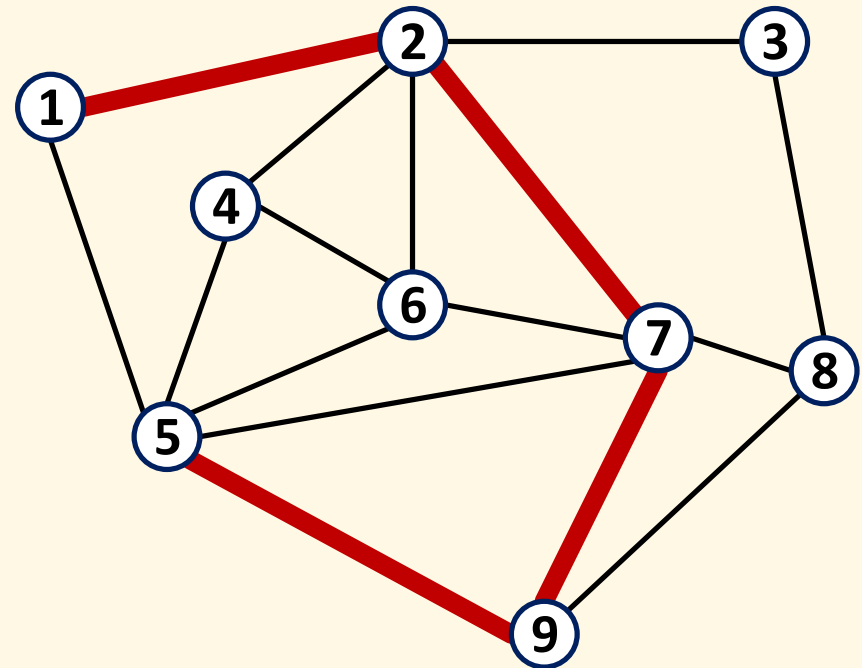
Neighborhood

- ▶ In undirected graph: vertex u is *neighbor* of v iff there exists an edge composed of those two vertices
 - ▶ *Degree* of a vertex: number of its neighbors
- ▶ In directed graph:
 - ▶ u is an *out-neighbor* of v iff there exists an edge $\langle v, u \rangle$
 - ▶ u is an *in-neighbor* of v iff there exists an edge $\langle u, v \rangle$
 - ▶ u is *neighbor* of v iff u is either an out-neighbor or an in-neighbor of v
 - ▶ *In-degree*, *out-degree*: numbers of in- and out-neighbors



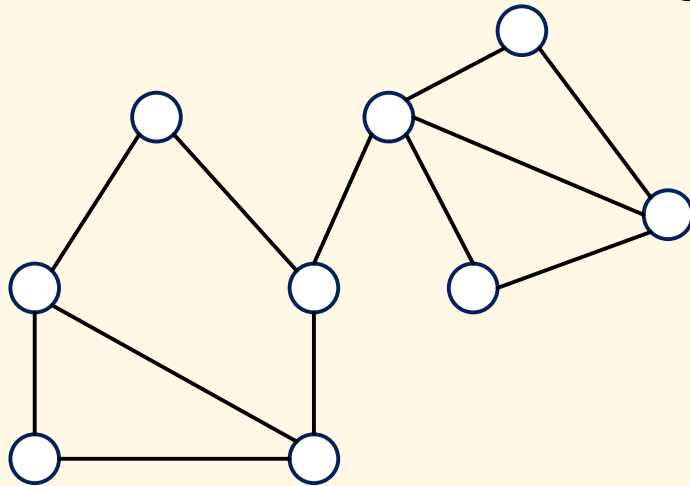
Paths

- ▶ A **path** p is an ordered list $\langle v_1, v_2, \dots, v_k \rangle$ of vertices, where v_{i+1} is neighbor (in undirected graph, or out-neighbor in directed graph) of v_i for any $i = 2..k$
 - ▶ p is called a path from v_1 to v_k
- ▶ **Length** of a path is the number of edges on it
- ▶ Example: $\langle 1, 2, 7, 9, 5 \rangle$
 - ▶ Length: 4

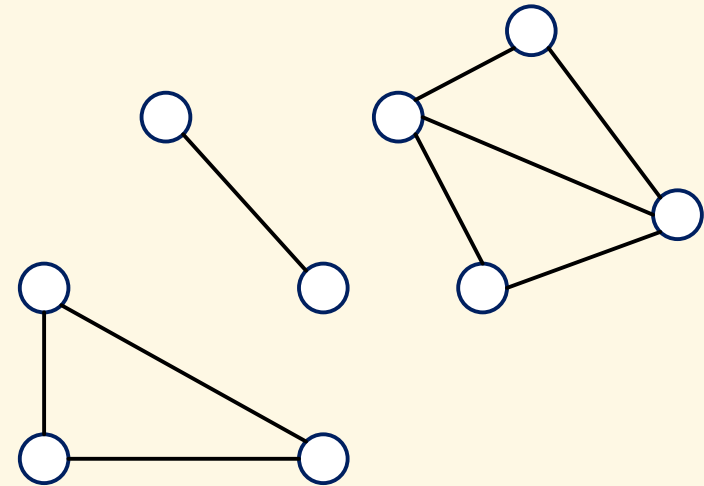


Connectivity

- ▶ Two vertices u, v are *connected* iff there exists at least one path from u to v ; otherwise, they are disconnected
- ▶ A graph is called *connected* if every pair of vertices are connected; otherwise, it is *disconnected*
- ▶ Trees are connected graphs



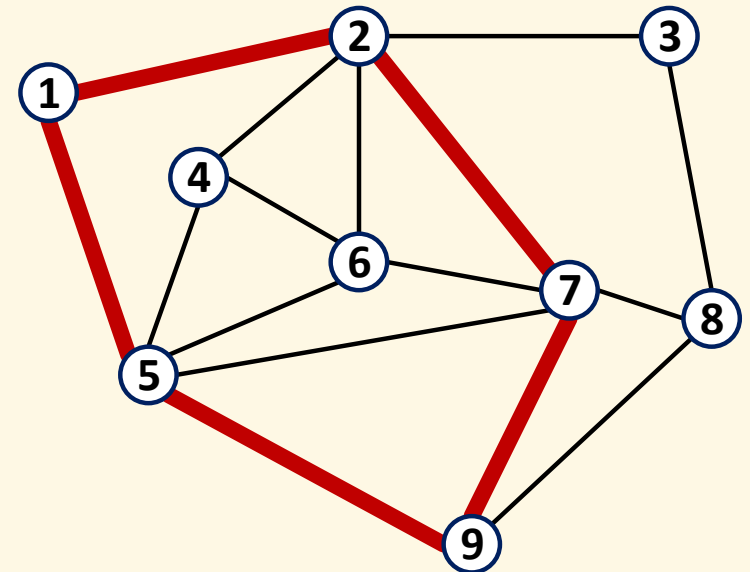
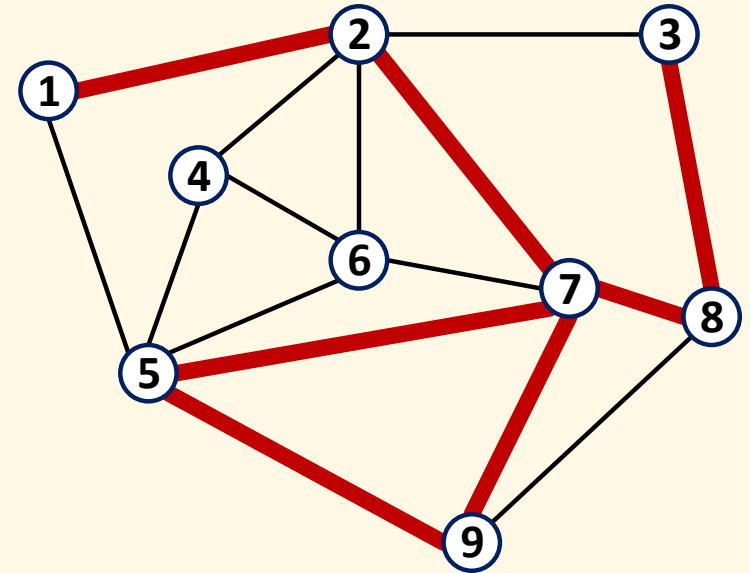
Connected graph



Disconnected graph

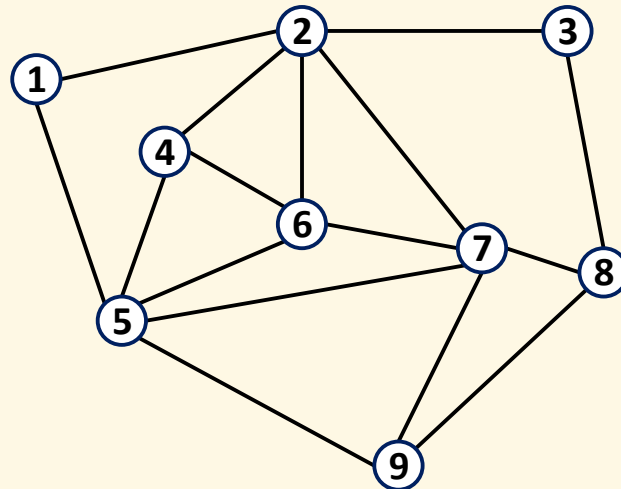
Simple Paths, Cycles

- ▶ A path is called *simple* iff all vertices, except possibly the first and the last, are distinct
 - ▶ Counter-example:
 $\langle 1, 2, 7, 9, 5, 7, 8, 3 \rangle$
- ▶ A *cycle* is a simple path in which the first and the last vertices are identical
 - ▶ Example: $\langle 1, 2, 7, 9, 5, 1 \rangle$

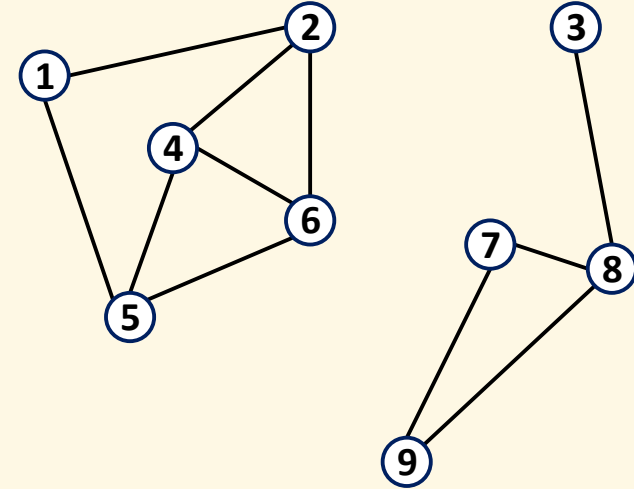


Cyclicity

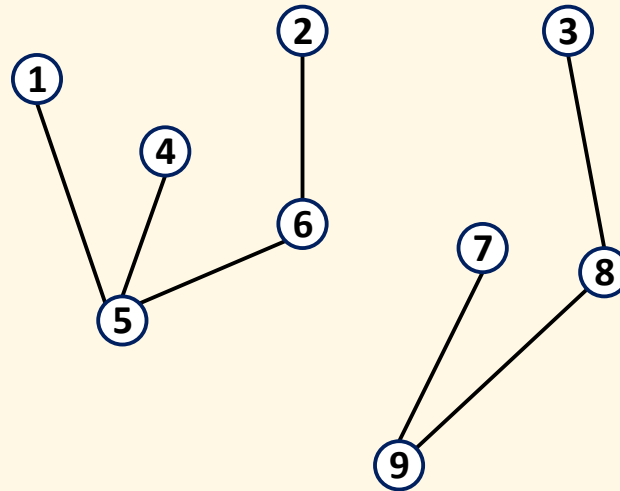
- ▶ A graph with cycles is called *cyclic*, otherwise, it is *acyclic*
- ▶ A connected acyclic graph is called *tree*
- ▶ A disconnected acyclic graph is called *forest*



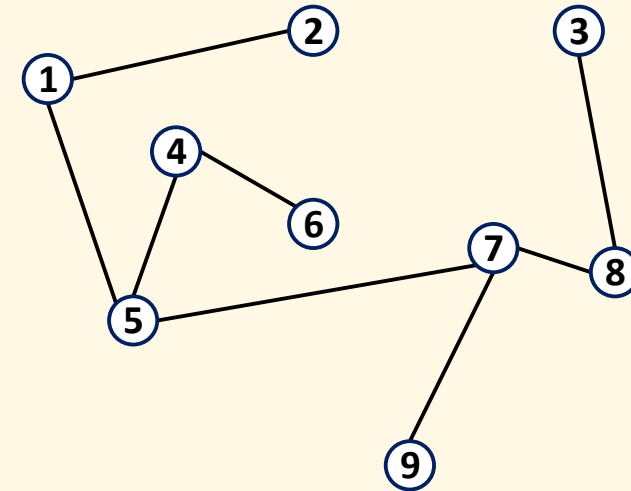
Connected cyclic graph



Disconnected cyclic graph



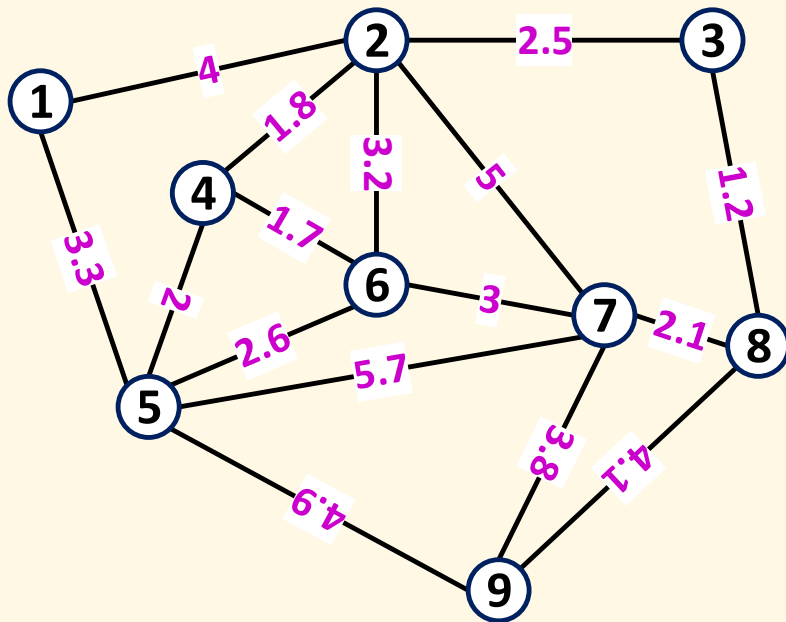
Disconnected acyclic graph (forest)



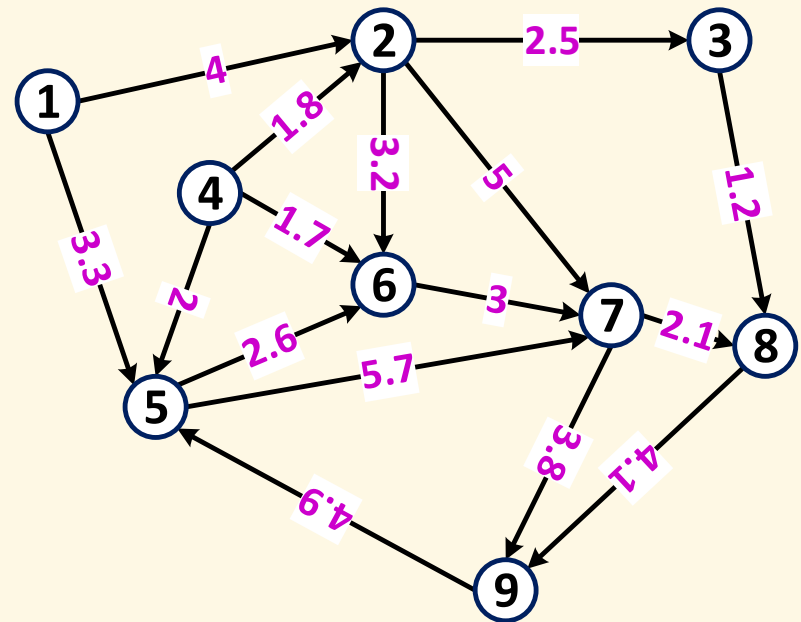
Connected acyclic graph (tree)

Weighted Graphs

- ▶ In a weighted graph, each edge is associated with a value called *weight* (or *cost*)



Undirected weighted graph

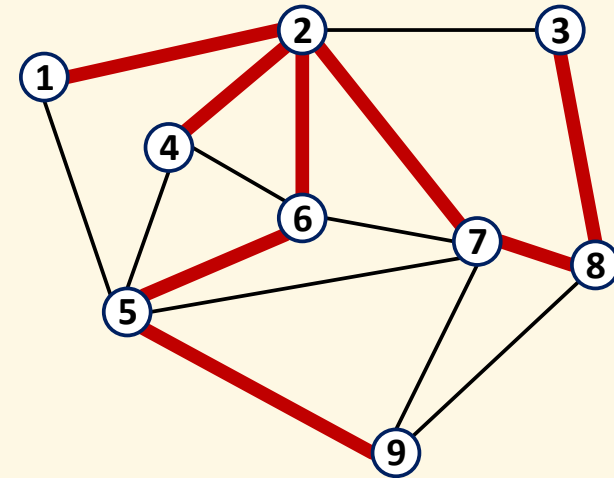
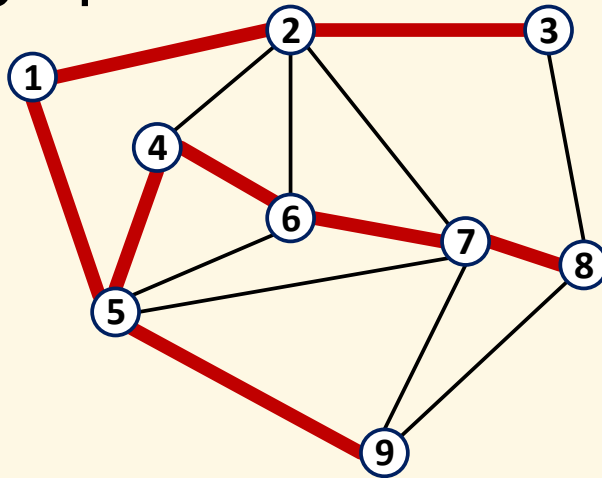


Directed weighted graph

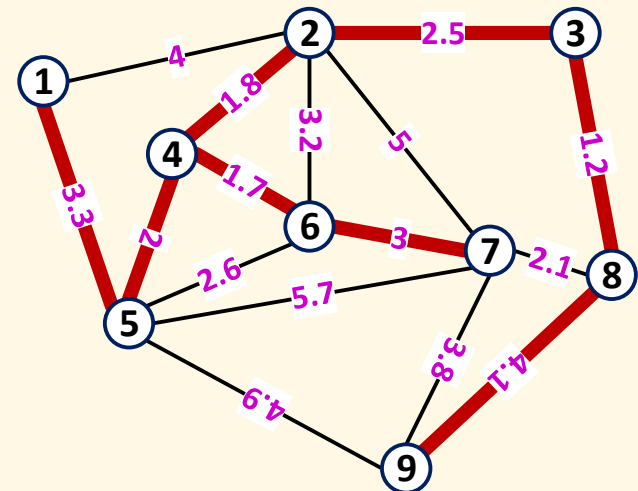
- ▶ *Cost of a path* is summation of all the member edges

Spanning Tree

- Given a connected graph G , a *spanning tree* is an acyclic subgraph of G which covers all vertices of G



- There may exist multiple spanning trees for a given graph
- Minimum spanning tree* is a spanning tree whose total edge weight is minimum

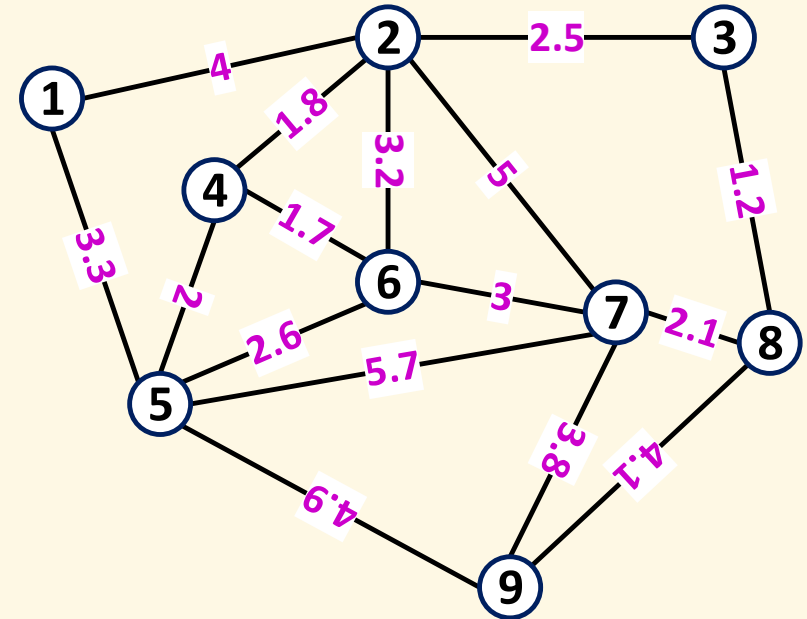


Graph Implementation using a Matrix

(Two-Dimensional Array, aka Adjacent Matrix)

Undirected Graph

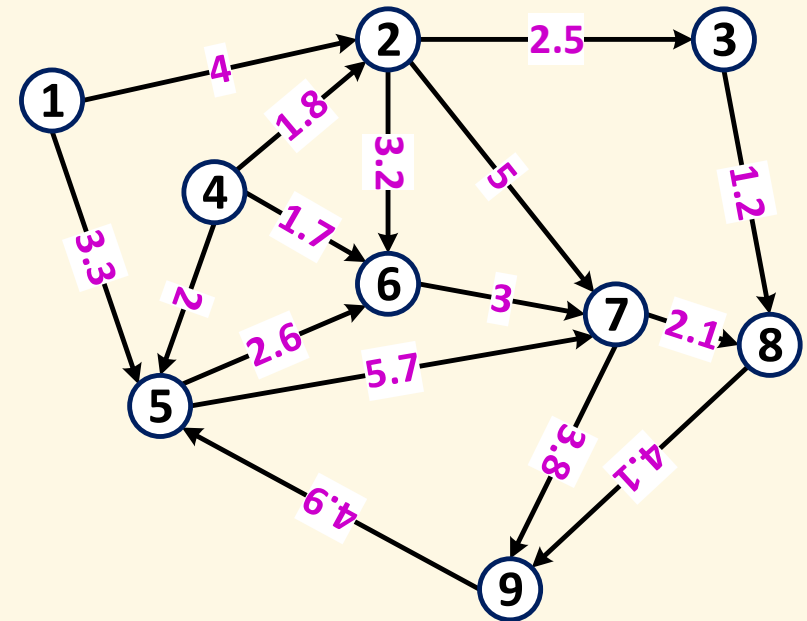
	1	2	3	4	5	6	7	8	9
1	0	4	∞	∞	3.3	∞	∞	∞	∞
2	4	0	2.5	1.8	∞	3.2	5	∞	∞
3	∞	2.5	0	∞	∞	∞	∞	1.2	∞
4	∞	1.8	∞	0	2	1.7	∞	∞	∞
5	3.3	∞	∞	2	0	2.6	5.7	∞	4.9
6	∞	3.2	∞	1.7	2.6	0	3	∞	∞
7	∞	5	∞	∞	5.7	3	0	2.1	3.8
8	∞	∞	1.2	∞	∞	∞	2.1	0	4.1
9	∞	∞	∞	∞	4.9	∞	3.8	4.1	0



- ▶ $a[i, j]$ represents the weight from vertex i to vertex j
 - ▶ Use a special value like -1 , NULL or NaN to represent ∞
 - ▶ Symmetricity: $a[i, j] = a[j, i]$

Directed Graph

	1	2	3	4	5	6	7	8	9
1	0	4	∞	∞	3.3	∞	∞	∞	∞
2	∞	0	2.5	∞	∞	3.2	5	∞	∞
3	∞	∞	0	∞	∞	∞	∞	1.2	∞
4	∞	1.8	∞	0	2	1.7	∞	∞	∞
5	∞	∞	∞	∞	0	2.6	5.7	∞	∞
6	∞	∞	∞	∞	∞	0	3	∞	∞
7	∞	∞	∞	∞	∞	∞	0	2.1	3.8
8	∞	∞	∞	∞	∞	∞	∞	0	4.1
9	∞	∞	∞	∞	4.9	∞	∞	∞	0



- Symmetricity not satisfied

Implementation

- ▶ Be aware that array indices start from 0

```
double G[9][9] = {  
    {0, 4, -1, -1, 3.3, -1, -1, -1, -1},  
    {4, 0, 2.5, 1.8, -1, 3.2, 5, -1, -1},  
    {-1, 2.5, 0, -1, -1, -1, -1, 1.2, -1},  
    // ...  
};
```

- ▶ To evaluate an edge from vertex i to j : $G[i][j]$

Exercise

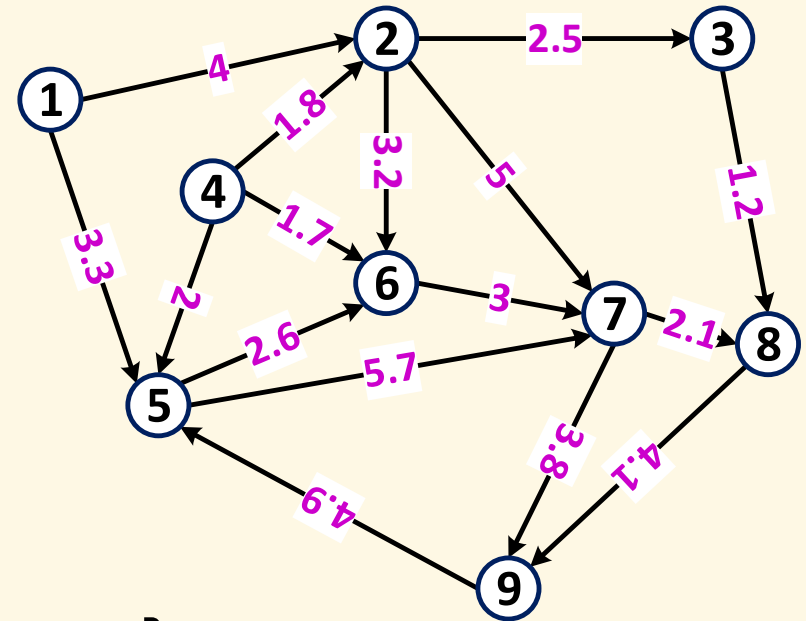
- ▶ Given a list of vertices: $P[0..k]$
 - ▶ Write the code to check if it is a path
 - ▶ Write the code to check if it is a simple path
 - ▶ Write the code to check if it is a cycle
 - ▶ Write the code to calculate the total cost of P
- ▶ Given two vertices i, j
 - ▶ Write the code to check if they are [in-, out-] neighbors
 - ▶ Write the code to check if they are connected
- ▶ Given a graph G
 - ▶ Write the code to check if it is cyclic
 - ▶ Write the code to check if it is connected

Graph Implementation using Separate Node and Edge Arrays/Lists

Example with Directed Graph

```
Vertex V[] = {  
    {.name = "1"},  
    {.name = "2"},  
    // ...  
};
```

```
Edge E[] = {  
    {.from = 0, .to = 1, .weight = 4},  
    {.from = 0, .to = 4, .weight = 3.3},  
    // ...  
};
```



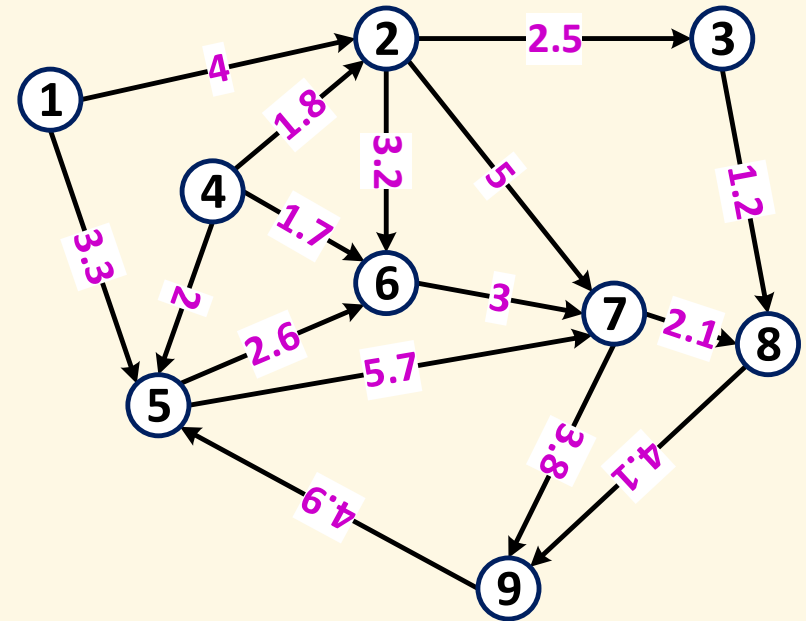
Exercise

- ▶ Same as for matrix-based method

Graph Implementation using a Node Array with Embedded Edges

Example with Directed Graph

```
Vertex V[] = {  
  {.name = "1", .edges = {  
    {.to = 1, .weight = 4},  
    {.to = 4, .weight = 3.3}  
  }},  
  
  {.name = "2", .edges = {  
    {.to = 2, .weight = 2.5},  
    {.to = 5, .weight = 3.2},  
    {.to = 6, .weight = 5}  
  }},  
  
  //...  
};
```



Exercise

- ▶ Same as for matrix-based method

Graph Search

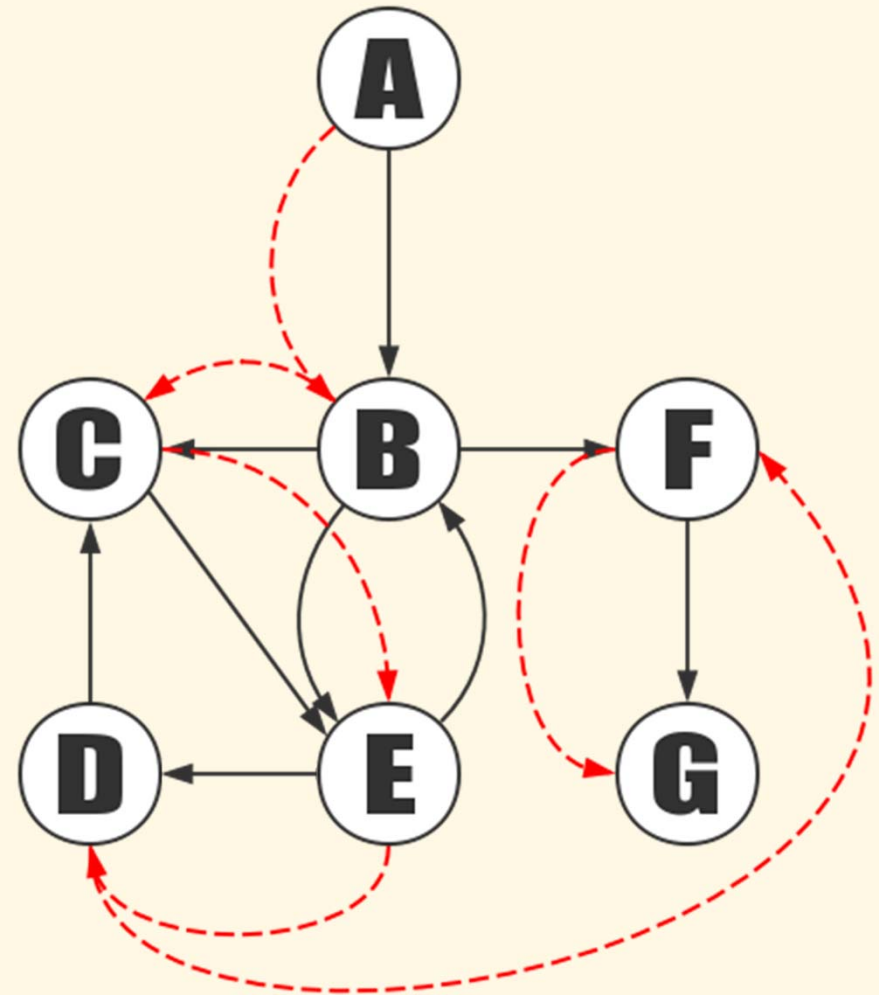
- ❑ Depth-first search (DFS)
- ❑ Breath-first search (BFS)
- ❑ Lowest-cost-first search (LCFS)

Graph Search Problem

- ▶ Given a graph G and a vertex v , find all vertices that can be reached from v , and (optionally) the best paths to those vertices

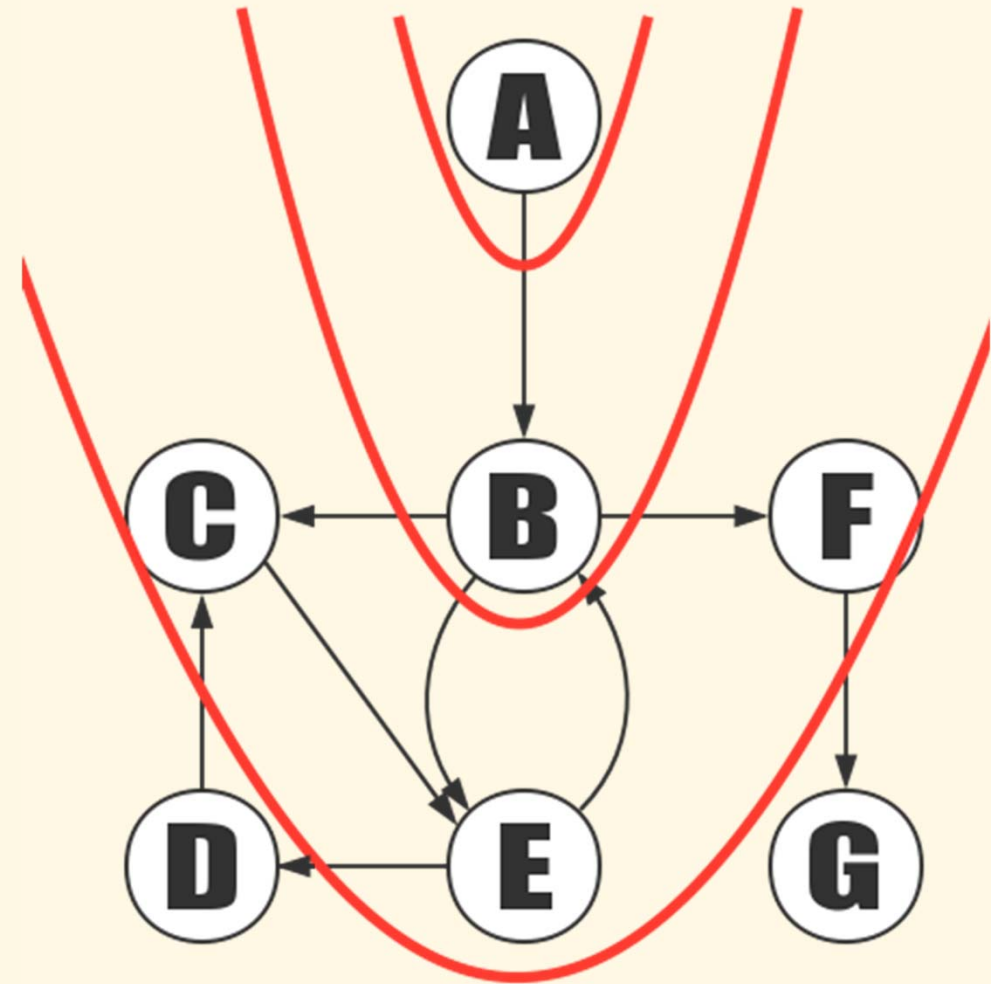
Depth-First Search (DFS)

- ▶ Algorithm:
 - ▶ Start with a vertex
 - ▶ Go to any vertex directly reachable from it that hasn't been visited
 - ▶ Mark the new vertex as visited
 - ▶ Explore the new vertex in a similar way
- ▶ 2 implementation methods:
 - ▶ Recursive
 - ▶ Iterative with help of a stack



Breadth-First Search (BFS)

- ▶ Algorithm:
 - ▶ Start with a vertex
 - ▶ Explore all directly reachable vertices from it that haven't been visited
 - ▶ Mark them as visited
 - ▶ Do similar process to these newly visited vertices
- ▶ Implementation method:
 - ▶ Iterative with help of a queue



Generic Search Algo

▶ 3 sets:

- ▶ Explored nodes (black set)
- ▶ Queued nodes (frontier, gray set)
- ▶ Unexplored nodes (white set)

▶ Algorithm:

Frontier $:= \{\langle s \rangle\}$

while Frontier $\neq \{\}$ **do**

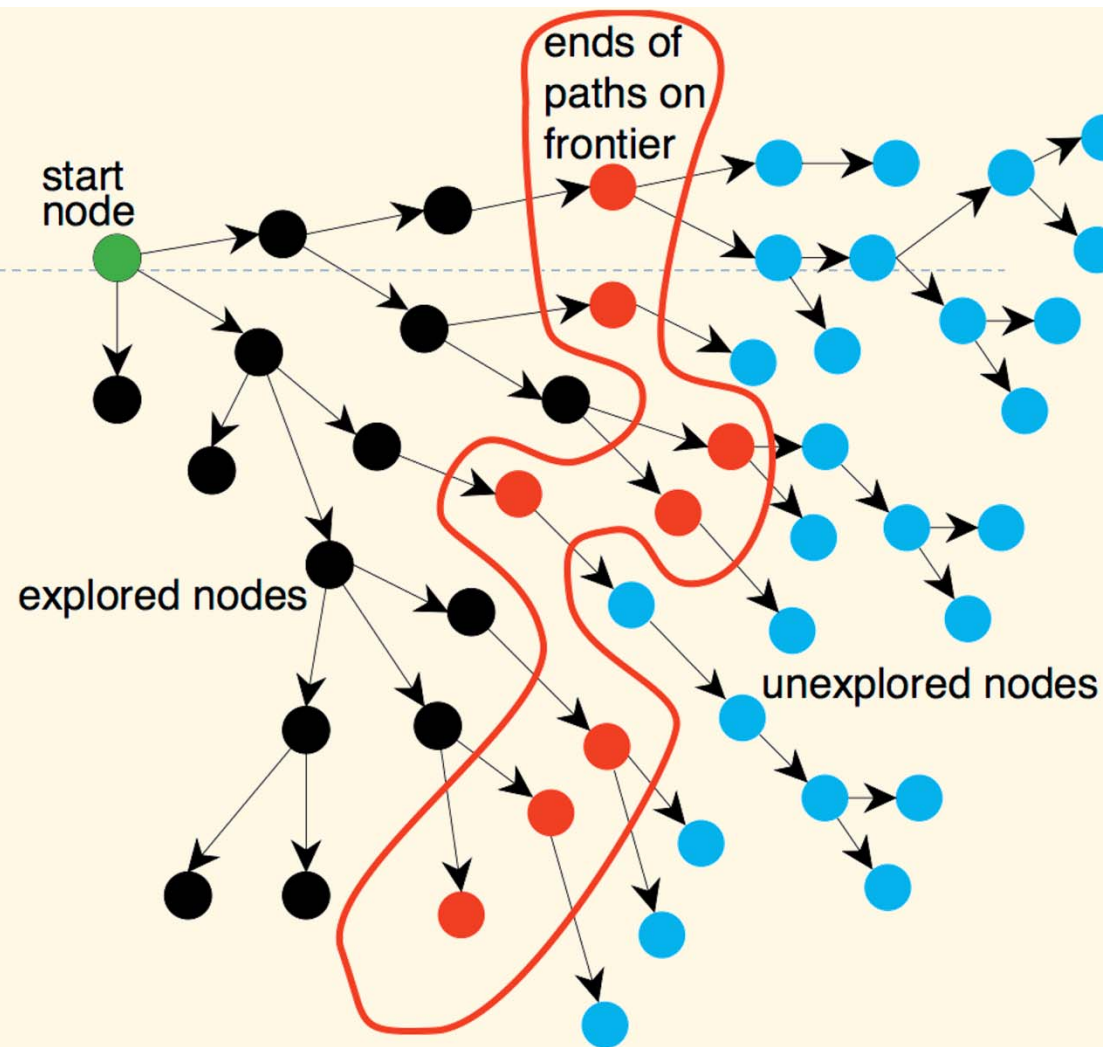
select and remove $\langle n_0, \dots, n_k \rangle$ from Frontier

if goal(n_k) **then**

return $\langle n_0, \dots, n_k \rangle$

 Frontier $:=$ Frontier $\cup \{\langle n_0, \dots, n_k, n \rangle : \langle n_k, n \rangle \in A\}$

return $\perp$

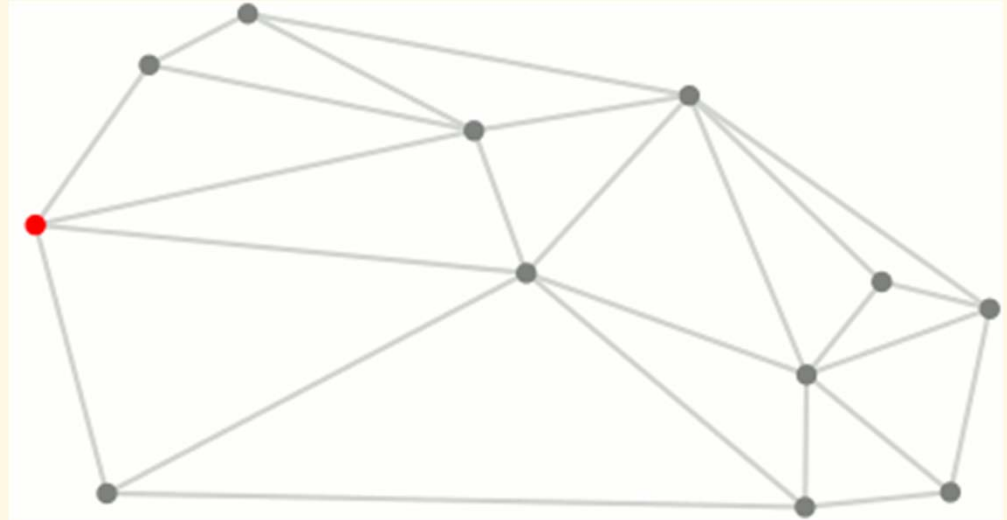


BFS and DFS as Special Cases

- ▶ From generic algorithm:
 - ▶ BFS: select = take first node from Frontier
 - ▶ DFS: select = take last node from Frontier

Lowest-Cost-First Search (Dijkstra's Algo)

- ▶ Idea: expand the wavefront to every possible paths by a constant speed until the target is reached
- ▶ From generic algorithm:
 - ▶ select = take node with lowest cost from Frontier
 - ▶ Frontier update: if a node is already included and the new cost is lower, replace its path
- ▶ Termination conditions:
 - ▶ When the target node is explored: to calculate path to one node
 - ▶ When Frontier is empty: to calculate path to every nodes



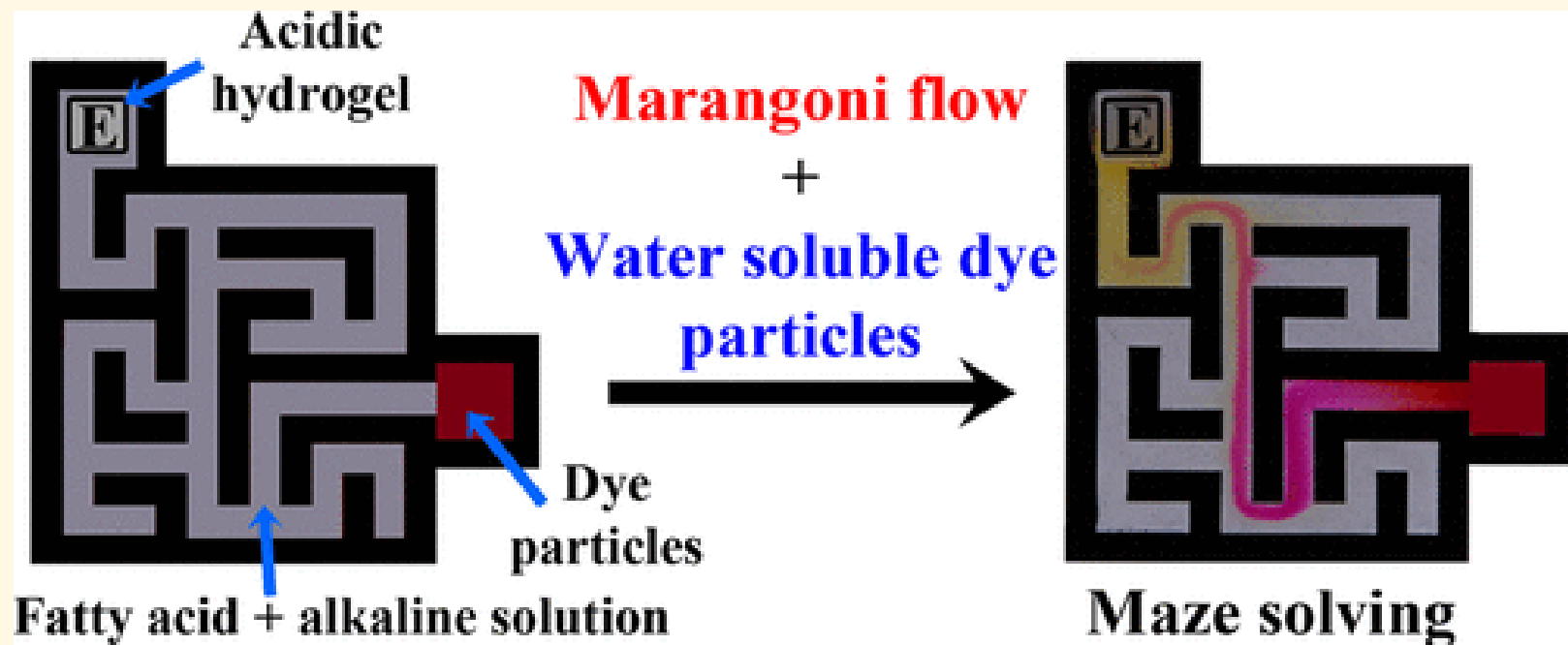
Dijkstra's Algo in Nature

- ▶ Lightning in extreme slow motion



Dijkstra's Algo in Nature

- ▶ Maze solving using fatty acid



Final Notes: Data Structures & Algorithms with C++ STL

Data Structures

- ▶ Dynamic array:
 - ▶ `std::vector<T>`
- ▶ Doubly linked list:
 - ▶ `std::list<T>`
- ▶ Stack and queue:
 - ▶ `std::stack<T>`
 - ▶ `std::queue<T>`
 - ▶ `std::deque<T>`
- ▶ Set:
 - ▶ `std::set<T>`
- ▶ Map:
 - ▶ `std::map<T>`
 - ▶ `std::multimap<T>`
- ▶ Tuple:
 - ▶ `std::pair<T>`
 - ▶ `std::tuple<T>`

Algorithms

▶ Sort:

- ▶ `std::sort<RandomAccessIterator>(first, last)`
- ▶ `std::sort<RandomAccessIterator, Compare>(first, last, comp)`
- ▶ `std::list<T>::sort()`
- ▶ `std::list<T>::sort<Compare>(comp)`

▶ Search:

- ▶ `std::find<InputIterator, T>(first, last, val)`
- ▶ `std::find_if<InputIterator, UnaryPredicate>(first, last, pred)`
- ▶ `std::search<ForwardIterator>(first1, last1, first2, last2)`
- ▶ `std::search<ForwardIterator, BinaryPredicate>(first1, last1, first2, last2, pred)`

Maps & Hash Functions

- ❑ Map (dictionary) data structure
- ❑ Hash functions
- ❑ Probing

Overview

- ▶ Arrays are integer indexed
 - ➔ How to generalize with different index types?
 - ➔ Pairs of $\langle key, value \rangle$ where *key* and *value* can be of any types

	KEYS	VALUES
	Jan	327.2
	Feb	368.2
	Mar	197.6
	Apr	178.6
	May	100.0
	Jun	69.9
	Jul	32.3
Aug	Aug	37.3
	Sep	19.0
	Oct	37.0
	Nov	73.2
	Dec	110.9

Terminology

- ▶ These terms may share the same similarities in different programming languages or libraries:
 - ▶ Map (C++, Java, Matlab)
 - ▶ Dictionary (C#, Python)
 - ▶ Associative array (JavaScript, PHP)
 - ▶ Lookup table (C#, Lua)
 - ▶ Hash table (Ruby)
- ▶ Maps are generally single valued (one key → one value)
 - ▶ Multi-valued maps can be considered as exercise

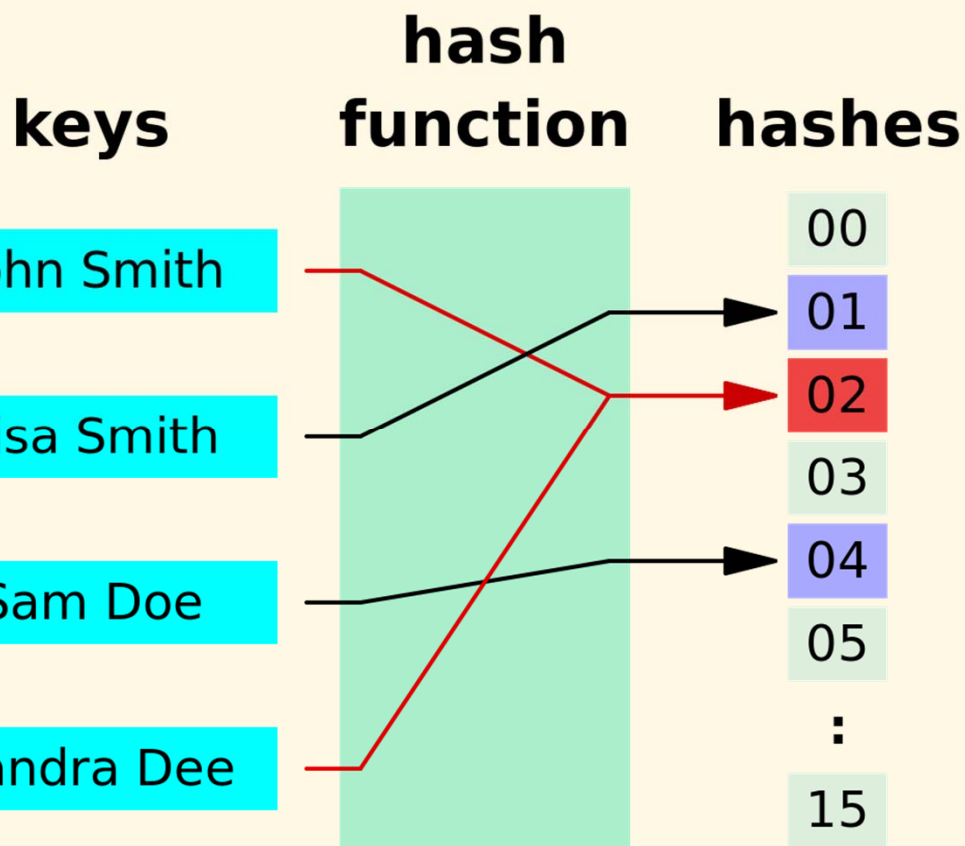
String-key dictionary

Introduction

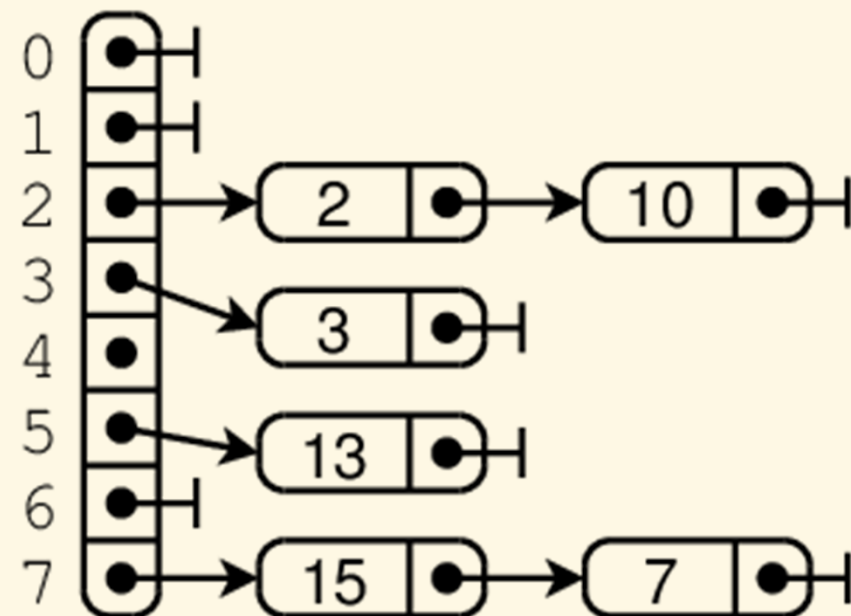
- ▶ A special case but very frequently used map data structure
- ▶ Mapping from **string keys** to other value types
- ▶ Main operations:
 - ▶ Insert elements
 - ▶ Remove elements
 - ▶ Find elements with their keys
- ▶ Applications:
 - ▶ Address book
 - ▶ Dictionary
 - ▶ Data lookup

Overview

- ▶ Arrange the data into groups with deterministic keys



Chaining



Declarations

```
typedef int (*STRING_HASH_FUNC)(char* key, int size);
```

```
typedef struct {  
    LLIST* entries;  
    int size;  
    STRING_HASH_FUNC hasher;  
} STR_MAP;
```

```
typedef struct {  
    char* key;  
    void* value;  
} STR_MAP_ITEM;
```


Operations

```
void mapInit(STR_MAP* m, int size, STRING_HASH_FUNC hasher);  
void mapDestroy(STR_MAP* m);
```

```
void mapInsert(STR_MAP* m, char* key, void* value, int sz);  
void* mapFind(STR_MAP* m, char* key);
```

```
typedef void (*MAP_CALLBACK_FUNC)  
    (STR_MAP_ITEM* item, void* user);  
void mapForEach(STR_MAP* m,  
    MAP_CALLBACK_FUNC callback, void* user);
```

```
void mapDelete(STR_MAP* m, char* key);  
void mapDeleteAll(STR_MAP* m);
```

Create map

```
void mapInit(STR_MAP* m, int size,
             STRING_HASH_FUNC hasher)
{
    m->size = size;
    m->hasher = hasher == NULL ? defaultHasher : hasher;
    m->entries = (void*)malloc(sizeof(LLIST) * m->size);

    for (int i = 0; i < m->size; i++)
        llInit(&m->entries[i]);
}
```

Destroy

```
void mapDestroy(STR_MAP* m)
{
    for (int i = 0; i < m->size; i++) {
        llForEach(m->entries[i], freeListElementData, NULL);
        llDestroy(&m->entries[i]);
    }

    free(m->entries);
}
```

Insert element

```
void mapInsert(STR_MAP* m, char* key, void* value, int sz) {
    LLIST* l = findEntry(m, key);
    LIST_ELEM* e = findElement(m, *l, key);

    if (e == NULL) {
        STR_MAP_ITEM item;
        int keylen = strlen(key);
        item.key = malloc(keylen+1);
        strcpy(item.key, key);

        item.value = malloc(sz);
        memcpy(item.value, value, sz);

        llInsertTail(l, &item, sizeof(item));
    } else {
        STR_MAP_ITEM* item = (STR_MAP_ITEM*)(e->data);
        free(item->value);
        item->value = malloc(sz);
        memcpy(item->value, value, sz);
    }
}
```

Find element

```
void* mapFind(STR_MAP* m, char* key)
{
    LLIST *l = findEntry(m, key);
    LIST_ELEM* e = findElement(m, *l, key);
    return e == NULL ? NULL :
        ((STR_MAP_ITEM*)(e->data))->value;
}
```

Delete element

```
void mapDelete(STR_MAP* m, char* key)
{
    LLIST *l = findEntry(m, key);
    if (*l == NULL) return;

    STR_MAP_ITEM* item =
        (STR_MAP_ITEM*)((*l)->data);
    if (strcmp(item->key, key) == 0) {
        free(item->key);
        free(item->value);
        llDeleteHead(l);
        return;
    }
}
```

```
LIST_ELEM* e;
for (e = *l; e->next != NULL;
     e = e->next)
{
    item = (STR_MAP_ITEM*)
        (e->next->data);
    if (strcmp(item->key, key) == 0)
    {
        free(item->key);
        free(item->value);
        llDeleteAfter(l, e);
        break;
    }
}
```

Iteration

```
void mapForEach(STR_MAP* m,
               MAP_CALLBACK_FUNC callback, void* user)
{
    for (int i = 0; i < m->size; i++) {
        llForEach(m->entries[i], (LLCALLBACK)callback, NULL);
    }
}
```

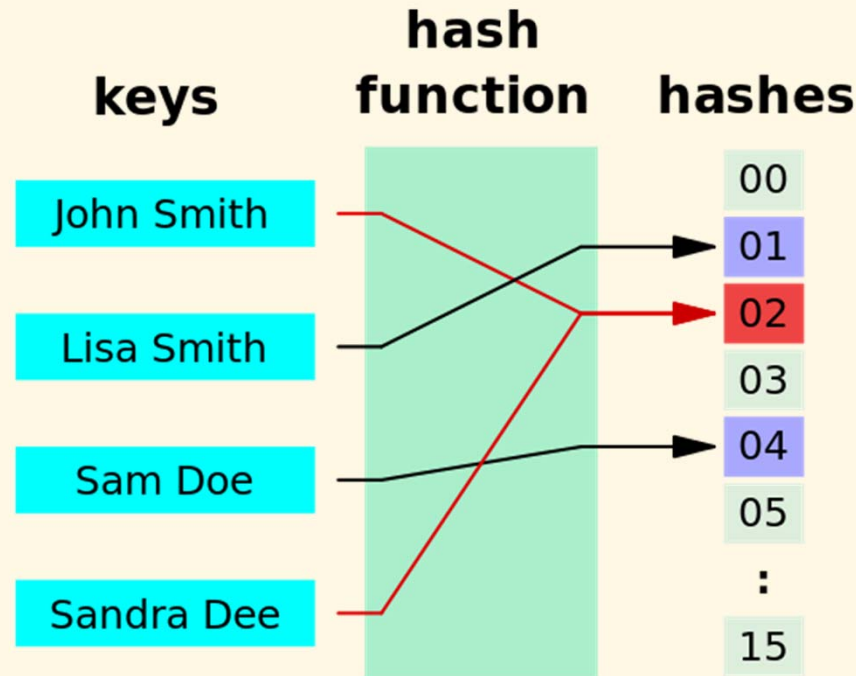
Delete all

```
void mapDeleteAll(STR_MAP* m)
{
    for (int i = 0; i < m->size; i++) {
        llForEach(m->entries[i], freeListElementData, NULL);
        llDeleteAll(&m->entries[i]);
    }
}
```

Hash functions

Definition

- ▶ Map from values of any type to fixed-size integers



- ▶ Usually composed of two functions
 - ▶ Hash code mapping: keys $\rightarrow$ integers
 - ▶ Compression mapping: integers $\rightarrow [0..N - 1]$

Good hash functions?

- ▶ Properties
 - ▶ Uniformity (probability distribution)
 - ▶ Efficiency
- ▶ Frequently used functions
 - ▶ Polynomial
 - ▶ Permutation
 - ▶ Logical operations (e.g., XOR)
- ▶ Attn: Variable-length keys can be resampled to become fixed-length before hashing

Generalized maps

Generalized key types

- ▶ Any type with following two operations can be used as key:
 - ▶ Hashing
 - ▶ Comparison for equality
- ▶ Examples:
 - ▶ Floating point values
 - ▶ 2D, 3D points
 - ▶ Structures
 - ▶ Binary data

Other map types

- ▶ Multi-valued maps
 - ▶ Aka multimap, multihash, multidict
- ▶ Two-way maps
 - ▶ Roles of keys and values are interchangeable

Exercises

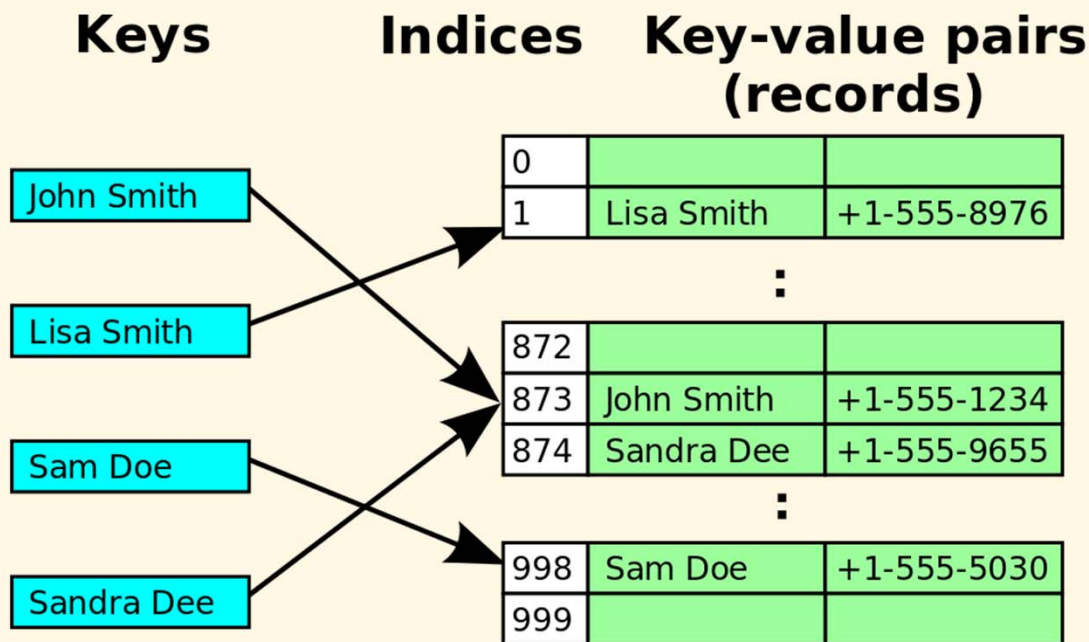
- ▶ Implement a generic hash table in C.
- ▶ Implement a generic hash table in C++.
- ▶ Implement a hash table where the keys are 3D points with `float`-typed components.

Probing

Introduction

- ▶ Linked-list based maps may be too complicated for implementation on resource-limited platforms or hardware

➔ Array based hash tables



- ▶ Need to resolve hash collisions?
➔ Probing methods

Linear probing: Insertion & Searching

- ▶ When insertion causes a collision, search the table for the closest following free location

[0]	72		[0]	72
[1]			[1]	15
[2]	18		[2]	18
[3]	43		[3]	43
[4]	36		[4]	36
[5]			[5]	10
[6]	6		[6]	6
[7]			[7]	5

Add the keys 10, 5, and 15 to the previous table .

Hash key = key % table size

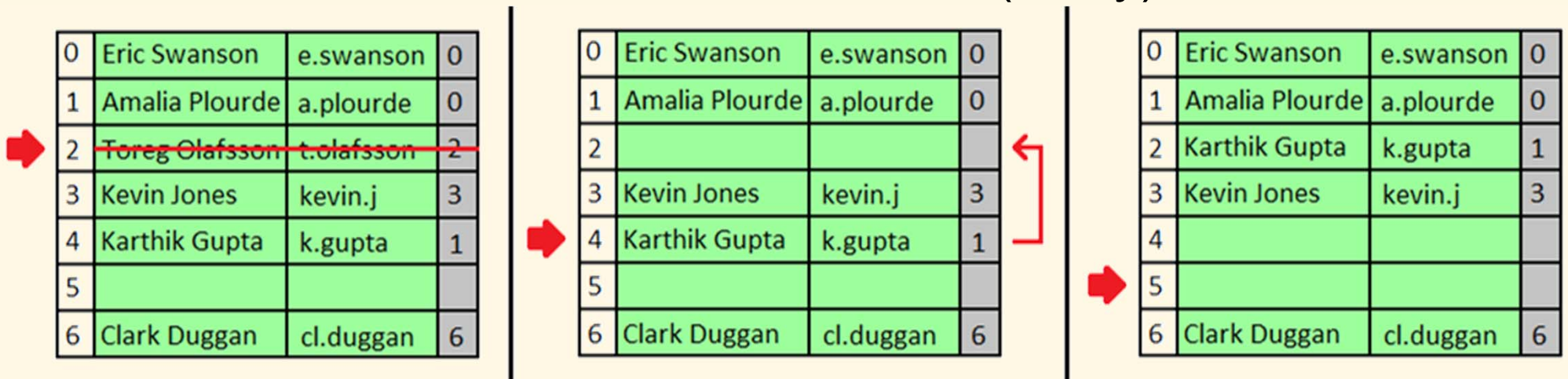
2 = 10 % 8

5 = 5 % 8

7 = 15 % 8

Linear probing: Deletion

- ▶ Deletion may cause problems to searching
- ▶ Solution 1:
 - ▶ Search forward for an element whose hash is $\leq$ that of the deleted element, and move it backward for replace
 - ▶ Do the same to the moved element (if any)...



- ▶ Solution 2: Lazy deletion with the help of a flag

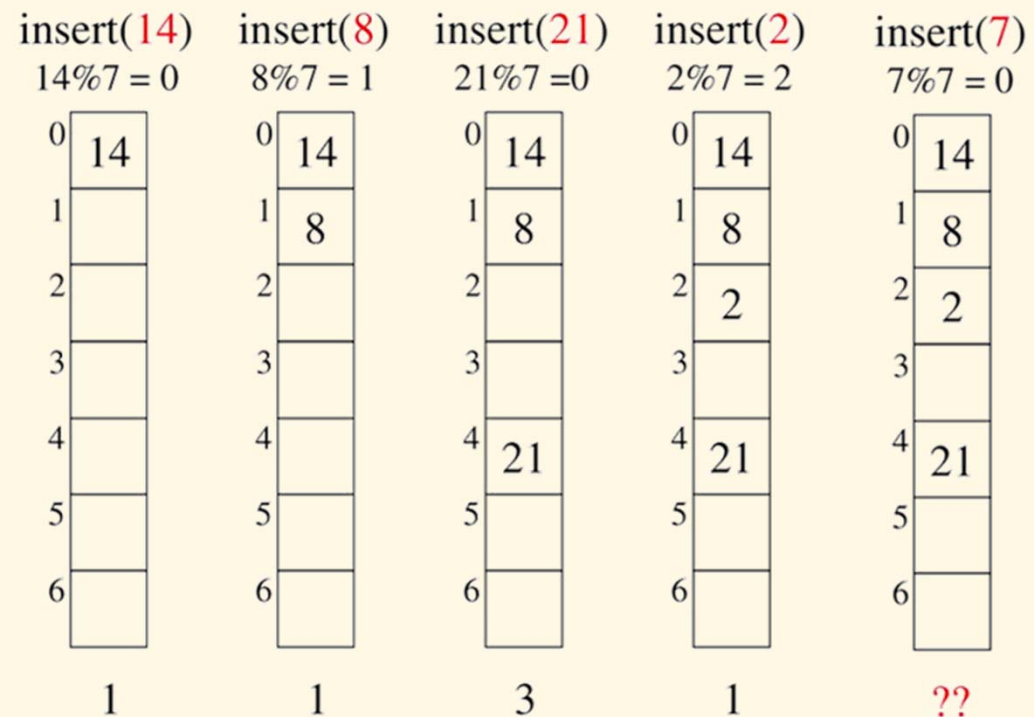
Quadratic probing

- ▶ Linear probing tends to have clustering problem
- ➔ Use a quadratic function to calculate the i^{th} probe position for a key k :

$$p(k, i) = (h(k) + i^2) \bmod N$$

where

- ▶ N : array size, better to be prime number
- ▶ $h(k)$: hash function for key k
- ▶ Not guaranteed to succeed when map is half full
- ▶ Must use lazy deletion



Double hashing

- ▶ Use a second hash function to resolve hash collisions:

$$p(k, i) = (h_1(k) + i \times h_2(k)) \bmod N$$

where

- ▶ $h_2(k)$ should never return 0

Lets say, Hash1 (key) = key % 13

Hash2 (key) = 7 - (key % 7)

Hash1(19) = 19 % 13 = 6

Hash1(27) = 27 % 13 = 1

Hash1(36) = 36 % 13 = 10

Hash1(10) = 10 % 13 = 10

Hash2(10) = 7 - (10%7) = 4

(Hash1(10) + 1*Hash2(10))%13= 1

(Hash1(10) + 2*Hash2(10))%13= 5

Collision

Homework

- ▶ Implement multiple hash tables for a student list so that we can find an item using different properties.